

PostgreSQL: Storing and Finding Data

Notes and exercises for all 44 chapters

Made on October 5, 2026

Contents

About this course

Level 1: The Basics

- 01 What Is a Database?
- 02 Tables, Rows and Columns
- 03 Installing PostgreSQL
- 04 Your First Database
- 05 Your First Table
- 06 Adding Data
- 07 Reading Data
- 08 Filtering Rows
- 09 Sorting and Limiting
- 10 Changing and Deleting Data
- 11 Project: My Book Collection

Level 2: Designing Good Tables

- 12 Data Types in Depth
- 13 Constraints
- 14 Primary Keys
- 15 Changing a Table's Shape
- 16 Relationships and Foreign Keys
- 17 Many-to-Many Relationships
- 18 Normalization
- 19 Project: Online Shop Database

Level 3: Asking Better Questions

- 20 Counting and Totals
- 21 Grouping
- 22 Joins
- 23 Outer Joins
- 24 Handy Built-in Functions
- 25 Subqueries
- 26 Common Table Expressions
- 27 Combining Results
- 28 Window Functions
- 29 Project: Sales Report

Level 4: Keeping Data Safe and Fast

- 30 Transactions

- 31 Indexes
- 32 Why Is My Query Slow?
- 33 Views
- 34 Users, Roles and Permissions
- 35 Backup and Restore
- 36 Project: Bank Account System

Level 5: PostgreSQL in Real Apps

- 37 JSON and Arrays
- 38 Functions and Triggers
- 39 Full-Text Search
- 40 Connecting from Node.js
- 41 Migrations
- 42 ORMs
- 43 PostgreSQL in the Cloud
- 44 Final Project

Hints

About this course

A **database** is an organized place to keep information so you can find it again quickly. Think of a well-run library, where every book has its place and you can find any one in seconds.

PostgreSQL (say "post-gres-Q-L", or just "Postgres") is a program that runs databases. It's free, it's **open source** (its code is public, so anyone can use it and check how it works), and it's one of the most popular databases in the world.

You talk to PostgreSQL in a language called **SQL** (Structured Query Language; say "S-Q-L" or "sequel"). Picture PostgreSQL as a librarian. SQL is the language you use to ask the librarian for things.

This course takes you from "what is a database?" all the way to designing, protecting, and connecting databases for real apps. There are 44 chapters in five levels. They go in order, and each one builds on the ones before it.

Before you start: nothing! This course assumes you've never touched a database. You don't need to know any programming language for Levels 1 to 4 either. SQL is its own small language, and you'll learn it from scratch here.

The hints for the exercises are in the [Hints section](#) at the end. Try each exercise on your own first, and read one hint at a time.

Level 1: The Basics

Goal: understand what a database is, and store, find, change, and delete data in a single table.

Chapters 01 and 02 need no setup at all. They're just reading and thinking.

01 What Is a Database?

What is it?

A **database** is an organized collection of information, stored so that it can be found, changed, and kept safe.

The program that looks after that information is called a **database management system** (people just say "database" for both). PostgreSQL is one of these programs. It runs on your computer (or a server somewhere), guards your data, and answers questions about it.

No setup in this chapter. It's all reading and thinking, so you know *what* you're installing before you install it.

Why does it matter?

Let's start with something you already know: a spreadsheet.

Imagine a small shop that keeps its orders in a spreadsheet. One row per order: customer name, address, what they bought, the price, the date. For 20 orders, this works fine.

Now imagine the shop grows. Three staff members, a website, 50,000 orders. Problems start piling up:

1. **It gets slow.** Scrolling and searching through 50,000 rows takes ages.
2. **People get in each other's way.** Two staff members open the file at once. Who wins when they both save?
3. **Mistakes slip in.** One row says `Kathmandu`, another `kathmandu`, another `Katmandu`. Someone types `ten` in the price column. Nothing stops them.
4. **The same fact is repeated.** A customer's address is typed out on every order. When they move house, someone has to fix it in 47 places, and they'll miss a few.

5. **Apps can't use it easily.** The website needs to save a new order in a fraction of a second, thousands of times a day, while people are still browsing.
6. **It's fragile.** Delete a column by accident, or lose power halfway through saving, and the data may be gone or broken.

A database is built to solve exactly these six problems. It's fast with millions of rows. It lets many people (and programs) work at the same time. It refuses bad data if you tell it the rules. It stores each fact once and links things together. And it's built so that a crash halfway through never leaves your data half-saved.

Spreadsheets aren't bad. They're great for small, personal, one-person data. But when many people, lots of data, or an app are involved, you want a database.

Real-world example

Think of the difference between a **pile of books** in a garage and a **library**.

Library	Database
The librarian	PostgreSQL, the program that manages everything
You ask the librarian for something	You send a request (a query)
The librarian's language	SQL , the language databases understand
The catalog, one card per book	A table , one row per item
Rules: every book has a shelf, you need a card to borrow	Constraints , the rules you give the database
You never wander into the back rooms yourself	You never open the database's files directly

You'll meet every one of these in the coming chapters.

How it works

Two parts: the server and you

A database has two sides.

The **server** is a program that starts when your computer starts and waits quietly in the background. It owns some files on your disk, and it's the *only* thing that touches them. You never open those files yourself.

You talk to the server through a **client**: a program that sends your requests and shows you the answers. In this course you'll use two clients: **psql** (you type commands) and **pgAdmin** (you click around in a window). Later, your own programs will be the client.



The word "server" can sound big and scary. Here it just means "the program that serves you answers". In this course, it runs on your own laptop.

Four things you do with data

Every app, from Instagram to your bank, does the same four things with its data:

Action	Everyday example	SQL command	Chapter
Create (add something new)	Sign up for an account	INSERT	06
Read (look something up)	Open your profile	SELECT	07
Update (change something)	Change your password	UPDATE	10
Delete (remove something)	Delete a post	DELETE	10

Together, these are called **CRUD**. If you can do these four things, you can build the data side of almost any app. Level 1 of this course is mostly about CRUD.

SQL: the language

SQL (Structured Query Language) is how you talk to the database. Here's one line of it:

```
SELECT title FROM books WHERE year < 1950;
```

Read it out loud: *select the title, from books, where the year is less than 1950*. SQL was designed to read like English, and mostly it does.

Notice what you **didn't** say: you didn't say *how* to find the books. No "start at the top, check each row, stop at the end". You just described what you wanted, and the database works out the fastest way to get it. That's a big part of why SQL is so pleasant to use.

If you've done the JavaScript course, you'll notice SQL feels quite different: there are no loops and no variables in a basic query. It's a language for *describing* data, not for giving step-by-step instructions.

Relational databases

PostgreSQL is a **relational database**. That means two things:

1. Data lives in **tables** (rows and columns, like a spreadsheet sheet). [Chapter 02](#) is all about them.
2. Tables can be **related** (linked) to each other: an order belongs to a customer, a customer has many orders. That's [chapter 16](#).

Other relational databases you'll hear about: MySQL, SQLite, Microsoft SQL Server, Oracle. They all speak SQL, with small differences in dialect, like British and American English. Learn PostgreSQL and you can read all of them.

There are also **non-relational** databases (often called NoSQL), like MongoDB, that organize data differently. Not this course. Relational databases are the most common kind by far, and the best place to start.

Where you'll meet databases

Everywhere:

- **A social app** stores users, posts, likes, and comments.
- **A bank** stores accounts and every transaction ever made.
- **An online shop** stores products, customers, orders, and reviews.
- **Your own React apps**. When a React app calls an API to fetch data (React chapter 18), the server on the other end is usually reading from a database like PostgreSQL. This is the "back end" that your front end talks to.

Common mistakes

1. Thinking a database is a file you open

You don't double-click a database like an Excel file. It's a program that's always running. You connect to it and send requests.

2. Mixing up PostgreSQL and SQL

SQL is the *language*. PostgreSQL is one *program* that understands the language. Like English and one particular person who speaks English.

3. Thinking you need to be good at maths or programming

You don't. What you need is clear thinking about how information is organized: "what things am I storing, and what do I want to know about them?" That's most of the skill.

4. Thinking spreadsheets are wrong

They aren't. A spreadsheet is the right tool for a small, personal list. A database is the right tool when many people, lots of data, or an app need to work with it.

Quick recap

- A database is an organized collection of information plus the program (PostgreSQL) that looks after it.
 - It solves the problems spreadsheets have at scale: speed, many users at once, bad data, repeated facts, apps, and safety.
 - The **server** runs in the background and owns the data. You talk to it through a **client** like psql or pgAdmin.
 - Almost everything you do with data is **CRUD**: create, read, update, delete.
 - **SQL** is the language. You describe what you want, and the database works out how to get it.
 - PostgreSQL is a **relational** database: data lives in tables, and tables can be linked.
-

Next: try the [exercises](#), then move on to [02 Tables, Rows and Columns](#).

Exercises

How to do these:

- No computer setup needed. These are thinking exercises.
 - Write your answers in a file: create a folder `PostgreSQL/playground/ch01/` and put an `answers.md` in it. You'll use this `playground` folder for the whole course.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your answers.
-

Exercise 1 (Easy): Spot the database

Pick three apps you use every day (for example a messaging app, a music app, a food delivery app).

For each one, list **three kinds of information** it must be storing somewhere. Think about what would be lost if the app's database vanished.

Example for a messaging app: users, messages, group chats.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): CRUD in real life

Take a to-do list app. Write down **one thing a user does** for each of the four CRUD actions:

- **Create:** ...
- **Read:** ...
- **Update:** ...
- **Delete:** ...

Then do the same for an online banking app.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): The spreadsheet problem

A school keeps student attendance in a single shared spreadsheet. Twelve teachers open it each morning and type in who is present. The head teacher uses it for end-of-year reports.

List **three problems** this spreadsheet will run into. For each problem, say which of the six database strengths from the notes would fix it.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Challenge): Read SQL before you learn it

You haven't learned SQL yet. But it's built from English words, so have a go. For each line below, write **in plain English** what you think it does. Don't worry about being exactly right.

```
SELECT name, price FROM products WHERE price < 10;
```

```
DELETE FROM tasks WHERE is_done = true;
```

```
UPDATE customers SET city = 'Pokhara' WHERE id = 7;
```

```
INSERT INTO books (title, author) VALUES ('Dune', 'Frank Herbert');
```

Bonus: which CRUD action is each one?

Stuck? The hint is in the [Hints section](#) at the end.

02 Tables, Rows and Columns

What is it?

A **table** stores one kind of thing: books, customers, tasks. Each **row** is one of those things. Each **column** is one detail that every row has.

That's nearly all there is to it. Almost everything in a relational database is a table.

Still no setup needed. This chapter is about learning to *see* tables in everyday information, which you'll do on paper.

Why does it matter?

Before you write any SQL, you need to look at real-world information and see the tables hiding in it. This is the skill that separates people who "know some SQL" from people who can build a database.

Get the tables right, and every question you ask later is easy. Get them wrong, and every question is a fight. So it's worth slowing down here.

Real-world example

A table is a lot like one sheet in a spreadsheet:

Spreadsheet	Database
One sheet	One table
One row	One row
A column heading, like "Title"	A column
One cell	One value

But there are some important differences, and they're what make a database more reliable:

1. **Every column has a fixed type.** The `year` column holds whole numbers, always. You can't type `nineteen` into it.
2. **Every row has exactly the same columns.** No row can have an extra cell on the end.
3. **No formatting.** No colors, no merged cells, no formulas. Just data.
4. **Rows have no fixed order.** This surprises people. When you want rows in a certain order, you ask for it when you read them ([chapter 09](#)).

5. **No empty grid.** A table only has the rows you put in. There's no "row 1000" waiting to be filled.

Another way to picture it: a box of index cards. The box is the table. Each card is a row. The labels printed on every card ("Title:", "Author:") are the columns.

How it works

The parts of a table

Here's a table called `books` :

column	column	column	column
id	title	author	year
1	The Hobbit	J.R.R. Tolkien	1937
2	Matilda	Roald Dahl	1988
3	Dune	Frank Herbert	1965

- The table has **4 columns** (`id`, `title`, `author`, `year`) and **3 rows**.
- Each row is one book. Each column is one fact about a book.
- Where a row and a column meet, there's exactly **one value**. `Frank Herbert` is the `author` of row 3.

One kind of thing per table

The `books` table holds only books. Not books *and* the people who borrowed them *and* the shelves they sit on. Each of those is a different kind of thing, so each gets its own table.

A quick test: can you describe every row with the same sentence? "This row is one book." If some rows would need a different sentence ("this row is a borrower"), you need another table.

Every column holds one type of value

Look at each column and ask "what kind of value goes here?":

Column	Kind of value
id	a whole number
title	text
author	text
year	a whole number

In [chapter 05](#), you'll tell PostgreSQL these kinds when you create a table. They're called **data types**. Once a column has a type, PostgreSQL refuses anything else. That's how the "someone typed `ten` in the price column" problem from chapter 01 disappears.

One value per cell

This is the most important rule in this chapter.

Each cell holds **one** value. Not a list. Not "Fantasy, Adventure". Not "555-1234 / 555-9876".

 Don't do this

 Each cell holds one value

title	genres
The Hobbit	Fantasy, Adventure

title	genre
The Hobbit	Fantasy

Why? If `genres` holds `Fantasy, Adventure`, then asking "which books are Adventure?" means pulling the text apart, and "how many Fantasy books are there?" becomes a mess. With one value per cell, both questions are a single easy line of SQL.

"But my book really has two genres!" True, and there's a proper way to handle that with a second table. It's [chapter 17](#). For now, just remember the rule.

Empty cells: NULL

Sometimes a value is missing. A book you haven't finished has no "date finished". The database has a special marker for this: **NULL**.

NULL means "there's no value here". It's not zero, and it's not empty text. It's the absence of a value, like a blank on a form. You'll learn how to work with it in [chapter 08](#).

Every row needs an identity

What if you own two different books both called *Dune*? Or two customers both named Sam Shrestha? You need a way to point at *exactly one row*.

That's what the `id` column is for. Every row gets its own number, and no two rows share one. It's the row's passport number, or a library barcode.

Nearly every table you make will have an `id` column, and PostgreSQL can fill it in for you automatically. [Chapter 14](#) explains the details.

Naming things

Names matter, because you'll type them hundreds of times. This course uses these conventions:

- **Lowercase, with underscores between words:** `published_year`, not `Published Year` or `publishedYear`. (This style is called **snake_case**, because the words look like they're joined by a snake.)
- **Table names are plural:** `books`, `customers`. A table holds many of them.
- **Column names are singular:** `title`, `email`. Each cell holds one.
- **Letters, numbers, and underscores only.** Start with a letter. No spaces.

Honest note: not every team does it this way. Some use singular table names (`book`). What matters most is picking one style and sticking to it.

From an app to tables: a worked example

Let's find the tables in a simple to-do app. Here's a method you can use every time:

Step 1: List the nouns. What *things* does the app deal with? Looking at a to-do app: tasks. That's it for a simple one.

Step 2: For each noun, list its details. A task has: a title, whether it's done, a due date, a priority.

Step 3: Turn each detail into a column with a type. Add an `id`.

Column	Type	Example
<code>id</code>	whole number	1
<code>title</code>	text	Buy milk
<code>is_done</code>	true or false	false
<code>due_date</code>	date	2026-10-03
<code>priority</code>	text	high

Step 4: Check the rules. One kind of thing per table? Yes, every row is one task. One value per cell? Yes.

Now suppose the app grows and adds **projects** ("Work", "Home") so tasks can be grouped. "Project" is a new noun, so it gets its own table: `projects` with `id` and `name`. Then each task needs to know which project it belongs to. You'd add a `project_id` column to `tasks` holding the project's `id`. That's a **relationship**, and it's the whole topic of [chapter 16](#). For now, just notice how a second noun became a second table.

Common mistakes

1. Lists in a cell

`tags = 'urgent, work, email'`. Covered above. One value per cell, always.

2. A column per item

`phone_1`, `phone_2`, `phone_3`. What happens when someone has four phones? Or when you want "everyone with this phone number"? This is the list-in-a-cell problem sideways. Same fix: a second table (chapter 16).

3. Mixing kinds of things in one table

A `products` table that holds books *and* DVDs *and* games, with columns for `author`, `director`, `platform`, mostly left blank. Sometimes a shared table is fine, but blank columns everywhere are a warning sign. [Chapter 18](#) gives you a way to decide.

4. Relying on row order

"The last row is the newest order." No. Rows have no order you can count on. If you care *when* something happened, store the date in a column.

5. Storing things you can work out

Storing `age` goes wrong every birthday. Store `birth_date` and work out the age when you need it. In general, store the facts, and calculate the rest.

Quick recap

- A **table** holds one kind of thing. A **row** is one of them. A **column** is one detail every row has.
- Every column holds one type of value, and every cell holds exactly **one** value. No lists.
- A missing value is **NULL**: not zero, not empty text, just "nothing here".
- Every row gets an **id**, so you can always point at exactly one row.
- Name things in lowercase with underscores. Tables plural, columns singular.
- To find tables in an app: list the nouns, list each noun's details, turn details into typed columns, then check the rules.

Next: try the [exercises](#), then move on to [03 Installing PostgreSQL](#).

Exercises

How to do these:

- Still no software needed. Write your answers in `playground/ch02/answers.md`.
- Draw tables the way the notes do, or as Markdown tables. Whatever is quickest.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your answers.

Exercise 1 (Easy): Label the parts

Here's a table called `products`:

id	name	price	in_stock
1	Notebook	3.50	true
2	Pen	1.20	true
3	Desk lamp	24.99	false
4	Stapler	8.00	true

Answer:

1. How many rows? How many columns?
2. What value is in the `price` column of the row where `name` is `Desk lamp`?
3. For each column, what kind of value does it hold (whole number, decimal number, text, or true/false)?
4. Which column is the row's identity?

Exercise 2 (Easy): Design a contacts table

You're building a simple contacts app, like the one on your phone. Design a `contacts` table.

- Give it an `id` and at least **five** other columns.
- For each column, write its name (following the naming conventions), the kind of value it holds, and one example value.

- Fill in two example rows.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Fix the bad table

A school made this `students` table. It has at least **three** problems from this chapter.

id	name	courses	phone_1	phone_2	age
1	Anita Rai	Maths, Art	9841000001	9841000002	15
2	Bikash Thapa	Maths	9841000003		16
3	Chandra Gurung	Art, Music, PE	9841000004		15

1. Find the problems and explain *why* each is a problem (what goes wrong later).
2. Suggest a fix for each. You don't have to know how to do the fix in SQL yet. Just describe it, like "move X to its own table".

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Find the tables in a library

A small library needs to keep track of its books, its members, and who has borrowed what.

1. List the nouns (the kinds of things).
2. Design a table for each, with columns and types.
3. Check each table against the rules: one kind of thing per table, one value per cell.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A cinema booking app

Design the tables for an app where people book cinema seats.

Think about: films, screenings (a film showing at a certain time in a certain room), rooms, seats, customers, and bookings.

1. List the tables and their columns.

2. Draw arrows showing which tables point to which (like `tasks.project_id` → `projects.id` in the notes).
3. Write one sentence describing what a single row in each table means. If you can't, that table is probably mixing things up.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

03 Installing PostgreSQL

What is it?

In this chapter you install three things, all in one go:

- 1. **The PostgreSQL server:** the program that stores your data and runs in the background.
- 2. **psql:** a client where you *type* commands. It looks like a plain black terminal window.
- 3. **pgAdmin:** a client where you *click*. It has menus, a tree of databases on the left, and a box to type queries in.

By the end, you'll have run your first SQL statement.

Why does it matter?

You can't practice on paper forever. From chapter 04 on, every chapter has you typing real SQL and seeing real answers. That needs a real database on your computer.

Why two clients? **psql** is what most developers use day to day. It's fast, it's on every server, and knowing it makes you look like you know what you're doing. **pgAdmin** is friendlier when you want to *look* at things: browse tables, see data in a grid, click through the structure. You'll use psql for most of this course and pgAdmin whenever you want a visual check.

Real-world example

Installing PostgreSQL is like setting up a new shop:

Setting up a shop	Installing PostgreSQL
Hire a manager who's always there	The server, which runs in the background from now on
Give the manager a master key	The <code>postgres</code> password you choose during install
A phone line to call the manager on	The port (5432), the number the server listens on
The counter where you speak to staff	psql and pgAdmin

You'll choose the master key during installation. **Write it down.** More on that below.

How it works

These steps are for **Windows**, which is what you're using. There's a short note for Mac at the end.

Honest warning: installer screens change a little between versions. If a screen looks slightly different from what's described here, read it and pick the default. The defaults are fine.

Step 1: Download the installer

1. Go to postgresql.org/download/windows.
2. Click **Download the installer**. This takes you to a page run by a company called EDB, which builds the official Windows installer.
3. Pick the newest version (18 or higher) for **Windows x86-64**, and download it.

The file is a few hundred megabytes. It contains everything: the server, psql, and pgAdmin.

Step 2: Run the installer

Double-click the downloaded file. Windows may ask if you want to allow it to make changes. Click yes. Then walk through the screens:

Screen	What to do
Installation Directory	Leave the default (<code>C:\Program Files\PostgreSQL\18</code>).
Select Components	Keep PostgreSQL Server , pgAdmin 4 , and Command Line Tools ticked. Untick Stack Builder (it's for extra add-ons you don't need).
Data Directory	Leave the default. This is where your databases will live.
Password	Choose a password for the <code>postgres</code> user. This is the master key. Write it down somewhere safe right now.
Port	Leave <code>5432</code> .
Advanced Options (Locale)	Leave <code>[Default locale]</code> .
Pre Installation Summary	Click Next.
Ready to Install	Click Next, and wait a few minutes.
Completing	If it offers to launch Stack Builder, untick that. Click Finish.

About that password. `postgres` is the name of the built-in administrator account, called a **superuser**. It can do anything. The password you chose is the only way in, and if you forget it, resetting it is fiddly. Save it in your password manager or write it on paper.

For a learning setup on your own laptop, something simple is fine. You're not protecting anything valuable yet.

Step 3: Meet `psql`

Open the Start menu and type **SQL Shell**. Open **SQL Shell (`psql`)**. A black window appears and asks you a few questions. **Press Enter for each one** to accept the default, then type your password:

```
Server [localhost]:          ← Enter
Database [postgres]:        ← Enter
Port [5432]:                 ← Enter
Username [postgres]:        ← Enter
Password for user postgres:  ← type your password (nothing shows while you type), then Enter
```

What those questions mean:

- **Server** `localhost` means "this computer". The server is running right here.
- **Database** `postgres` is a default database that always exists. It's just somewhere to start.
- **Port** `5432` is the number the server listens on.
- **Username** `postgres` is the superuser account.

If it worked, you'll see something like:

```
psql (18.0)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

postgres=#
```

That warning is normal on Windows and you can ignore it. It's about how the terminal shows special characters like `é` or `ñ`.

The important part is the last line: `postgres=#`. This is the **prompt**. It's `psql` saying "I'm connected to the database called `postgres`, and I'm ready. Type something."

Step 4: Your first SQL

At the prompt, type this exactly, including the semicolon, and press Enter:

```
SELECT version();
```

You'll get back something like:

```
          version
-----
PostgreSQL 18.0 on x86_64-windows, compiled by msvc-19.44.35213, 64-bit
(1 row)
```

You just sent a request to the server, and it answered. That's the whole client-server picture from chapter 01, happening for real.

Try one more:

```
SELECT 2 + 2;
```

```
?column?
-----
         4
(1 row)
```

The database does maths too. (The `?column?` heading means "you didn't name this column". You'll learn to name it in [chapter 07](#).)

Two rules that will save you frustration:

1. **Every SQL statement ends with a semicolon ;**. If you press Enter without one, psql thinks you're not finished and waits for more. The prompt changes to `postgres-#` (a dash instead of `=`). Just type `;` and press Enter.
2. **Uppercase doesn't matter to PostgreSQL.** `SELECT` and `select` are the same. This course writes SQL words in UPPERCASE and your own names in lowercase, because it's easier to read.

To leave psql, type `\q` and press Enter. To come back, open SQL Shell again.

Step 5: Meet pgAdmin

Open the Start menu, type **pgAdmin**, and open **pgAdmin 4**. It takes a moment to start and opens in a window (or a browser tab).

1. The first time, it asks you to set a **master password**. This is pgAdmin's *own* password, used to protect the server passwords it saves for you. It doesn't have to match your `postgres` password, but you can use the same one to keep life simple.

2. On the left, you'll see a tree. Expand **Servers**. There's an entry called something like **PostgreSQL 18**. Click it.
3. It asks for the password of the `postgres` user. Type the one you chose during install, tick **Save Password**, and click OK.
4. Expand **PostgreSQL 18** → **Databases**. You'll see the `postgres` database.
5. Click `postgres`, then in the top menu choose **Tools** → **Query Tool**. A text area opens.
6. Type `SELECT version();` and press **F5** (or click the ► **Execute** button). The result shows in a grid underneath.

Same server, same answer, different window. That's the point: the clients are just different ways to talk to the same database.

What's running now?

The PostgreSQL server is now a **Windows service**. It starts when your computer starts and sits quietly in the background, using very little memory. You don't need to start or stop it yourself.

If you're ever curious, open the Start menu, type **Services**, and look for `postgresql-x64-18`. It should say "Running".

Optional: run psql from any terminal

The SQL Shell shortcut works fine for this course. But if you'd like to type `psql` in any terminal (like the one in VS Code), Windows needs to know where to find it:

1. Open the Start menu, type **environment variables**, and choose **Edit the system environment variables**.
2. Click **Environment Variables...**
3. Under **User variables**, select **Path** and click **Edit**.
4. Click **New** and paste: `C:\Program Files\PostgreSQL\18\bin` (change `18` if your version is different).
5. Click OK on every window. Then **close and reopen** any terminal.

Now `psql -U postgres` in any terminal connects you, after asking for your password. (`-U` means "as this user".)

If you're on a Mac

The easiest route is [Postgres.app](https://postgres.app), which gives you the server and psql. For pgAdmin, download it separately from pgadmin.org. Everything else in this course is the same.

Common mistakes

1. Forgetting the `postgres` password

The most common problem by far. If you're reading this after forgetting it: the simplest fix is to uninstall PostgreSQL, delete the data folder (`C:\Program Files\PostgreSQL\18\data`), and reinstall. You'll lose your practice databases, which is fine this early. Then write the password down.

2. Mixing up the two passwords in pgAdmin

pgAdmin asks for a **master password** (its own) when it opens, and the **postgres password** (the server's) when you connect to the server. If you used the same one for both, this never bites you.

3. Waiting forever after typing a statement

You forgot the semicolon. Look at the prompt: `postgres-#` means "waiting for more". Type `;` and press Enter.

4. "Port 5432 is already in use" during install

Something else is using that number, usually an older PostgreSQL install. Either uninstall the old one first, or let the installer use a different port like `5433`, and remember to type it whenever `psql` asks for the port.

5. Panicking about the code page warning

The `Console code page` warning in `psql` is harmless. Everything works.

Quick recap

- One installer gives you the **server, psql** (type), and **pgAdmin** (click).
- The `postgres` user is the superuser. Its password is your master key. **Write it down.**
- The server listens on port `5432` and runs in the background as a Windows service.
- In `psql`, every statement ends with `;`. If the prompt shows `-#`, it's waiting for one.
- `SELECT version();` proves you're connected. `\q` leaves `psql`.

Next: try the [exercises](#), then move on to [04 Your First Database](#).

Exercises

How to do these:

- These make sure your setup works and get your fingers used to `psql`. There's nothing to save this time.
- Try on your own first. Only open a hint if you've been stuck for a while.
- If anything doesn't work, tell Claude exactly what you typed and what appeared on screen.

Exercise 1 (Easy): In and out

1. Open SQL Shell (psql) and connect with all the defaults.
 2. Run `SELECT version();` and read which version you have.
 3. Quit with `\q`.
 4. Open it again and connect again. This should feel boring by the third time. That's the goal.
-

Exercise 2 (Easy): The database does maths

In psql, run each of these and look at the answers:

```
SELECT 10 * 5;  
SELECT 100 / 8;  
SELECT 100 / 8.0;
```

One of those answers might surprise you. Write down which one, and your guess at why. (You'll get the real explanation in [chapter 12](#).)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Easy): The semicolon trap

On purpose, type `SELECT 1 + 1` **without** a semicolon and press Enter.

1. What does the prompt look like now?
2. Type `;` and press Enter. What happens?

Getting stuck like this will happen to you by accident many times. Now you know how to get out.

Exercise 4 (Medium): Text and time

Run these:

```
SELECT 'Hello, database!';  
SELECT now();  
SELECT current_date;
```

Notice that text goes inside **single quotes** `'...'`. Now try it with double quotes:

```
SELECT "Hello, database!";
```

You'll get an error. Copy the first line of the error into your notes. In [chapter 06](#), you'll learn exactly why single and double quotes mean different things in SQL.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Medium): The same thing in pgAdmin

1. Open pgAdmin, connect to your server, and open the Query Tool on the `postgres` database.
2. Run `SELECT now();` there.
3. Run `SELECT 'psql' AS client;` in psql, and `SELECT 'pgAdmin' AS client;` in pgAdmin. Look at how each one shows the result.

Which do you prefer for reading results? There's no right answer. You'll use both.

04 Your First Database

What is it?

One PostgreSQL server can hold many **databases**. Each one is a separate, walled-off collection of tables. In this chapter you create your own database called `practice`, connect to it, and learn your way around `psql`.

Why does it matter?

Right now you're connected to the `postgres` database, which is a built-in one that PostgreSQL uses for its own housekeeping. You *could* practice there, but it's like doing your homework on the kitchen table where everyone eats. Better to have your own room.

You'll also learn the handful of `psql` commands that you'll use every single day: list the databases, connect to one, look around, get help, quit. Knowing these makes everything else faster.

Real-world example

Think of the server as an **apartment building**:

Apartment building	PostgreSQL
The building	The server
One apartment	One database
The rooms and furniture inside	The tables and data
Walking into a different apartment	Connecting to a different database

Apartments don't share furniture. A table in one database is invisible from another. That's useful: your practice database can't mess up a real one.

How it works

Creating a database

Open `psql` (SQL Shell) and connect with the defaults. At the `postgres=#` prompt, type:

```
CREATE DATABASE practice;
```

PostgreSQL answers:

```
CREATE DATABASE
```

That's it. PostgreSQL repeats the command name back when a command succeeds and has nothing else to say. It looks like an echo, but it means "done".

Listing databases: \l

Type `\l` (that's a backslash and a lowercase L) and press Enter:

```
                                List of databases
   Name   |  Owner   | Encoding | ...
-----+-----+-----+-----
postgres | postgres | UTF8     | ...
practice  | postgres | UTF8     | ...
template0 | postgres | UTF8     | ...
template1 | postgres | UTF8     | ...
(4 rows)
```

Your `practice` database is there. The others:

- `postgres`: the default database you've been connected to.
- `template0` and `template1`: blueprints PostgreSQL copies when you create a new database. Leave them alone.

The real output has more columns. Don't worry about them.

Two kinds of commands

You've now seen both kinds of thing you can type in psql, and it's important to tell them apart:

	SQL statements	psql commands
Example	<code>CREATE DATABASE practice;</code>	<code>\l</code>
Start with	A word like <code>SELECT</code> , <code>CREATE</code>	A backslash <code>\</code>
End with	A semicolon <code>;</code>	Nothing, just press Enter
Who understands it	The PostgreSQL server	Only psql

	SQL statements	psql commands
Case	Doesn't matter	Matters: <code>\l</code> and <code>\L</code> are different

SQL is the real language, and it works everywhere: psql, pgAdmin, your future apps. The backslash commands are psql's own shortcuts for common tasks. They're called **meta-commands**, and pgAdmin doesn't understand them. It has buttons instead.

Connecting to a database: `\c`

You created `practice`, but you're still inside `postgres`. Look at the prompt. To move:

```
\c practice
```

```
You are now connected to database "practice" as user "postgres".
practice=#
```

The prompt changed to `practice=#`. From now on, everything you type happens inside `practice`. **Always glance at the prompt before you create or delete anything.** It tells you where you are.

Reading the prompt

The prompt is packed with information:

```
practice=#
|         |
|         | └─ # means you're a superuser (the all-powerful postgres account)
|         └─ = means "ready for a new statement"
└────────── the database you're connected to
```

When the second-to-last character isn't `=`, psql is waiting for you to finish something:

Prompt	What psql is waiting for
<code>practice=#</code>	Nothing. Ready.
<code>practice-#</code>	A semicolon. You started a statement and didn't end it.
<code>practice(#</code>	A closing bracket <code>)</code> .
<code>practice'#</code>	A closing single quote <code>'</code> .
<code>practice"#</code>	A closing double quote <code>"</code> .

If you get stuck and don't know what it wants, press **Ctrl+C**. That throws away what you typed and gives you a fresh `=#` prompt.

Getting help

Two commands you'll come back to often:

- `\?` lists every psql meta-command. It's long. Press the **space bar** to page through, and **q** to stop.
- `\h` followed by an SQL command shows its syntax. Try `\h CREATE DATABASE`.

The output of `\h` looks scary at first, full of square brackets and `|` signs. Square brackets mean "optional", and `|` means "or". You'll get used to reading it. And you can always ask Claude instead.

Quitting: `\q`

`\q` quits psql. Your database and data stay exactly as they were. The server keeps running in the background.

Deleting a database

Make a throwaway one, then delete it:

```
CREATE DATABASE scratch;  
DROP DATABASE scratch;
```

```
CREATE DATABASE  
DROP DATABASE
```

`DROP` is SQL's word for "delete this whole thing". It's instant and there's no undo, no recycle bin. It asks no questions.

You can't drop the database you're currently connected to:

```
DROP DATABASE practice;
```

```
ERROR:  cannot drop the currently open database
```

That's PostgreSQL being careful. Connect somewhere else first (`\c postgres`), then drop.

Writing SQL that's easy to read

Three habits worth starting now:

1. Comments. Anything after two dashes `--` is a note for humans. PostgreSQL ignores it.

```
-- This makes a database for the SQL course.  
CREATE DATABASE practice;
```

2. UPPERCASE for SQL words, lowercase for your names. PostgreSQL doesn't care (`create database practice;` works), but your eyes will thank you.

3. Several lines are fine. PostgreSQL only cares about the semicolon. You can spread one statement over many lines:

```
CREATE DATABASE  
practice;
```

Set up your playground

You'll write a lot of SQL in this course, and it's worth saving. Create this folder now, inside your `PostgreSQL` folder:

```
PostgreSQL/  
└─ playground/  
    └─ ch01/      (you may already have these from the earlier exercises)  
    └─ ch02/  
    └─ ch04/
```

For each chapter, save your queries in `.sql` files, like `playground/ch04/ex1.sql`. VS Code understands `.sql` files and colors the SQL words for you.

Running a `.sql` file: `\i`

Create `playground/ch04/hello.sql` in VS Code with this inside:

```
-- My first SQL file  
SELECT 'Hello from a file!' AS greeting;
```

Then in `psql`, run it with `\i` and the full path. **Use forward slashes**, even on Windows:

```
\i C:/Users/YourName/Downloads/Projects/Sandip/docs/PostgreSQL/playground/ch04/hello.sql
```



```
greeting
-----
Hello from a file!
(1 row)
```

`\i` means "include": run every statement in this file as if I'd typed it. This is how you'll run your exercise files.

In **pgAdmin**, the Query Tool has a folder icon (Open File) that loads a `.sql` file into the editor. Then press F5.

Connecting straight from a terminal

If you did the optional PATH step in chapter 03, you can skip the SQL Shell questions and connect directly from any terminal:

```
psql -U postgres -d practice
```

`-U` is the user, `-d` is the database. It asks for your password, and you land in `practice=#`.

Common mistakes

1. Putting a semicolon on a meta-command

`\c practice;` sometimes works by luck and sometimes gives strange results. Meta-commands don't take semicolons.

2. Forgetting the semicolon on SQL, then typing the next command

```
practice=# CREATE DATABASE scratch
practice=# \l
```

Now `\l` gets tangled up with the unfinished `CREATE`. Press Ctrl+C, and start again with the semicolon.

3. Working in the wrong database

You created a table, and it "disappeared". Check the prompt. You're probably in `postgres` when you meant `practice`, or the other way round.

4. Typing `\L` or `\C`

Meta-commands are case-sensitive. `\l` lists databases. `\L` does something else. Always lowercase unless told otherwise.

5. Using backslashes in `\i` paths

`\i C:\Users\...` confuses psql, because `\` is its special character. Use `C:/Users/...`.

Quick recap

- One server holds many **databases**, each a walled-off collection of tables. `CREATE DATABASE name;` makes one.
 - **SQL statements** end with `;` and work everywhere. **Meta-commands** start with `\`, have no semicolon, and only work in psql.
 - `\l` lists databases, `\c name` connects to one, `\q` quits, `\?` and `\h` give help.
 - The prompt tells you **where you are** and **what psql is waiting for**. `=#` means ready. **Ctrl+C** escapes a stuck prompt.
 - `DROP DATABASE` deletes a database instantly, with no undo.
 - Save your work in `playground/chNN/*.sql` and run files with `\i C:/path/with/forward/slashes.sql`.
-

Next: try the [exercises](#), then move on to [05 Your First Table](#).

Exercises

How to do these:

- Work in psql. Save each answer in `playground/ch04/ex1.sql`, `ex2.sql`, and so on, and run it with `\i`.
 - Meta-commands (`\l`, `\c`) can't go in a `.sql` file you run from pgAdmin, but they're fine in a file you run with `\i` in psql.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Make, list, connect, leave

1. Create a database called `sandbox`.
2. List all databases and confirm it's there.
3. Connect to it. Check the prompt says `sandbox=#`.
4. Connect back to `practice`.

Which of those four steps were SQL, and which were meta-commands?

Exercise 2 (Easy): Clean up

Delete the `sandbox` database.

Then try to delete `practice` **while connected to it**, and read the error. Don't actually delete `practice`. You need it for the rest of the course.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Easy): Escape the trap

Type each of these, exactly as written, one at a time. After each, look at the prompt and work out what `psql` is waiting for. Then give it what it wants.

1. `SELECT 'hello`
2. `SELECT (1 + 2`
3. `SELECT 5 * 5`

Write down the three different prompts you saw.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): A file with several statements

Create `playground/ch04/ex4.sql` containing **three** `SELECT` statements, each on its own line, each with a comment above it saying what it does. For example, one could work out how many days are in 12 weeks.

Run the file with `\i`. You should see three separate results.

Expected shape of the output:

```
?column?
-----
      84
(1 row)

...two more results...
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Read the help

Run `\h CREATE DATABASE` and look at the syntax it prints.

1. Everything in square brackets `[...]` is optional. How many optional parts can you count?
2. One of them is `OWNER`. Based on the name, what do you think it does?
3. Try creating a database using **one** of the optional parts, then drop it again. If you get an error, read it, and try to work out what went wrong.

Stuck? The hint is in the [Hints section](#) at the end.

05 Your First Table

What is it?

`CREATE TABLE` tells PostgreSQL: "I want a new table called *this*, with *these* columns, and each column holds *this kind* of value."

Those kinds of value are called **data types**. This chapter introduces the four you'll use most. [Chapter 12](#) covers the rest.

Why does it matter?

In chapter 02 you designed tables on paper. Now you make them real. And unlike a spreadsheet, where a column is just a heading, a database column is a **promise**: "this column will only ever hold whole numbers." PostgreSQL enforces that promise from the moment you create the table.

That's where a lot of the safety comes from. Bad data doesn't get in, because the table's shape won't allow it.

Real-world example

A table is like a **form** with labeled boxes:

A paper form	A table
The form design: which boxes, what labels	<code>CREATE TABLE</code>
"Year of birth: [][][][]" — four digit boxes	A column with type <code>integer</code>
A box that says "Name" with a long line	A column with type <code>text</code>
A tick box	A column with type <code>boolean</code>
One filled-in copy of the form	One row

Designing the form is separate from filling it in. `CREATE TABLE` designs it. `INSERT` (next chapter) fills it in.

How it works

Creating the table

Connect to `practice` (`\c practice`), and create the `books` table from the last few chapters:

```
CREATE TABLE books (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  title text,
  author text,
  year integer
);
```

CREATE TABLE

Let's take it apart:

- `CREATE TABLE books ( ... );` says "make a table called `books`". The columns go inside the brackets, separated by commas.
- Each column is a **name** followed by a **type**: `title text`, `year integer`.
- The first line is special. Read on.

The `id` line: a recipe for now

```
id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
```

Every table needs a column that identifies each row (chapter 02). This line makes one. In plain English: "a column called `id`, holding whole numbers, that identifies each row, and PostgreSQL fills it in automatically: 1, 2, 3, ..."

You don't need to understand every word of it yet. **Copy this line as the first column of every table you make.** [Chapter 14](#) explains exactly what `PRIMARY KEY` and `IDENTITY` mean. For now, it's a recipe, and the payoff is that you never have to type an `id` yourself.

The four types you'll use most

Type	Holds	Examples
<code>text</code>	Any text, any length	<code>'The Hobbit'</code> , <code>'Kathmandu'</code> , <code>''</code>
<code>integer</code>	Whole numbers, positive or negative	<code>1937</code> , <code>0</code> , <code>-5</code>
<code>boolean</code>	<code>true</code> or <code>false</code>	<code>true</code> , <code>false</code>
<code>date</code>	A calendar date	<code>'2026-09-30'</code>

Two things to notice already:

- Text and dates are written inside **single quotes**. Numbers and booleans are not.

- Dates are always written **year-month-day**: '2026-09-30'. This is the international standard (ISO 8601), and PostgreSQL will never misread it. '30/09/2026' and '09/30/2026' are ambiguous, so avoid them.

Money and decimals need a different type (`numeric`), because computers are surprisingly bad at decimals. That's in chapter 12. For now, if you need a price, use `numeric(10, 2)`: up to 10 digits, 2 of them after the point.

Looking at a table: `\d`

You can't see what you just made. Ask psql to **describe** it:

```
\d books
```

```

              Table "public.books"
  Column | Type   | Collation | Nullable |          Default
-----+-----+-----+-----+-----
  id     | integer |           | not null | generated always as identity
  title  | text    |           |          |
  author | text    |           |          |
  year   | integer |           |          |
Indexes:
    "books_pkey" PRIMARY KEY, btree (id)

```

Reading it:

- One line per column, with its type.
- `Nullable` shows `not null` for `id`: it can never be empty. The others *can* be empty (NULL), because you didn't say otherwise. Chapter 13 shows how to say otherwise.
- `Default` shows how `id` gets its value: automatically.
- The `Indexes` part is bookkeeping for `PRIMARY KEY`. Ignore it until chapter 31.
- `"public.books"`: `public` is the default section inside a database where tables live. Every table you make in this course goes there. You can ignore it.

To list all the tables in the current database:

```
\dt
```

```

      List of relations
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | books | table | postgres
(1 row)

```

PostgreSQL calls tables "relations" in a few places. Same thing.

In **pgAdmin**, expand your database → Schemas → public → Tables in the tree on the left. Right-click the table and choose **Properties** to see the columns, or **View/Edit Data** → **All Rows** to see what's in it.

Naming rules

Table and column names must:

- start with a letter or underscore,
- contain only letters, numbers, and underscores,
- avoid SQL's own words like `select`, `table`, `order`, `user`.

Follow the chapter 02 conventions (lowercase, `snake_case`, plural tables, singular columns) and you'll never have a naming problem.

If you break a rule, PostgreSQL usually tells you at once:

```
CREATE TABLE order (id integer);
```

```
ERROR:  syntax error at or near "order"
LINE 1: CREATE TABLE order (id integer);
                        ^
```

`order` is an SQL word (you'll meet `ORDER BY` in chapter 09). Use `orders` instead. Notice how the error points at the exact spot with `^`. PostgreSQL's error messages are good. Read them.

Deleting a table

```
DROP TABLE books;
```

```
DROP TABLE
```

Gone, along with every row in it. No undo. Same as `DROP DATABASE`, just smaller.

Try to create a table that already exists and you get:

```
ERROR:  relation "books" already exists
```

While you're learning, you'll often want to wipe a table and start fresh. Two helpers:


```
DROP TABLE IF EXISTS books;    -- no error if it isn't there
CREATE TABLE IF NOT EXISTS books ( ... );    -- no error if it is
```

`IF EXISTS` is handy at the top of a `.sql` file: drop, then create, so the file can be run again and again.

A second table

Let's make a `tasks` table for a to-do app, using all four types:

```
CREATE TABLE tasks (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  title text,
  is_done boolean,
  due_date date,
  priority integer
);
```

Then `\dt` shows two tables, and `\d tasks` shows its five columns. That's the whole rhythm: design on paper, `CREATE TABLE`, `\d` to check.

Creating a table in pgAdmin

You can also click your way there: right-click **Tables** → **Create** → **Table**, then fill in the name and add columns on the **Columns** tab. Before you click Save, look at the **SQL** tab. pgAdmin shows you the `CREATE TABLE` statement it's about to run. It's a nice way to check your understanding, but typing the SQL yourself is what makes it stick.

Common mistakes

1. Forgetting the commas between columns

```
CREATE TABLE books (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  title text
);
```

```
ERROR:  syntax error at or near "title"
```

Every column needs a comma after it, except the last one.

2. A comma after the last column

```
year integer,  
);
```

```
ERROR: syntax error at or near ")"
```

The opposite problem. No comma before the closing bracket.

3. Using a type that doesn't exist

```
title string
```

```
ERROR: type "string" does not exist
```

It's `text`. Also: `int` works as a short form of `integer`, `bool` for `boolean`. But `string`, `number`, and `varchar` without a length are either wrong or a bad idea.

4. Spaces or capitals in names

`CREATE TABLE My Books` fails. `CREATE TABLE "My Books"` works, but now you have to type the double quotes and exact capitals *every time you use it*. Stick to `my_books`.

5. Forgetting the `id` line

The table will work, but soon you'll have no way to point at one row. Copy the recipe every time.

Quick recap

- `CREATE TABLE name ( column type, column type, ... );` makes a table. Commas between columns, none after the last.
- The four everyday types: `text`, `integer`, `boolean`, `date`. Text and dates go in single quotes; dates are `'YYYY-MM-DD'`.
- Start every table with the recipe `id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,` for an automatic row number.
- `\d table_name` describes a table. `\dt` lists all tables.
- `DROP TABLE` deletes a table and everything in it. `IF EXISTS` / `IF NOT EXISTS` make files re-runnable.
- Read error messages. The `^` points at the problem.

Next: try the [exercises](#), then move on to [06 Adding Data](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. Save each answer in `playground/ch05/ex1.sql`, `ex2.sql`, and so on, and run it with `\i`.
 - Start each file with `DROP TABLE IF EXISTS ...;` so you can run it as many times as you like.
 - After every `CREATE TABLE`, check it with `\d`.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Fix the broken statements

Each of these has one mistake. Find it, fix it, and run the corrected version.

```
CREATE TABLE pets (  
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY  
  name text,  
  species text  
);
```

```
CREATE TABLE cities (  
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  name string,  
  population integer  
);
```

```
CREATE TABLE movies (  
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  title text,  
  release date,  
);
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): The contacts table, for real

In chapter 02 exercise 2 you designed a `contacts` table on paper. Create it now, with the `id` recipe and at least five other columns. Use at least three different types.

Run `\d contacts` and check every column has the type you meant.

Exercise 3 (Medium): A shopping cart

An online shop needs a table of products. Each product has a name, a price, how many are in stock, whether it's currently on sale, and the date it was added to the shop.

1. Create the `products` table. Use the right type for each column. (Reread the note about money in the notes.)
2. Run `\d products`.
3. Then create a second table, `cart_items`, for things a customer has put in their basket: which product (by its id), how many, and when it was added.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Read the description

Run `\d products` from Exercise 3 and answer, in a comment at the bottom of your `ex4.sql` file:

1. Which column can never be empty (NULL), and why?
 2. What does the `Default` column say for `id`, and what does it mean in plain English?
 3. What is `products_pkey`?
-

Exercise 5 (Challenge): The library, in SQL

In chapter 02 exercise 4 you designed tables for a library: books, members, and loans. Create all three in one `.sql` file that:

- starts by dropping all three tables if they exist,
- creates the three tables,
- can be run with `\i` over and over without errors.

Then run `\dt` and make sure all three are listed.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

06 Adding Data

What is it?

`INSERT` puts a new row into a table. It's the **C** in CRUD: create.

Why does it matter?

An empty table is a form nobody has filled in. Every sign-up, every order, every saved message in every app you've used was an `INSERT`. It's also where you first see PostgreSQL *checking* your data against the column types, and refusing what doesn't fit.

Real-world example

Filling in one copy of the form from chapter 05:

Paper form	SQL
Pick up a blank <code>books</code> form	<code>INSERT INTO books</code>
Which boxes you're filling in	<code>(title, author, year)</code>
What you write in them, in the same order	<code>VALUES ('The Hobbit', 'J.R.R. Tolkien', 1937)</code>
Hand it in	<code>;</code>
The clerk stamps a number on it	<code>id</code> gets filled in for you

How it works

Make sure you have the `books` table from [chapter 05](#), empty. If you're not sure, recreate it:

```
DROP TABLE IF EXISTS books;

CREATE TABLE books (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  title text,
  author text,
  year integer
);
```

Inserting one row

```
INSERT INTO books (title, author, year)
VALUES ('The Hobbit', 'J.R.R. Tolkien', 1937);
```

```
INSERT 0 1
```

The reply means "1 row inserted". (The `0` is a leftover from old versions of PostgreSQL. Ignore it.)

Read the statement in two halves:

1. `INSERT INTO books (title, author, year)`: which table, and which columns you're giving values for.
2. `VALUES ('The Hobbit', 'J.R.R. Tolkien', 1937)`: the values, **in the same order** as the columns.

The first column matches the first value, the second the second, and so on. Three columns, three values.

You didn't mention `id`. PostgreSQL filled it in: this row is number 1.

Peeking at the table

You'll learn `SELECT` properly next chapter, but you need a way to see what you've done. This shows every row:

```
SELECT * FROM books;
```

```
id | title | author | year
---+-----+-----+-----
 1 | The Hobbit | J.R.R. Tolkien | 1937
(1 row)
```

There it is, with its automatic `id`.

Inserting several rows at once

Put more brackets after `VALUES`, separated by commas:

```
INSERT INTO books (title, author, year)
VALUES
('Matilda', 'Roald Dahl', 1988),
('Dune', 'Frank Herbert', 1965),
('Pride and Prejudice', 'Jane Austen', 1813),
('The Martian', 'Andy Weir', 2011);
```

```
INSERT 0 4
```

Four rows in one go. This is faster than four separate statements, and easier to read. Note the commas *between* the bracket groups and none after the last one, the same rule as columns in `CREATE TABLE`.

Quotes: single for values, double for names

This trips up everyone, so let's be very clear:

- **Single quotes** `'...'` wrap a **value**: `'The Hobbit'`, `'2026-09-30'`.
- **Double quotes** `"..."` wrap a **name** (a table or column). You almost never need them.

Get it backwards and PostgreSQL thinks your book title is a column name:

```
INSERT INTO books (title) VALUES ("Dune");
```

```
ERROR: column "Dune" does not exist
LINE 1: INSERT INTO books (title) VALUES ("Dune");
                                         ^
```

That's the error from chapter 03's exercise 4, explained. Values get single quotes. Always.

Numbers and booleans don't need quotes at all: `1937`, `true`. Wrapping a number in quotes usually still works (PostgreSQL converts `'1937'` to a number), but it's a habit worth avoiding, because it hides mistakes.

Text with an apostrophe in it

If a value contains a single quote, like *Charlotte's Web*, write the quote **twice**:

```
INSERT INTO books (title, author, year)
VALUES ('Charlotte's Web', 'E.B. White', 1952);
```

Two single quotes in a row, `' '`, mean "one real apostrophe inside the text". It's not a double-quote character. It's stored as `Charlotte's Web`, with one apostrophe.

If you forget, the prompt turns into `practice'#`, waiting for the closing quote. Ctrl+C, then fix it.

Leaving columns out: NULL

You don't have to give every column a value. *Beowulf* is a poem with no known author and no known year:

```
INSERT INTO books (title) VALUES ('Beowulf');
```

The columns you didn't mention get **NULL**, the "no value" marker from chapter 02.

```
SELECT * FROM books;
```

id	title	author	year
1	The Hobbit	J.R.R. Tolkien	1937
2	Matilda	Roald Dahl	1988
3	Dune	Frank Herbert	1965
4	Pride and Prejudice	Jane Austen	1813
5	The Martian	Andy Weir	2011
6	Charlotte's Web	E.B. White	1952
7	Beowulf		

(7 rows)

psql shows NULL as an empty space, which is easy to miss and easy to confuse with empty text. Make it visible:

```
\pset null '[NULL]'
```

Now the `Beowulf` row shows `[NULL]` in both empty cells. This setting lasts until you quit psql. It's worth typing at the start of every session.

You can also write `NULL` on purpose, as a value. This would have done exactly the same thing (don't run it now, or you'll have two Beowulfs):

```
INSERT INTO books (title, author, year) VALUES ('Beowulf', NULL, NULL);
```

`NULL` has no quotes; it's a keyword, not text. Writing it out is useful when you insert several rows at once and only some of them have a missing value.

Getting the new row back: RETURNING

Often you want to know the `id` PostgreSQL just gave you. Add `RETURNING`:

```
INSERT INTO books (title, author, year)
VALUES
  ('The BFG', 'Roald Dahl', 1982),
  ('A Brief History of Time', 'Stephen Hawking', 1988)
RETURNING id, title;
```



```

id |          title
----+-----
 8 | The BFG
 9 | A Brief History of Time
(2 rows)

INSERT 0 2

```

`RETURNING *` gives back every column of the new rows. Apps use this constantly: insert a new order, get its `id` back, show it to the customer.

When the value doesn't fit

The table's types are promises, and `INSERT` is where PostgreSQL checks them:

```

INSERT INTO books (title, author, year)
VALUES ('Emma', 'Jane Austen', 'eighteen fifteen');

```

```

ERROR:  invalid input syntax for type integer: "eighteen fifteen"
LINE 2: VALUES ('Emma', 'Jane Austen', 'eighteen fifteen');
                                         ^

```

Nothing was inserted. The row is rejected as a whole. That's the "someone typed `ten` in the price column" problem from chapter 01, gone for good.

Trying to set the `id` yourself

```

INSERT INTO books (id, title) VALUES (50, 'Emma');

```

```

ERROR:  cannot insert a non-DEFAULT value into column "id"
DETAIL:  Column "id" is an identity column defined as GENERATED ALWAYS.
HINT:   Use OVERRIDING SYSTEM VALUE to override.

```

This is the `id` recipe doing its job. PostgreSQL owns that column. Leave it out of your inserts and let it count.

Adding rows in pgAdmin

Right-click the table → **View/Edit Data** → **All Rows**. A grid opens. You can type into the empty last row and press the **Save** (💾) button. Behind the scenes, pgAdmin writes an `INSERT` for you. Handy for a quick fix. But type the SQL yourself while you're learning.

Common mistakes

1. Wrong number of values

```
INSERT INTO books (title, author, year) VALUES ('Emma', 'Jane Austen');
```

ERROR: INSERT has more target columns than expressions

Three columns named, two values given. Count them. (The opposite, more values than columns, gives `INSERT` has more expressions than target columns.)

2. Values in the wrong order

```
INSERT INTO books (title, author, year) VALUES ('Jane Austen', 'Emma', 1815);
```

No error, because both are text. But now you have a book called *Jane Austen* by someone named Emma. PostgreSQL can only check types, not meaning. Match your values to your column list carefully.

3. Skipping the column list

```
INSERT INTO books VALUES ('Emma', 'Jane Austen', 1815);
```

Without a column list, PostgreSQL assumes you mean *every* column in table order, starting with `id`. Here it tries to put `'Emma'` into `id` and fails. Always write the column list. It's clearer, and it keeps working if the table changes later.

4. Double quotes around text

Covered above, but it will happen again. `"Dune"` is a column. `'Dune'` is a value.

5. Confusing NULL with empty text

`''` (two quotes with nothing between) is empty text: a value that happens to be blank. `NULL` is *no value at all*. They behave differently in chapter 08. If you don't know something, use `NULL`.

Quick recap

- `INSERT INTO table (col1, col2) VALUES (val1, val2);` adds a row. Values match columns by position.
- Insert many rows with `VALUES (...), (...), (...)`.
- **Single quotes** for values. **Double quotes** for names. Numbers and booleans have no quotes.
- An apostrophe inside text is written as two single quotes: `'Charlotte's Web'`.
- Columns you leave out become **NULL**. `\pset null '[NULL]'` makes `NULL` visible in `psql`.

- `RETURNING id` hands back the automatic id (or any columns) of the new rows.
 - If a value doesn't match the column's type, the whole row is rejected. That's the point.
-

Next: try the [exercises](#), then move on to [07 Reading Data](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. Save each answer in `playground/ch06/ex1.sql`, `ex2.sql`, and so on, and run it with `\i`.
- Run `\pset null '[NULL]'` first so you can see NULLs.
- After each insert, run `SELECT * FROM your_table;` to check.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Important: the next few chapters' exercises use the exact data below, so the expected outputs match. Use these values as written.

Exercise 1 (Easy): Fill the to-do list

You made a `tasks` table in chapter 05 (`title`, `is_done`, `due_date`, `priority`). If it's gone, recreate it exactly like that.

Insert these five tasks. Use **one** `INSERT` for the first two, and **one** `INSERT` for the last three.

title	is_done	due_date	priority
Buy milk	false	2026-10-01	2
Finish SQL chapter 6	true	2026-09-30	1
Call the dentist	false	2026-10-05	3
Water the plants	false	<i>(no due date)</i>	3
Renew passport	false	2026-11-15	1

Expected `SELECT * FROM tasks;`:

id	title	is_done	due_date	priority
1	Buy milk	f	2026-10-01	2
2	Finish SQL chapter 6	t	2026-09-30	1
3	Call the dentist	f	2026-10-05	3
4	Water the plants	f	[NULL]	3
5	Renew passport	f	2026-11-15	1

(5 rows)

(psql shows booleans as `t` and `f`.)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Fix the broken inserts

Each of these fails. Run it, read the error, fix it, run it again. Use a throwaway table:

```
DROP TABLE IF EXISTS notes;
CREATE TABLE notes (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  body text,
  written_on date,
  stars integer
);
```

```
INSERT INTO notes (body, written_on, stars) VALUES ("Great day", '2026-09-30', 5);
```

```
INSERT INTO notes (body, written_on, stars) VALUES ('Rainy', '2026-09-31', 3);
```

```
INSERT INTO notes (body, written_on) VALUES ('Tired', '2026-09-29', 2);
```

```
INSERT INTO notes (body, written_on, stars) VALUES ('Sam's birthday', '2026-10-02', 5);
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Stock the shop

To keep later chapters matching, recreate `products` with **exactly** these columns:

```

DROP TABLE IF EXISTS products;
CREATE TABLE products (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text,
  price numeric(10, 2),
  stock integer,
  on_sale boolean,
  added_on date
);

```

Insert all six products in **one** statement, and use `RETURNING` to show the `id` and `name` of each new row.

name	price	stock	on_sale	added_on
Notebook	3.50	120	false	2026-09-01
Pen	1.20	500	false	2026-09-01
Desk lamp	24.99	15	true	2026-09-10
Stapler	8.00	40	false	2026-09-12
Backpack	45.00	8	true	2026-09-20
Sticky notes	2.75	0	false	2026-09-25

Expected output of the `RETURNING` :

```

id |      name
----+-----
 1 | Notebook
 2 | Pen
 3 | Desk lamp
 4 | Stapler
 5 | Backpack
 6 | Sticky notes
(6 rows)

INSERT 0 6

```

Exercise 4 (Medium): Break the promises

Using the `products` table, try to insert:

1. A product whose `price` is the text `'cheap'`.
2. A product whose `stock` is `12.5`.
3. A product whose `on_sale` is `'maybe'`.
4. A product with an `id` of `100`.

None should work. Write the first line of each error message as a comment in your `ex4.sql`. Then run `SELECT * FROM products;` and confirm you still have exactly 6 rows.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A re-runnable seed file

A **seed file** is a `.sql` file that fills a database with starting data. Developers use them to get a fresh database into a known state.

Write `playground/ch06/seed.sql` that:

1. Drops `tasks` and `products` if they exist.
2. Creates both tables.
3. Inserts all the data from Exercises 1 and 3.
4. Ends with `SELECT * FROM tasks;` and `SELECT * FROM products;` so you can see the result.

Run it with `\i` twice in a row. It should work both times with no errors, and you should end up with 5 tasks and 6 products, ids starting from 1.

You'll use this file at the start of the next few chapters whenever you want a clean start.

Stuck? The hint is in the [Hints section](#) at the end.

07 Reading Data

What is it?

`SELECT` gets data out of a table. It's the **R** in CRUD: read.

You'll spend more time writing `SELECT` than everything else in SQL put together. This chapter covers the basics: which columns, renaming them, and doing a bit of maths on the way out. The next two chapters add filtering and sorting.

Why does it matter?

Data you can't get back out is worthless. Every screen in every app is a `SELECT` underneath: your inbox, your order history, a product page, a leaderboard. Learning to ask the database precise questions is *the* core skill of this whole course.

Real-world example

Asking the librarian:

You say	SQL
"Show me everything about every book."	<code>SELECT * FROM books;</code>
"Just the titles, please."	<code>SELECT title FROM books;</code>
"Titles and authors."	<code>SELECT title, author FROM books;</code>
"And call that column 'Name' on the printout."	<code>SELECT title AS name FROM books;</code>

Important: the librarian brings you a **photocopy**. Reading never changes what's on the shelves. You can `SELECT` all day and nothing in the table moves.

How it works

You need the `books` table with the 9 rows from [chapter 06](#). Run `\pset null '[NULL]'` too.

Everything: `SELECT *`

```
SELECT * FROM books;
```

id	title	author	year
1	The Hobbit	J.R.R. Tolkien	1937
2	Matilda	Roald Dahl	1988
3	Dune	Frank Herbert	1965
4	Pride and Prejudice	Jane Austen	1813
5	The Martian	Andy Weir	2011
6	Charlotte's Web	E.B. White	1952
7	Beowulf	[NULL]	[NULL]
8	The BFG	Roald Dahl	1982
9	A Brief History of Time	Stephen Hawking	1988

(9 rows)

`*` means "all columns". `FROM books` says which table. You get every row, every column.

The rows came back in `id` order this time. **That's luck, not a promise.** Remember from chapter 02: rows have no fixed order. Chapter 09 shows how to ask for one.

Just some columns

List the columns you want, separated by commas:

```
SELECT title, year FROM books;
```

title	year
The Hobbit	1937
Matilda	1988
...	

The result has only those two columns, **in the order you listed them**. Swap them (`SELECT year, title`) and the columns swap.

This matters more than it looks. In a real app, a table might have 40 columns, and a screen might need 3. Asking for only what you need is faster and clearer.

Renaming columns: `AS`

The name in the result doesn't have to match the table:


```
SELECT title AS book, year AS published FROM books;
```

book	published
The Hobbit	1937
...	

AS gives the column an **alias**: a temporary name, just for this result. The table itself is unchanged.

If you want spaces or capitals in an alias, you need double quotes, because now you're naming something:

```
SELECT title AS "Book Title" FROM books;
```

This is the one place you'll use double quotes regularly. Notice the rule still holds: double quotes for **names**, single quotes for **values**.

Doing maths in SELECT

You can put an expression where a column goes:

```
SELECT title, 2026 - year AS age FROM books;
```

title	age
The Hobbit	89
Matilda	38
Dune	61
Pride and Prejudice	213
The Martian	15
Charlotte's Web	74
Beowulf	[NULL]
The BFG	44
A Brief History of Time	38

(9 rows)

Two things to see here:

1. The maths runs **once per row**, using that row's `year`.
2. *Beowulf* has a NULL `year`, so `2026 - NULL` is... NULL. **Anything combined with NULL gives NULL**. "2026 minus I-don't-know" is "I don't know". You'll meet this again in the next chapter.

Without an alias, PostgreSQL labels a calculated column `?column?`. Always give expressions a name with AS.

The usual operators work: `+`, `-`, `*`, `/`. One catch from chapter 03's exercise: dividing two whole numbers gives a whole number (`7 / 2` is `3`). Chapter 12 explains why and how to get `3.5` .

Joining text together: `||`

Two vertical bars glue text end to end:

```
SELECT title || ' by ' || author AS description FROM books;
```

```

      description
-----
The Hobbit by J.R.R. Tolkien
Matilda by Roald Dahl
Dune by Frank Herbert
Pride and Prejudice by Jane Austen
The Martian by Andy Weir
Charlotte's Web by E.B. White
[NULL]
The BFG by Roald Dahl
A Brief History of Time by Stephen Hawking
(9 rows)
```

Once again NULL spreads: *Beowulf* has a NULL author, so the whole description is NULL. Chapter 24 has a fix (`COALESCE`), but notice the pattern now.

A first look at functions

A **function** is a ready-made tool that takes a value and gives you something back. You've used two already: `version()` and `now()` . Here are two for text:

```
SELECT upper(title) AS shouty, length(title) AS letters FROM books;
```

```

      shouty      | letters
-----+-----
THE HOBBIT       |      10
MATILDA          |       7
...
```

`upper()` makes text uppercase. `length()` counts its characters. The value goes inside the brackets. [Chapter 24](#) is a whole tour of these. For now, just know they exist and that this is what they look like.

SELECT without a table

You've done this since chapter 03:

```
SELECT 2 + 2;  
SELECT now();
```

No `FROM`, so no rows to work through. PostgreSQL just works out the value once and hands it back. Useful as a calculator and for testing an expression before you use it on real data.

The result is a temporary table

Every `SELECT` gives you back something shaped like a table: columns and rows. But it's a **result**, not a stored table. It exists only on your screen, and the next query replaces it. Nothing you do in `SELECT` changes the data in `books`.

This idea, "a query gives back a table-shaped answer", becomes powerful later, when you'll feed one query's result into another (chapters 25 and 26).

Wide results: `\x`

When a table has many columns, rows wrap and become unreadable. Toggle **expanded display**:

```
\x
```

Now each row prints as a vertical list of `column | value`. Type `\x` again to switch back. `\x auto` lets `psql` decide based on width. `pgAdmin`'s grid doesn't have this problem, which is one reason to keep it around.

Common mistakes

1. A missing comma turns a column into an alias

```
SELECT title author FROM books;
```

No error! PostgreSQL reads `title author` as `title AS author` (the `AS` is optional). You get one column of titles, labeled "author". If a column seems to have vanished, look for a missing comma.

2. Forgetting `FROM`

```
SELECT title;
```

```
ERROR: column "title" does not exist
```

Without a table, `title` means nothing. "Column does not exist" often means "you forgot which table" or "you misspelled the name".

3. Double quotes around a value

```
SELECT title || " by " || author FROM books;
```

```
ERROR: zero-length delimited identifier at or near """"
```

Scary message, simple cause: `" by "` in double quotes is treated as a (badly formed) name. Single quotes for text.

4. Using `SELECT *` everywhere

Fine while learning and exploring. In a real app, ask for the columns you need. It's faster, and your code won't break when someone adds a column.

5. Expecting `NULL` to behave like zero or empty text

`2026 - NULL` is `NULL`, not `2026`. `'a' || NULL` is `NULL`, not `'a'`. `NULL` means "unknown", and unknown stays unknown.

Quick recap

- `SELECT columns FROM table;` reads data. `*` means all columns. Reading never changes the table.
- List columns in the order you want them. Rename with `AS alias`. Aliases with spaces need double quotes.
- Any expression can be a column: `2026 - year`, `title || ' by ' || author`, `upper(title)`. Always alias it.
- **NULL spreads:** any calculation involving `NULL` gives `NULL`.
- `SELECT` without `FROM` is a calculator. `\x` makes wide results readable in `psql`.
- The result of a `SELECT` is a temporary table on your screen, not stored anywhere.

Next: try the [exercises](#), then move on to [08 Filtering Rows](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. You need the `tasks` and `products` tables with the data from chapter 06. If in doubt, run your `playground/ch06/seed.sql`.
 - Run `\pset null '[NULL]'` first.
 - Save each answer in `playground/ch07/ex1.sql`, `ex2.sql`, and so on.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Pick your columns

Write three queries on `products`:

1. Every column of every product.
2. Only `name` and `price`.
3. `price` first, then `name`, then `stock`.

Expected output of query 2:

```
   name   | price
-----+-----
Notebook  |  3.50
Pen       |  1.20
Desk lamp | 24.99
Stapler   |  8.00
Backpack  | 45.00
Sticky notes | 2.75
(6 rows)
```

Exercise 2 (Easy): Nicer headings

Show each task's title and due date, but label the columns `Task` and `Due` (with capital letters).

Expected output:

Task	Due
Buy milk	2026-10-01
Finish SQL chapter 6	2026-09-30
Call the dentist	2026-10-05
Water the plants	[NULL]
Renew passport	2026-11-15

(5 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Stock value

For each product, show its name and the **total value of its stock** (price times how many you have), labeled `stock_value`.

Expected output:

name	stock_value
Notebook	420.00
Pen	600.00
Desk lamp	374.85
Stapler	320.00
Backpack	360.00
Sticky notes	0.00

(6 rows)

Then add a third column: the price with 20% added (for tax), labeled `price_with_tax`. Check the Desk lamp comes out as `29.988`. (Chapter 24 will show you how to round that. For now, just get the maths right.)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): A sentence per row

Make one text column called `summary` that reads like a sentence for each product. For example, the first row should be:

Notebook costs 3.50 and we have 120 in stock

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Days until due

For each task, show its title and how many days are left until its due date, counting from **2026-09-30**, labeled `days_left`.

Expected output:

title	days_left
Buy milk	1
Finish SQL chapter 6	0
Call the dentist	5
Water the plants	[NULL]
Renew passport	46
(5 rows)	

Then: why is `Water the plants` NULL? Write the reason as a comment in your file.

Bonus: replace the fixed date with `current_date` so the answer changes every day. Your numbers will be different from the expected output above, and that's fine.

Stuck? The hint is in the [Hints section](#) at the end.

08 Filtering Rows

What is it?

`WHERE` picks out only the rows you care about. "Books written before 1950." "Tasks that aren't done." "Products on sale that are in stock."

Without `WHERE`, `SELECT` gives you every row. With it, you get exactly the ones that match your condition.

Why does it matter?

Real tables have thousands or millions of rows. Nobody wants all of them. Every search box, every filter, every "show me my orders" is a `WHERE`. And it's not only for reading: in chapter 10, `WHERE` decides which rows get changed or deleted, so getting it right really matters.

Real-world example

Asking the librarian, but with a condition:

You say	SQL
"Every book"	<code>SELECT * FROM books</code>
"...written before 1950"	<code>WHERE year < 1950</code>
"...by Roald Dahl"	<code>WHERE author = 'Roald Dahl'</code>
"...by Dahl, and written after 1985"	<code>WHERE author = 'Roald Dahl' AND year > 1985</code>
"...whose title starts with The"	<code>WHERE title LIKE 'The %'</code>

The librarian checks each card against your condition. Cards that pass go on the pile. Cards that fail, or where the answer is "can't tell", stay in the drawer.

How it works

You need the 9-row `books` table from [chapter 06](#), and `\pset null '[NULL]'`.

A simple condition

```
SELECT title, year FROM books WHERE year < 1950;
```

title	year
The Hobbit	1937
Pride and Prejudice	1813
(2 rows)	

`WHERE year < 1950` is a **condition**: something that's true or false for each row. PostgreSQL checks every row and keeps the ones where it's true.

`WHERE` goes after `FROM`. The order is always `SELECT ... FROM ... WHERE ...`.

Comparing things

Operator	Meaning	Example
<code>=</code>	equal to	<code>author = 'Roald Dahl'</code>
<code><></code>	not equal to (also written <code>!=</code>)	<code>author <> 'Roald Dahl'</code>
<code><</code>	less than	<code>year < 1950</code>
<code>></code>	greater than	<code>year > 2000</code>
<code><=</code>	less than or equal	<code>year <= 1988</code>
<code>>=</code>	greater than or equal	<code>year >= 1988</code>

Note that "equals" is a single `=`. If you've done the JavaScript course, you're used to `===`. Not here.

Text comparisons follow the quote rule from chapter 06: the value goes in single quotes.

```
SELECT title FROM books WHERE author = 'Roald Dahl';
```

title	
Matilda	
The BFG	
(2 rows)	

And they are **exact**, including capital letters:

```
SELECT title FROM books WHERE author = 'roald dahl';
```

```
title
-----
(0 rows)
```

Zero rows, no error. 'roald dahl' and 'Roald Dahl' are different text. Later in this chapter, `ILIKE` gives you a way to ignore case.

Combining conditions: `AND`, `OR`, `NOT`

`AND` : both must be true.

```
SELECT title, year FROM books
WHERE author = 'Roald Dahl' AND year > 1985;
```

```
title | year
-----+-----
Matilda | 1988
(1 row)
```

`OR` : at least one must be true.

```
SELECT title, year FROM books
WHERE year < 1900 OR year > 2000;
```

```
title | year
-----+-----
Pride and Prejudice | 1813
The Martian | 2011
(2 rows)
```

`NOT` : flips a condition.

```
SELECT title FROM books WHERE NOT author = 'Roald Dahl';
```

That gives 6 rows, not 7. *Beowulf* is missing, even though its author is certainly not Roald Dahl. Hold that thought until the NULL section below.

Brackets when you mix AND and OR

AND is checked before OR, the way multiplication happens before addition. This surprises people:

```
-- Means: (Dahl books written after 1985) OR (any Austen book)
WHERE author = 'Roald Dahl' AND year > 1985 OR author = 'Jane Austen'
```

If you meant "Dahl or Austen, but only after 1985", you need brackets:

```
WHERE (author = 'Roald Dahl' OR author = 'Jane Austen') AND year > 1985
```

Rule of thumb: **whenever you mix AND and OR, use brackets.** Even when you don't strictly need them. It makes your intention obvious to the next person, who is usually you next week.

Ranges: BETWEEN

```
SELECT title, year FROM books WHERE year BETWEEN 1950 AND 1990;
```

title	year
Matilda	1988
Dune	1965
Charlotte's Web	1952
The BFG	1982
A Brief History of Time	1988

(5 rows)

BETWEEN 1950 AND 1990 is the same as `year >= 1950 AND year <= 1990`. Both ends are **included**. It works with dates too: `due_date BETWEEN '2026-10-01' AND '2026-10-31'`.

Lists: IN

Instead of chaining OR s:

```
SELECT title FROM books WHERE author IN ('Roald Dahl', 'Jane Austen', 'Andy Weir');
```

```

      title
-----
Matilda
Pride and Prejudice
The Martian
The BFG
(4 rows)

```

`IN (...)` means "matches any value in this list". `NOT IN (...)` means none of them.

Patterns: `LIKE` and `ILIKE`

For "starts with", "ends with", or "contains", use `LIKE` with two special characters:

- `%` matches **any number** of characters, including none.
- `_` matches **exactly one** character.

```
SELECT title FROM books WHERE title LIKE 'The %';
```

```

      title
-----
The Hobbit
The Martian
The BFG
(3 rows)

```

`'The %'` means: `The`, a space, then anything. Some more patterns:

Pattern	Matches
<code>'The %'</code>	starts with "The "
<code>'%Time'</code>	ends with "Time"
<code>'%of%'</code>	contains "of" anywhere
<code>'D_ne'</code>	D, any one character, ne: "Dune", "Dane", "Done"
<code>'%'</code>	anything at all (not very useful)

`LIKE` is case-sensitive, like `=`. `ILIKE` (the `I` is for "insensitive") ignores case:

```
SELECT title FROM books WHERE title LIKE '%the%';    -- 0 rows: no lowercase "the"
SELECT title FROM books WHERE title ILIKE '%the%';    -- 3 rows: The Hobbit, The Martian, The BFG
```

`ILIKE` is a PostgreSQL extra; most other databases don't have it. It's very handy for search boxes.

Missing values: `IS NULL`

Here's the trap everyone falls into once:

```
SELECT title FROM books WHERE year = NULL;
```

```
title
-----
(0 rows)
```

Zero rows, even though *Beowulf* has a NULL year. Why?

NULL means "unknown". Asking "is this year equal to unknown?" can't be answered yes or no. The answer is itself unknown. And `WHERE` only keeps rows where the condition is **definitely true**. Unknown isn't true, so the row is dropped.

The same thing explains the `NOT author = 'Roald Dahl'` puzzle from earlier. "Is unknown not equal to Roald Dahl?" Unknown. Dropped.

To ask about NULL, use the special words `IS NULL` and `IS NOT NULL`:

```
SELECT title FROM books WHERE year IS NULL;
```

```
title
-----
Beowulf
(1 row)
```

```
SELECT title FROM books WHERE year IS NOT NULL;    -- the other 8
```

Remember this one: `= NULL` **never works**. Use `IS NULL`.

Filtering on columns you don't show

The `WHERE` is checked against the whole row, before `SELECT` picks which columns to display. So you can filter on `year` and show only `title`:

```
SELECT title FROM books WHERE year > 2000;
```

The filter column doesn't have to appear in the result.

Putting it together

```
SELECT title, author, year
FROM books
WHERE (author = 'Roald Dahl' OR title LIKE 'The %')
      AND year IS NOT NULL
      AND year BETWEEN 1930 AND 1990;
```

title	author	year
The Hobbit	J.R.R. Tolkien	1937
Matilda	Roald Dahl	1988
The BFG	Roald Dahl	1982

(3 rows)

Long conditions are easier to read spread over several lines, one condition per line, with `AND` / `OR` at the start of each. PostgreSQL doesn't care about the line breaks.

Common mistakes

1. `= NULL`

Returns nothing, always. Use `IS NULL` or `IS NOT NULL`.

2. Forgetting the quotes on text

```
WHERE author = Roald Dahl
```

```
ERROR:  syntax error at or near "Dahl"
```

Or worse, `WHERE author = Dahl` gives `column "dahl" does not exist`. Unquoted words are treated as column names.

3. AND when you mean OR

"Show me books by Dahl and Austen" sounds like `AND`, but no single book is by both. You want `author = 'Roald Dahl' OR author = 'Jane Austen', OR IN`.

4. Using `=` with a pattern

```
WHERE title = 'The %'
```

That looks for a book literally called "The %". Patterns only work with `LIKE` and `ILIKE`.

5. Mixing `AND` and `OR` without brackets

You'll get rows you didn't expect, with no error. Add brackets.

6. Case

`'Dahl'` doesn't match `'dahl'`. Use `ILIKE` when case shouldn't matter, or make sure your data is stored consistently (chapter 13 helps with that).

Quick recap

- `WHERE condition` keeps only rows where the condition is **true**. It goes after `FROM`.
- Compare with `=`, `<>`, `<`, `>`, `<=`, `>=`. Text comparisons are exact and case-sensitive.
- Combine with `AND`, `OR`, `NOT`. **Use brackets** whenever `AND` and `OR` meet.
- `BETWEEN a AND b` is an inclusive range. `IN (...)` matches a list.
- `LIKE` matches patterns: `%` is anything, `_` is one character. `ILIKE` ignores case.
- `NULL` is unknown. `= NULL` **never matches**. Use `IS NULL` / `IS NOT NULL`.

Next: try the [exercises](#), then move on to [09 Sorting and Limiting](#).

Exercises

How to do these:

- Work in `psql`, connected to `practice`, with the `tasks` and `products` data from chapter 06 (run `playground/ch06/seed.sql` for a clean start).
- Run `\pset null '[NULL]'` first.
- Save each answer in `playground/ch08/ex1.sql`, `ex2.sql`, and so on.
- Try on your own first. Only open a hint if you've been stuck for a while.

- When you're done, ask Claude to check your work.

Exercise 1 (Easy): One condition each

Write a query for each:

1. The names and prices of products that are on sale.
2. The titles of tasks that are **not** done.
3. The names of products that cost less than 5.00.

Expected output of query 3:

```
name
-----
Notebook
Pen
Sticky notes
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Two conditions

1. Tasks that are not done **and** have priority 1.
2. Products that are on sale **or** cost less than 3.00.
3. Tasks due in October 2026 (use `BETWEEN`).

Expected output of query 3:

```
title | due_date
-----+-----
Buy milk | 2026-10-01
Call the dentist | 2026-10-05
(2 rows)
```

Exercise 3 (Medium): Patterns

1. Products whose name contains "note", ignoring case. (Both `Notebook` and `Sticky notes` should appear.)

2. Tasks whose title starts with a `B` or a `C`.
3. Products whose name is exactly three letters long. (There's one.)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Where's the NULL?

1. Tasks that have **no** due date.
2. Tasks that **do** have a due date and are not done.
3. Now run this and explain, in a comment, why the result is wrong:

```
SELECT title FROM tasks WHERE due_date <> '2026-10-01';
```

You'd expect 4 tasks (everything except "Buy milk"). How many do you get, and which one is missing? Fix the query so that all 4 appear.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): The shop's front page

The shop wants to show products that are **either on sale or cost less than 3.00**, but **only** if there are at least 10 in stock (so the front page never shows something about to sell out).

Write the query, with brackets, showing `name`, `price`, `stock`, and `on_sale`.

Expected output:

```
name      | price | stock | on_sale
-----+-----+-----+-----
Pen       | 1.20  | 500   | f
Desk lamp | 24.99 | 15    | t
(2 rows)
```

Then remove the brackets and run it again. Which extra product sneaks in, and why?

Stuck? The hint is in the [Hints section](#) at the end.

09 Sorting and Limiting

What is it?

Three small tools that shape your results:

- `ORDER BY` puts rows in an order you choose.
- `LIMIT` gives you only the first few.
- `DISTINCT` removes duplicate rows.

Why does it matter?

Remember: rows in a table have **no order**. When you ran `SELECT * FROM books` and got the rows in `id` order, that was PostgreSQL being convenient, not making a promise. Add a row, delete a row, and the order can change.

So any time order matters ("newest first", "cheapest first", "top 10"), you have to ask for it. And most of the time, order matters.

Real-world example

A results page in an online shop:

The shop shows	SQL
"Sort by: price, low to high"	<code>ORDER BY price</code>
"Sort by: newest first"	<code>ORDER BY added_on DESC</code>
"Showing 20 of 3,400 results"	<code>LIMIT 20</code>
"Page 3"	<code>LIMIT 20 OFFSET 40</code>
"Filter by brand" dropdown, listing each brand once	<code>SELECT DISTINCT brand</code>

How it works

You need the 9-row `books` table from [chapter 06](#), and `\pset null '[NULL]'`.

Sorting: ORDER BY

```
SELECT title, year FROM books ORDER BY year;
```

title	year
Pride and Prejudice	1813
The Hobbit	1937
Charlotte's Web	1952
Dune	1965
The BFG	1982
Matilda	1988
A Brief History of Time	1988
The Martian	2011
Beowulf	[NULL]

(9 rows)

Oldest first. Smallest to largest is the default, called **ascending**. You can say it out loud with `ASC`, but nobody does.

Two things to notice:

1. **NULL came last.** PostgreSQL treats NULL as bigger than everything when sorting ascending.
2. **Matilda and A Brief History of Time both say 1988.** Which comes first? PostgreSQL doesn't promise. Run it again after adding a row and they might swap. A tie is a gap in your instructions.

Largest first: DESC

```
SELECT title, year FROM books ORDER BY year DESC;
```

title	year
Beowulf	[NULL]
The Martian	2011
Matilda	1988
A Brief History of Time	1988
...	

`DESC` is **descending**: biggest first. And now NULL is at the top, because "bigger than everything" gets flipped too. That's rarely what you want. Tell PostgreSQL where to put them:

```
SELECT title, year FROM books ORDER BY year DESC NULLS LAST;
```

`NULLS FIRST` also exists.

Breaking ties: sort by more than one column

Give a second column to use when the first one is equal:

```
SELECT title, year FROM books ORDER BY year DESC NULLS LAST, title;
```

title	year
The Martian	2011
A Brief History of Time	1988
Matilda	1988
The BFG	1982
...	

Now the two 1988 books are in title order, every time. Read it as "sort by year, newest first; when years match, sort by title A to Z."

Each column gets its own direction. `ORDER BY year DESC, title` means year descending, **title ascending**. `DESC` only applies to the column right before it.

Sorting text

```
SELECT title FROM books ORDER BY title;
```

title
A Brief History of Time
Beowulf
Charlotte's Web
Dune
Matilda
Pride and Prejudice
The BFG
The Hobbit
The Martian

(9 rows)

Alphabetical, as you'd hope. Honest note: the fine details (where capital letters go, whether spaces and punctuation count) depend on your computer's language settings, which PostgreSQL picked up during install. It's called the **collation**. For everyday use you won't notice. If a sort ever looks slightly odd, that's why.

One real gotcha: numbers stored as **text** sort as text. '10' comes before '9', because '1' comes before '9'. That's one more reason to give columns the right type.

Sorting by something you calculated

You can order by an alias or an expression from your `SELECT`:

```
SELECT title, 2026 - year AS age FROM books ORDER BY age DESC NULLS LAST;
```

Or by a column you *aren't* showing:

```
SELECT title FROM books ORDER BY year;
```

Both work.

Only the first few: `LIMIT`

```
SELECT title, year FROM books ORDER BY year LIMIT 3;
```

title	year
Pride and Prejudice	1813
The Hobbit	1937
Charlotte's Web	1952

(3 rows)

The three oldest books. `LIMIT` goes last, after `ORDER BY`.

`LIMIT` **without** `ORDER BY` means "any 3 rows", in whatever order PostgreSQL happens to find them.

Sometimes that's fine ("show me a few rows so I can see what this table looks like"). But "the 3 cheapest" or "the 10 newest" always need an `ORDER BY`.

Skipping some: `OFFSET`

```
SELECT title, year FROM books ORDER BY year, title LIMIT 3 OFFSET 3;
```

title	year
Dune	1965
The BFG	1982
A Brief History of Time	1988

(3 rows)

`OFFSET 3` skips the first 3, then `LIMIT 3` takes the next 3. Books 4, 5 and 6. This is how apps do **pages**: page 1 is `LIMIT 20`, page 2 is `LIMIT 20 OFFSET 20`, page 3 is `LIMIT 20 OFFSET 40`.

Notice the `, title` tie-breaker. Without it, the two 1988 books could land on different pages on different days.

Removing duplicates: `DISTINCT`

```
SELECT author FROM books;
```

lists Roald Dahl twice. If you want each author once:

```
SELECT DISTINCT author FROM books ORDER BY author;
```

author
Andy Weir
E.B. White
Frank Herbert
J.R.R. Tolkien
Jane Austen
Roald Dahl
Stephen Hawking
[NULL]

(8 rows)

8 rows: 7 authors plus NULL, which `DISTINCT` treats as one value.

With several columns, `DISTINCT` removes rows where the **whole combination** repeats:

```
SELECT DISTINCT author, year FROM books;
```

Roald Dahl appears twice here (1988 and 1982), because those are different combinations.

The order of the clauses

Every part has a fixed place. You can leave parts out, but you can't rearrange them:

```
SELECT  DISTINCT title, year    -- 1. which columns (DISTINCT goes right after SELECT)
FROM    books                  -- 2. which table
WHERE   year IS NOT NULL       -- 3. which rows
ORDER BY year DESC, title      -- 4. in what order
LIMIT   5 OFFSET 0;           -- 5. how many
```

Say it as a sentence: *select these columns, from this table, where this is true, ordered like this, only this many.*

Common mistakes

1. LIMIT without ORDER BY

"The 5 most expensive products" with `LIMIT 5` but no `ORDER BY price DESC` gives 5 random products. No error, wrong answer.

2. Clauses in the wrong order

```
SELECT title FROM books ORDER BY year WHERE year > 1950;
```

```
ERROR:  syntax error at or near "WHERE"
```

`WHERE` always comes before `ORDER BY`.

3. Expecting DESC to apply to every column

`ORDER BY year, title DESC` sorts year **ascending** and title descending. Put the direction after each column that needs it.

4. Forgetting about NULL

Sorting descending puts NULLs first. Add `NULLS LAST` when a column can be empty.

5. Ties

If two rows can have the same value in your sort column, add a tie-breaker (usually `id` or a name). Otherwise the order can change between runs, which is a nasty bug in a paginated list.

Quick recap

- Rows have no natural order. **Always** `ORDER BY` when order matters.
 - `ORDER BY col` is ascending. `DESC` flips it. Add more columns to break ties: `ORDER BY year DESC, title`.
 - `NULLs` sort last ascending, first descending. Control it with `NULLS LAST` / `NULLS FIRST`.
 - `LIMIT n` takes the first `n` rows. `OFFSET m` skips `m` first. Together they make pages. Both need `ORDER BY` to mean anything.
 - `DISTINCT` removes duplicate rows (whole-row duplicates, when you list several columns).
 - Clause order: `SELECT`, `FROM`, `WHERE`, `ORDER BY`, `LIMIT / OFFSET`.
-

Next: try the [exercises](#), then move on to [10 Changing and Deleting Data](#).

Exercises

How to do these:

- Work in `psql`, connected to `practice`, with the `tasks` and `products` data from chapter 06 (run `playground/ch06/seed.sql` for a clean start).
 - Run `\pset null '[NULL]'` first.
 - Save each answer in `playground/ch09/ex1.sql`, `ex2.sql`, and so on.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Cheapest first, soonest first

1. All products, cheapest first. Show `name` and `price`.
2. All tasks, soonest due date first, with the task that has no due date at the **bottom**. Show `title` and `due_date`.

Expected output of query 2:

title	due_date
Finish SQL chapter 6	2026-09-30
Buy milk	2026-10-01
Call the dentist	2026-10-05
Renew passport	2026-11-15
Water the plants	[NULL]

(5 rows)

Exercise 2 (Easy): Top three

1. The three most expensive products (`name` , `price`).
2. The two products with the most stock (`name` , `stock`).
3. The single newest product (by `added_on`).

Expected output of query 1:

```
name      | price
-----+-----
Backpack  | 45.00
Desk lamp | 24.99
Stapler   |  8.00
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Priority, then date

Show all tasks ordered by priority (1 is most urgent, so 1 first), and within the same priority, by due date, soonest first. Show `title` , `priority` , `due_date` .

Expected output:

```
title          | priority | due_date
-----+-----+-----
Finish SQL chapter 6 |      1 | 2026-09-30
Renew passport    |      1 | 2026-11-15
Buy milk          |      2 | 2026-10-01
Call the dentist  |      3 | 2026-10-05
Water the plants  |      3 | [NULL]
(5 rows)
```

Exercise 4 (Medium): Each value once

1. List each distinct `priority` value in `tasks` , smallest first.
2. List each distinct `added_on` date in `products` , newest first. How many rows do you get, and why isn't it 6?
3. List the distinct combinations of `on_sale` and `added_on` in `products` .

Exercise 5 (Challenge): Pages and a combined query

1. The shop shows products **2 per page**, sorted by name A to Z. Write the query for **page 2**.

Expected output:

```
name | price
-----+-----
Notebook | 3.50
Pen      | 1.20
(2 rows)
```

2. Write one query that answers: "What's the cheapest product that's in stock and **not** on sale?" It should return exactly one row.
3. Write a query showing the tasks that aren't done, most urgent first, but only the top 2. Then explain in a comment why `LIMIT 2` **without** `ORDER BY` would be a bug here, even though it also returns 2 rows.

Stuck? The hint is in the [Hints section](#) at the end.

10 Changing and Deleting Data

What is it?

`UPDATE` changes values in rows that already exist. `DELETE` removes rows. They're the **U** and **D** in CRUD.

Both use `WHERE` to decide *which* rows. And both do something new for you: they change data permanently, with no undo button. So this chapter is also about being careful.

Why does it matter?

Data changes. A customer moves house. A task gets ticked off. A price goes up. An account gets closed. Every one of those is an `UPDATE` or a `DELETE`.

And here's the thing that makes them different from everything so far: a **`SELECT` with a mistake gives you a wrong answer. An `UPDATE` with a mistake gives you wrong data**, for everyone, from now on. So professionals have habits for these two commands, and you'll learn them here.

Real-world example

Back to the library's index cards:

Librarian's job	SQL
Correct a spelling mistake on one card	<code>UPDATE books SET author = ... WHERE id = 10</code>
Cross out every card for books that were thrown away	<code>DELETE FROM books WHERE ...</code>
Photocopy the drawer before making changes, in case	<code>BEGIN ... ROLLBACK</code>
Find the cards first, <i>then</i> pick up the pen	<code>SELECT</code> with the same <code>WHERE</code> , then <code>UPDATE</code>

A careful librarian checks which cards they're about to change *before* writing on them. So will you.

How it works

You need the 9-row `books` table from [chapter 06](#), and `\pset null '[NULL]'`.

Adding a row with a mistake in it

Let's add a book, with two deliberate errors: the author's name is misspelled and the year is off by one.

```
INSERT INTO books (title, author, year)
VALUES ('Emma', 'Jane Austin', 1816)
RETURNING *;
```

id	title	author	year
10	Emma	Jane Austin	1816

Now let's fix it.

Changing a value: UPDATE

```
UPDATE books SET author = 'Jane Austen' WHERE id = 10;
```

```
UPDATE 1
```

Read it: *update books, set author to 'Jane Austen', where id is 10.*

- `SET column = value` says what to change.
- `WHERE` says which rows. Here, exactly one.
- The reply `UPDATE 1` tells you **how many rows changed**. Always read this number. If you expected 1 and see 47, something went wrong.

Using the old value in the new one

The right-hand side of `SET` can be an expression, using the row's current values:

```
UPDATE books SET year = year - 1 WHERE id = 10;
```

```
UPDATE 1
```

"Set year to the current year minus one." 1816 becomes 1815. This is how you do "raise every price by 10%" (`SET price = price * 1.1`) or "add one to the view count" (`SET views = views + 1`).

Several columns at once

Separate them with commas:

```
UPDATE books
SET author = 'Jane Austen',
    year = 1815
WHERE id = 10;
```

That's both fixes in one statement. Commas between the `SET` parts, not `AND`.

Getting the changed rows back

`RETURNING` works here too:

```
UPDATE books SET title = 'Emma (a novel)' WHERE id = 10 RETURNING id, title;
```

```
id |      title
----+-----
 10 | Emma (a novel)
(1 row)

UPDATE 1
```

Handy for checking what you did without a second query.

The disaster: forgetting `WHERE`

Here's what every database developer has done at least once. Without `WHERE`, `UPDATE` changes **every row**:

```
UPDATE books SET year = 2000;
```

```
UPDATE 10
```

Every book is now from the year 2000. There's no undo. If this were a real shop's product table, you'd have just set every price to the same number.

Don't run that. Instead, let's learn the safety net that lets you try it without consequences.

The safety net: `BEGIN` and `ROLLBACK`

PostgreSQL can treat a group of statements as a **draft**. Nothing becomes permanent until you say so:

```
BEGIN;
```

```
BEGIN
```

Now try the disaster:

```
UPDATE books SET year = 2000;  
SELECT title, year FROM books LIMIT 3;
```

```
UPDATE 10
```

```
  title  | year  
-----+-----  
The Hobbit | 2000  
Matilda   | 2000  
Dune      | 2000  
(3 rows)
```

It looks done. But it's a draft. Throw it away:

```
ROLLBACK;
```

```
ROLLBACK
```

```
SELECT title, year FROM books LIMIT 3;
```

```
  title  | year  
-----+-----  
The Hobbit | 1937  
Matilda   | 1988  
Dune      | 1965  
(3 rows)
```

Everything is back. Here's the vocabulary:

Command	Meaning
<code>BEGIN;</code>	Start a draft. From now on, changes are pending.
<code>ROLLBACK;</code>	Throw the draft away. Nothing you did since <code>BEGIN</code> happened.
<code>COMMIT;</code>	Make the draft permanent.

This draft is called a **transaction**, and it's one of the most important ideas in databases. [Chapter 30](#) covers it fully. For now, use it as a seatbelt: **when you're about to run an UPDATE or DELETE you're not sure about, type BEGIN; first.** Check the result. If it's right, COMMIT; . If not, ROLLBACK; .

Without BEGIN , every statement commits itself the moment it finishes. That's why the disaster above would have been permanent.

While you're in a transaction, the psql prompt changes to practice=## . The * means "you have a draft open". If you see it and don't know why, ROLLBACK; is the safe choice.

Removing rows: DELETE

```
DELETE FROM books WHERE id = 10;
```

```
DELETE 1
```

Emma is gone. DELETE FROM table WHERE condition removes every row that matches. Same shape as UPDATE , same rule: **read the number in the reply.**

Two things to know:

1. **The id is not reused.** The next book you insert gets id 11, not 10. Gaps in ids are normal and harmless.
2. **DELETE without WHERE deletes every row.** The table stays, empty. Same danger as UPDATE , same safety net.

RETURNING works with DELETE too, giving you one last look at what you removed:

```
DELETE FROM books WHERE author = 'Roald Dahl' RETURNING title;
```

(Wrap that in BEGIN / ROLLBACK if you try it. You want to keep those books for the project.)

Emptying a table: TRUNCATE

To remove every row on purpose, there's a faster command:

```
TRUNCATE books;
```

It's like DELETE FROM books with no WHERE , but quicker on big tables. Add RESTART IDENTITY to also reset the automatic id counter to 1. Don't run it now.

Command	Removes	Keeps
<code>DELETE FROM t WHERE ...</code>	Some rows	The table, the other rows
<code>DELETE FROM t / TRUNCATE t</code>	All rows	The empty table
<code>DROP TABLE t</code>	Everything	Nothing. The table is gone.

When nothing matches

```
UPDATE books SET year = 1900 WHERE id = 999;
```

```
UPDATE 0
```

Not an error. Zero rows matched, zero rows changed. Which is exactly why you read the number: `UPDATE 0` when you expected `UPDATE 1` means your `WHERE` is wrong, usually a typo or a case mismatch.

The habits of careful people

1. **SELECT first.** Write your `WHERE`, run it as a `SELECT`, look at the rows. If those are the rows you mean, change `SELECT *` to `UPDATE ... SET` or `DELETE`, keeping the `WHERE` identical.
2. **Prefer** `WHERE id = ...` for single rows. It can't match the wrong row.
3. **BEGIN when unsure.** Then `COMMIT` or `ROLLBACK`.
4. **Read the row count.** Every time.
5. **Never type** `UPDATE` or `DELETE` **without a** `WHERE` unless you really do mean every row, and you've said so out loud.

In pgAdmin

View/Edit Data → All Rows opens a grid. Double-click a cell to change it, or select a row and click the bin icon to delete it. Then press **Save**. pgAdmin writes the `UPDATE` or `DELETE` for you, always with `WHERE id = ...`. Fine for quick fixes.

Common mistakes

1. No `WHERE`

Every row changed or gone. Covered above. It will happen to you eventually; make sure it's inside a `BEGIN`.

2. `AND` between `SET` parts


```
UPDATE books SET author = 'Jane Austen' AND year = 1815 WHERE id = 10;
```

ERROR: argument of AND must be type boolean, not type text

Commas separate the things you're setting. `AND` is only for conditions.

3. UPDATE `0` because of case or spelling

`WHERE title = 'the hobbit'` matches nothing. Check with a `SELECT` first, or use `ILIKE`.

4. Confusing `DELETE` and `DROP`

`DELETE FROM books` empties the table. `DROP TABLE books` destroys it. Very different.

5. Leaving a transaction open

If the prompt shows `*`, you have uncommitted changes. Other people (and other windows) can't see them, and some things may hang waiting for you. `COMMIT` or `ROLLBACK`, then carry on.

Quick recap

- `UPDATE table SET col = value, col2 = value2 WHERE ...;` changes matching rows. The value can use the old one: `SET year = year - 1`.
- `DELETE FROM table WHERE ...;` removes matching rows. Ids are never reused.
- **Without `WHERE`, both hit every row.** No undo.
- `BEGIN;` opens a draft. `ROLLBACK;` throws it away. `COMMIT;` keeps it. Use it as a seatbelt.
- Read the reply: `UPDATE 1, DELETE 3, UPDATE 0`. It's PostgreSQL telling you what just happened.
- Habit: **`SELECT` with the same `WHERE` first**, then change it to `UPDATE` or `DELETE`.

Next: try the [exercises](#), then move on to your first project: [11 Project: My Book Collection](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. Run `playground/ch06/seed.sql` first so `tasks` and `products` are in their starting state.
- Run `\pset null '[NULL]'`.
- **Do the exercises in order.** Each one changes the data the next one uses.

- For every `UPDATE` and `DELETE` : write the `SELECT` with the same `WHERE` first, look, then change. Save both in your file.
- Save each answer in `playground/ch10/ex1.sql` , `ex2.sql` , and so on.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Tick it off

You bought the milk. Mark the "Buy milk" task as done. Use `RETURNING` to show the changed row.

Expected output:

```
id | title  | is_done | due_date | priority
---+-----+-----+-----+-----
  1 | Buy milk | t       | 2026-10-01 |      2
(1 row)

UPDATE 1
```

Exercise 2 (Easy): Sale prices

Everything that's on sale gets **10% added** to its price (the sale is ending). Update the prices in one statement, then show all products sorted by name.

Expected prices afterwards: Desk lamp `27.49` , Backpack `49.50` . The rest unchanged.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Delivery day

A delivery arrives: 200 Sticky notes and 25 Desk lamps.

1. Add 200 to the Sticky notes stock and 25 to the Desk lamp stock. You'll need two statements (or one clever one; two is fine).
2. The sale is over: set `on_sale` to false for **every** product, in one statement. Yes, this one really does mean every row. Say so in a comment above it.
3. Show `name` , `stock` , `on_sale` for all products.

Expected output of step 3 (sorted by name):

name	stock	on_sale
Backpack	8	f
Desk lamp	40	f
Notebook	120	f
Pen	500	f
Stapler	40	f
Sticky notes	200	f

(6 rows)

Exercise 4 (Medium): Clear the done tasks

1. Delete every task that is done. Use `RETURNING title` so you see what went.
2. Insert a new task: 'Book a haircut', not done, due `2026-10-10`, priority 3.
3. Show all tasks.

What `id` did the new task get? Write a comment explaining why it isn't 1 or 2, even though those ids are now free.

Expected output of step 3:

id	title	is_done	due_date	priority
3	Call the dentist	f	2026-10-05	3
4	Water the plants	f	[NULL]	3
5	Renew passport	f	2026-11-15	1
6	Book a haircut	f	2026-10-10	3

(4 rows)

Exercise 5 (Challenge): The seatbelt

1. Start a transaction. Set **every** product's price to `0.00`, on purpose. Run a `SELECT` to see the damage. Then roll back, and run the `SELECT` again to confirm everything is back.
2. Now a real change: the shop has stopped selling Staplers. Inside a transaction, delete the Stapler. Check with `SELECT` that only the Stapler is gone and 5 products remain. If so, commit. If not, roll back and try again.
3. Finally, run this and explain in a comment what went wrong and how you'd fix it. **Don't** "fix" the data, just explain:

```
UPDATE products SET stock = 0 WHERE name = 'backpack';
```

Stuck? The hint is in the [Hints section](#) at the end.

11 Project: My Book Collection

What you'll build

A database of your own books, and a set of saved queries that answer questions about them: what haven't you read yet, what did you rate highest, what's the longest thing on your shelf.

It's the whole of Level 1 in one go: plan a table, create it, fill it, ask it questions, keep it up to date, and do all of that in files you can re-run from scratch.

By the end, you'll have:

- a `my_books` database with a `books` table you designed yourself,
- at least 15 books in it,
- a `queries.sql` file answering 10 questions about your collection,
- an `updates.sql` file with safe changes,
- a folder you can rebuild with three `\i` commands.

Before you start

- Finish chapters 01 to 10.
- Make a folder `playground/ch11/`. **Every statement you type in this project goes in a file there**, even the one-liners. That's the point of Milestone 6.
- Look at the starter folder. It has 20 ready-made books you can load if you don't want to type your own, or want more data to play with.
- Run `\pset null '[NULL]'` in every psql session.

Milestone 1: Plan on paper

Goal: decide what you want to know about each book, before touching SQL ([chapter 02](#)).

Create `playground/ch11/plan.md` and write a table with three columns: **column name**, **type**, **example**. Start from this list and add anything you care about:

- the title and author,
- when it was published, and how long it is,
- what kind of book it is,
- whether you've finished it, when, and what you thought of it (1 to 5).

For each column, ask: **can this be empty?** An unread book has no rating and no finished date. That's NULL, not zero and not `1900-01-01`.

If you plan to use the starter data, your column names need to match the ones listed in the starter README. You can add extra columns on top.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 2: Create the database and table

Goal: turn the plan into a real table ([chapters 04](#) and [05](#)).

1. In psql, `CREATE DATABASE my_books;` then `\c my_books`.
2. Create `playground/ch11/schema.sql`. Put your `CREATE TABLE books (...)` in it, with `DROP TABLE IF EXISTS books;` above it, and a comment at the top saying what the file is.
3. Run it with `\i`, then `\d books`.

Check: `\d books` shows every column from your plan, with the type you meant. The `id` line is your recipe from chapter 05.

Milestone 3: Add your books

Goal: at least 15 rows ([chapter 06](#)).

Two ways, and you can mix them:

- **Your own books.** Create `playground/ch11/my_books.sql` and write the `INSERT`s. Use one multi-row `INSERT` rather than 15 separate ones.
- **The starter books.** Run `\i` on `starter/books.sql` (full path, forward slashes).

Make sure your data has some variety, because the queries in the next milestone need it: some finished books with ratings and dates, some unread ones with NULLs, at least two by the same author, and at least one title with an apostrophe.

Check: `SELECT * FROM books;` and read the row count at the bottom. 15 or more.

Milestone 4: Ask questions

Goal: one query per question, saved in `playground/ch11/queries.sql`, each with a comment above it saying which question it answers ([chapters 07](#), [08](#), [09](#)).

1. Every book, alphabetical by title. Just `title` and `author`.
2. Books you haven't finished.
3. Your **5 highest-rated** finished books, best first. Break ties by title.
4. All books in one genre of your choice, newest first.

5. Books whose author's name contains a word, ignoring case (for example 'weir').
6. The **3 longest** books.
7. Books published before 1950. (What happens to a book with an unknown year?)
8. Books you finished in 2026, in the order you finished them.
9. A data check: books that have a rating **but no finished date**, or a finished date but no rating. With clean data this returns nothing, which is the point.
10. One sentence per book, like `The Hobbit by J.R.R. Tolkien (1937)` , for finished books only.

Run the whole file with `\i` and read through the results one by one.

Check, if you used the starter data: question 2 gives 5 books. Question 3 gives Dune, Project Hail Mary, The Hobbit, The Name of the Wind, then Educated. Question 6 starts with The Name of the Wind. Question 8 gives 6 books.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 5: Keep it up to date

Goal: safe changes, saved in `playground/ch11/updates.sql` ([chapter 10](#)).

1. **You finished a book.** Pick an unread one. In a single `UPDATE` , set it to finished, give it a rating, and set the finished date to today (`current_date`). Wrap it in `BEGIN; ... COMMIT;` , with a `SELECT` in between to check.
2. **Fix a mistake.** Change one book's genre or year. Use `WHERE id = ...` .
3. **You gave a book away.** Delete it, with `RETURNING title` so you see it go.
4. **Rerun question 2 and question 9** from Milestone 4. Do the results make sense after your changes?

Above each statement, write the `SELECT` you used to check the `WHERE` first, as a comment or as a real statement.

Check: every reply says `UPDATE 1` or `DELETE 1` . If you ever see a bigger number, that's a `ROLLBACK` moment.

Milestone 6: Rebuild from scratch

Goal: prove your files are complete.

In `psql`, connected to `my_books` , run in order:

```
\i .../playground/ch11/schema.sql
\i .../playground/ch11/my_books.sql      (or the starter books.sql)
\i .../playground/ch11/queries.sql
```

If all three run without an error and the queries show sensible results, you have a **reproducible** database: anyone (including you, on a new laptop) can rebuild it from your files. This is how real projects work. The database is disposable; the files are what matter.

If something fails, fix the *file*, not the database, and run all three again.

Common mistakes

1. Using 0 or a made-up date for "not yet"

A rating of 0 for an unread book will show up in "lowest-rated books". A finished date of 1900-01-01 will show up in "books finished before 1950". Use NULL. It exists for exactly this.

2. Lists in a cell

`genre = 'Fantasy, Adventure'` breaks question 4. One genre per book for now. Chapter 17 fixes this properly.

3. Typing straight into psql

It works, and then it's gone. If it's not in a file, Milestone 6 will fail. Type it in VS Code, run it with `\i`.

4. Skipping the `SELECT` before an `UPDATE`

You'll get away with it nine times. The tenth time, you'll change every row. Build the habit while the stakes are low.

5. Booleans as text

`is_finished` text with values 'yes' and 'Yes' and 'y'. Then `WHERE is_finished = 'yes'` misses two thirds of your books. Use `boolean`.

Quick recap

- Plan first: columns, types, and **what can be NULL**.
- `schema.sql` creates, `my_books.sql` fills, `queries.sql` asks. Three files, one `\i` each.
- Variety in your data (NULLs, repeats, apostrophes) is what makes your queries worth testing.
- Every change goes through: `SELECT` first, `BEGIN`, change, check, `COMMIT`.
- If you can rebuild from files, you've done it right.

Congratulations! You've finished Level 1. 🎉 You can now store, find, change, and delete data in a table, which is more SQL than most people ever learn. Try the [exercises](#) for extra features, then head back to the roadmap. Level 2 is about designing tables well, and linking them together.

Exercises

How to do these:

- These are extra features for your book collection. Finish all 6 milestones in the notes first.
 - Keep working in `playground/ch11/`. After each exercise, the rebuild from Milestone 6 must still work.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to review your files.
-

Exercise 1 (Easy): A wishlist

You want to track books you *want* but don't own yet. Add a column `is_owned boolean` to your table.

You haven't learned how to add a column to an existing table (that's [chapter 15](#)). But you don't need to: edit `schema.sql`, add the column, and **rebuild** with your three `\i` commands. Then add two wishlist books with `is_owned = false`, and write a query that shows only the wishlist.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Recent reads

Write a query showing the books you finished in the **last 90 days**, counting from today, most recent first.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Re-reads

Some books you read more than once. Add a column `times_read integer` (rebuild, as in Exercise 1). Set it to `1` for finished books and `0` for unfinished ones, with two `UPDATE` s.

Then "re-read" one book: bump its `times_read` by one **without** typing the new number yourself, and update its `finished_on` to today, in one statement.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Export to a spreadsheet

psql can write a query's result to a file. This is a psql meta-command, so it goes on **one line** with no semicolon at the end:

```
\copy (SELECT title, author, published_year, rating FROM books ORDER BY title) TO
'C:/Users/YourName/Desktop/books.csv' WITH (FORMAT csv, HEADER)
```

Run it (with your own path), then open `books.csv` in Excel or VS Code. Each row of your table is a line, with commas between values, and the first line is the column names.

Now change the query inside the brackets so the export only includes finished books, best rated first.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Reading speed

Add a column `started_on date` (rebuild), and fill it in for your finished books. Make up dates if you have to, but make each one earlier than the book's `finished_on`.

Then write a query showing, for each finished book: `title`, how many **days** it took you, and roughly how many **pages per day** you read. Fastest first.

Two traps to find and solve:

1. Dividing two whole numbers gives a whole number (chapter 03, exercise 2). `320 / 45` is `7`, not `7.1`. How do you get the decimal?
2. A book started and finished on the same day gives `0` days. What happens when you divide by zero? Keep those books out of the result.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Level 2: Designing Good Tables

Goal: plan tables that keep your data clean, and link them together.

12 Data Types in Depth

What is it?

A **data type** says what kind of value a column can hold. You've been using four: `text`, `integer`, `boolean`, `date`. This chapter fills in the rest of the everyday set, explains the traps (money, time zones, dividing whole numbers), and gives you a simple guide for picking the right one.

Why does it matter?

The type you pick decides three things:

1. **What PostgreSQL refuses.** An `integer` column refuses `'ten'`. That's the safety you've seen since chapter 06.
2. **How values behave.** `'10' < '9'` is true for text and false for numbers. `7 / 2` is `3` for whole numbers. Dates can be subtracted, text can't.
3. **What can go wrong later.** Store money as the wrong kind of number and totals will be off by a cent. Store a moment in time without a time zone and it'll be wrong for anyone in another country.

Choosing types is a small decision you make once, and live with forever. Worth ten minutes of thought.

Real-world example

Think of the containers in a kitchen:

Kitchen	Data types
A measuring jug with exact markings	<code>numeric</code> : exact decimals, for money

Kitchen	Data types
A kitchen scale that's good to about a gram	<code>double precision</code> : approximate decimals, for measurements
An egg carton with 12 slots	<code>integer</code> : whole numbers only
A label maker	<code>text</code> : any words, any length
A light switch	<code>boolean</code> : on or off
A calendar on the wall	<code>date</code>
A clock that also knows which city it's in	<code>timestampz</code>

You wouldn't measure flour in an egg carton. Pick the container that fits the thing.

How it works

Work in `practice`. Everything here can be tried with `SELECT` alone, no tables needed, unless it says otherwise.

Text

Type	What it is	Use it?
<code>text</code>	Any length	Yes, by default
<code>varchar(n)</code>	At most <code>n</code> characters	Only when there's a real limit
<code>char(n)</code>	Exactly <code>n</code> characters, padded with spaces	No

In PostgreSQL, `text` is fast and there's no penalty for using it. Reach for `varchar(50)` only when a limit is a genuine rule of your data, like a 2-letter country code. Then PostgreSQL enforces it:

```
CREATE TABLE codes (country varchar(2));
INSERT INTO codes VALUES ('NPL');
```

```
ERROR: value too long for type character varying(2)
```

Avoid `char(n)`. It pads short values with spaces, which causes strange comparison bugs.

A habit you'll see in older code and tutorials: `varchar(255)` on everything. It's a leftover from other databases. In PostgreSQL, just write `text`.

Whole numbers

Type	Range	Use for
<code>smallint</code>	about ± 32 thousand	Rarely. Tiny counts.
<code>integer</code>	about ± 2.1 billion	Most whole numbers: counts, years, quantities
<code>bigint</code>	about ± 9 quintillion (9 with 18 zeros)	Huge counters, ids in very large tables

Go past the top and you get an error, not a wrong answer:

```
SELECT 2147483647 + 1;
```

```
ERROR: integer out of range
```

`integer` is the everyday choice. For an `id` column on a table that might one day have billions of rows (every click on a big website), many teams use `bigint` from the start, because changing later is painful. Your practice tables are fine with `integer`.

Dividing whole numbers

This is the surprise from chapter 03's exercises:

```
SELECT 7 / 2;      -- 3      whole ÷ whole = whole, remainder thrown away
SELECT 7 / 2.0;    -- 3.5    a decimal anywhere makes the answer a decimal
SELECT 7 % 2;      -- 1      % gives the remainder
```

If you divide two integer *columns* and want a decimal, turn one into a decimal first. You'll see how in the casting section below.

Decimals: exact or approximate?

There are two families, and picking the wrong one is one of the most common database mistakes in the world.

`numeric(precision, scale)` stores decimals **exactly**.

- `precision` is the total number of digits, `scale` is how many are after the point.
- `numeric(10, 2)` holds up to 99999999.99: 10 digits, 2 of them decimals.
- Plain `numeric` with no brackets holds any size and any number of decimals.

```
SELECT 0.1 + 0.2;  -- 0.3    (numbers you type are numeric by default)
```

`double precision` (and its smaller sibling `real`) store decimals **approximately**, the way most programming languages do. Fast, but:

```
SELECT 0.1::double precision + 0.2::double precision;
```

```
?column?
-----
0.30000000000000004
```

That tiny error is harmless when you're measuring temperature. It's a disaster when you're adding up invoices. A cent goes missing, the totals don't match, and someone spends a day finding out why.

Rule: money is always `numeric`. Use `numeric(10, 2)` for ordinary prices, or wider (`numeric(12, 2)`, `numeric(14, 4)`) if you need bigger numbers or more decimal places.

Two things `numeric(10, 2)` does that you should know:

```
CREATE TABLE prices (amount numeric(10, 2));
INSERT INTO prices VALUES (12.345);           -- stored as 12.35: rounded to fit
INSERT INTO prices VALUES (123456789.00);      -- ERROR: numeric field overflow
```

Too many decimals get rounded. Too many digits in total are refused.

PostgreSQL also has a type literally called `money`. **Don't use it.** Its formatting depends on the computer's language settings, and it's a well-known source of bugs. `numeric` does the job properly.

Booleans

`boolean` holds `true`, `false`, or `NULL` (unknown). PostgreSQL is relaxed about how you write them:

```
SELECT true, 'yes'::boolean, 'no'::boolean, '1'::boolean, 'off'::boolean;
```

```
bool | bool | bool | bool | bool
-----+-----+-----+-----+-----
t    | t    | f    | t    | f
```

`t`, `f`, `yes`, `no`, `on`, `off`, `1`, `0`, `y`, `n` are all accepted as input. But it's stored as a proper true/false and shown as `t` / `f`. Write `true` and `false` in your own SQL; save the aliases for reading other people's.

A boolean column can be used directly as a condition: `WHERE is_done` and `WHERE NOT is_done`, as you saw in chapter 08's exercises.

Dates and times

Type	Holds	Example	Use for
<code>date</code>	A calendar day	<code>'2026-09-30'</code>	Birthdays, due dates, publication dates
<code>time</code>	A time of day, no date	<code>'14:30'</code>	Opening hours. Rare.
<code>timestamp</code>	Date and time, no time zone	<code>'2026-09-30 14:30'</code>	Avoid for events. See below.
<code>timestampz</code>	Date and time, with time zone	<code>'2026-09-30 14:30+05:45'</code>	When something happened: created, updated, logged in
<code>interval</code>	A length of time	<code>'3 days'</code> , <code>'2 hours 30 minutes'</code>	Durations

Always write dates as `YYYY-MM-DD` . `'2026-09-30'` . No exceptions. `'30/09/2026'` and `'09/30/2026'` mean different things in different countries, and PostgreSQL may guess wrong.

`timestamp` vs `timestampz` . This one matters. `timestampz` (the `tz` is for time zone) stores an exact **moment**, the same instant for everyone on Earth, and shows it to you in your computer's time zone:

```
SELECT now();
```

```
now
-----
2026-09-30 14:30:12.48211+05:45
```

That `+05:45` is Nepal time. Someone running the same query in London would see the same moment as `08:45:12+01` . Plain `timestamp` drops the zone, so `'2026-09-30 14:30'` is just some 2:30pm, somewhere. When your React app has users in two countries, that's a bug.

Rule: for "when did this happen", use `timestampz` . For calendar dates with no time, use `date` .

Date maths

Dates and times are numbers underneath, so you can do arithmetic:

```

SELECT current_date;           -- today
SELECT current_date + 30;      -- 30 days from today
SELECT date '2026-12-25' - current_date; -- days until Christmas, an integer
SELECT now() + interval '2 hours'; -- two hours from now
SELECT now() - interval '3 days';  -- three days ago
SELECT date '2026-10-01' - date '2026-09-01'; -- 30

```

- `date + integer` gives a date.
- `date - date` gives an integer (days).
- `timestampz + interval` gives a timestampz.
- `timestampz - timestampz` gives an interval.

Intervals read like English: `interval '1 year'`, `interval '90 minutes'`, `interval '1 month 2 weeks'`.

`current_date` and `now()` are the two you'll use most. `now()` is the moment the current statement started, as a `timestampz`.

Casting: changing one type into another

Sometimes you have a value of one type and need another. That's a **cast**, and PostgreSQL has a short syntax for it: two colons.

```

SELECT '42'::integer + 8;      -- 50    text to number
SELECT 42::text || ' items';   -- 42 items  number to text
SELECT '2026-09-30'::date + 1; -- 2026-10-01
SELECT 7::numeric / 2;         -- 3.5000000000000000

```

That last one is the answer to the integer division problem: cast one side to `numeric` and the division becomes decimal. (Chapter 24 shows `round()` to tidy up all those zeros.)

The long form is `CAST('42' AS integer)`. It means the same thing and works in every database. The `::` form is PostgreSQL's, and it's what you'll see most.

A cast that can't work fails loudly:

```
SELECT 'hello'::integer;
```

```
ERROR:  invalid input syntax for type integer: "hello"
```

Types you'll meet later

Just so the names aren't strange when you see them:

- `uuid`: a long random id like `a3f1...-...`, an alternative to numbered ids. [Chapter 14](#) mentions it.
- `serial`: an older way to write an auto-numbered column. Also chapter 14.
- `jsonb`: stores a whole JavaScript-style object in one cell. [Chapter 37](#).
- `text[]`: an array (list) in one cell, which is sometimes acceptable despite chapter 02's rule. Also chapter 37.

Choosing a type: the short guide

You're storing	Use
Names, emails, addresses, descriptions	<code>text</code>
Counts, quantities, years, ratings	<code>integer</code>
Ids on a table that might get enormous	<code>bigint</code>
Money, prices, percentages	<code>numeric(10, 2)</code> or wider
Scientific measurements, coordinates	<code>double precision</code>
Yes/no	<code>boolean</code>
A calendar date	<code>date</code>
The moment something happened	<code>timestampz</code>
A duration	<code>interval</code>
A phone number, postcode, product code	<code>text</code> (see mistakes below)

Common mistakes

1. `double precision` or `real` for money

Cents go missing. Always `numeric`.

2. Numbers that aren't really numbers

A phone number like `01-4412345` or `+977 98...` isn't something you add or divide. Store it as `text`. Same for postcodes (`'0800'` loses its zero as an integer), and product codes. If you'd never do maths on it, it's text.

3. Dates as text

`'30/09/2026'` in a `text` column can't be sorted, compared, or have 30 days added. Use `date`, and write `YYYY-MM-DD`.

4. `timestamp` when you meant `timestamptz`

Works fine until your first user in another time zone. Use `timestamptz` for events.

5. Booleans as text

`status text` holding `'yes'`, `'Yes'`, `'Y'`, `'true'`. Use `boolean`.

6. Forgetting integer division

`total_pages / days` in a query gives a whole number and nobody notices for a month. Cast one side:
`total_pages::numeric / days`.

Quick recap

- **Text:** `text`, nearly always. `varchar(n)` only for real limits.
- **Whole numbers:** `integer` by default, `bigint` for huge counts. `7 / 2` is `3`.
- **Decimals:** `numeric(p, s)` is exact, use it for **money**. `double precision` is approximate, for measurements.
- **Dates:** `date` for days, `timestamptz` for moments. Always `YYYY-MM-DD`. Dates can be added and subtracted.
- **Casting:** `value::type` changes the type. `x::numeric / y` fixes integer division.
- Phone numbers, postcodes and codes are `text`, because you never do maths on them.

Next: try the [exercises](#), then move on to [13 Constraints](#).

Exercises

How to do these:

- Work in `psql`, connected to `practice`. Save each answer in `playground/ch12/ex1.sql`, `ex2.sql`, and so on.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Predict, then run

Before running them, write down what you think each line returns. Then run them and compare.

```

SELECT 10 / 4;
SELECT 10 / 4.0;
SELECT 10 % 4;
SELECT 10::numeric / 4;
SELECT '5' + 3;
SELECT '5' || 3;
SELECT 0.1 + 0.2;
SELECT 0.1::double precision + 0.2::double precision;
SELECT date '2026-03-01' - date '2026-02-01';
SELECT date '2026-03-01' - 1;

```

Two of them might surprise you. Write a comment next to each surprise explaining what happened.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Pick the type

For each piece of information, write the type you'd use and one sentence saying why:

1. A customer's phone number
2. A postcode
3. A birthday
4. An account balance
5. The moment a user signed up
6. Whether a user is an admin
7. A parcel's weight in kilograms, from a scale
8. A product code like `SKU-00421`
9. A blog post's body
10. How many likes a post has

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Watch `numeric` work

```

DROP TABLE IF EXISTS measurements;
CREATE TABLE measurements (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  exact_value numeric(6, 2),
  approx_value double precision
);

```

1. Insert a row with `1234.567` in **both** columns. Select it back. What did each column store?
2. Insert a row with `12345.67` in both. What happens, and why does only one column complain?
3. Insert `0.1` in both columns. Then run a query that multiplies each column by `3`. Compare the two answers.

Write a comment explaining, in your own words, when you'd choose each type.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Date arithmetic

Write one `SELECT` for each, using `current_date` or `now()` so the answers stay correct tomorrow:

1. The date 100 days from today.
2. How many days until the next New Year's Day (1 January 2027).
3. The date and time exactly 6 hours ago.
4. How old, in days, someone born on `2000-01-01` is today.
5. The date 3 months from today. (Use an `interval`.)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Flight times

Create a `flights` table with `flight_number` text, `departs_at` `timestampz`, `arrives_at` `timestampz`.

Insert this flight from Kathmandu (time zone `+05:45`) to Dubai (time zone `+04:00`):

- Departs `2026-10-01 08:00` Kathmandu time.
- Arrives `2026-10-01 10:30` Dubai time.

Write each time with its own offset, like `'2026-10-01 08:00+05:45'`.

1. Select both columns. PostgreSQL shows them in **your** time zone. Are the numbers what you typed? Why or why not?
2. Write a query showing the flight number and how long the flight took (`arrives_at - departs_at`). If you did it right, the answer is 4 hours 15 minutes, even though the clock times are only 2.5 hours apart.
3. Now redo it with `timestamp` (no tz) columns, inserting `'2026-10-01 08:00'` and `'2026-10-01 10:30'`. What duration do you get, and why is it wrong?

Stuck? The hint is in the [Hints section](#) at the end.

13 Constraints

What is it?

A **constraint** is a rule you attach to a table. Every time a row is added or changed, PostgreSQL checks the rule, and refuses anything that breaks it.

The four you'll use every day:

Constraint	The rule
NOT NULL	This column must always have a value.
UNIQUE	No two rows may have the same value here.
CHECK (...)	This condition must be true for every row.
DEFAULT ...	If no value is given, use this one. (Not strictly a constraint, but it lives with them.)

Two more, `PRIMARY KEY` and `FOREIGN KEY`, get their own chapters next.

Why does it matter?

Types stop `'ten'` from going in a price column. But types don't stop:

- a price of `-5`,
- a product with no name,
- two users with the same email,
- a rating of `11` on a 1-to-5 scale.

Constraints do. And here's the key point: the check happens **in the database**, so it applies to *everyone* who ever writes to that table. Your React app, a colleague's Python script, a hurried fix typed into psql at midnight. Every one of them hits the same rule. An app can forget to validate. The database never forgets.

Think of it as the last line of defence. Your app should check things too, so it can show a friendly message. But the database is the one that can't be bypassed.

Real-world example

A bouncer at the door of a club:

The bouncer says	Constraint
"You must be a person, not a dog"	The column's type
"No entry without ID"	NOT NULL
"That ticket's already been used"	UNIQUE
"You must be 18 or over"	CHECK (age >= 18)
"No coat? Here's a house one"	DEFAULT

One bouncer, one door, every guest checked the same way.

How it works

Work in `practice`. Let's rebuild the `products` table with rules this time. (Your seed version of `products` will be replaced; you can always recreate it with your chapter 06 seed file.)

```
DROP TABLE IF EXISTS products;

CREATE TABLE products (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text NOT NULL,
  sku text NOT NULL UNIQUE,
  price numeric(10, 2) NOT NULL CHECK (price >= 0),
  stock integer NOT NULL DEFAULT 0 CHECK (stock >= 0),
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

(A **SKU**, "stock keeping unit", is a shop's own code for a product, like `PEN-001`.)

Constraints go right after the type, and a column can have several. Let's break each one on purpose.

NOT NULL

```
INSERT INTO products (sku, price) VALUES ('PEN-001', 1.20);
```

```
ERROR: null value in column "name" of relation "products" violates not-null constraint
DETAIL: Failing row contains (1, null, PEN-001, 1.20, 0, t, 2026-09-30 10:15:02.11+05:45).
```

No name given, so `name` would be NULL, and the rule says it can't be. The whole row is refused. Notice the `DETAIL` line shows you the row PostgreSQL *would* have inserted, with the defaults filled in. That's handy for debugging.

(One small thing: even a failed insert uses up an id number. The next successful row will be `2`, not `1`. Gaps like this are normal.)

When to use it: almost always. A column that's *allowed* to be NULL should be a deliberate decision: "this really can be unknown" (a finished date on an unread book). Everything else gets `NOT NULL`.

UNIQUE

```
INSERT INTO products (name, sku, price) VALUES ('Pen', 'PEN-001', 1.20);
INSERT INTO products (name, sku, price) VALUES ('Blue pen', 'PEN-001', 1.50);
```

```
INSERT 0 1
ERROR:  duplicate key value violates unique constraint "products_sku_key"
DETAIL:  Key (sku)=(PEN-001) already exists.
```

The first insert works. The second tries to reuse the SKU and is refused. PostgreSQL named the constraint for you: `products_sku_key`, from the table and column.

Two things to know about `UNIQUE` :

- It's **case-sensitive**: `'PEN-001'` and `'pen-001'` are different values. For emails, that's a problem you'll solve in [chapter 24](#) by storing them in lowercase.
- On its own, it **allows several NULLs**, because two unknowns aren't "the same value". That's why you'll usually see `NOT NULL UNIQUE` together.

CHECK

```
INSERT INTO products (name, sku, price) VALUES ('Free pen', 'PEN-002', -1.00);
```

```
ERROR:  new row for relation "products" violates check constraint "products_price_check"
DETAIL:  Failing row contains (3, Free pen, PEN-002, -1.00, 0, t, 2026-09-30 10:16:40.52+05:45).
```

`CHECK` takes any condition that's true or false, the same kind you write after `WHERE` :

```

CHECK (price >= 0)
CHECK (rating BETWEEN 1 AND 5)
CHECK (status IN ('draft', 'published', 'archived'))
CHECK (email LIKE '%@%')
CHECK (length(name) > 0)

```

That third one, `IN (...)`, is how you make a column that only accepts a fixed set of words. Very common.

The NULL catch. A `CHECK` only refuses rows where the condition is **false**. If the column is `NULL`, the condition is *unknown*, and unknown is let through. So `CHECK (rating BETWEEN 1 AND 5)` on its own still allows a `NULL` rating. If you don't want that, add `NOT NULL` too. This is the same three-way logic you met in [chapter 08](#).

DEFAULT

```

INSERT INTO products (name, sku, price) VALUES ('Stapler', 'STA-001', 8.00)
RETURNING *;

```

id	name	sku	price	stock	is_active	created_at
4	Stapler	STA-001	8.00	0	t	2026-09-30 10:18:05.30411+05:45

You gave three values and got seven. `stock` became `0`, `is_active` became `true`, and `created_at` became the current moment, all from their defaults.

`DEFAULT now()` is the one you'll write most. Every table that records "things happening" wants a `created_at timestamptz NOT NULL DEFAULT now()`. The `now()` runs at the moment of each insert, so every row gets its own time.

Two rules about defaults:

- A default is used only when you **leave the column out** (or write the word `DEFAULT` in its place). If you write `NULL` explicitly, you get `NULL`, and then `NOT NULL` will refuse it.
- `DEFAULT` and `NOT NULL` are a natural pair: "must have a value, and here's a sensible one if you don't say".

Rules that involve two columns

A constraint that compares columns goes at the **end** of the table, after the last column. You can also give it your own name with `CONSTRAINT`:


```
CREATE TABLE events (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text NOT NULL,
  starts_at timestamptz NOT NULL,
  ends_at timestamptz NOT NULL,
  CONSTRAINT ends_after_start CHECK (ends_at > starts_at)
);
```

Now an event that ends before it starts gets:

```
ERROR: new row for relation "events" violates check constraint "ends_after_start"
```

Your own name makes the error mean something. Naming is optional for single-column constraints, where the automatic names are fine.

`UNIQUE` can also cover a **combination** of columns:

```
UNIQUE (room_id, booked_on)
```

That means "the same room can't be booked twice on the same day", while still letting the room appear on many days and many rooms appear on one day. You'll use this a lot in [chapter 17](#).

Seeing the rules: \d

```
\d products
```

Column	Type	Collation	Nullable	Default
id	integer		not null	generated always as identity
name	text		not null	
sku	text		not null	
price	numeric(10,2)		not null	
stock	integer		not null	0
is_active	boolean		not null	true
created_at	timestamp with time zone		not null	now()

Indexes:

```
"products_pkey" PRIMARY KEY, btree (id)
"products_sku_key" UNIQUE CONSTRAINT, btree (sku)
```

Check constraints:

```
"products_price_check" CHECK (price >= 0::numeric)
"products_stock_check" CHECK (stock >= 0)
```

Everything is listed: `not null` in the Nullable column, defaults, the `UNIQUE` (shown under Indexes, because that's how PostgreSQL enforces it), and the checks. `timestamp with time zone` is the long name for `timestampz`.

Constraints apply to `UPDATE` too

Rules aren't only checked on the way in:

```
UPDATE products SET stock = stock - 5 WHERE sku = 'STA-001';
```

```
ERROR: new row for relation "products" violates check constraint "products_stock_check"
```

Stock was 0; 0 minus 5 is negative; refused. This is how a `CHECK (balance >= 0)` on a bank account stops an overdraft, no matter which app tries it.

Adding rules to an existing table

You can add or remove constraints later with `ALTER TABLE` ([chapter 15](#)). The catch: every existing row has to pass the new rule, or the `ALTER` is refused. So clean the data first, then add the rule.

What constraints can't do

They check **shape**, not **meaning**. A title of `'Jane Austen'` in a `title text NOT NULL` column passes every rule. `'Kathmandu'` and `'kathmandu'` are both valid text. Constraints stop nonsense; they don't stop mistakes that look like valid data. For that you need good design (chapters 16 to 18) and careful people.

Common mistakes

1. Forgetting `NOT NULL`

The most common one. A `CHECK` lets `NULL` through, `UNIQUE` lets several `NULL`s through, and then your "required" column is half empty. Make `NOT NULL` your default habit.

2. Trusting the app to validate

"Our React form already checks the price is positive." Then someone writes a script, or edits in pgAdmin, or a second app appears. Put the rule in the database too.

3. A `CHECK` that's too strict for real life

`CHECK (length(phone) = 10)` breaks the moment someone from another country signs up. `CHECK (age >= 18)` blocks a legitimate 17-year-old with parental consent. Only encode rules you're sure about.

4. UNIQUE on something that legitimately repeats

Two customers can share a name. `UNIQUE` belongs on emails, usernames, SKUs, and codes, not on names.

5. DEFAULT 'now' instead of DEFAULT now()

With quotes, `'now'` is a piece of text that PostgreSQL converts to a time **once, when the table is created**, and then every row gets that same stale time. With brackets, `now()` is a function that runs for each insert. Always the brackets.

6. Expecting a default to rescue an explicit NULL

`INSERT INTO products (name, sku, price, stock) VALUES ('x', 'X-1', 1, NULL)` puts NULL in `stock`, and `NOT NULL` refuses it. Defaults only fill in what you leave out.

Quick recap

- Constraints are rules the database checks on every `INSERT` and `UPDATE`, for everyone, always.
- `NOT NULL`: must have a value. Use it on almost every column.
- `UNIQUE`: no repeats. Case-sensitive, and allows several NULLs unless combined with `NOT NULL`.
- `CHECK (condition)`: any true/false rule. NULL passes, so pair with `NOT NULL`. `IN (...)` gives a fixed set of allowed values.
- `DEFAULT value`: used when the column is left out. `DEFAULT now()` for timestamps, with brackets.
- Two-column rules and your own names go at the end: `CONSTRAINT name CHECK (...)`, `UNIQUE (a, b)`.
- `\d table` shows every rule. Read the error's `DETAIL` line when something is refused.

Next: try the [exercises](#), then move on to [14 Primary Keys](#).

Exercises

How to do these:

- Work in `psql`, connected to `practice`. Save each answer in `playground/ch13/ex1.sql`, `ex2.sql`, and so on.
- Start each file with `DROP TABLE IF EXISTS ...;` so it can be run again.
- When an exercise says "try to break it", run the failing statement, and paste the **first line of the error** into your file as a comment.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): A users table with rules

Create a `users` table where:

- `email` and `username` are required and can't repeat,
- `is_admin` is required and defaults to false,
- `created_at` is required and defaults to the current moment,
- `display_name` is optional.

Then run these five inserts. **Before** running each, write a comment predicting whether it succeeds or fails, and which rule it hits.

```
INSERT INTO users (email, username) VALUES ('asha@example.com', 'asha');
INSERT INTO users (email, username) VALUES ('asha@example.com', 'asha2');
INSERT INTO users (email) VALUES ('bikram@example.com');
INSERT INTO users (email, username, is_admin) VALUES ('chandra@example.com', 'chandra', NULL);
INSERT INTO users (email, username, display_name) VALUES ('dev@example.com', 'dev', NULL);
```

Finish with `SELECT * FROM users;`. You should have exactly 2 rows.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Read the error, name the rule

Someone ran statements against a table you can't see, and got these errors. For each, write which kind of constraint was hit, and what the table's rule probably is:

ERROR: duplicate key value violates unique constraint "employees_badge_number_key"

ERROR: null value in column "hired_on" of relation "employees" violates not-null constraint

ERROR: new row for relation "employees" violates check constraint "employees_salary_check"

ERROR: new row for relation "employees" violates check constraint "leaves_after_joins"

Bonus: which one of the four was given a name by a person rather than by PostgreSQL? How can you tell?

Exercise 3 (Medium): Reviews

Create a `reviews` table for product reviews:

- `product_name` required.
- `rating` required, a whole number from 1 to 5.
- `body` required, and at least 10 characters long. Give this rule your own name, `body_long_enough`.
- `is_verified` required, default false.
- `written_at` required, default now.

Try to break each rule with a separate insert (rating 0, rating 6, a 5-character body, a missing rating). Then insert two valid reviews.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): No overdrafts

Create an `accounts` table with `owner` text NOT NULL and `balance` numeric(12, 2) NOT NULL DEFAULT 0 CHECK (balance >= 0).

1. Insert an account for 'Asha' with no balance given. What does it start at?
2. Add 500 to it with an `UPDATE`.
3. Take 200 out.
4. Try to take 400 out. Read the error.
5. Now do step 4 again, but inside `BEGIN; ... ROLLBACK;`. Does the constraint still fire? What does the balance show after the rollback?

Write a comment: why is this `CHECK` more trustworthy than a "you can't withdraw more than your balance" check written in an app?

Exercise 5 (Challenge): Meeting rooms

A company books meeting rooms by the day. Create `room_bookings` with:

- `room_name` required.
- `booked_on` a required date.
- `booked_by` required.
- `status` required, default 'pending', and only ever 'pending', 'confirmed', or 'cancelled'.
- **A room can only be booked once per day.**

Then:

1. Book 'Everest' for 2026-10-03 by Asha. Book 'Everest' for 2026-10-04 by Bikram. Book 'Annapurna' for 2026-10-03 by Chandra. All three should succeed.
2. Try to book 'Everest' for 2026-10-03 again. It must fail.
3. Try to insert a booking with status 'maybe' . It must fail.
4. Confirm Asha's booking with an UPDATE . Then try to set its status to 'done' . It must fail.
5. Run \d room_bookings and find where PostgreSQL lists your "once per day" rule.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

14 Primary Keys

What is it?

A **primary key** is the column (or combination of columns) that identifies each row. Two rules come built in: the value must be **unique**, and it can **never be NULL**. Every table gets exactly one primary key.

You've been making one since chapter 05 with the recipe line. This chapter explains what every word of it does, and when you'd write something different.

Why does it matter?

Let's see what goes wrong without one. Work in `practice`:

```
CREATE TABLE guests (name text, table_number integer);
INSERT INTO guests VALUES ('Sam', 4), ('Sam', 4), ('Priya', 2);
```

Two guests called Sam, both at table 4. Now one of them moves to table 7:

```
UPDATE guests SET table_number = 7 WHERE name = 'Sam';
```

```
UPDATE 2
```

Both Sams moved. There is **no way** to say "this Sam, not that one". They're identical in every column. You can't update one, you can't delete one, and no other table can point at one. The row has no identity.

A primary key fixes this once and for all: every row gets a value that belongs to it alone.

```
DROP TABLE guests;
```

Real-world example

Two people called Sita Sharma live in the same town. The government doesn't tell them apart by name. It gives each a **citizenship number**. That number:

- is different for every person (unique),
- everyone has one (not null),
- never changes, even if she changes her name (stable),
- is what every other office uses to refer to her (other tables point at it).

That's a primary key. A library barcode does the same for two copies of the same book.

How it works

The recipe, word by word

```
id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY
```

Part	Meaning
id	The column's name. Just a convention; you could call it anything.
integer	Whole numbers.
PRIMARY KEY	This column identifies rows. Implies <code>UNIQUE</code> and <code>NOT NULL</code> , and creates an index (chapter 31) so lookups by id are fast.
GENERATED ALWAYS AS IDENTITY	PostgreSQL keeps a counter and fills in the next number on every insert. <code>ALWAYS</code> means you're not allowed to supply your own.

That counter is called a **sequence**. It's a tiny object that lives next to the table and remembers "the next number is 10". You saw it in `\d books as generated always as identity`.

Natural keys and surrogate keys

There are two ways to pick a primary key:

- A **natural key** is a real-world identifier that already exists: an email address, an ISBN, a citizenship number, a country code.
- A **surrogate key** is a made-up number with no meaning outside the database: `id`.

Natural keys sound appealing. Why invent a number when a book already has an ISBN? Here's why:

Problem with natural keys	Example
They change	People change their email. Then every table pointing at the old one breaks.
They're sometimes missing	Old books have no ISBN. <i>Beowulf</i> has no author.
They repeat in surprising ways	The same book gets a new ISBN for each edition. Two products can share a barcode across countries.

Problem with natural keys	Example
They're long and easy to mistype	978-0-547-92822-7 vs 7

A surrogate `id` has none of these problems: always present, never changes, short.

The advice this course follows: give nearly every table a surrogate `id`, and put `UNIQUE NOT NULL` on the natural candidates (like `email`) so they're still protected. You get identity *and* the real-world rule.

The exception: tiny reference tables where the natural key really is stable and universal, like a `countries` table keyed by its two-letter code. You'll try that in the exercises.

ALWAYS vs BY DEFAULT

```
id integer PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY
```

`BY DEFAULT` means "fill it in for me *unless* I give a value". It's useful once: when you're importing old data that already has ids. Otherwise it's a trap:

```
CREATE TABLE tickets (
  id integer PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  subject text NOT NULL
);
INSERT INTO tickets (id, subject) VALUES (1, 'Printer broken'); -- you chose 1
INSERT INTO tickets (subject) VALUES ('Login not working'); -- counter gives... 1
```

```
ERROR: duplicate key value violates unique constraint "tickets_pkey"
DETAIL: Key (id)=(1) already exists.
```

The counter didn't know you'd used 1. After an import, you have to tell it where to continue:

```
ALTER TABLE tickets ALTER COLUMN id RESTART WITH 100;
```

Stick with `ALWAYS` unless you're importing. It can't get out of step, because it never lets you choose.

serial : the old way

You'll see this in tutorials, older code, and Stack Overflow answers:

```
id serial PRIMARY KEY
```

`serial` does the same job as `GENERATED BY DEFAULT AS IDENTITY`, in an older style with a few quirks around permissions and copying tables. Read it as "auto-numbered id". In your own code, write the `IDENTITY` form. It's the standard, and it's what PostgreSQL recommends.

Gaps are normal

Ids will have gaps. A failed insert uses a number. A rolled-back transaction uses a number. Deleting a row leaves a hole. Your table might have ids 1, 2, 5, 6, 9.

This is fine. An id is a name, not a position. Don't try to "fix" gaps, don't reuse a deleted id, and don't assume the biggest id equals the number of rows. If you need row numbers for display, chapter 28 has a proper tool for that.

integer or bigint?

`integer` runs out at about 2.1 billion. For a table of books, customers, or products, you'll never get there. For a table that logs every click, message, or sensor reading on a busy system, you might. Changing the type later is possible but slow on a huge table, so many teams write `bigint` for every id from day one:

```
id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY
```

Either is fine for this course. Just know why you'd see both.

Composite primary keys

Sometimes no single column identifies a row, but a **combination** does. A student can take many courses and a course has many students, but each (student, course) pair happens once:

```
CREATE TABLE enrollments (  
  student_id integer NOT NULL,  
  course_id integer NOT NULL,  
  PRIMARY KEY (student_id, course_id)  
);
```

The primary key goes at the end, listing both columns. Now the same student can't enroll in the same course twice, but can enroll in many courses. This is the main tool of [chapter 17](#).

UUIDs: another kind of surrogate key

Instead of counting 1, 2, 3, some systems use a **UUID**: a random 128-bit value that looks like `a3f1c2d4-8e5b-4c7a-9f2e-1b3d5a7c9e0f`.

```
id uuid PRIMARY KEY DEFAULT gen_random_uuid()
```

Why? Two reasons. Ids can be generated anywhere (a phone app that's offline, two databases that later merge) without clashing. And they can't be guessed: with numbered ids, `/orders/1043` invites someone to try `/orders/1044`.

The cost: they're long, they don't sort by creation order, and they're horrible to type in psql. For this course, and for most apps, numbered ids are the right call.

Using the key

Everything you learned in chapter 10 leans on the primary key:

```
UPDATE books SET year = 1938 WHERE id = 1;  
DELETE FROM books WHERE id = 7;
```

`WHERE id = ...` can never match the wrong row, because no two rows share an id. That's why careful people prefer it.

Common mistakes

1. A table with no primary key

The two Sams. Every table gets one, even tiny ones, even "temporary" ones.

2. Using something that can change

Email, username, phone number. The day it changes, everything pointing at it breaks. Surrogate `id`, plus `UNIQUE` on the natural column.

3. Treating ids as row numbers

"Show the 5th customer" is `ORDER BY ... LIMIT 1 OFFSET 4`, not `WHERE id = 5`. Ids have gaps.

4. Supplying ids by hand

With `ALWAYS`, PostgreSQL refuses, which is the point. With `BY DEFAULT`, it lets you, and the counter gets out of step. Let the database count.

5. A primary key on `name`

Names repeat, change, and have spelling variations. Never.

6. Two "primary keys"

A table has exactly one. If two columns each need to be unique, one is the `PRIMARY KEY` and the other gets `UNIQUE`.

Quick recap

- A primary key identifies each row: **unique, never NULL**, one per table.
 - `id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY` is a **surrogate key**: PostgreSQL counts for you, and you can't override it.
 - Prefer surrogate ids over natural keys (email, ISBN), and protect natural candidates with `UNIQUE NOT NULL` instead.
 - Gaps in ids are normal. Never reuse or "fix" them.
 - `serial` is the old spelling. `BY DEFAULT` is for imports only. `bigint` for tables that might get enormous. `uuid` when ids are made in many places.
 - A **composite** primary key, `PRIMARY KEY (a, b)`, identifies rows by a pair. Chapter 17 uses it heavily.
-

Next: try the [exercises](#), then move on to [15 Changing a Table's Shape](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. Save each answer in `playground/ch14/ex1.sql`, `ex2.sql`, and so on.
 - Start each file with `DROP TABLE IF EXISTS ...;`
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): The two Sams

1. Recreate the `guests` table from the notes, **without** a primary key. Insert the two Sams and Priya.
2. Try to delete **only one** Sam. Write a comment explaining why you can't.
3. Drop the table and recreate it with the `id` recipe. Insert the same three guests.

4. Now delete one Sam, by id. Confirm the other is still there.

Exercise 2 (Easy): Natural or surrogate?

For each table, say whether you'd use a natural key or a surrogate `id`, and which column(s) you'd protect with `UNIQUE` :

1. `customers` (name, email, phone)
2. `countries` (two-letter code, name)
3. `books` (title, isbn, author)
4. `cars` (license plate, make, model, owner)
5. `employees` (name, badge number, national id number)
6. `currencies` (three-letter code like NPR, USD; name)

Write one sentence of reasoning for each.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A natural key that works

Create a `countries` table with a **natural** primary key: `code text PRIMARY KEY`, plus `name text NOT NULL UNIQUE`. Add a `CHECK` so the code is always exactly 2 characters.

1. Insert Nepal (`NP`), India (`IN`), and the United States (`US`).
2. Try to insert `'NPL'`. Try to insert `'NP'` again. Try to insert a country with no code.
3. Run `\d countries`. What's the primary key's index called?

Then a thought experiment, as a comment: suppose 40 other tables refer to countries by this code, and one day a country changes its code. What has to happen? (Chapter 16 will show a tool that helps, but the pain is the point.)

Exercise 4 (Medium): The import trap

1. Create a `tickets` table with `GENERATED BY DEFAULT AS IDENTITY` and a `subject text NOT NULL`.
2. "Import" three old tickets with ids 1, 2, 3 by supplying the ids yourself.
3. Insert a new ticket **without** an id. Read the error.
4. Fix the counter with `ALTER TABLE ... RESTART WITH ...`, so the next automatic id is 4. Insert the new ticket again and check its id.

- Drop the table and recreate it with `GENERATED ALWAYS`. Try step 2 again. What happens, and why is that better most of the time?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Composite keys at the cinema

A cinema sells seats for screenings. Create `seat_reservations` with `screening_id integer NOT NULL`, `seat_number text NOT NULL`, `customer_name text NOT NULL`, `reserved_at timestamptz NOT NULL DEFAULT now()`, and a **composite** primary key on the screening and the seat.

- Reserve seat `'A1'` and `'A2'` for screening 1, and `'A1'` for screening 2. All should work.
- Try to reserve `'A1'` for screening 1 again. It must fail. Copy the error's `DETAIL` line.
- Answer in comments:
 - Why doesn't this table need an `id` column?
 - Someone suggests adding an `id` anyway "so it's like our other tables". If you did, what constraint would you **have** to keep, or the double-booking bug comes back?

Stuck? The hint is in the [Hints section](#) at the end.

15 Changing a Table's Shape

What is it?

`ALTER TABLE` changes a table that already exists: add a column, remove one, rename things, change a type, add or drop a rule. It's how a table keeps up with a changing app without being thrown away and rebuilt.

This chapter also tidies up the three ways to get rid of things: `DELETE`, `TRUNCATE`, and `DROP`.

Why does it matter?

So far, when a table was wrong, you dropped it and created it again. That works when the table holds 9 practice rows. It does not work when it holds three years of customer orders.

Real tables live for years and requirements change constantly. "We need a phone number on customers." "Prices need more decimal places." "Rename `on_sale` to `is_on_sale` to match the others." Every one of those is an `ALTER TABLE`, run while the data stays put.

In a team, these changes are written as scripts and run in order on every copy of the database. Those scripts are called **migrations**, and [chapter 41](#) is about them. `ALTER TABLE` is what's inside.

Real-world example

Renovating a house **while people live in it**:

Renovation	ALTER TABLE
Add a room	<code>ADD COLUMN</code>
Knock a room down (and everything in it is gone)	<code>DROP COLUMN</code>
Put a new name on the door	<code>RENAME</code>
Rewire a room, carefully, with the lights still on	<code>ALTER COLUMN ... TYPE</code>
Add a house rule: "no shoes indoors"	<code>ADD CONSTRAINT</code>

Some renovations are trivial (an empty new room). Some need care (rewiring a room that's in use). PostgreSQL will tell you which is which, by refusing anything that would break the existing data.

How it works

Work in `practice`, on the 9-row `books` table from [chapter 06](#). If yours has changed, recreate it with the chapter 06 statements. Run `\pset null '[NULL]'`.

Adding a column

```
ALTER TABLE books ADD COLUMN pages integer;
```

```
ALTER TABLE
```

Every existing row now has a `pages` column, holding NULL, because nothing was there before. Fill it with `UPDATE` :

```
UPDATE books SET pages = 310 WHERE id = 1;
```

Adding a column that can't be NULL

```
ALTER TABLE books ADD COLUMN is_read boolean NOT NULL;
```

```
ERROR: column "is_read" of relation "books" contains null values
```

PostgreSQL can't add a required column to 9 existing rows without knowing what to put in it. Give it a default:

```
ALTER TABLE books ADD COLUMN is_read boolean NOT NULL DEFAULT false;
```

Now every existing row gets `false`, and the rule holds. This is the normal way to add a required column to a table that already has rows.

The other way, when the right value differs per row: add it nullable, fill it with `UPDATE` s, then lock it down with `SET NOT NULL` (below).

Removing a column

```
ALTER TABLE books DROP COLUMN pages;
```

Gone, and so is every value that was in it. No undo, unless you're inside a transaction (keep reading).

Renaming

```
ALTER TABLE books RENAME COLUMN year TO published_year;
ALTER TABLE books RENAME TO my_books;
ALTER TABLE my_books RENAME TO books;
```

Renaming is instant and safe for the data. It's **not** safe for everything that uses the old name: your saved `.sql` files, your future apps. Any query that says `year` is now broken. Renaming is a decision to go and update those too.

(Keep the column as `published_year` for the rest of this chapter. It's a better name anyway.)

Changing a column's type

Some changes PostgreSQL can do on its own, because every value converts cleanly:

```
ALTER TABLE books ALTER COLUMN published_year TYPE smallint;    -- all years fit
ALTER TABLE books ALTER COLUMN published_year TYPE text;         -- numbers become text
```

Going back is harder. Text to number isn't automatic, because text *might* contain `'hello'`:

```
ALTER TABLE books ALTER COLUMN published_year TYPE integer;
```

```
ERROR: column "published_year" cannot be cast automatically to type integer
HINT: You might need to specify "USING published_year::integer".
```

The hint tells you what to do. `USING` says *how* to convert each existing value:

```
ALTER TABLE books ALTER COLUMN published_year TYPE integer USING published_year::integer;
```

If any value can't convert, the whole `ALTER` fails and nothing changes. That's the lights-on rewiring: PostgreSQL won't leave you half done.

Widening is always easy (`numeric(10, 2)` to `numeric(12, 2)`, `integer` to `bigint`). Narrowing fails if any value won't fit.

Defaults and NOT NULL on existing columns

```
ALTER TABLE books ALTER COLUMN author SET DEFAULT 'Unknown';
ALTER TABLE books ALTER COLUMN author DROP DEFAULT;
```

Defaults only affect **future** inserts. Existing rows don't change.

```
ALTER TABLE books ALTER COLUMN author SET NOT NULL;
```

```
ERROR: column "author" of relation "books" contains null values
```

Beowulf has no author. Same story as before: the data must already obey the rule. Fix the data, then add the rule:

```
UPDATE books SET author = 'Unknown' WHERE author IS NULL;  
ALTER TABLE books ALTER COLUMN author SET NOT NULL;
```

And the reverse, `DROP NOT NULL`, always works.

Adding and removing constraints

```
ALTER TABLE books ADD CONSTRAINT year_is_sensible  
CHECK (published_year BETWEEN 1 AND 2100);
```

Existing rows are checked. If one fails:

```
ERROR: check constraint "year_is_sensible" of relation "books" is violated by some row
```

Clean the data first, then try again. To remove a rule, use its name:

```
ALTER TABLE books DROP CONSTRAINT year_is_sensible;
```

You can see constraint names with `\d books`. Automatic names look like `books_title_key` or `books_published_year_check`.

`UNIQUE` works the same way:

```
ALTER TABLE books ADD CONSTRAINT books_title_key UNIQUE (title);
```

Several changes in one statement

Separate actions with commas:

```
ALTER TABLE books
  ADD COLUMN pages integer,
  ADD COLUMN isbn text,
  ALTER COLUMN is_read SET DEFAULT true;
```

Either all of them happen or none do.

A PostgreSQL superpower: structure changes can be rolled back

In many databases, `ALTER TABLE` and `DROP TABLE` are permanent the instant you press Enter. In PostgreSQL, they obey transactions:

```
BEGIN;
ALTER TABLE books DROP COLUMN is_read;
\d books                -- is_read is gone
ROLLBACK;
\d books                -- is_read is back, data and all
```

This is a big deal. **Every risky `ALTER` should start with `BEGIN;`** . Make the change, look at `\d` and a `SELECT` , and only `COMMIT` when it's right.

The three ways to remove things

Command	What goes	What stays	Undo?
<code>DELETE FROM t WHERE ...</code>	Some rows	Table, structure, other rows	Only inside a transaction
<code>TRUNCATE t</code>	All rows	Empty table, structure	Only inside a transaction
<code>DROP COLUMN c</code>	One column and all its values	The rest of the table	Only inside a transaction
<code>DROP TABLE t</code>	Everything	Nothing	Only inside a transaction

`DROP TABLE` has one extra: if another table points at this one (next chapter), PostgreSQL refuses, and you'll need `DROP TABLE t CASCADE` to take the dependents down with it. More on that in [chapter 16](#).

The recipe for a safe change

1. **Write the `ALTER` in a `.sql` file**, not straight into `psql`. You'll want it again on the next copy of the database.
2. `BEGIN`;
3. Run the file.
4. Check: `\d table`, and a `SELECT` on the affected column.
5. `COMMIT`; if it's right, `ROLLBACK`; if not.

That file is your first migration. In chapter 41 you'll learn how teams number and track them.

In pgAdmin

Right-click a table → **Properties** → **Columns** tab. You can add, remove, and rename columns with clicks, and the **SQL** tab shows the `ALTER TABLE` it's about to run. It's a good way to check your understanding of the syntax. But for anything that matters, write the SQL yourself, in a file, inside a transaction.

Common mistakes

1. `DROP COLUMN` outside a transaction

The data is gone. A `BEGIN` costs nothing and gives you a way back.

2. Adding a `NOT NULL` column with no `DEFAULT` to a full table

Fails every time. Add a default, or add nullable and fill it first.

3. Renaming and forgetting who uses the old name

The `ALTER` works. Your `queries.sql` from chapter 11 and your future app code break. Search your files for the old name before you rename.

4. Type change without `USING`

Read the `HINT`. It usually hands you the exact `USING` you need.

5. Changing the table and not saving the statement

The database on your laptop now differs from the one in your files. Next time you rebuild from `schema.sql`, the change is missing. Every `ALTER` goes in a file.

6. Confusing `DROP COLUMN` with `DELETE`

`DELETE` removes rows (data). `DROP COLUMN` removes a column (structure, and the data in it). Very different sizes of mistake.

Quick recap

- `ALTER TABLE t ADD COLUMN c type` adds a column; existing rows get NULL, or the `DEFAULT` if you give one.
 - `DROP COLUMN`, `RENAME COLUMN a TO b`, `RENAME TO`, `ALTER COLUMN c TYPE t [USING ...]`, `SET/DROP DEFAULT`, `SET/DROP NOT NULL`, `ADD/DROP CONSTRAINT`.
 - **Existing data must already obey any new rule**, or the `ALTER` is refused. Clean, then constrain.
 - In PostgreSQL, structure changes **can be rolled back**. Wrap every risky `ALTER` in `BEGIN ... COMMIT / ROLLBACK`.
 - `DELETE` removes rows, `TRUNCATE` empties, `DROP COLUMN` removes a column, `DROP TABLE` removes everything.
 - Save every `ALTER` in a file. That's a migration.
-

Next: try the [exercises](#), then move on to [16 Relationships and Foreign Keys](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. Run `playground/ch06/seed.sql` first so `products` is in its 6-row starting state.
 - Save each answer in `playground/ch15/ex1.sql`, `ex2.sql`, and so on. **Do them in order**: each exercise changes the table the next one uses.
 - Wrap every change in `BEGIN; ... COMMIT;`, with a `\d products` or `SELECT` in between to check.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): A category for every product

1. Add a `category text` column to `products`.
2. Fill it in with `UPDATE s: Notebook, Pen, Sticky notes and Stapler are 'Stationery'; Desk lamp is 'Furniture'; Backpack is 'Bags'`. Try to do it in three statements, not six.
3. Now make the column required (`NOT NULL`).
4. Prove it worked: try to insert a product with no category.

Expected `\d products` afterwards: `category | text | | not null |`.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Better names, better types

1. Rename `on_sale` to `is_on_sale`, to match the `is_` style of boolean columns in this course.
2. Rename `added_on` to `added_at` and change its type to `timestampz`. Select the column afterwards: what time of day did the old dates become?
3. Write a comment listing which of your earlier exercise files (chapters 07 to 10) would now be broken by these renames.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Rules the data must already follow

1. Add a constraint that `price` must be `>= 0`. This should succeed. Why?
2. Try to add a constraint that `stock` must be `> 0`. Read the error. Which row is the problem?
3. Decide: is the right fix to change the *data* or to change the *rule*? Do it, and add the constraint successfully.
4. Add a constraint named `name_not_blank` so `name` can't be an empty string. Test it.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Rewiring with the lights on

1. Widen `price` from `numeric(10, 2)` to `numeric(12, 2)`. Confirm the values are unchanged.
2. Inside a transaction that you will **roll back**, change `price` to `integer`. PostgreSQL will tell you it needs a `USING`. Give it one that rounds. Select the prices, and write a comment about which information would be lost forever if you committed. Then roll back.
3. Still thinking: `stock` is an `integer`. Someone suggests making it `numeric(10, 2)` "in case we sell half a notebook". Would the `ALTER` succeed? Would it be a good idea? Two sentences.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Your first migration, and its undo

Write **two** files.

`playground/ch15/001_add_sku.sql` should, inside a single transaction:

1. Add a `sku text` column.

2. Fill it with a unique code per product, like `STA-001` , `STA-002` , `FUR-001` , `BAG-001` .
3. Make it `NOT NULL` and `UNIQUE` .
4. Drop the `is_on_sale` column (the shop has decided sales will be handled differently).
5. `COMMIT` .

`playground/ch15/001_add_sku_down.sql` should reverse every step, so running it after the first file leaves the table as it was before (an `is_on_sale` column back, with `false` everywhere, and no `sku`).

Run up, check `\d products` , run down, check again, run up again. Both should work every time in that order.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

16 Relationships and Foreign Keys

What is it?

A **relationship** links a row in one table to a row in another: this order belongs to that customer.

A **foreign key** is how you build the link. It's a column in one table that holds the primary key of a row in another table, plus a rule that says the link must always point at a real row.

This is the "relational" in relational database. It's the most important chapter in Level 2.

Why does it matter?

Think back to chapter 01's spreadsheet problem number 4: the same fact repeated. Here's an `orders` table built the obvious way:

id	customer_name	customer_email	total
1	Asha Rai	asha@example.com	1200.00
2	Asha Rai	asha@exmaple.com	350.50
3	Bikram Shrestha	bikram@example.com	89.99

Asha's email is wrong on order 2. When she changes her email, someone has to update every order. "How many customers do we have?" is unanswerable, because Asha might be one person or two. The customer isn't a *thing* here, just some text copied onto each order.

The fix is to make customers a thing: a `customers` table, where Asha exists **once**, with an `id`. Then each order stores only her `id`. Her email lives in one place. Fix it there, and every order sees the fix.

That `customer_id` column on orders is the foreign key. And the rule that comes with it stops the next problem: an order whose `customer_id` is `99`, when there's no customer 99. Without the rule, that order points at nobody. It's called an **orphan**, and foreign keys make orphans impossible.

Real-world example

Your phone's call history doesn't store your friend's name and number in each entry. It stores **which contact** the call was with. Rename the contact, and every past call shows the new name. Delete the contact, and the phone has to decide: keep the calls as "unknown number", or remove them too?

Phone	Database
Contacts	The <code>customers</code> table (the "one" side)
Call history	The <code>orders</code> table (the "many" side)
Each call remembers which contact	<code>orders.customer_id</code>
You can't log a call to a contact that doesn't exist	The foreign key rule
"Delete this contact and their calls?"	<code>ON DELETE</code> behaviour

How it works

Work in `practice`. We'll build customers and orders properly.

The parent table first

```
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS customers;

CREATE TABLE customers (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text NOT NULL,
  email text NOT NULL UNIQUE
);
```

Nothing new here. `customers` is the **parent**: the table being pointed at. (You'll also hear "the one side".)

The child table, with the foreign key

```
CREATE TABLE orders (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  customer_id integer NOT NULL REFERENCES customers (id),
  ordered_at timestampz NOT NULL DEFAULT now(),
  total numeric(10, 2) NOT NULL CHECK (total >= 0)
);
```

The new part:

```
customer_id integer NOT NULL REFERENCES customers (id)
```

Read it as: "customer_id holds whole numbers, must always be filled in, and every value must be the id of a real row in customers."

- REFERENCES customers (id) creates the foreign key. orders is the **child** (the "many side").
- The type must match the parent's primary key: integer here, because customers.id is integer.
- NOT NULL because every order must have a customer. Leave it off when the link is optional (a task that may or may not belong to a project).

This is a **one-to-many** relationship: one customer, many orders. Each order has exactly one customer.

Add some data

```
INSERT INTO customers (name, email) VALUES
('Asha Rai', 'asha@example.com'),
('Bikram Shrestha', 'bikram@example.com'),
('Chandra Gurung', 'chandra@example.com');

INSERT INTO orders (customer_id, total) VALUES
(1, 1200.00),
(1, 350.50),
(2, 89.99);
```

Asha (id 1) has two orders, Bikram (id 2) has one, Chandra (id 3) has none. Her email is stored exactly once.

The rule in action: no orphans

```
INSERT INTO orders (customer_id, total) VALUES (99, 10.00);
```

```
ERROR: insert or update on table "orders" violates foreign key constraint
"orders_customer_id_fkey"
DETAIL: Key (customer_id)=(99) is not present in table "customers".
```

There's no customer 99, so there can be no order for customer 99. Refused. The constraint got an automatic name: orders_customer_id_fkey.

The rule works in the other direction too:

```
DELETE FROM customers WHERE id = 1;
```

```
ERROR: update or delete on table "customers" violates foreign key constraint
"orders_customer_id_fkey" on table "orders"
DETAIL: Key (id)=(1) is still referenced from table "orders".
```

Asha has orders. Deleting her would orphan them. Refused. **A foreign key protects both ends of the link.**

Deciding what happens on delete

Refusing is the default, and often right. But sometimes you want something else. You choose with `ON DELETE` :

Option	When the parent row is deleted...	Good for
<code>ON DELETE RESTRICT</code> (or <code>NO ACTION</code> , the default)	Refuse the delete.	Orders, payments, anything that's a record you must keep
<code>ON DELETE CASCADE</code>	Delete the child rows too.	Comments on a post, items in a cart, anything meaningless without its parent
<code>ON DELETE SET NULL</code>	Keep the children, clear the link to NULL. The column must be nullable.	A task whose project was deleted, a post whose author left

For example, comments that should vanish with their post:

```
post_id integer NOT NULL REFERENCES posts (id) ON DELETE CASCADE
```

A task that can outlive its project:

```
project_id integer REFERENCES projects (id) ON DELETE SET NULL
```

Pick deliberately, every time. `CASCADE` on the wrong table quietly deletes things you wanted. For `orders`, the default refusal is correct: a business never wants sales records to disappear. If a customer leaves, you mark them inactive instead ([chapter 19](#) does exactly this).

There's also `ON UPDATE CASCADE`, for when a parent's primary key changes. With surrogate ids that never change, you'll rarely need it. It matters with natural keys, like the country codes in chapter 14's exercises.

Seeing the links: `\d`

```
\d orders
```

At the bottom:

Foreign-key constraints:

```
"orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(id)
```

And the parent knows about it too:

```
\d customers
```

Referenced by:

```
TABLE "orders" CONSTRAINT "orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(id)
```

Order matters

Because children depend on parents:

- **Create** parents before children. `CREATE TABLE orders` fails with `relation "customers" does not exist` if `customers` isn't there yet.
- **Insert** parents before children. You can't order for customer 3 before customer 3 exists.
- **Delete** children before parents, unless `ON DELETE CASCADE` does it for you.
- **Drop** children before parents. That's why the top of this chapter dropped `orders` first. Or use `DROP TABLE customers CASCADE`, which drops the foreign key constraints on dependent tables for you (but not the tables themselves).

Adding a foreign key to an existing table

```
ALTER TABLE orders
  ADD CONSTRAINT orders_customer_id_fkey
  FOREIGN KEY (customer_id) REFERENCES customers (id);
```

Every existing `customer_id` must already match a real customer, or the `ALTER` is refused. Same principle as chapter 15: clean the data, then add the rule.

Naming convention

The foreign key column is the parent table's name, made singular, plus `_id`: `customer_id`, `project_id`, `post_id`. Follow this and anyone reading your tables knows instantly which table each column points at.

One-to-one

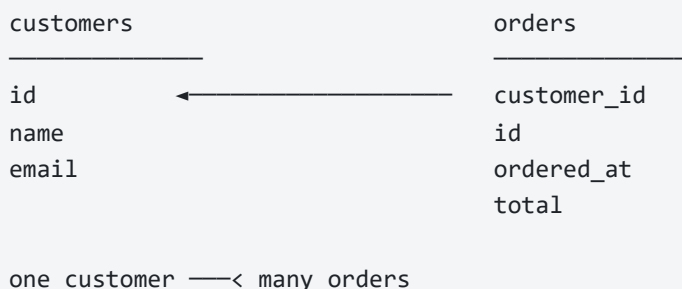
Sometimes each parent has at most **one** child: a customer has one profile with their address and preferences. That's a foreign key with `UNIQUE` on it:

```
CREATE TABLE customer_profiles (  
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  customer_id integer NOT NULL UNIQUE REFERENCES customers (id) ON DELETE CASCADE,  
  address text,  
  newsletter boolean NOT NULL DEFAULT false  
);
```

`UNIQUE` means no customer can appear twice, so it's one profile each. People use one-to-one to split rarely used or sensitive columns away from the main table. You won't need it often.

Drawing it

A picture helps when there are more than two tables:



The arrow goes from the foreign key to the primary key it references. The `—<` with a fork on the "many" end is the standard way to draw one-to-many; you'll see it in every database diagram.

A sneak peek: seeing both tables together

You've got the customer's name in one table and their orders in another. How do you see them side by side? With a `JOIN`:

```
SELECT orders.id, customers.name, orders.total  
FROM orders  
JOIN customers ON customers.id = orders.customer_id;
```

id	name	total
1	Asha Rai	1200.00
2	Asha Rai	350.50
3	Bikram Shrestha	89.99

(3 rows)

`JOIN customers ON customers.id = orders.customer_id` says "for each order, find the customer whose `id` matches the order's `customer_id`, and line them up." Chandra has no orders, so Chandra doesn't appear.

This is a preview. [Chapter 22](#) explains joins properly, and [chapter 23](#) shows how to include Chandra. For now, it's enough to know the link you built can be followed.

Common mistakes

1. Copying the parent's data into the child

`customer_name` in `orders`. That's the spreadsheet all over again. Store the `customer_id`, and look up the name when you need it.

2. An id column with no REFERENCES

`customer_id integer` on its own is just a number. Nothing stops `99`. The word `REFERENCES` is what makes it a relationship.

3. Type mismatch

```
customer_id text REFERENCES customers (id)
```

```
ERROR: foreign key constraint "orders_customer_id_fkey" cannot be implemented
DETAIL: Key columns "customer_id" and "id" are of incompatible types: text and integer.
```

Match the parent's type exactly.

4. The foreign key on the wrong side

Putting `order_id` in `customers`. A customer has *many* orders. Which one would it hold? The foreign key always goes on the **many** side.

5. CASCADE by habit

Every `ON DELETE CASCADE` is a decision that deleting one row should silently delete others. Right for comments. Catastrophic for orders. Choose each time.

6. Creating or dropping in the wrong order

relation "customers" does not exist when creating; cannot drop table customers because other objects depend on it when dropping. Parents first to create, children first to drop.

Quick recap

- A **foreign key** is a column that holds another table's primary key, plus the rule that it must point at a real row. Written `column type REFERENCES parent (id)`.
 - It prevents **orphans** in both directions: no child without a parent, no deleting a parent that still has children.
 - **One-to-many**: the foreign key goes on the **many** side. Name it `parent_id`.
 - `ON DELETE RESTRICT` (default) refuses, `CASCADE` deletes children too, `SET NULL` unlinks them. Choose on purpose.
 - Parents before children when creating and inserting; children before parents when deleting and dropping.
 - `JOIN ... ON` follows the link to show both tables together. Chapter 22 covers it fully.
-

Next: try the [exercises](#), then move on to [17 Many-to-Many Relationships](#).

Exercises

How to do these:

- Work in psql, connected to `practice`. Save each answer in `playground/ch16/ex1.sql`, `ex2.sql`, and so on.
 - Start each file by dropping its tables **children first**, with `IF EXISTS`.
 - When something should fail, run it, and paste the error's first line as a comment.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Authors get their own table

Since chapter 06, `books.author` has been text, and Roald Dahl is typed out twice. Fix it properly.

1. Create an `authors` table (`id`, `name NOT NULL`, `country`).
2. Create a **new** `library_books` table with `title`, `published_year`, and an `author_id` that references `authors` and can't be NULL.

3. Insert three authors and five books, with at least two books by the same author.
4. Try to insert a book with `author_id = 42`. Try to delete an author who has books. Both must fail.
5. Use the sneak-peek `JOIN` from the notes to list each book's title next to its author's name.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Optional links

Back in chapter 02, tasks could belong to a project, but didn't have to.

1. Create `projects` (`id`, `name` NOT NULL UNIQUE) and `project_tasks` (`id`, `title` NOT NULL, `is_done` NOT NULL DEFAULT false, `project_id` referencing `projects`, **nullable**, with ON DELETE SET NULL).
 2. Insert two projects, two tasks in each, and one task with no project.
 3. Delete one project. Select all tasks. What happened to the two tasks that were in it?
 4. In a comment: why would `ON DELETE CASCADE` have been a worse choice here? And why would `NOT NULL` on `project_id` have made step 3 fail?
-

Exercise 3 (Medium): Three ways to say goodbye

Create a parent table `teams` and **three** child tables, each referencing `teams (id)` with a different `ON DELETE`:

- `team_messages` with `CASCADE`,
- `team_invoices` with `RESTRICT`,
- `team_members` with `SET NULL` (so `team_id` must be nullable).

Insert one team, one row in each child table pointing at it. **Before** running anything else, write a comment predicting what `DELETE FROM teams WHERE id = 1;` will do.

Then run it. Were you right? Fix whatever stopped it (in the way a real business would), and run the delete again. Finally, select from all three child tables and describe what you see.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Fix the backwards design

A colleague designed this:


```
CREATE TABLE customers (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text NOT NULL,
  order_id integer
);

CREATE TABLE orders (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  total numeric(10, 2) NOT NULL
);
```

1. In a comment, explain the two problems: what happens when Asha places her second order, and what stops `order_id` being 999?
2. Rewrite both tables correctly, with the foreign key on the right side, the right `NOT NULL`, and a sensible `ON DELETE`.
3. Insert one customer with three orders to prove the new design handles it.

Exercise 5 (Challenge): A small blog

Design and build:

- `users` (`id`, `username` NOT NULL UNIQUE).
- `posts` (`id`, `author_id` → `users`, required, `title`, `body`, `published_at` DEFAULT `now()`).
- `comments` (`id`, `post_id` → `posts`, required, `author_id` → `users`, **optional**, `body`, `written_at` DEFAULT `now()`).

Decide the `ON DELETE` for each of the three foreign keys, and justify each in a comment. Think about: what should happen to a user's posts and comments when they delete their account? What should happen to a post's comments when the post is deleted?

Then:

1. Insert two users, two posts (one each), and three comments (one of them by the post's own author, one by the other user, one on the other post).
2. Delete one user. Select from `posts` and `comments`. Does the result match your justification?
3. Use the sneak-peek `JOIN` to list each comment's body next to the title of the post it's on.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

17 Many-to-Many Relationships

What is it?

A **many-to-many** relationship is when each row on *both* sides can be linked to many rows on the other. A student takes many courses, and a course has many students. A book has many genres, and a genre has many books.

You can't build this with one foreign key. You build it with a **third table** that sits in the middle and holds the pairs. People call it a **junction table**, a **join table**, or a **linking table**. Same thing.

Why does it matter?

Chapter 02 gave you a rule: one value per cell, no `'Fantasy, Adventure'`. Chapter 16 gave you one-to-many: put a foreign key on the "many" side. But a book with two genres has no "many side". Both sides are many. Where does the foreign key go?

Nowhere. If you put `genre_id` on `books`, a book gets one genre. If you put `book_id` on `genres`, a genre gets one book. Either way, something is lost.

This comes up constantly: products in orders, tags on posts, actors in films, users with roles, students in courses, friends on a social network. Once you can see the pattern and build the middle table, a whole class of design problems becomes routine.

Real-world example

A school needs to record who is in which class.

- One page per class, listing its students? Then "which classes is Asha in?" means flipping through every page.
- One page per student, listing their classes? Then "who's in SQL Basics?" means flipping through every student.

Schools solved this long ago with **enrollment slips**: one slip per (student, class) pair. "Asha, SQL Basics." "Asha, Web Design." "Bikram, SQL Basics." The stack of slips answers both questions, and each slip can carry extra facts, like the date enrolled and the grade.

The stack of slips is the junction table.

How it works

Work in `practice`.

The two sides

```
DROP TABLE IF EXISTS enrollments;
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;

CREATE TABLE students (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text NOT NULL
);

CREATE TABLE courses (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  title text NOT NULL UNIQUE
);
```

Nothing links them yet. Neither table mentions the other. That's correct.

The table in the middle

```
CREATE TABLE enrollments (
  student_id integer NOT NULL REFERENCES students (id) ON DELETE CASCADE,
  course_id integer NOT NULL REFERENCES courses (id) ON DELETE CASCADE,
  enrolled_on date NOT NULL DEFAULT current_date,
  grade text,
  PRIMARY KEY (student_id, course_id)
);
```

Take it apart:

- **Two foreign keys**, one to each side. Each row is one "slip": this student, this course.
- **PRIMARY KEY (student_id, course_id)**: the pair identifies the row. This is the composite key from [chapter 14](#). It means the same student can't be enrolled in the same course twice, while still allowing many courses per student and many students per course.
- **ON DELETE CASCADE on both**. If a student leaves, their enrollment slips should go too. A slip without a student is meaningless. This is the one place `CASCADE` is almost always right.
- **enrolled_on and grade** are facts about the *pair*. Asha's grade in SQL Basics isn't a fact about Asha alone or SQL Basics alone. The junction table is exactly where such facts belong.

Add data

```
INSERT INTO students (name) VALUES ('Asha Rai'), ('Bikram Shrestha'), ('Chandra Gurung');
INSERT INTO courses (title) VALUES ('SQL Basics'), ('Web Design'), ('Maths');

INSERT INTO enrollments (student_id, course_id, grade) VALUES
  (1, 1, 'A'),      -- Asha, SQL Basics
  (1, 2, NULL),     -- Asha, Web Design (no grade yet)
  (2, 1, 'B'),      -- Bikram, SQL Basics
  (3, 1, NULL),     -- Chandra, SQL Basics
  (3, 3, 'A');      -- Chandra, Maths
```

Asha is in two courses. SQL Basics has three students. Both directions, no lists in cells.

The composite key at work

```
INSERT INTO enrollments (student_id, course_id) VALUES (1, 1);
```

```
ERROR: duplicate key value violates unique constraint "enrollments_pkey"
DETAIL: Key (student_id, course_id)=(1, 1) already exists.
```

Asha is already in SQL Basics. Refused. And the foreign keys still do their job: (1, 99) fails because there's no course 99.

Following the links

To answer "which courses is Asha in?", you walk from `students` through `enrollments` to `courses`. That's two joins (sneak peek again, full story in [chapter 22](#)):

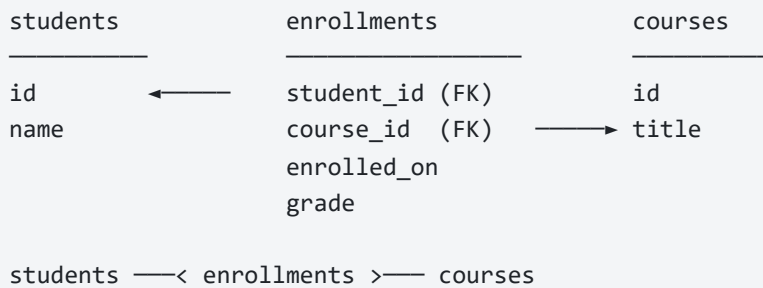
```
SELECT students.name, courses.title, enrollments.grade
FROM enrollments
JOIN students ON students.id = enrollments.student_id
JOIN courses  ON courses.id  = enrollments.course_id
ORDER BY students.name, courses.title;
```

name	title	grade
Asha Rai	SQL Basics	A
Asha Rai	Web Design	[NULL]
Bikram Shrestha	SQL Basics	B
Chandra Gurung	Maths	A
Chandra Gurung	SQL Basics	[NULL]

(5 rows)

Read it as: start from the slips, look up each slip's student, look up each slip's course. Add `WHERE students.name = 'Asha Rai'` for just her, or `WHERE courses.title = 'SQL Basics'` for the class list. Same query, two questions.

Drawing it



The middle table has a fork on **both** ends. When you see that shape, you're looking at a many-to-many.

An alternative shape: `id` plus `UNIQUE`

Some teams give every table an `id`, even junction tables, and protect the pair with `UNIQUE` instead:

```

CREATE TABLE enrollments (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  student_id integer NOT NULL REFERENCES students (id) ON DELETE CASCADE,
  course_id integer NOT NULL REFERENCES courses (id) ON DELETE CASCADE,
  enrolled_on date NOT NULL DEFAULT current_date,
  grade text,
  UNIQUE (student_id, course_id)
);
  
```

Same protection, and now each enrollment has its own `id`, which is handy if other tables need to point at an enrollment (an attendance record, say), or if your tools expect every table to have an `id` column ([chapter 42](#) will). Both designs are correct. Pick one style for a project and stick to it.

Second example: products in orders

This one you'll build in the [Level 2 project](#). An order has many products; a product appears in many orders. The slip is an **order item**:

```
CREATE TABLE order_items (  
  order_id integer NOT NULL REFERENCES orders (id) ON DELETE CASCADE,  
  product_id integer NOT NULL REFERENCES products (id),  
  quantity integer NOT NULL CHECK (quantity > 0),  
  unit_price numeric(10, 2) NOT NULL,  
  PRIMARY KEY (order_id, product_id)  
);
```

Two things to notice:

- **quantity** is a fact about the pair: how many of *this product* in *this order*. Right where it belongs. The composite key means the same product can't appear twice in one order; you raise the quantity instead.
- **unit_price** looks like a copy of `products.price`. It isn't. It's the price the customer *paid*, at the moment of ordering. Prices change; receipts mustn't. This is a **snapshot**, and it's deliberate. [Chapter 18](#) talks about when copying a value is a mistake and when it's exactly right.

Also notice the `ON DELETE` choices: cascading from `orders` (an item without its order is meaningless), but **not** from `products`. You can't delete a product that's been sold; you mark it inactive.

Naming the middle table

Two conventions:

- Name it for the real-world thing it records when one exists: `enrollments`, `order_items`, `memberships`, `bookings`.
- Otherwise, join the two table names: `book_genres`, `post_tags`, `user_roles`.

Prefer the first when you can. "An enrollment" is easier to talk about than "a student_course".

Fixing the book genres, at last

The problem from chapter 02 and the project in chapter 11:

```
CREATE TABLE genres (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  name text NOT NULL UNIQUE
);

CREATE TABLE book_genres (
  book_id integer NOT NULL REFERENCES books (id) ON DELETE CASCADE,
  genre_id integer NOT NULL REFERENCES genres (id) ON DELETE CASCADE,
  PRIMARY KEY (book_id, genre_id)
);
```

The Hobbit gets two rows in `book_genres` : one for Fantasy, one for Adventure. One value per cell, every genre spelled exactly once in `genres` , and "which books are Adventure?" is a simple join. You'll build this in the exercises.

Common mistakes

1. Comma-separated lists

`genres = 'Fantasy, Adventure'` . This whole chapter exists to replace that.

2. Numbered columns

`genre_1` , `genre_2` , `genre_3` . The same problem sideways. What about the fourth genre? How do you search all three at once?

3. The foreign key on one side only

`course_id` on `students` . Now Asha can take exactly one course. If you catch yourself writing a foreign key and thinking "but they could have several...", you need a junction table.

4. No composite key or `UNIQUE` on the pair

Without it, Asha can be enrolled in SQL Basics twice. Every junction table needs `PRIMARY KEY (a, b)` or `UNIQUE (a, b)` .

5. Pair facts stored on one side

`grade` on `students` . Which course is that the grade for? Facts about the pair live on the junction table.

6. Cascading from the wrong side

`order_items` cascading from `products` means deleting a product silently rewrites every past order. Cascade from the side that *owns* the pair (orders), restrict the other.

Quick recap

- When both sides can have many, you need a **junction table** holding one row per pair.
 - It has **two foreign keys**, one to each side, and a **composite primary key** (or `UNIQUE`) on the pair so no pair repeats.
 - Facts about the pair (`grade` , `quantity` , `enrolled_on`) belong on the junction table.
 - `ON DELETE CASCADE` is usually right on a junction table, but think about each side separately.
 - Follow the links with two joins: from the middle table out to each side.
 - `unit_price` on an order item is a deliberate **snapshot**, not a copying mistake.
-

Next: try the [exercises](#), then move on to [18 Normalization](#).

Exercises

How to do these:

- Work in psql, connected to `practice` . Save each answer in `playground/ch17/ex1.sql` , `ex2.sql` , and so on.
 - Start each file by dropping its tables, junction table first, with `IF EXISTS` .
 - Use the two-join query from the notes whenever you need to see names instead of ids.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Books and genres, properly

You need the 9-row `books` table from chapter 06 (or any `books` table with an `id`).

1. Create `genres` and `book_genres` as shown at the end of the notes.
2. Insert genres: Fantasy, Adventure, Children, Science Fiction, Classic, Poetry.
3. Link *The Hobbit* to Fantasy **and** Adventure. Link *Matilda* and *The BFG* to Children. Link *Dune* and *The Martian* to Science Fiction. Link *Beowulf* to Poetry.
4. Try to link *The Hobbit* to Fantasy again. It must fail.
5. Write the two-join query that lists every book title next to each of its genres, sorted by title. *The Hobbit* should appear twice.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Users and roles

An app has users, and each user can have several roles (`admin` , `editor` , `viewer`). A role is held by many users.

1. Create `app_users` , `roles` , and a junction table. The junction table should record **when** each role was granted.
 2. Give one user all three roles, another user just `viewer` , and leave a third user with no roles.
 3. In a comment: `granted_on` is a fact about what? Why can't it live on `app_users` or on `roles` ?
 4. Remove `editor` from the first user by deleting one junction row.
-

Exercise 3 (Medium): Playlists

A music app has playlists and songs. The same song can be in many playlists. Within a playlist, songs have a **position** (1st, 2nd, 3rd), and the **same song can appear twice** in one playlist at different positions.

1. Design the junction table. Think hard about the primary key: `(playlist_id, song_id)` is **wrong** here. Why? What uniqueness rule is right?
2. Build all three tables, insert one playlist with four entries (one song twice), and prove that two songs can't share a position in the same playlist.
3. Write the query that shows one playlist in order: position, song title.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Actors and films

An actor appears in many films; a film has many actors. Each appearance has a **role name**, and an actor can play **two roles in the same film** (it happens).

1. Design the junction table so that the same (actor, film, role) can't be recorded twice, but (actor, film) with a different role can.
2. Build it, and insert one actor playing two roles in one film, plus two more actors in that film.
3. Write a query listing the film's cast: actor name and role, alphabetical by actor.
4. In a comment: compare your primary key here with the one in Exercise 3. Both are "more than two columns" or "not the obvious pair". What's the general lesson about choosing a junction table's key?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Order items with a price snapshot

Build the `orders` / `products` / `order_items` design from the notes. You can reuse `customers` and `orders` from chapter 16 and `products` from chapter 13, or create fresh ones.

1. Insert three products with prices, one customer, and one order for that customer.
2. Add three items to the order, copying each product's **current** price into `unit_price`. Try to add the same product to the same order twice. It must fail.
3. Write a query showing, for that order: product name, quantity, unit price, and a `line_total` column (quantity × unit price).
4. Now **raise the price** of one of those products in `products`. Run the query from step 3 again. Did the order's line total change? Write a comment explaining why that's correct, and what would have gone wrong if `order_items` had no `unit_price` column.
5. Try to delete a product that's in the order. What happens, and why is that the right behaviour?

Stuck? The hint is in the [Hints section](#) at the end.

18 Normalization

What is it?

Normalization is organizing your tables so that each fact is stored **exactly once**, in the one place it belongs.

You've been doing it informally since chapter 02: one kind of thing per table, one value per cell, link instead of copy. This chapter gives the ideas their names, shows a step-by-step method, and explains when it's right to break the rules on purpose.

Why does it matter?

Here's a single-table design for an online shop, the kind that starts life as a spreadsheet:

order_id	order_date	customer_name	customer_email	customer_city	products	quantity
1	2026-09-05	Asha Rai	asha@example.com	Kathmandu	Notebook, Pen	2, 5
2	2026-09-12	Bikram Shrestha	bikram@example.com	Pokhara	Desk lamp	1
3	2026-09-20	Asha Rai	asha@exmaple.com	Kathmandu	Backpack	1
4	2026-09-28	Chandra Gurung	chandra@example.com	lalitpur	Sticky notes	3

It *works*, in the sense that the data is in there. But look at what goes wrong. Each of these has a name:

- **Update anomaly.** Asha changes her email. It's on orders 1 and 3 (and order 3 already has a typo). You must find and fix every copy, and you'll miss one.
- **Insert anomaly.** A new customer signs up but hasn't ordered yet. Where do they go? There's no row for a customer without an order.
- **Delete anomaly.** Order 2 is cancelled and deleted. Bikram's name, email, and city vanish with it. You just lost a customer by deleting an order.
- **Inconsistency.** Kathmandu and lalitpur: free text, typed by hand, every time. "How many customers in Lalitpur?" now depends on spelling.
- **Lists in cells.** Notebook, Pen and 2, 5. "How many Pens have we sold?" needs you to pull text apart and line two lists up by position.

Every one of these is the same disease: **a fact stored in more than one place, or facts about different things crammed into one row**. Normalization is the cure.

Real-world example

Two ways to keep a birthday card list:

- **The messy way:** a notebook where each page is a card you sent, and you write the friend's full address on every page. Your friend moves house: you have twelve pages to correct, and you'll miss some.
- **The normalized way:** an address book with one entry per friend, and a card list that just says "Sent to Priya, 2026". Priya moves: one line in the address book changes, and every card record is automatically right.

Normalization is the address book. One fact, one place, and everything else points at it.

How it works

Rule 1: one value per cell

Also known as **first normal form** (1NF). You know this one. No lists, no numbered column groups (`product_1` , `product_2`).

Applied to the shop table, each product in an order becomes its own row:

order_id	order_date	customer_name	customer_email	...	product	quantity
1	2026-09-05	Asha Rai	asha@example.com	...	Notebook	2
1	2026-09-05	Asha Rai	asha@example.com	...	Pen	5
2	2026-09-12	Bikram Shrestha	bikram@example.com	...	Desk lamp	1

Better: "how many Pens sold?" is now easy. Worse: Asha's email is now repeated *within* one order. Rule 1 alone isn't enough. That's expected; it's the first step, not the last.

Rule 2: every column must be about the whole key

This is **second normal form** (2NF), and it only matters when a table's key is a combination of columns.

The rows above are identified by (`order_id` , `product`) . Now ask, for each column: *is this a fact about the whole pair, or about just part of it?*

- `quantity` : about the pair. 2 Notebooks *in order 1*. Stays.
- `order_date` : about `order_id` only. It's the same for every product in the order.
- `customer_name` , `customer_email` : about `order_id` only (really about the customer, but we'll get there).

Columns that depend on only *part* of the key get moved out to a table keyed by that part. `order_date` and the customer columns move to an `orders` table keyed by `order_id`. What's left, `(order_id, product, quantity)`, is `order_items`.

Rule 3: every column must be about the key, and nothing else

This is **third normal form** (3NF). Now look at the `orders` table:

<code>order_id</code>	<code>order_date</code>	<code>customer_name</code>	<code>customer_email</code>	<code>customer_city</code>
-----------------------	-------------------------	----------------------------	-----------------------------	----------------------------

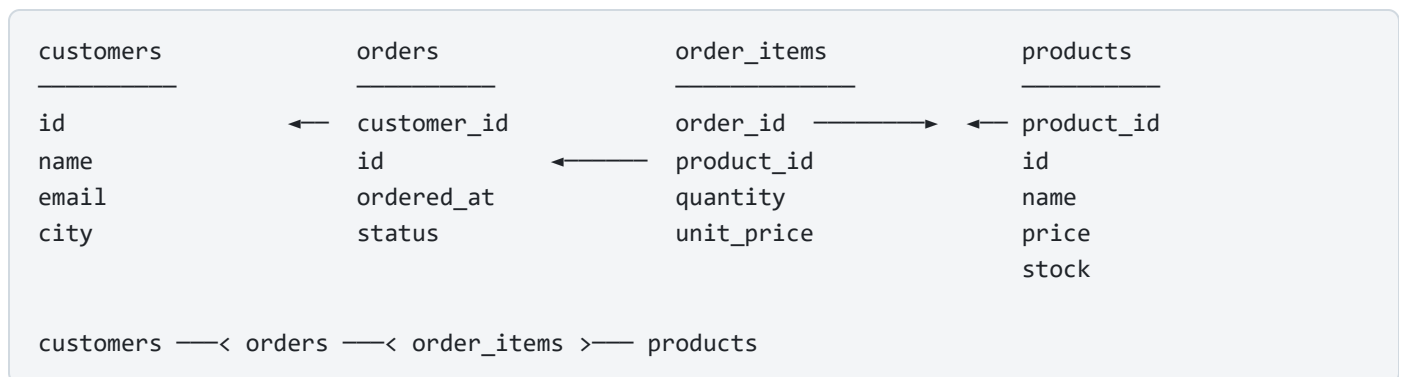
`order_date` is a fact about the order. But `customer_email`? That's a fact about the *customer*, who happens to be on this order. It depends on `customer_name`, not on `order_id`. Columns that describe some *other* thing get moved to that thing's table: `customers`, with its own `id`, and `orders` keeps just a `customer_id`.

The classic phrase for rules 2 and 3 together: **every column must depend on the key, the whole key, and nothing but the key.**

- *the key*: each row has a primary key, and columns describe that row.
- *the whole key*: not just part of a composite key (rule 2).
- *nothing but the key*: not some other column that's really a different thing (rule 3).

The result

Four tables, each about exactly one thing:



Check against the anomalies:

- Asha's email changes: one `UPDATE` on one row of `customers`.
- A new customer with no orders: a row in `customers`. Fine.

- Order 2 deleted: Bikram stays in `customers`.
- Lalitpur spelled wrong: still possible in `city`, but now it's one cell per customer rather than one per order. (For a fixed list of cities, a `cities` table with a foreign key fixes it completely. Whether that's worth it is a judgment call. See "when to stop" below.)
- Lists in cells: gone. One `order_items` row per product per order.

This is exactly the design you'll build in the [Level 2 project](#).

A method you can follow

1. **Write it messy first.** Imagine the data as one wide spreadsheet. It's much easier to fix a bad table than to design a perfect one from nothing.
2. **Split lists into rows** (rule 1).
3. **For each column, ask "what is this a fact about?"** Write the answer next to it: "the order", "the customer", "the product", "this product in this order". Each distinct answer is a table.
4. **Give each table an `id`**, and replace copied facts with a foreign key to where the fact now lives.
5. **Check each table with one sentence:** "each row is one ____." If you need "and", split again.
6. **Look at anything that still appears to be duplicated**, and decide: mistake, or snapshot? (Next section.)

Duplication versus snapshots

Look at `order_items.unit_price` next to `products.price`. Isn't that a fact stored twice?

No. They're **two different facts**: what the product costs *now*, and what the customer *paid then*. If the shop raises the Notebook price next month, the current price changes and the old receipt doesn't. Store both, because both are true.

A **snapshot** is a value copied on purpose because it records history. Shipping addresses on orders are another one: the customer may move, but the parcel went to the old address. Snapshots are not a normalization failure.

The test: *if the source changes, should this copy change too?* If yes, it's duplication: remove it and use a foreign key. If no, it's a snapshot: keep it, and name it clearly (`unit_price` , not `price`).

Things you can work out

`total` on the original spreadsheet is the sum of quantity × unit price over the order's items. If you store it, it can disagree with the items. If you don't, you calculate it every time (easy, with [chapter 20](#)).

The default is **don't store what you can calculate**. The exception is when calculating is slow or the value must be frozen (a tax total on an issued invoice). If you do store a calculated value, it's your job to keep it right, which is what [triggers](#) are for.

When to stop

Taken to extremes, normalization gives you a `cities` table, a `first_names` table, and a `postcodes` table that determines cities. Each is technically "more normal". Most are not worth it.

Aim for the three rules above (3NF). Then ask of each further split: *does this fact actually change, or get spelled wrong, in a way that would hurt?* A list of 50 cities you filter and report by: yes, make a table. A free-text "notes" column: leave it alone.

Deliberately storing data in a less normalized shape, for speed or convenience, is called **denormalization**. Real systems do it, especially for reporting. The rule is: normalize first, denormalize only for a measured reason, and know how you'll keep the copies in sync.

The names, honestly

You'll hear "first, second, third normal form", and also BCNF, 4NF, 5NF. Interviewers like asking about them. In practice:

- 1NF, 2NF, 3NF are the three rules in this chapter. Aim for them.
- The higher ones handle rare, specific situations. You can look them up if a design ever feels wrong in a way the three rules don't explain.

You don't need to recite definitions. You need the method and the habit of asking "what is this a fact about?"

Common mistakes

1. Under-normalizing "for convenience"

`customer_name` on `orders` "so we don't have to join". Three months later, two spellings of every name. Join. It's one line.

2. Over-normalizing

A table for every noun in sight. A `genders` table with two rows. A `first_names` table. If a split doesn't prevent a real anomaly, it's just more joins.

3. "Fixing" snapshots

Someone sees `unit_price` and `price`, decides it's duplication, and deletes `unit_price`. Every historical receipt now shows today's prices. Ask the "if the source changes, should this change?" question first.

4. Storing calculated values without a plan

A `total` on `orders` that nobody updates when an item is removed. Either don't store it, or make its maintenance automatic.

5. Normalizing a table with no primary key

The rules are all about "the key". No key, no rules. Give every table an `id` first.

Quick recap

- Normalization: each fact **once**, in the table that's about that thing. It cures update, insert, and delete anomalies.
- **Rule 1:** one value per cell. **Rule 2:** every column depends on the whole key. **Rule 3:** every column depends on the key and nothing else. "The key, the whole key, and nothing but the key."
- Method: write it messy, split lists, ask "what is this a fact about?" for each column, make a table per answer, link with foreign keys.
- A **snapshot** (`unit_price`) is a deliberate copy that records history. Not a mistake. Test: should it change when the source changes?
- Don't store what you can calculate, unless you have a reason and a plan to keep it right.
- Aim for the three rules; split further only when it prevents a real problem.

Next: try the [exercises](#), then move on to the Level 2 project: [19 Project: Online Shop Database](#).

Exercises

How to do these:

- Exercises 1, 2 and 4 are on paper (or in `playground/ch18/answers.md`). Exercises 3 and 5 end in real `CREATE TABLE` statements in `.sql` files.
- Draw tables as Markdown tables or ASCII, whichever is faster.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your answers.

Exercise 1 (Easy): Name the anomaly

A company keeps this `employees` table:

id	name	department	department_phone	manager_name
1	Asha	Engineering	01-5551000	Dev Shrestha
2	Bikram	Engineering	01-5551000	Dev Shrestha

id	name	department	department_phone	manager_name
3	Chandra	Sales	01-5552000	Maya Rai
4	Dipesh	Engineering	01-5551001	Dev Shrestha

1. Something's already gone wrong in the data. Find it.
2. Describe one **update anomaly**, one **insert anomaly**, and one **delete anomaly** this table will suffer. Be specific: "if X happens, then Y goes wrong".
3. Which rule (1, 2, or 3) is being broken? Which columns are facts about something other than the employee?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Rule 1

Fix this table so it obeys rule 1 (one value per cell, no repeating groups). Show the resulting table(s):

student	phone_numbers	course_1	course_2	course_3
Asha	9841000001, 9841000002	Maths	Art	
Bikram	9841000003	Maths		

You'll need more than one new table. Say which rows each one holds.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Normalize an invoice sheet

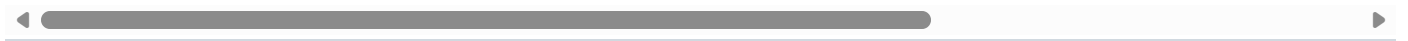
A freelancer tracks work in one spreadsheet:

invoice_no	invoice_date	client_name	client_email	client_address	service	hours
101	2026-09-01	Himal Tech	hi@himal.example	Lazimpat, Kathmandu	Web design	10
101	2026-09-01	Himal Tech	hi@himal.example	Lazimpat, Kathmandu	Hosting setup	5

invoice_no	invoice_date	client_name	client_email	client_address	service	hours
102	2026-09-15	Everest Cafe	owner@everest.example	Thamel, Kathmandu	Logo	4

1. Follow the method: for each column, write what it's a fact about.
2. Design the tables. Decide which of `hourly_rate`, `line_total`, and `invoice_total` to **store**, which to treat as a **snapshot**, and which to **calculate**. Justify each in one sentence.
3. Write the `CREATE TABLE` statements in `playground/ch18/ex3.sql`, with primary keys, foreign keys, and sensible constraints, and run them in `practice`.

Stuck? The hint is in the [Hints section](#) at the end.



Exercise 4 (Medium): Duplication or snapshot?

For each situation, decide: is the copied value a **duplication mistake** (remove it, use a foreign key) or a **deliberate snapshot** (keep it)? One sentence each, using the test from the notes.

1. `orders.customer_email`, copied from `customers.email` when the order is placed, "so we can email the receipt".
2. `orders.shipping_address`, copied from the customer's address when the order is placed.
3. `order_items.product_name`, copied from `products.name`.
4. `employees.department_name`, copied from `departments.name`.
5. `payslips.salary`, copied from `employees.salary` on the day the payslip is generated.
6. `posts.author_username`, copied from `users.username`.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A school timetable

A school's timetable is a spreadsheet with one row per lesson:

day	period	class	subject	teacher	teacher_email	room	room_capacity
Mon	1	7A	Maths	R. Thapa	rthapa@school.example	101	30
Mon	1	7B	Science	S. Karki	skarki@school.example	Lab 1	24
Mon	2	7A	Science	S. Karki	skarki@school.example	Lab 1	24

Design a normalized set of tables, then write `CREATE TABLE` statements in `playground/ch18/ex5.sql` that enforce **all** of these rules with constraints:

- A room can't host two lessons at the same day and period.
- A teacher can't teach two lessons at the same day and period.
- A class can't have two lessons at the same day and period.
- `period` is between 1 and 8. `day` is one of Mon to Fri.

Insert the three lessons above, then prove each of the first three rules by trying to break it.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

19 Project: Online Shop Database

What you'll build

A complete database for a small online shop, from a client's plain-English brief to a set of files that build it from nothing. Categories, products, customers, orders, and the items in each order. Rules that stop bad data. Links that stop orphans. And then, because this is how real projects go, the client changes their mind, and you change the tables without losing anything.

It's all of Level 2 in one piece of work: types, constraints, primary keys, `ALTER TABLE`, one-to-many, many-to-many, and normalization.

By the end, you'll have, in `playground/ch19/`:

- `plan.md`: the tables you found in the brief, with columns, types, rules, and a diagram,
- `schema.sql`: creates every table, in the right order, with every constraint,
- `seed.sql`: fills the shop with the sample data,
- `checks.sql`: a list of things that **must fail**, and proof that they do,
- `changes.sql`: the client's change requests, as `ALTER TABLE` statements,
- `queries.sql`: questions about the shop, answered.

Before you start

- Finish chapters 12 to 18.
- Read the client brief in the starter folder. It's written the way a real client talks: clear about what they want, vague about how. The sample data is in there too.
- Make a folder `playground/ch19/`.
- You'll work in a new database called `shop`. Create it in Milestone 4.
- Run `\pset null '[NULL]'` in each session.

The golden rule, again: every statement goes in a file. Milestone 9 will rebuild the whole thing from your files, and anything you typed straight into `psql` will be missing.

Milestone 1: Find the things

Goal: turn the brief into a list of tables ([chapter 02](#), [chapter 18](#)).

Read the brief and underline the nouns. For each one, decide: is this a *kind of thing* the shop needs to keep rows of, or just a detail of something else? Write your list in `plan.md`, with one sentence per table: "each row is one ____."

You should end up with five tables. If you have four, you've missed the one in the middle. If you have seven, you've probably made tables for details.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 2: Columns, types, and rules

Goal: for each table, decide every column, its type, and its constraints ([chapters 12](#) and [13](#)).

Extend `plan.md` with a table per table: column, type, rules (`NOT NULL ?` `UNIQUE ?` `CHECK ?` `DEFAULT ?`), and an example value. Go back to the brief for every rule: "email must be unique", "we never sell for a negative price", "an order starts as pending".

Some things to decide, and write down your reasoning:

- Which columns can be NULL? (Very few.)
- What type is money? What type is "when the order was placed"?
- Where does the price the customer paid live, and why isn't it just `products.price` ?
- What are the allowed order statuses, exactly?

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 3: Draw the links

Goal: decide every relationship and every `ON DELETE` ([chapters 16](#) and [17](#)).

Draw the five tables in `plan.md` the way chapter 18 does, with arrows from each foreign key to the primary key it points at, and a `—<` on each "many" end. Then, for each foreign key, write the `ON DELETE` you've chosen and **why**, in one sentence. The brief has opinions about this. Find them.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 4: Build it

Goal: `schema.sql` creates everything from scratch, in the right order ([chapter 05](#), [chapter 16](#)).

1. In psql: `CREATE DATABASE shop;` then `\c shop` .
2. Write `schema.sql` :
 - A comment at the top saying what it is.
 - `DROP TABLE IF EXISTS` for every table, **children first**.
 - `CREATE TABLE` for every table, **parents first**, with every constraint from your plan.
3. Run it with `\i` . Then run it **again**. It must work both times.

4. `\d` every table and compare with `plan.md`.

Check: `\dt` lists five tables. `\d order_items` shows two foreign keys and a primary key (or unique constraint) on the pair.

Milestone 5: Load the data

Goal: `seed.sql` fills the shop with the sample data from the brief ([chapter 06](#)).

Insert in dependency order: categories, then products, then customers, then orders, then order items. On a freshly built database, ids start from 1 in each table, so you can predict them. Say so in a comment, because it's an assumption.

For each order item, copy the product's price **at the time of the order** into your snapshot column. For these sample orders, that's the current price.

Check: `SELECT * FROM order_items;` shows 6 rows. The `JOIN` preview from chapter 16, adapted, shows each order's customer name. Try: `SELECT orders.id, customers.name, orders.status FROM orders JOIN customers ON customers.id = orders.customer_id;`

Milestone 6: Try to break it

Goal: `checks.sql` proves every rule works ([chapter 13](#)).

Write one statement for each of these. **Every one must fail.** Above each, write a comment saying which rule should stop it. Below each, after you've run it, paste the first line of the error.

1. A product with a negative price.
2. A product with a SKU that's already used.
3. A second customer with an existing email.
4. An order for a customer who doesn't exist.
5. An order item for a product that doesn't exist.
6. An order item with a quantity of 0.
7. An order with status `'lost'`.
8. The same product added twice to the same order.
9. Deleting a customer who has orders.
10. Deleting a product that's in an order.

Run them **one at a time**, not inside a transaction. (A failed statement inside a transaction puts the whole transaction in an error state, and everything after it is ignored until you `ROLLBACK`. Try it once, to see what that looks like.)

If any of the ten **succeeds**, you've found a missing rule. Fix `schema.sql`, rebuild, reseed, and run the checks again.

Milestone 7: The client changes their mind

Goal: `changes.sql` applies the client's change requests without dropping anything ([chapter 15](#)).

The brief ends with a list of changes the owner thought of after you'd built everything. This always happens. For each one, write an `ALTER TABLE` (or whatever it takes), inside a `BEGIN ... COMMIT`, with a check in between.

One of the changes is a trap: it asks you to remove a product that's already been sold. Your own Milestone 6 check number 10 says that's impossible, and the brief says the shop never loses order records. Find the solution that satisfies both, and explain it in a comment. It's a pattern you'll use for the rest of your career.

Check: after running `changes.sql`, every row of every table is still there, and `\d` shows the new shape.

Milestone 8: Ask questions

Goal: `queries.sql` answers these, one query per question, using everything from Level 1 plus the join preview:

1. All active products in the Stationery category, cheapest first.
2. Every order, with the customer's name and the order status, newest first.
3. The items in order 1: product name, quantity, unit price, and the line total.
4. Products with fewer than 20 in stock, that are still active.
5. Orders that are `pending` and older than a given date.
6. All orders placed by Asha Rai.

You can't yet total an order or find customers with no orders. Those need [chapters 20](#) and [23](#). Write them down as "TODO" comments at the bottom of the file; you'll come back for them.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 9: Rebuild from nothing

Goal: prove the files are complete.

Connect to `postgres`, `DROP DATABASE shop;`, `CREATE DATABASE shop;`, `\c shop`. Then:

```
\i .../playground/ch19/schema.sql
\i .../playground/ch19/seed.sql
\i .../playground/ch19/changes.sql
\i .../playground/ch19/queries.sql
```

Four files, no errors, sensible output. If anything fails, fix the file, not the database, and start again from the drop.

Then look at what you've made: a database that any developer could rebuild on any machine in under a minute, from a handful of text files, with every rule from the client's brief enforced. That's professional work.

Common mistakes

1. Copying names instead of ids

`customer_name` on `orders`. The whole of chapter 18 says why not. If you see yourself typing a name into any table except the one that owns it, stop.

2. Forgetting the snapshot

`order_items` with no `unit_price`, "because the price is in `products`". Then the client's change request to raise prices silently rewrites every past order. The brief warns you about this.

3. CASCADE on the wrong key

Deleting a product that cascades into `order_items` quietly removes lines from sold orders. Restrict, and deactivate instead.

4. Wrong order in `schema.sql`

`relation "customers" does not exist` because `orders` came first. Parents first to create, children first to drop.

5. Hard-coding ids without saying so

`seed.sql` assumes the first category is id 1. On a fresh build that's true. Write the assumption as a comment, so future-you knows why it broke if it ever does.

6. Running `checks.sql` inside one transaction

The first failure poisons everything after it. One statement at a time, or `ROLLBACK` between them.

Quick recap

- Brief → nouns → tables. "Each row is one ____."
- Columns and rules come from the brief. Money is `numeric`, "when" is `timestampz`, statuses are a `CHECK (... IN ...)`.
- Four foreign keys, each with a deliberate `ON DELETE`. Restrict where records must survive; deactivate instead of delete.
- `schema.sql`, `seed.sql`, `checks.sql`, `changes.sql`, `queries.sql`. Files are the project. The database is disposable.

- Prove rules by trying to break them. Every check must fail.
 - Change the shape with `ALTER TABLE` in a transaction. Never drop a table with real data to "fix" it.
-

Congratulations! You've finished Level 2. 🎉 You can design a multi-table database from a brief, with the rules and links that keep its data honest. Try the [exercises](#) for extra features, then head back to the roadmap. Level 3 teaches you to ask the hard questions across all these tables: totals, groupings, and joins.

Exercises

How to do these:

- These are extra features for your shop. Finish all 9 milestones in the notes first.
 - Each one is a new file in `playground/ch19/`, like `002_reviews.sql`, written as a migration: `BEGIN`, changes, checks, `COMMIT`. After each, the Milestone 9 rebuild (now with one more file) must still work.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to review your files.
-

Exercise 1 (Easy): Product reviews

Customers can review products. A review has a rating (1 to 5), optional text, and when it was written. **A customer can review a product only once.**

Design and add the table. Decide what happens to reviews when a customer is deleted, and when a product is deleted (remember products aren't deleted, they're deactivated, so what does that mean here?). Insert two reviews, then try to add a second review by the same customer for the same product.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Multiple addresses

The brief gave each customer one city. Real customers have several addresses (home, work, a parent's house). Move addresses into their own table: a customer has many addresses, each with a label, street, city, and whether it's the default.

Migrate the existing `city` data into the new table (one address per customer, labeled `'home'`, marked default), then drop the old column. All inside one transaction.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Tags

Products can have any number of tags (`eco` , `bestseller` , `new` , `clearance`), and a tag applies to many products. Add `tags` and the junction table. Tag at least three products, one with three tags. Then write a query listing every product that has the `bestseller` tag.

Then, in a comment: the brief's `categories` are one-to-many (a product has one category) while tags are many-to-many. Give one example of a real shop where categories should be many-to-many too, and one where they shouldn't.

Exercise 4 (Medium): Stock movements

Right now `products.stock` is a single number that gets overwritten. The owner wants to know *why* it changed: a delivery of 50, a sale of 2, a stock-take correction of -1.

1. Add a `stock_movements` table: which product, how many (positive or negative), a reason from a fixed list, and when.
2. Record the movements that would explain each product's current stock, starting from zero.
3. In a comment: `products.stock` is now a value you could calculate from `stock_movements` . Chapter 18 says "don't store what you can calculate." Should you drop `products.stock` ? Argue both sides in two or three sentences, and pick one. (Chapter 38 will show how to keep a stored total correct automatically, which may change your answer.)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Order status history

The owner asks: "When did order 2 get shipped? And who marked it paid?" Your `status` column can't answer that; it only knows the current value.

1. Add an `order_status_history` table: which order, the status it changed **to**, when, and a `changed_by` text . Keep `orders.status` as the current value (a deliberate snapshot of the latest history row).
2. Backfill the history for the four sample orders with plausible timestamps: every order started as `pending` ; paid and shipped orders have those steps too; the cancelled one went pending → cancelled.
3. Write a query showing order 2's full history in time order.

4. Now change order 3 from `pending` to `paid` the **right way**: insert a history row and update `orders.status`, in one transaction. Then explain in a comment what could go wrong if someone updates `orders.status` directly and forgets the history. Chapter 38 (triggers) is the fix; name it as a TODO.

Stuck? The hint is in the [Hints section](#) at the end.

Level 3: Asking Better Questions

Goal: answer real questions by combining, counting, and summarizing data across many tables.

20 Counting and Totals

What is it?

An **aggregate function** takes many rows and boils them down to one number. The five you'll use constantly:

Function	Answers
<code>count()</code>	How many?
<code>sum()</code>	What's the total?
<code>avg()</code>	What's the average?
<code>min()</code>	What's the smallest / earliest / first alphabetically?
<code>max()</code>	What's the biggest / latest / last alphabetically?

Until now, every query gave you back rows. Aggregates give you back **answers**.

Why does it matter?

"How many orders did we get this month?" "What's our total revenue?" "What's the average price?" "When was the last order?" These are the questions people actually ask a database. Nobody wants to see 50,000 order rows. They want one number.

Level 3 is about answering real business questions, and aggregates are the first tool. Chapter 21 adds "per group" (total *per customer*, count *per month*), and chapter 22 adds joins so the answers can span several tables.

Real-world example

A shopkeeper at the end of the day:

The shopkeeper	SQL
Counts the receipts	<code>count(*)</code>
Adds up the money	<code>sum(total)</code>
Works out the average sale	<code>avg(total)</code>
Finds the smallest and biggest sale	<code>min(total)</code> , <code>max(total)</code>

They don't want to see every receipt. They want the totals at the bottom.

How it works

The Level 3 dataset

Chapters 20 to 29 share one dataset: a small stationery shop with three months of orders. It lives in the chapter 29 project's starter folder (`PostgreSQL/29-project-sales-report/starter/seed.sql`), because that project uses it too. Set it up once:

```
CREATE DATABASE sales;
\c sales
\i C:/Users/YourName/Downloads/Projects/Sandip/docs/PostgreSQL/29-project-sales-report/starter/seed.sql
```

Five tables, the same shape as your chapter 19 shop: `categories`, `products`, `customers`, `orders`, `order_items`. Spend two minutes looking around: `\dt`, then `SELECT * FROM ...` on each. The important facts: 11 products (one discontinued, one never sold), 6 customers (one never ordered), 12 orders from July to September 2026 (one cancelled), 21 order lines.

Run `\pset null '[NULL]'` as always.

`count(*)` : how many rows?

```
SELECT count(*) FROM orders;
```

```
count
-----
      12
(1 row)
```

One row back, with one number in it. `count(*)` counts every row the query would otherwise have returned. Add a `WHERE` and it counts only the matches:

```
SELECT count(*) FROM products WHERE is_active;
```

```
count
-----
      10
(1 row)
```

`count(column)` : how many non-empty values?

Here's a distinction that trips people up:

```
SELECT count(*) AS all_orders, count(shipped_at) AS shipped_orders FROM orders;
```

```
all_orders | shipped_orders
-----+-----
          12 |              7
(1 row)
```

`count(*)` counts rows. `count(shipped_at)` counts rows where `shipped_at` **is not NULL**. Five orders haven't shipped yet, so their `shipped_at` is NULL, and they're skipped.

This is handy and dangerous in equal measure. Handy: "how many orders have shipped?" is just `count(shipped_at)`. Dangerous: `count(some_column)` can silently give you a smaller number than you expected. When you mean "how many rows", write `count(*)`.

`count(DISTINCT column)` : how many different values?

```
SELECT count(customer_id) AS rows_with_customer,
       count(DISTINCT customer_id) AS distinct_customers
FROM orders;
```

rows_with_customer	distinct_customers
12	5

(1 row)

12 orders, but only 5 different customers placed them. `DISTINCT` inside `count()` means "count each value once". It's how you answer "how many customers have ever ordered?"

`sum()` : add it up

```
SELECT sum(stock) AS units_in_stock FROM products;
```

units_in_stock
1249

(1 row)

`sum()` takes an expression, not just a column. The most useful one in this whole dataset:

```
SELECT sum(quantity * unit_price) AS revenue FROM order_items;
```

revenue
709.16

(1 row)

For each line, multiply quantity by price, then add all the results. That's the shop's total sales (including one cancelled order; you'll fix that in chapter 22).

`avg()` : the average

```
SELECT avg(price) FROM products;
```

avg
27.44363636363636

(1 row)

That's a lot of decimal places. `avg()` gives you the exact answer, and it's your job to tidy it:

```
SELECT round(avg(price), 2) AS average_price FROM products;
```

```
average_price
-----
          27.44
(1 row)
```

`round(value, 2)` rounds to 2 decimal places. You'll meet it properly in [chapter 24](#), but it's so useful with `avg()` that you need it now.

Two things to know about `avg()` :

- It **ignores NULLs**. The average of `{10, NULL, 20}` is 15, not 10. Missing values aren't counted as zero.
- It always returns a decimal, even for whole-number columns. `avg(quantity)` on our order lines is `3.0000000000000000`. Round it.

`min()` and `max()` : the extremes

```
SELECT min(price) AS cheapest, max(price) AS priciest FROM products;
```

```
cheapest | priciest
-----+-----
      1.20 |    149.00
(1 row)
```

They work on anything that can be sorted: numbers, dates, text.

```
SELECT min(ordered_at) AS first_order, max(ordered_at) AS latest_order FROM orders;
```

```
first_order | latest_order
-----+-----
2026-07-03 10:15:00+05:45 | 2026-09-26 18:05:00+05:45
(1 row)
```

On text, `min()` is the first alphabetically and `max()` the last: `min(name)` on customers gives `Asha Rai`, `max(name)` gives `Farhan Ali`.

Several at once

You can ask for as many aggregates as you like in one query:


```
SELECT count(*) AS products,
       round(avg(price), 2) AS avg_price,
       min(price),
       max(price),
       sum(stock) AS stock
FROM products;
```

```
products | avg_price | min  | max   | stock
-----+-----+-----+-----+-----
      11 |    27.44 | 1.20 | 149.00 | 1249
(1 row)
```

Still one row. Every aggregate looks at the same set of rows (all 11 products) and reports its own answer.

Aggregates and WHERE

WHERE picks the rows first, then the aggregates work on what's left:

```
SELECT count(*) AS items, sum(quantity) AS units, sum(quantity * unit_price) AS total
FROM order_items
WHERE order_id = 5;
```

```
items | units | total
-----+-----+-----
      3 |      5 | 56.24
(1 row)
```

Order 5 has three lines, five items in total, worth 56.24. That's an order total, and it's exactly what a receipt needs at the bottom.

When there's nothing to count

```
SELECT count(*) AS items, sum(quantity * unit_price) AS total
FROM order_items
WHERE order_id = 999;
```

```
items | total
-----+-----
      0 | [NULL]
(1 row)
```

No order 999, so no rows. `count` of nothing is `0`. But `sum` of nothing is **NULL**, not 0. Same for `avg`, `min`, `max`. If you need a zero, wrap it: `coalesce(sum(...), 0)`, which chapter 24 explains. For now, just know that an empty total shows up as NULL.

The error everyone hits

```
SELECT name, count(*) FROM products;
```

```
ERROR: column "products.name" must appear in the GROUP BY clause or be used in an aggregate function
```

Think about what you asked for. `count(*)` is one number for the whole table. `name` is eleven different values. PostgreSQL can't fit eleven names into a one-row answer, so it refuses. You either want *one* answer (drop `name`) or *one answer per name* (that's `GROUP BY`, and it's the whole of the [next chapter](#)).

A PostgreSQL bonus: `FILTER`

Sometimes you want two counts with different conditions, side by side:

```
SELECT count(*) FILTER (WHERE is_active) AS active,
       count(*) FILTER (WHERE NOT is_active) AS inactive
FROM products;
```

```
active | inactive
-----+-----
      10 |         1
(1 row)
```

`FILTER (WHERE ...)` applies a condition to one aggregate only. It works on any aggregate. Other databases make you do this with `CASE` (chapter 24); PostgreSQL gives you the clean version.

Aggregates on dates

Subtracting two timestamps gives an interval, and you can average those too:

```
SELECT avg(shipped_at - ordered_at) AS avg_time_to_ship
FROM orders
WHERE shipped_at IS NOT NULL;
```

```
avg_time_to_ship
-----
1 day 23:25:00
(1 row)
```

Just under two days from order to dispatch. Notice the `WHERE` : without it, the `NULL shipped_at` values would be skipped by `avg()` anyway, but saying so makes the query honest.

Common mistakes

1. `count(column)` when you mean `count(*)`

You'll get a smaller number with no warning, because `NULLs` are skipped. Use `count(*)` for "how many rows".

2. Expecting `sum()` of nothing to be 0

It's `NULL`. Wrap it in `coalesce(..., 0)` when you need a real zero.

3. Mixing a plain column with an aggregate

`SELECT name, count(*)` . Covered above. Either drop the column or wait for `GROUP BY` .

4. Averaging something that shouldn't be averaged

`avg(id)` runs fine and means nothing. `avg(price)` over *products* is the average list price, not the average price *sold*; for that you'd weight by quantity. Think about what the number means before you report it.

5. Forgetting that aggregates ignore `NULL`

`avg(rating)` on 10 books where 4 are unrated is the average of the 6 ratings. Usually that's what you want. Sometimes it isn't. Know which.

6. Not rounding `avg()`

Sixteen decimal places helps nobody. `round(avg(x), 2)` .

Quick recap

- Aggregates turn many rows into one answer: `count` , `sum` , `avg` , `min` , `max` .
- `count(*)` counts rows. `count(col)` counts non-`NULL` values. `count(DISTINCT col)` counts different values.
- `sum()` and `avg()` take expressions: `sum(quantity * unit_price)` .
- Every aggregate except `count` **ignores `NULL`** and returns `NULL` when there are no rows.
- `WHERE` filters first; aggregates work on what's left.

- A plain column next to an aggregate is an error, until `GROUP BY`.
 - `FILTER (WHERE ...)` gives one aggregate its own condition.
-

Next: try the [exercises](#), then move on to [21 Grouping](#).

Exercises

How to do these:

- Work in psql, connected to `sales` (set up in the notes, from the chapter 29 seed file (`PostgreSQL/29-project-sales-report/starter/seed.sql`)).
 - Run `\pset null '[NULL]'` first.
 - Save each answer in `playground/ch20/ex1.sql`, `ex2.sql`, and so on. Give every aggregate a name with `AS`.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Three counts

Write three queries:

1. How many customers are there?
2. How many products are active?
3. How many orders have actually been shipped? (Use the `shipped_at` column, not `status`.)

Expected answers: `6`, `10`, `7`.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): The product shelf

In **one** query on `products`, show:

- the total number of units in stock,
- the cheapest and most expensive price,
- the average price, rounded to 2 decimal places.

Expected output:

```

units_in_stock | cheapest | priciest | avg_price
-----+-----+-----+-----
          1249 |       1.20 |      149.00 |       27.44
(1 row)

```

Exercise 3 (Medium): Receipt totals

For order **8**, show how many lines it has, how many units in total, and the order total. Then do the same for order **11**.

Expected output for order 8:

```

lines | units | total
-----+-----+-----
      2 |      2 | 173.99
(1 row)

```

For order 11 you should get 2 lines, 22 units, **29.50**.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Distinct things

1. How many **different** customers have placed at least one order?
2. How many **different** products have ever been sold?
3. Using the answer to question 1 and the total number of customers from Exercise 1, how many customers have **never** ordered? (Work it out by hand for now. Chapter 23 shows how to make the database do it.)

Expected: **5**, **10**, and one customer.

Exercise 5 (Challenge): Real business questions

1. Total revenue **excluding** the cancelled order. First find the cancelled order's id with a `SELECT`, then use it in a `WHERE` on `order_items`. (Chapter 22 will let you do this in one query with a join.) Expected: **664.16**.
2. The average value of a single order line, rounded to 2 places. Expected: **33.77**.
3. The slowest and fastest time from order to shipping. Expected: **2 days 02:40:00** and **1 day 20:30:00**.

4. In one query, using `FILTER` : how many orders are `shipped` , how many are `pending` , and how many in total. Expected: `6` , `2` , `12` .

Stuck? The hint is in the [Hints section](#) at the end.

21 Grouping

What is it?

`GROUP BY` splits the rows into groups and runs the aggregates **once per group**. Instead of "how many orders?", you get "how many orders *per status*?" Instead of "total revenue?", "total revenue *per month*?"

`HAVING` filters the groups afterwards: "only statuses with more than one order".

Why does it matter?

Almost every report is a `GROUP BY`. Sales per region. Users per sign-up month. Average rating per product. Orders per customer. If chapter 20 gave you the number at the bottom of the receipt, this chapter gives you the whole summary table, one row per thing you care about.

It's also the fix for the error you met last chapter. `SELECT name, count(*)` failed because PostgreSQL didn't know how to get one count for eleven names. `GROUP BY name` tells it: one count *for each* name.

Real-world example

Sorting a pile of receipts into stacks:

You	SQL
Make one stack per payment method	<code>GROUP BY payment_method</code>
Count each stack	<code>count(*)</code>
Add up each stack	<code>sum(total)</code>
Throw away stacks with fewer than 5 receipts	<code>HAVING count(*) >= 5</code>
Only sort receipts from this week in the first place	<code>WHERE ...</code> (before stacking)

The result is one line per stack. That's what `GROUP BY` returns: one row per group.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

One group per value

```
SELECT status, count(*) FROM orders GROUP BY status ORDER BY status;
```

status	count
cancelled	1
paid	3
pending	2
shipped	6

(4 rows)

Read it as: "put the orders into one pile per status, then count each pile." Four statuses in the data, four rows back.

The rule that makes this work: **every column in SELECT must be either in the GROUP BY or inside an aggregate.** `status` is in the `GROUP BY`. `count(*)` is an aggregate. Both allowed.

Any aggregate, per group

```
SELECT category_id, count(*) AS products, round(avg(price), 2) AS avg_price
FROM products
GROUP BY category_id
ORDER BY category_id;
```

category_id	products	avg_price
1	5	3.53
2	2	87.00
3	2	32.25
4	2	22.87

Everything from chapter 20 works per group. (Category *names* would be nicer than ids. That needs a join, next chapter.)

The query you'll write most: totals per order

```
SELECT order_id, sum(quantity * unit_price) AS total
FROM order_items
GROUP BY order_id
ORDER BY order_id;
```


order_id	total
1	13.00
2	32.99
3	149.00
4	45.00
5	56.24
6	23.00
7	35.25
8	173.99
9	90.00
10	29.99
11	29.50
12	31.20

(12 rows)

Twenty-one lines in, twelve totals out. This is the receipt total for every order at once. You'll build on this exact query for the rest of Level 3.

Sorting by the aggregate

You can `ORDER BY` an aggregate or its alias:

```
SELECT customer_id, count(*) AS orders
FROM orders
GROUP BY customer_id
ORDER BY orders DESC, customer_id;
```

customer_id	orders
1	4
2	2
3	2
4	2
6	2

(5 rows)

Customer 1 (Asha) is the most frequent buyer. Notice customer 5 is missing: she has no orders, so there's no group for her. `GROUP BY` can only group rows that exist. Including zeroes is a job for [chapter 23](#).

Filtering groups: `HAVING`

`WHERE` filters rows *before* grouping. To filter *groups*, after the aggregates are computed, use `HAVING`:

```
SELECT order_id, sum(quantity * unit_price) AS total
FROM order_items
GROUP BY order_id
HAVING sum(quantity * unit_price) > 50
ORDER BY total DESC;
```

```
order_id | total
-----+-----
      8 | 173.99
      3 | 149.00
      9 |  90.00
      5 |  56.24
(4 rows)
```

"Group the lines by order, total each one, and keep only the orders over 50." You can't do that with `WHERE`, because the total doesn't exist until after the grouping.

Note that `HAVING` repeats the whole expression. `HAVING total > 50` doesn't work, because aliases from `SELECT` aren't available yet at that stage (`ORDER BY` runs later, which is why `ORDER BY total` *does* work).

WHERE and HAVING together

They do different jobs, and you'll often need both:

```
SELECT customer_id, count(*) AS orders
FROM orders
WHERE status <> 'cancelled'      -- 1. drop cancelled orders (rows)
GROUP BY customer_id           -- 2. one group per customer
HAVING count(*) >= 2            -- 3. keep customers with 2+ orders (groups)
ORDER BY customer_id;
```

```
customer_id | orders
-----+-----
          1 |      3
          2 |      2
          3 |      2
          4 |      2
          6 |      2
(5 rows)
```

Asha's count dropped from 4 to 3, because `WHERE` removed her cancelled order *before* the counting. A rule of thumb: **if the condition is about a single row's values, use `WHERE`. If it's about a group's total, use `HAVING`.**

The order things happen in

This is worth memorizing, because it explains every "why doesn't this work" in grouping:

```
FROM      1. which table(s)
WHERE     2. keep some rows
GROUP BY  3. put rows into groups
HAVING    4. keep some groups
SELECT    5. compute the output columns (aggregates happen here)
ORDER BY  6. sort the result
LIMIT    7. cut it down
```

You *write* `SELECT` first, but it *runs* fifth. That's why `WHERE` can't see aggregates (they don't exist yet), `HAVING` can't see aliases (same reason), and `ORDER BY` can see both.

Grouping by an expression

You don't have to group by a plain column. Group orders by month:

```
SELECT date_trunc('month', ordered_at)::date AS month, count(*) AS orders
FROM orders
GROUP BY month
ORDER BY month;
```

month	orders
2026-07-01	4
2026-08-01	4
2026-09-01	4

(3 rows)

`date_trunc('month', ...)` chops a timestamp down to the first of its month ([chapter 24](#)). PostgreSQL lets you `GROUP BY` the alias `month`, which many other databases don't; you can also write `GROUP BY 1`, meaning "the first output column". Both are fine. Repeating the whole expression in `GROUP BY` is the most portable.

Grouping by two columns

One group per **combination**:

```
SELECT category_id, is_active, count(*)
FROM products
GROUP BY category_id, is_active
ORDER BY category_id, is_active;
```

category_id	is_active	count
1	f	1
1	t	4
2	t	2
3	t	2
4	t	2

(5 rows)

Category 1 splits into two rows, one for the discontinued Stapler and one for the four active products. Add more columns to `GROUP BY` and the groups get finer.

Top N groups

`GROUP BY` plus `ORDER BY` plus `LIMIT` is the "top 3" pattern:

```
SELECT product_id, sum(quantity) AS units
FROM order_items
GROUP BY product_id
ORDER BY units DESC
LIMIT 3;
```

product_id	units
2	35
1	8
3	6

(3 rows)

Pens, notebooks, sticky notes. The three best-selling products by units.

Two more aggregates worth knowing

`FILTER` from chapter 20 shines with grouping, for side-by-side counts per group:

```
SELECT date_trunc('month', ordered_at)::date AS month,
       count(*) AS orders,
       count(*) FILTER (WHERE status = 'shipped') AS shipped
FROM orders
GROUP BY month
ORDER BY month;
```

month	orders	shipped
2026-07-01	4	2
2026-08-01	4	3
2026-09-01	4	1

(3 rows)

`string_agg` glues a group's text values into one string:

```
SELECT city, string_agg(name, ', ' ORDER BY name) AS people
FROM customers
GROUP BY city
ORDER BY city;
```

city	people
Kathmandu	Asha Rai, Dipesh Thapa
Lalitpur	Chandra Gurung, Elina Maharjan
Pokhara	Bikram Shrestha, Farhan Ali

(3 rows)

Great for reports. The `ORDER BY` inside the brackets controls the order of the names.

Common mistakes

1. A column that's neither grouped nor aggregated

```
SELECT name, count(*) FROM products GROUP BY category_id;
```

ERROR: column "products.name" must appear in the GROUP BY clause or be used in an aggregate function

Five products in category 1, one count. Which name should go in the row? PostgreSQL won't guess. Either add `name` to `GROUP BY` (changing what the groups mean) or aggregate it (`string_agg(name, ', ')` , `min(name)`).

2. WHERE after GROUP BY

```
SELECT status, count(*) FROM orders GROUP BY status WHERE count(*) > 1;
```

ERROR: syntax error at or near "WHERE"

Filtering groups is `HAVING`. `WHERE` goes before `GROUP BY`, and it can't use aggregates.

3. An alias in `HAVING`

`HAVING total > 50` gives `column "total" does not exist`. Repeat the expression: `HAVING sum(quantity * unit_price) > 50`.

4. Expecting missing groups to show as zero

Customer 5 doesn't appear in "orders per customer". No rows, no group. Chapter 23.

5. Using `HAVING` for row conditions

`HAVING status = 'shipped'` sometimes works (if `status` is grouped) but it's doing `WHERE`'s job badly. Row conditions go in `WHERE`; it's clearer and faster.

6. Forgetting `ORDER BY`

Grouped results come back in whatever order PostgreSQL built the groups. Always sort a report.

Quick recap

- `GROUP BY column` makes one group per value and runs aggregates once per group. One row out per group.
- Every `SELECT` column must be in the `GROUP BY` or inside an aggregate.
- `WHERE` filters **rows** before grouping. `HAVING` filters **groups** after. Use both when you need both.
- Execution order: `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`. It explains what each clause can see.
- Group by expressions (`date_trunc('month', ...)`) and by several columns (one group per combination).
- `ORDER BY` aggregate `DESC LIMIT n` is the "top N" pattern.
- Groups only exist for rows that exist. Zero-rows groups need an outer join (chapter 23).

Next: try the [exercises](#), then move on to [22 Joins](#).

Exercises

How to do these:

- Work in `psql`, connected to `sales`. Run `\pset null '[NULL]'`.
- Save each answer in `playground/ch21/ex1.sql`, `ex2.sql`, and so on. Always `ORDER BY` your grouped results.

- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): One row per value

1. How many customers are in each city?
2. How many orders are there in each status?

Expected output of query 1:

```

city | customers
-----+-----
Kathmandu |      2
Lalitpur  |      2
Pokhara   |      2
(3 rows)

```

Exercise 2 (Easy): Per category

For each `category_id`, show the number of products, the cheapest price, and the most expensive price.

Expected output:

```

category_id | products | min  | max
-----+-----+-----+-----
1 |      5 | 1.20 | 8.00
2 |      2 | 24.99 | 149.00
3 |      2 | 19.50 | 45.00
4 |      2 | 15.75 | 29.99
(4 rows)

```

Exercise 3 (Medium): HAVING

1. Which orders have **more than one** line? Show the order id and the number of lines.
2. Which products have sold **3 or more units** in total? Show `product_id` and units, best seller first.
3. Which customers have placed **at least 2 orders that weren't cancelled**? Show `customer_id` and the count.

Expected output of query 2:

```
product_id | units
-----+-----
          2 |    35
          1 |     8
          3 |     6
          8 |     3
(4 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Months

1. How many orders were placed in each month? Use `date_trunc('month', ordered_at)::date`.
2. Extend it: in the same query, also show how many of each month's orders are `shipped`, using `FILTER`.
3. Now group by **month and status** together. How many rows do you get, and why is it more than 3?

Expected output of query 2:

```
month      | orders | shipped
-----+-----+-----
2026-07-01 |      4 |      2
2026-08-01 |      4 |      3
2026-09-01 |      4 |      1
(3 rows)
```

Exercise 5 (Challenge): A customer summary

For each customer who has ordered, show their `customer_id`, how many orders they've placed, the date of their **first** order, and the date of their **latest** order. Most recent latest-order first.

Expected output:

customer_id	orders	first_order	last_order
1	4	2026-07-03	2026-09-26
6	2	2026-08-15	2026-09-19
4	2	2026-08-02	2026-09-11
3	2	2026-07-18	2026-09-04
2	2	2026-07-10	2026-08-09

(5 rows)

Then, as a comment: one customer is missing from this list. Who, and why can't `GROUP BY` include them?

Stuck? The hint is in the [Hints section](#) at the end.

22 Joins

What is it?

A **join** combines rows from two tables into one result, matching them up by a shared value. Usually that value is a foreign key on one side and a primary key on the other: `orders.customer_id` matches `customers.id`.

You've seen a sneak peek twice already (chapters 16 and 17). Now it gets its proper chapter.

Why does it matter?

Level 2 taught you to split data into tables so each fact lives once. The cost is that the data you want to see is spread out: the order has a `customer_id`, but the name is in another table. Joins pay that cost back. They let you store data in the clean, normalized way and still read it as one combined picture, whenever you need to.

Every real report is a join. "Revenue per category" needs `order_items`, `products`, and `categories`. "Orders with customer names" needs `orders` and `customers`. If `GROUP BY` is the most-written clause in reporting, `JOIN` is a close second.

Real-world example

Two filing cabinets in an office: one with customer cards (numbered), one with order forms, each form showing only a customer number. A new clerk asked for "orders with the customer's name on them" would:

1. Take an order form.
2. Read the customer number.
3. Find the card with that number.
4. Write the name onto a combined sheet.
5. Repeat for every form.

That's a join. The "customer number on the form matches the number on the card" rule is the `ON` clause.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

The basic join

```
SELECT orders.id, customers.name, orders.status
FROM orders
JOIN customers ON customers.id = orders.customer_id
ORDER BY orders.id;
```

id	name	status
1	Asha Rai	paid
2	Bikram Shrestha	shipped
3	Chandra Gurung	shipped
4	Asha Rai	cancelled
5	Dipesh Thapa	shipped
...		

(12 rows)

Read it as: "from `orders`, join in `customers`, matching each order to the customer whose `id` equals the order's `customer_id`." Twelve orders in, twelve rows out, each now carrying a name.

Columns are written as `table.column`, because both tables have an `id` and PostgreSQL needs to know which one you mean.

Table aliases

Typing `customers.` and `orders.` everywhere gets old. Give each table a short nickname right after its name:

```
SELECT o.id, c.name, o.status
FROM orders o
JOIN customers c ON c.id = o.customer_id
ORDER BY o.id;
```

Same result. `o` means `orders`, `c` means `customers`, for this query only. Almost everyone writes joins this way. Pick aliases that are obvious: first letter, or a short abbreviation (`oi` for `order_items`).

Ambiguous columns

```
SELECT id, name FROM orders JOIN customers ON customers.id = orders.customer_id;
```

```
ERROR: column reference "id" is ambiguous
```

Both tables have an `id`. Which one? Always prefix columns in a join, even when only one table has that column. It's clearer, and it won't break when someone adds a column later.

Products with their category names

The query chapter 21 couldn't write:

```
SELECT p.name, c.name AS category, p.price
FROM products p
JOIN categories c ON c.id = p.category_id
ORDER BY c.name, p.name;
```

name	category	price
Backpack	Bags	45.00
Laptop sleeve	Bags	19.50
USB hub	Electronics	29.99
Wireless mouse	Electronics	15.75
Desk lamp	Furniture	24.99
Office chair	Furniture	149.00
Notebook	Stationery	3.90
...		

(11 rows)

Both tables have a `name`, so the category's gets an alias. Same pattern as before: foreign key on one side, primary key on the other.

Three tables

A line on a receipt needs the item, the order, and the product. Chain the joins:

```
SELECT o.id AS order_id, p.name AS product, oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_total
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
JOIN products p ON p.id = oi.product_id
WHERE o.id = 5
ORDER BY p.name;
```

order_id	product	quantity	unit_price	line_total
5	Notebook	3	3.50	10.50
5	USB hub	1	29.99	29.99
5	Wireless mouse	1	15.75	15.75

(3 rows)

Each `JOIN ... ON` adds one table. Start from the table in the middle (`order_items`, which has foreign keys to both) and reach out. The `WHERE` and `ORDER BY` can use columns from any joined table.

Four tables

Everything Asha has ever bought:

```
SELECT o.id AS order_id, c.name AS customer, p.name AS product, oi.quantity
FROM orders o
JOIN customers c ON c.id = o.customer_id
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON p.id = oi.product_id
WHERE c.name = 'Asha Rai'
ORDER BY o.id, p.name;
```

order_id	customer	product	quantity
1	Asha Rai	Notebook	2
1	Asha Rai	Pen	5
4	Asha Rai	Backpack	1
8	Asha Rai	Desk lamp	1
8	Asha Rai	Office chair	1
12	Asha Rai	Laptop sleeve	1
12	Asha Rai	Notebook	3

(7 rows)

Four tables, three `ON` clauses, each one following a foreign key. Draw the tables and arrows from chapter 18 and the joins write themselves: every arrow is an `ON`.

Joins and `GROUP BY` together

This is where Level 3 comes together. Revenue per category, excluding the cancelled order:

```

SELECT c.name AS category, sum(oi.quantity) AS units, sum(oi.quantity * oi.unit_price) AS
    revenue
FROM order_items oi
JOIN products p ON p.id = oi.product_id
JOIN categories c ON c.id = p.category_id
JOIN orders o ON o.id = oi.order_id
WHERE o.status <> 'cancelled'
GROUP BY c.name
ORDER BY revenue DESC;

```

category	units	revenue
Furniture	4	347.98
Bags	4	129.00
Stationery	50	95.70
Electronics	4	91.48

(4 rows)

Join to reach every column you need, `WHERE` to drop rows, `GROUP BY` the column you want one row per. Furniture sells few units but makes the most money; stationery is the opposite.

The join-and-count trap

Here's a mistake that gives a wrong answer with no error. Orders and spending per customer:

```

SELECT c.name, count(o.id) AS orders, sum(oi.quantity * oi.unit_price) AS spent
FROM customers c
JOIN orders o ON o.customer_id = c.id
JOIN order_items oi ON oi.order_id = o.id
WHERE o.status <> 'cancelled'
GROUP BY c.name
ORDER BY spent DESC;

```

name	orders	spent
Chandra Gurung	2	239.00
Asha Rai	6	218.19
Dipesh Thapa	4	86.23
...		

Asha has 3 orders, not 6. What happened? Joining `order_items` turned each order into one row *per line*. Asha's three orders have six lines between them, so `count(o.id)` counted six. The `sum` is right (every line counted once), but the count is wrong.

The fix is `count(DISTINCT o.id)`:

```
SELECT c.name, count(DISTINCT o.id) AS orders, sum(oi.quantity * oi.unit_price) AS spent
...
```

name	orders	spent
Chandra Gurung	2	239.00
Asha Rai	3	218.19
Dipesh Thapa	2	86.23
Farhan Ali	2	64.75
Bikram Shrestha	2	55.99

(5 rows)

Whenever you join to a "many" table and count, ask: am I counting the thing I think I'm counting, or the lines? `count(DISTINCT ...)` is the usual answer. [Chapter 26](#) shows a cleaner way: total each order first, then join.

The old way: commas

You'll see this in older code:

```
SELECT o.id, c.name FROM orders o, customers c WHERE c.id = o.customer_id;
```

It's the same inner join, with the match condition hiding in the `WHERE`. It works, but when you have four tables and six conditions it's hard to tell which conditions are joins and which are filters. Write `JOIN ... ON`. Recognize the comma version when you meet it.

Forgetting the match: `CROSS JOIN`

What if there's no `ON`? Every row of one table gets paired with every row of the other:

```
SELECT count(*) FROM orders CROSS JOIN customers;
```

```
count
-----
    72
(1 row)
```

12 orders × 6 customers = 72 nonsense rows. This is called a **cross join** or Cartesian product. Occasionally useful (every size with every colour, chapter 23), usually a mistake. If a join gives you far more rows than you expected,

you probably lost an `ON` condition, or wrote the comma version and forgot the `WHERE`.

Joining a table to itself

A table can join to itself when rows refer to other rows in the same table. Employees and their managers:

```
CREATE TEMP TABLE employees (  
  id integer PRIMARY KEY,  
  name text,  
  manager_id integer REFERENCES employees (id)  
);  
  
INSERT INTO employees VALUES  
  (1, 'Maya Rai', NULL), (2, 'Dev Shrestha', 1), (3, 'Asha Thapa', 2),  
  (4, 'Bikram Karki', 2), (5, 'Sita Gurung', 1);  
  
SELECT e.name AS employee, m.name AS manager  
FROM employees e  
JOIN employees m ON m.id = e.manager_id  
ORDER BY e.name;
```

employee	manager
Asha Thapa	Dev Shrestha
Bikram Karki	Dev Shrestha
Dev Shrestha	Maya Rai
Sita Gurung	Maya Rai

(4 rows)

The same table, two aliases, two roles. `e` is "the employee", `m` is "the manager". Maya has no manager, so she's missing; the next chapter fixes that.

(`CREATE TEMP TABLE` makes a table that disappears when you quit `psql`. Handy for experiments.)

Common mistakes

1. Ambiguous column names

Always `alias.column` in a join. Every time.

2. Counting lines instead of orders

The trap above. `count(DISTINCT o.id)`, or total the lines first (chapter 26).

3. Missing `ON`

ERROR: syntax error at or near "ORDER"

A `JOIN` without `ON` is a syntax error (unless you write `CROSS JOIN` on purpose). If the error points just after a table name, you forgot the `ON`.

4. Joining on the wrong columns

`ON c.id = o.id` instead of `ON c.id = o.customer_id`. No error, nonsense result: order 3 gets customer 3 regardless of who placed it. Check each `ON` follows a real foreign key.

5. Row explosion

Thousands of rows from a small query means a join condition is too loose or missing. Count the rows before you trust the sums.

6. Filtering in `ON` instead of `WHERE`

For inner joins, `ON c.id = o.customer_id AND o.status = 'shipped'` and putting the status in `WHERE` give the same answer. For outer joins (next chapter) they don't. Get in the habit: `ON` for matching, `WHERE` for filtering.

Quick recap

- `FROM a JOIN b ON b.id = a.b_id` lines up rows from two tables by a shared value. Every foreign key arrow is an `ON`.
- Give tables short aliases and prefix every column: `o.id`, `c.name`.
- Chain joins to reach more tables. Start from the table in the middle.
- `JOIN + WHERE + GROUP BY` answers real reporting questions across tables.
- Joining a "many" table multiplies rows. Use `count(DISTINCT ...)` or total first.
- A join with no `ON` pairs everything with everything. If you get far too many rows, look for a lost condition.
- A table can join to itself with two aliases.

Next: try the [exercises](#), then move on to [23 Outer Joins](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
- Save each answer in `playground/ch22/ex1.sql`, `ex2.sql`, and so on. Use table aliases and prefix every column.

- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Names instead of ids

1. Every order with its customer's name and status, in order id order.
2. Every product with its category name and price, sorted by category then product name.
3. Every order placed by a customer from **Pokhara**: order id, customer name, order date (as a date).

Expected output of query 3:

```
id |      name      | ordered_at
---+-----+-----
 2 | Bikram Shrestha | 2026-07-10
 6 | Bikram Shrestha | 2026-08-09
 7 | Farhan Ali      | 2026-08-15
11 | Farhan Ali      | 2026-09-19
(4 rows)
```

Exercise 2 (Easy): A receipt

Show the lines of order **8**: product name, quantity, unit price, and line total.

Expected output:

```
name | quantity | unit_price | line_total
-----+-----+-----+-----
Desk lamp | 1 | 24.99 | 24.99
Office chair | 1 | 149.00 | 149.00
(2 rows)
```

Exercise 3 (Medium): Reports across tables

1. For each category (by **name**), the number of products and the average price, most expensive category first.
2. Total units sold per category name, most units first. Include every order, cancelled or not.
3. Each customer's spending, **excluding** cancelled orders: name, number of orders, total spent. Highest spender first. Make sure the order count is right (reread the trap in the notes).

Expected output of query 3:

name	orders	spent
Chandra Gurung	2	239.00
Asha Rai	3	218.19
Dipesh Thapa	2	86.23
Farhan Ali	2	64.75
Bikram Shrestha	2	55.99

(5 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Questions with a `WHERE`

1. Which **distinct** products were sold in September 2026? Names only, alphabetical.
2. Which customers have bought an **Office chair**? Names only, each once.
3. All active Stationery products, cheapest first. (Filter on the category's name, show the product's name and price.)

Expected output of query 2:

name
Asha Rai
Chandra Gurung

(2 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Receipts, all at once

1. One row per order showing: order id, customer name, status, and the order **total**. In order id order. (This is the chapter 21 totals query, plus a join. Think about what goes in `GROUP BY`.)
2. Build the `employees` table from the notes, and add a row for `'Ram Lama'` whose manager is Asha Thapa. Then write a self-join showing each employee next to their manager **and their manager's manager**. Ram's row should read `Ram Lama | Asha Thapa | Dev Shrestha`.

Expected first rows of query 1:

id	name	status	total
1	Asha Rai	paid	13.00
2	Bikram Shrestha	shipped	32.99
3	Chandra Gurung	shipped	149.00
...			

(12 rows)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

23 Outer Joins

What is it?

The joins in chapter 22 only keep rows that **match** on both sides. They're called **inner joins**, and `JOIN` is short for `INNER JOIN`.

An **outer join** also keeps rows that *don't* match, filling the missing side with NULLs. `LEFT JOIN` keeps everything from the left table. `RIGHT JOIN` keeps everything from the right. `FULL JOIN` keeps everything from both.

Why does it matter?

Some of the most useful questions are about what's **missing**:

- Which customers have never ordered?
- Which products have never sold?
- Which categories are empty?
- Which months had zero sales?

An inner join can't answer any of these, because a customer with no orders has nothing to match. They simply vanish from the result, silently. You've seen it twice: customer 5 missing from "orders per customer", Maya missing from "employees and managers". Outer joins bring them back.

Real-world example

A teacher has a class list and a pile of handed-in homework.

- **Inner join:** "line up each homework with the student who handed it in." Students who handed nothing in don't appear. Neither does homework with no name on it.
- **Left join** (class list on the left): "go down the class list and attach each student's homework, if any." Every student appears. Those who handed nothing in get a blank. **Now you can see who's missing.**
- **Full join:** every student *and* every piece of homework, including the anonymous ones.

The question "who didn't do the homework?" is a left join followed by "where the homework is blank".

How it works

Work in `sales`, with `\pset null '[NULL]'`.

What the inner join loses

```
SELECT c.name, o.id AS order_id
FROM customers c
JOIN orders o ON o.customer_id = c.id
ORDER BY c.name, o.id;
```

12 rows, 5 names. Elina Maharjan, who has never ordered, isn't there.

LEFT JOIN : keep everything on the left

```
SELECT c.name, o.id AS order_id, o.status
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
ORDER BY c.name, o.id;
```

name	order_id	status
Asha Rai	1	paid
Asha Rai	4	cancelled
Asha Rai	8	shipped
Asha Rai	12	pending
Bikram Shrestha	2	shipped
Bikram Shrestha	6	shipped
Chandra Gurung	3	shipped
Chandra Gurung	9	shipped
Dipesh Thapa	5	shipped
Dipesh Thapa	10	paid
Elina Maharjan	[NULL]	[NULL]
Farhan Ali	7	paid
Farhan Ali	11	pending

(13 rows)

Thirteen rows now. Elina appears once, with NULL in every column that comes from `orders`, because there was nothing to match.

"Left" means the table written **before** the `JOIN` keyword: here, `customers`. Every row from it survives. Rows from the right table (`orders`) appear only where they match.

Finding what's missing

Elina's row is the only one with a NULL `order_id`. So:

```
SELECT c.name, c.email
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

name	email
Elina Maharjan	elina@example.com

(1 row)

This is **the** outer join pattern. Left join, then `WHERE right_table.id IS NULL`. Say it as "customers with no matching order." The same shape answers "products never sold":

```
SELECT p.name, p.sku
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
WHERE oi.order_id IS NULL;
```

name	sku
Whiteboard marker	STA-005

(1 row)

Always test for NULL on a column that **can't** be NULL in the right table when there's a match: its primary key, or a NOT NULL column. `o.id` and `oi.order_id` are both perfect. If you tested `o.shipped_at IS NULL`, you'd also catch real orders that haven't shipped.

Counting with a LEFT JOIN

Orders per customer, including zero:

```
SELECT c.name, count(o.id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.name
ORDER BY c.name;
```

name	orders
Asha Rai	4
Bikram Shrestha	2
Chandra Gurung	2
Dipesh Thapa	2
Elina Maharjan	0
Farhan Ali	2
(6 rows)	

Elina shows 0. Now the chapter 20 lesson pays off: it has to be `count(o.id)`, not `count(*)`. Elina's group has one row (her own, with NULLs). `count(*)` would count that row and report 1:

```
SELECT c.name, count(*) AS orders ...
```

```
Elina Maharjan | 1 ← wrong
```

`count(o.id)` skips the NULL and gives 0. **With outer joins, count a column from the right table, never `*`.**

Sums with a LEFT JOIN

```
SELECT p.name, sum(oi.quantity) AS units_sold
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
GROUP BY p.name
ORDER BY units_sold DESC NULLS LAST, p.name;
```

The Whiteboard marker comes out with `[NULL]`, because the sum of nothing is NULL. To show a proper zero, wrap it in `coalesce()`, which returns the first non-NULL value it's given:

```
SELECT p.name, coalesce(sum(oi.quantity), 0) AS units_sold
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
GROUP BY p.name
ORDER BY units_sold DESC, p.name;
```


name	units_sold
Pen	35
Notebook	8
Sticky notes	6
Backpack	3
Desk lamp	2
Laptop sleeve	2
Office chair	2
USB hub	2
Wireless mouse	2
Stapler	1
Whiteboard marker	0
(11 rows)	

A complete sales table, every product present. `coalesce` gets its full introduction in [chapter 24](#); for now, `coalesce(x, 0)` means "x, or 0 if x is NULL".

The ON vs WHERE difference

This is the subtle part of outer joins, and it matters.

Suppose you want every customer, with their **shipped** orders, keeping customers who have none. Put the condition in the `ON`:

```
SELECT c.name, o.id AS shipped_order
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'shipped'
ORDER BY c.name, o.id;
```

name	shipped_order
Asha Rai	8
Bikram Shrestha	2
Bikram Shrestha	6
Chandra Gurung	3
Chandra Gurung	9
Dipesh Thapa	5
Elina Maharjan	[NULL]
Farhan Ali	[NULL]
(8 rows)	

All six customers. Elina and Farhan (whose two orders are paid and pending, not shipped) show NULL.

Now put the same condition in the `WHERE` instead:

```
SELECT c.name, o.id AS shipped_order
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'shipped'
ORDER BY c.name, o.id;
```

name	shipped_order
Asha Rai	8
Bikram Shrestha	2
Bikram Shrestha	6
Chandra Gurung	3
Chandra Gurung	9
Dipesh Thapa	5

(6 rows)

Elina and Farhan are gone. Why? The `LEFT JOIN` gave them rows with NULL status. Then `WHERE o.status = 'shipped'` asked "is NULL equal to shipped?", got unknown, and dropped them. The `WHERE` quietly turned your outer join back into an inner one.

The rule: **conditions about the right table go in the `ON`. Conditions about the left table go in the `WHERE`.** The one exception is the deliberate `WHERE right.id IS NULL` pattern for finding what's missing.

RIGHT JOIN : the mirror image

```
SELECT c.name, o.id AS order_id
FROM orders o
RIGHT JOIN customers c ON o.customer_id = c.id
ORDER BY c.name, o.id;
```

Exactly the same 13 rows as the first `LEFT JOIN`. A `RIGHT JOIN` keeps everything from the table **after** `JOIN`. Any right join can be rewritten as a left join by swapping the tables, and nearly everyone does, because "the table I start with is the one I keep" is easier to read. You'll rarely write `RIGHT JOIN`; just recognize it.

FULL JOIN : keep both sides

Sometimes neither table is "the main one". The shop has a newsletter list that isn't tied to customer accounts:

```

CREATE TEMP TABLE newsletter (email text PRIMARY KEY, subscribed_on date NOT NULL);
INSERT INTO newsletter VALUES
  ('asha@example.com', '2026-03-01'),
  ('elina@example.com', '2026-07-01'),
  ('guest@example.com', '2026-08-15');

SELECT c.name, c.email AS customer_email, n.email AS subscriber_email
FROM customers c
FULL JOIN newsletter n ON n.email = c.email
ORDER BY c.name NULLS LAST, n.email;

```

name	customer_email	subscriber_email
Asha Rai	asha@example.com	asha@example.com
Bikram Shrestha	bikram@example.com	[NULL]
Chandra Gurung	chandra@example.com	[NULL]
Dipesh Thapa	dipesh@example.com	[NULL]
Elina Maharjan	elina@example.com	elina@example.com
Farhan Ali	farhan@example.com	[NULL]
[NULL]	[NULL]	guest@example.com

(7 rows)

Customers who aren't subscribed (NULL on the right), a subscriber who isn't a customer (NULL on the left), and the two who are both. `FULL JOIN` is less common than `LEFT JOIN`, but it's exactly right for comparing two lists.

CROSS JOIN : every combination, on purpose

Chapter 22 called this a mistake. Sometimes it's the goal:

```

CREATE TEMP TABLE sizes (size text);
INSERT INTO sizes VALUES ('S'), ('M'), ('L');
CREATE TEMP TABLE colours (colour text);
INSERT INTO colours VALUES ('red'), ('blue');

SELECT size, colour FROM sizes CROSS JOIN colours ORDER BY size, colour;

```

```

size | colour
-----+-----
L    | blue
L    | red
M    | blue
M    | red
S    | blue
S    | red
(6 rows)

```

Every size with every colour: six product variants. No `ON`, because there's nothing to match. Write `CROSS JOIN` explicitly so nobody thinks you forgot the `ON`.

The family

Join	Keeps	Use for
<code>JOIN</code> (inner)	Only matching rows	Most queries
<code>LEFT JOIN</code>	All left rows, matching right rows	"All X, with their Y if any"; "X with no Y"
<code>RIGHT JOIN</code>	All right rows, matching left rows	Rare; rewrite as <code>LEFT JOIN</code>
<code>FULL JOIN</code>	Everything from both	Comparing two lists
<code>CROSS JOIN</code>	Every combination	Generating variants

Common mistakes

1. `count(*)` with an outer join

Gives `1` for things that have nothing. Count a right-table column: `count(o.id)`.

2. A right-table condition in `WHERE`

Turns the outer join back into an inner join. Move it to the `ON`.

3. Testing the wrong column for `NULL`

`WHERE o.shipped_at IS NULL` catches unshipped orders *and* customers with no orders. Test the right table's primary key, or a `NOT NULL` column.

4. Forgetting `coalesce` on sums

`sum()` over a group of `NULL`s is `NULL`. `coalesce(sum(...), 0)` for a clean zero.

5. Chaining outer joins carelessly

`customers LEFT JOIN orders JOIN order_items` makes the second join inner, and the inner join on `order_items` drops Elina's NULL row again. If you want to keep the left side all the way through, every join in the chain must be `LEFT JOIN`.

6. Writing `RIGHT JOIN` when you could swap

Not wrong, just harder to read. Start from the table you want to keep, and use `LEFT JOIN`.

Quick recap

- `JOIN` keeps only matches. `LEFT JOIN` keeps every row from the left table, filling the right side with NULL when there's no match.
 - **Find what's missing:** `LEFT JOIN` then `WHERE right.id IS NULL`.
 - **Count with outer joins** using a right-table column (`count(o.id)`), and wrap sums in `coalesce(..., 0)`.
 - **ON vs WHERE :** right-table conditions go in `ON`, or you lose the unmatched rows.
 - `RIGHT JOIN` is a `LEFT JOIN` with the tables swapped. `FULL JOIN` keeps both sides. `CROSS JOIN` makes every combination.
 - In a chain of joins, every link after a `LEFT JOIN` must also be `LEFT JOIN` to keep the unmatched rows.
-

Next: try the [exercises](#), then move on to [24 Handy Built-in Functions](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
 - Save each answer in `playground/ch23/ex1.sql`, `ex2.sql`, and so on.
 - For each exercise, decide **which table is the one you want to keep**, and start your `FROM` with it.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): What's missing

1. Which customers have never placed an order? Name and email.
2. Which products have never been sold? Name and SKU.

Expected output of query 2:

```
      name      | sku
-----+-----
Whiteboard marker | STA-005
(1 row)
```

Exercise 2 (Easy): Counting with zeroes

1. Orders per customer, **including** customers with none, alphabetical by name.
2. Run it again with `count(*)` instead. Which number changes, and why? Write the answer as a comment.

Expected output of query 1:

```
      name      | orders
-----+-----
Asha Rai       |      4
Bikram Shrestha |      2
Chandra Gurung  |      2
Dipesh Thapa    |      2
Elina Maharjan  |      0
Farhan Ali      |      2
(6 rows)
```

Exercise 3 (Medium): Complete tables

1. Units sold per product, **every** product listed, zero for unsold ones. Most units first, then by name.
2. Add a category called `'Books'` (no products in it). Then list every category with its product count, including Books with 0. Afterwards, delete the Books category so the dataset is back to normal.
3. Each customer's **last order date**, including customers who have none (they should show `[NULL]` at the bottom). Most recent first.

Expected output of query 3:

name		last_order
Asha Rai		2026-09-26
Farhan Ali		2026-09-19
Dipesh Thapa		2026-09-11
Chandra Gurung		2026-09-04
Bikram Shrestha		2026-08-09
Elina Maharjan		[NULL]

(6 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): ON or WHERE ?

1. Every customer with their **pending** orders, keeping customers who have none. Elina, Bikram, Chandra, and Dipesh should all appear with NULL.
2. Now: which customers have **no** pending orders? Names only, alphabetical. You should get four.
3. Write the "wrong" version of query 1, with the status condition in `WHERE`, and explain in a comment what happened to the rows.

Expected output of query 2:

name
Bikram Shrestha
Chandra Gurung
Dipesh Thapa
Elina Maharjan

(4 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Two lists, and a chain

1. Create the `newsletter` table from the notes. Write a `FULL JOIN` that shows **only** the mismatches: customers not subscribed, and subscribers who aren't customers. You should get five rows.
2. Spending per customer, **including** Elina with `0`, counting every order (cancelled too). Name, order count, total spent. Think carefully: this chains `customers` → `orders` → `order_items`, and the order count must be right.

Expected output of query 2:

name	orders	spent
Asha Rai	4	263.19
Chandra Gurung	2	239.00
Dipesh Thapa	2	86.23
Farhan Ali	2	64.75
Bikram Shrestha	2	55.99
Elina Maharjan	0	0.00

(6 rows)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

24 Handy Built-in Functions

What is it?

A **function** takes one or more values and gives you back a new one: `upper('hello')` gives `'HELLO'`, `round(27.44363, 2)` gives `27.44`, `date_trunc('month', some_date)` gives the first of that month.

PostgreSQL has hundreds. This chapter is the two dozen you'll actually use, grouped by what they work on: text, numbers, dates, and two special tools for handling NULL and "if this then that" (`COALESCE` and `CASE`).

Why does it matter?

Real data is messy. Emails arrive in mixed case. Names have stray spaces. You need "the month" out of a timestamp, "the first letter" out of a name, "N/A" instead of a blank, and "cheap / mid / premium" instead of a raw price. Functions turn stored data into the shape a report or a screen needs, right inside the query.

You've already used a few: `length()`, `upper()`, `round()`, `now()`, `date_trunc()`. This chapter fills in the set, so you stop reaching for a programming language to do what SQL does in one line.

Real-world example

A kitchen drawer of tools. Each one does one small job well:

Tool	Function
A peeler: strips the outside off	<code>trim()</code> strips spaces from the ends
A knife: cuts a piece out	<code>substring()</code> , <code>left()</code> , <code>split_part()</code>
Measuring cups: round to the nearest mark	<code>round()</code> , <code>floor()</code> , <code>ceil()</code>
A calendar: which month is it?	<code>date_trunc()</code> , <code>extract()</code> , <code>to_char()</code>
A substitute ingredient when one is missing	<code>coalesce()</code>
A recipe step: "if it's too thick, add water"	<code>CASE WHEN</code>

You don't memorize the whole drawer. You learn where the common tools are, and look up the rest.

How it works

Work in `sales`, with `\pset null '[NULL]'`. Most examples here work with `SELECT` and no table, so you can try variations freely.

Text

Changing case and measuring:

```
SELECT upper('hello') AS up, lower('HELLO') AS low, initcap('asha rai') AS cap,
       length('Kathmandu') AS len;
```

up	low	cap	len
HELLO	hello	Asha Rai	9

`lower()` matters more than it looks. `'Asha@example.com'` and `'asha@example.com'` are different text to PostgreSQL, so a `UNIQUE` on `email` won't catch them. Store emails as `lower(email)`, and compare with `lower()` too.

Cleaning up spaces:

```
SELECT trim(' hello ') AS trimmed, trim(both '-' from '--hi--') AS dashes;
```

trimmed	dashes
hello	hi

User input nearly always has stray spaces. `trim()` before you store it.

Joining text:

```
SELECT 'Hello' || ' ' || 'World' AS pipes,
       concat('Hello', ' ', 'World') AS concat,
       concat_ws(', ', 'Rai', 'Asha') AS with_sep;
```

pipes	concat	with_sep
Hello World	Hello World	Rai, Asha

`||` and `concat()` look the same until a NULL turns up. `'a' || NULL` is NULL (chapter 07). `concat('a', NULL, 'b')` is `'ab'`: it skips NULLs. `concat_ws` ("with separator") puts the separator between the parts. For building display text from columns that might be empty, `concat()` is the safer choice.

Cutting pieces out:

```
SELECT left('Kathmandu', 4) AS l, right('Kathmandu', 5) AS r, substring('Kathmandu' from 5 for 3) AS mid;
```

l	r	mid
Kath	mandu	man

`substring(text from start for length)`. Positions start at 1, not 0.

Finding and replacing:

```
SELECT replace('2026-09-30', '-', '/') AS replaced,
       position('@' in 'asha@example.com') AS at_pos,
       split_part('asha@example.com', '@', 2) AS domain;
```

replaced	at_pos	domain
2026/09/30	5	example.com

`split_part(text, separator, n)` is wonderful: split on the separator and take piece number *n*. Email domains, the second word of a name, parts of a product code.

Padding and repeating:

```
SELECT lpad('42', 5, '0') AS padded, repeat('=', 10) AS line, reverse('abc') AS rev;
```

padded	line	rev
00042	=====	cba

`lpad` makes `'00042'`-style invoice numbers.

On real data:

```
SELECT name, left(name, 1) || left(split_part(name, ' ', 2), 1) AS initials
FROM customers ORDER BY name;
```

name	initials
Asha Rai	AR
Bikram Shrestha	BS
Chandra Gurung	CG
...	

Functions nest: the result of `split_part` goes straight into `left`.

Numbers

```
SELECT round(27.4436, 2) AS r2, round(27.4436) AS r0, ceil(27.1) AS up, floor(27.9) AS down,
       trunc(27.4436, 2) AS cut;
```

r2	r0	up	down	cut
27.44	27	28	27	27.44

- `round(x, n)` rounds to n decimal places. `round(x)` rounds to a whole number. Halves round away from zero: `round(2.5)` is 3.
- `ceil()` always goes up, `floor()` always goes down.
- `trunc()` chops without rounding.

```
SELECT abs(-5) AS absolute, 17 % 5 AS remainder, power(2, 10) AS pow, sqrt(144) AS root,
       greatest(3, 9, 4) AS big, least(3, 9, 4) AS small;
```

absolute	remainder	pow	root	big	small
5	2	1024	12	9	3

`greatest()` and `least()` pick the biggest or smallest **of several values in one row**. Don't confuse them with `max()` and `min()`, which work *down* a column across rows.

And the integer-division fix, one more time, because it bites everyone:

```
SELECT 10 / 3 AS int_div, round(10.0 / 3, 2) AS rounded;
```

int_div	rounded
3	3.33

Dates and times

Chopping to a unit: `date_trunc`

```
SELECT date_trunc('month', timestampz '2026-09-26 18:05+05:45') AS month_start;
```

month_start
2026-09-01 00:00:00+05:45

`date_trunc('month', ...)` turns any moment into the first instant of its month. Also `'year'`, `'week'`, `'day'`, `'hour'`. You used it in chapter 21 to group orders by month. Add `::date` when you only want the day part.

Pulling a part out: `extract`

```
SELECT extract(year from date '2026-09-26') AS yr,
       extract(month from date '2026-09-26') AS mon,
       extract(dow from date '2026-09-26') AS dow;
```

yr	mon	dow
2026	9	6

`dow` is day of week, 0 = Sunday, so 6 is Saturday. `extract(epoch from interval)` turns an interval into seconds, which is how you get "hours to ship" as a plain number:

```
SELECT id, shipped_at - ordered_at AS time_to_ship,
       round(extract(epoch from shipped_at - ordered_at) / 3600) AS hours
FROM orders WHERE shipped_at IS NOT NULL ORDER BY id LIMIT 3;
```

id	time_to_ship	hours
1	1 day 22:45:00	47
2	1 day 21:00:00	45
3	2 days 00:55:00	49

Formatting for humans: `to_char`

```
SELECT to_char(date '2026-09-26', 'DD Mon YYYY') AS a,
       to_char(date '2026-09-26', 'FMDay') AS b,
       to_char(timestampz '2026-09-26 18:05+05:45', 'HH24:MI') AS c,
       to_char(date '2026-09-26', 'YYYY-MM') AS d;
```

a	b	c	d
26 Sep 2026	Saturday	18:05	2026-09

The pattern letters: `YYYY` year, `MM` month number, `Mon` short month name, `Month` full name, `DD` day, `Dy / Day` weekday, `HH24:MI` 24-hour time. `FM` in front strips padding spaces. `to_char` works on numbers too:

`to_char(1234567.891, 'FM999,999,990.00')` gives `1,234,567.89`.

`to_char(ordered_at, 'YYYY-MM')` is a popular alternative to `date_trunc` for grouping by month, because the label is already readable. Either works.

Going the other way: `to_date`, `make_date`

```
SELECT to_date('26/09/2026', 'DD/MM/YYYY') AS parsed, make_date(2026, 12, 25) AS xmas;
```

parsed	xmas
2026-09-26	2026-12-25

`to_date` is how you import dates written in some other format. Tell it the pattern once, and it never guesses.

Differences: `age`

```
SELECT age(date '2026-09-30', date '2000-01-15') AS age;
```

age
26 years 8 mons 15 days

`age(later, earlier)` gives a human-friendly interval. Subtracting dates gives plain days; `age` gives years, months, and days.

Arithmetic, recap from chapter 12:

```
SELECT date '2026-09-30' + 7 AS next_week,
       (date '2026-09-30' + interval '1 month')::date AS next_month;
```

```
next_week | next_month
-----+-----
2026-10-07 | 2026-10-30
```

CASE : if this, then that

`CASE` lets you compute a value based on conditions, row by row. Price bands:

```
SELECT name, price,
       CASE
         WHEN price < 5 THEN 'cheap'
         WHEN price < 50 THEN 'mid'
         ELSE 'premium'
       END AS tier
FROM products ORDER BY price;
```

name	price	tier
Pen	1.20	cheap
Whiteboard marker	1.80	cheap
Sticky notes	2.75	cheap
Notebook	3.90	cheap
Stapler	8.00	mid
...		
Backpack	45.00	mid
Office chair	149.00	premium

(11 rows)

Conditions are checked top to bottom; the first true one wins. `ELSE` catches everything else. Without an `ELSE`, unmatched rows get `NULL`.

`CASE` is an expression, so it goes anywhere a value goes. In a `GROUP BY`:

```
SELECT CASE WHEN status IN ('shipped', 'cancelled') THEN 'closed' ELSE 'open' END AS state,
       count(*)
FROM orders GROUP BY state ORDER BY state;
```

state	count
closed	7
open	5

Inside a `sum()`, to count conditionally (the portable version of `FILTER`):

```
SELECT sum(CASE WHEN status = 'shipped' THEN 1 ELSE 0 END) AS shipped FROM orders;
```

In an `ORDER BY`, to put one value first: `ORDER BY CASE WHEN status = 'pending' THEN 0 ELSE 1 END, ordered_at`.

COALESCE : the first thing that isn't NULL

```
SELECT id, status, coalesce(shipped_at::date::text, 'not shipped') AS shipped
FROM orders ORDER BY id LIMIT 5;
```

id	status	shipped
1	paid	2026-07-05
2	shipped	2026-07-12
3	shipped	2026-07-20
4	cancelled	not shipped
5	shipped	2026-08-04

`coalesce(a, b, c, ...)` returns the first argument that isn't NULL. Three everyday uses:

- **Display:** `coalesce(author, 'Unknown'), coalesce(phone, 'no phone')`.
- **Maths:** `coalesce(sum(x), 0)` so an empty total is `0`, not NULL ([chapter 23](#)).
- **Fallbacks:** `coalesce(nickname, first_name, 'friend')`.

All arguments must be the same type, which is why `shipped_at` was cast to text above before being paired with `'not shipped'`.

NULLIF : turn a value into NULL

The opposite tool. `nullif(a, b)` returns NULL if `a` equals `b`, otherwise `a`. Its classic job is stopping division by zero:

```
SELECT 10 / nullif(0, 0) AS safe_division;
```

```
safe_division
-----
[NULL]
```

`10 / 0` is an error that kills the whole query. `10 / nullif(x, 0)` gives NULL for that row instead and lets the rest through. Also handy for treating empty strings as missing: `nullif(trim(notes), '')`.

Finding more

- In psql, `\df upper` shows a function's signatures; `\df *date*` lists everything with "date" in the name.
- The official docs chapter "Functions and Operators" is the full catalogue. It's long. Skim the section headings, and search when you need something.
- Ask Claude "is there a PostgreSQL function that...". There nearly always is.

Common mistakes

1. `||` with a column that can be NULL

The whole string goes NULL. Use `concat()`, or wrap the column in `coalesce()`.

2. Positions starting at 0

SQL counts from 1. `substring('abc' from 1 for 1)` is `'a'`.

3. `max()` when you mean `greatest()`

`max(a, b)` is an error. Across rows is `max()`; within a row is `greatest()`.

4. `CASE` without `ELSE`

Rows that match nothing become NULL, often silently. Add an `ELSE` unless NULL is what you want.

5. Mismatched types in `coalesce`

`coalesce(shipped_at, 'not shipped')` fails: a timestamp and a text can't share a column. Cast first.

6. Guessing `to_char` patterns

`MM` is month, `MI` is minutes, `mm` is also month (case only affects *names* like `Mon / MON`). When a date comes out strange, check the pattern letters.

Quick recap

- **Text:** `lower / upper`, `trim`, `length`, `concat / concat_ws` (NULL-safe), `left / right / substring`, `replace`, `split_part`, `lpad`.
- **Numbers:** `round(x, n)`, `ceil`, `floor`, `trunc`, `abs`, `%`, `greatest / least` (within a row).
- **Dates:** `date_trunc` to chop, `extract` to pull a part out, `to_char` to format, `to_date` to parse, `age` for human differences.
- `CASE WHEN ... THEN ... ELSE ... END` computes a value from conditions, anywhere a value can go.
- `coalesce(a, b)` is the first non-NULL. `nullif(a, b)` makes NULL on purpose, especially to avoid dividing by zero.

- `\df name` and the docs for everything else.

Next: try the [exercises](#), then move on to [25 Subqueries](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
- Save each answer in `playground/ch24/ex1.sql`, `ex2.sql`, and so on.
- When you're not sure what a function does, try it on a literal first: `SELECT split_part('a-b-c', '-', 2);`
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Text clean-up

1. Show each customer's name and their **initials** (first letter of each of the two names), like `AR` for Asha Rai.
2. Show each customer's email **domain** (the part after `@`), and how many customers share each domain.
3. Show each product's name in **UPPERCASE** and its length, the three longest names first.

Expected output of query 3:

```
      name      | len
-----+-----
WHITEBOARD MARKER | 17
WIRELESS MOUSE   | 14
LAPTOP SLEEVE    | 13
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Dates for humans

For the first four orders (by id), show the order id, the date as `26 Sep 2026`-style text, the month as `Jul 2026`-style text, and the weekday as `Fri`.

Expected output:

id	day	month_label	weekday
1	03 Jul 2026	Jul 2026	Fri
2	10 Jul 2026	Jul 2026	Fri
3	18 Jul 2026	Jul 2026	Sat
4	25 Jul 2026	Jul 2026	Sat

(4 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Price bands and status groups

1. Label every product `cheap` (under 5), `mid` (5 to under 50), or `premium` (50 and up), then **count how many products are in each band**.
2. Group orders into `closed` (shipped or cancelled) and `open` (everything else), with a count of each.
3. Without `FILTER`, using `sum(CASE ...)`, show in one row: how many orders are shipped, and how many aren't.

Expected output of query 1:

tier	products
cheap	4
mid	6
premium	1

(3 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Filling in blanks

1. List every order's id, status, and shipped date, showing the text `not shipped` where there's no date.
2. For every order that **has** shipped, show the id and the time to ship in **whole hours**.
3. Every customer's name and city, but show `(local)` instead of the city for customers in Kathmandu. Use `nullif` and `coalesce` together, not `CASE`.

Expected first rows of query 2:

```

id | hours
----+-----
1  |    47
2  |    45
3  |    49
5  |    51
...

```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A monthly report, nicely formatted

Produce one row per month with: the month as `Jul 2026`-style text, the number of orders, the number shipped, and the **percentage shipped** rounded to one decimal place and shown with a `%` sign, like `50.0%`.

Expected output:

```

month | orders | shipped | pct
-----+-----+-----+-----
Jul 2026 |    4 |    2 | 50.0%
Aug 2026 |    4 |    3 | 75.0%
Sep 2026 |    4 |    1 | 25.0%
(3 rows)

```

Two traps: the months must come out in **date order**, not alphabetical (Aug, Jul, Sep would be wrong), and the division must not be integer division.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

25 Subqueries

What is it?

A **subquery** is a query inside another query. You put a whole `SELECT` in brackets and use its result as if it were a value, a list, or a table.

```
SELECT name, price FROM products
WHERE price > (SELECT avg(price) FROM products);
```

The inner query works out the average. The outer query uses it. One statement, two steps.

Why does it matter?

Some questions have two parts: "find the average, then find the products above it." "Find the customers who have orders, then list everyone *else*." "Total each order, then average the totals." Until now you'd run the first query, read the answer, and type it into the second. A subquery lets the database do both parts in one go, and the answer stays right when the data changes.

Subqueries also give you new ways to ask about existence ("customers who have *any* cancelled order") and absence ("products that appear in *no* order"), which are often clearer than the outer-join tricks of chapter 23.

Real-world example

Asking the librarian a two-part question:

You say	SQL
"What's the average page count?" (pause) "OK, show me every book longer than that."	<code>WHERE pages > (SELECT avg(pages) ...)</code>
"Which members have overdue books? Right, send <i>them</i> a letter."	<code>WHERE id IN (SELECT member_id FROM loans WHERE ...)</code>
"Is there <i>any</i> loan for this book? If so, don't put it in the sale."	<code>WHERE EXISTS (SELECT 1 FROM loans WHERE ...)</code>

The bracketed part is you waiting for the first answer before asking the second.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

A subquery as a single value

When a subquery returns exactly one row and one column, you can use it anywhere a value goes:

```
SELECT name, price
FROM products
WHERE price > (SELECT avg(price) FROM products)
ORDER BY price DESC;
```

name	price
Office chair	149.00
Backpack	45.00
USB hub	29.99

(3 rows)

The average is 27.44, and three products beat it. This is called a **scalar subquery** (scalar means "a single value"). If the data changes, so does the average, and the query stays right.

A scalar subquery works in `SELECT` too:

```
SELECT name, price, round(price - (SELECT avg(price) FROM products), 2) AS vs_average
FROM products
ORDER BY vs_average DESC
LIMIT 4;
```

name	price	vs_average
Office chair	149.00	121.56
Backpack	45.00	17.56
USB hub	29.99	2.55
Desk lamp	24.99	-2.45

(4 rows)

If a scalar subquery returns more than one row, PostgreSQL refuses:

```
ERROR: more than one row returned by a subquery used as an expression
```

That error means "you used a list where a single value was needed". Either add `LIMIT 1` (if any one will do), make the subquery an aggregate, or switch to `IN`.

A subquery as a list: `IN`

When the inner query returns many rows of one column, use it with `IN`:

```
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE status = 'pending')
ORDER BY name;
```

```
name
-----
Asha Rai
Farhan Ali
(2 rows)
```

"Customers whose id is in the list of customer ids with a pending order." Read it from the inside out: the inner query makes the list, the outer picks from it.

`NOT IN` gives the opposite:

```
SELECT name
FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);
```

```
name
-----
Elina Maharjan
(1 row)
```

The chapter 23 "never ordered" question, without a join. Many people find this version easier to read.

The `NOT IN` trap. If the inner list contains even one `NULL`, `NOT IN` returns *nothing*, with no error. Why? `id NOT IN (1, 2, NULL)` means `id <> 1 AND id <> 2 AND id <> NULL`, and that last part is unknown, which poisons the whole condition. Here `customer_id` is `NOT NULL`, so we're safe. When the column can be `NULL`, use `NOT EXISTS` instead. That's next.

Does one exist? EXISTS

EXISTS (subquery) is true if the subquery returns at least one row. It doesn't care *what* the rows are, only whether there are any:

```
SELECT c.name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id AND o.status = 'cancelled');
```

```
name
-----
Asha Rai
(1 row)
```

"Customers for whom there exists at least one cancelled order." The `SELECT 1` is a convention: since only existence matters, select the simplest thing.

Notice the inner query mentions `c.id`, a column from the **outer** query. That makes it a **correlated subquery**: it's re-run for each customer, with that customer's id plugged in. More on that below.

`NOT EXISTS` is the safest way to ask "has none":

```
SELECT p.name
FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.id);
```

```
name
-----
Whiteboard marker
(1 row)
```

Products with no order lines at all. Unlike `NOT IN`, `NOT EXISTS` is never confused by NULLs, so prefer it for "has none" questions.

Correlated subqueries

A correlated subquery refers to the current row of the outer query. Each product next to its own category's average:


```

SELECT p.name, p.price,
       (SELECT round(avg(p2.price), 2) FROM products p2 WHERE p2.category_id = p.category_id) AS
       category_avg
FROM products p
ORDER BY p.category_id, p.price DESC;

```

name	price	category_avg
Stapler	8.00	3.53
Notebook	3.90	3.53
Sticky notes	2.75	3.53
Whiteboard marker	1.80	3.53
Pen	1.20	3.53
Office chair	149.00	87.00
Desk lamp	24.99	87.00
...		

(11 rows)

For each product `p`, the inner query averages the products `p2` in the same category. Same table, two aliases, like the self-join in chapter 22. Then filter on it to find products above their category's average:

```

SELECT p.name, p.price
FROM products p
WHERE p.price > (SELECT avg(p2.price) FROM products p2 WHERE p2.category_id = p.category_id)
ORDER BY p.category_id;

```

name	price
Notebook	3.90
Stapler	8.00
Office chair	149.00
Backpack	45.00
USB hub	29.99

(5 rows)

Correlated subqueries are easy to read but can be slow on big tables, because the inner query runs once per outer row. [Chapter 28](#) shows a faster way to do "compare to the group" calculations.

A subquery as a table: in FROM

A subquery can stand in for a table. Average order value: first total each order, then average the totals.

```
SELECT round(avg(total), 2) AS avg_order_value
FROM (
  SELECT order_id, sum(quantity * unit_price) AS total
  FROM order_items
  GROUP BY order_id
) AS order_totals;
```

```
avg_order_value
-----
              59.10
(1 row)
```

The inner query is the chapter 21 "totals per order" result. The outer query treats it as a table called `order_totals` and averages its `total` column. This is called a **derived table**.

You can't write `avg(sum(...))` directly; aggregates don't nest. Two steps is the honest way, and a derived table is how you write two steps in one statement.

Give derived tables a name (`AS order_totals`). Older PostgreSQL versions required it, and even where it's optional, a name tells the reader what the inner result *is*.

Subqueries in `INSERT`, `UPDATE`, `DELETE`

Remember chapter 17, where you typed a product's price into an order item by hand? Let the database look it up:

```
BEGIN;
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (12, 2, 4, (SELECT price FROM products WHERE id = 2))
RETURNING *;
ROLLBACK;
```

```
order_id | product_id | quantity | unit_price
-----+-----+-----+-----
      12 |          2 |         4 |        1.20
```

The scalar subquery fetches the current price at the moment of insert: a proper snapshot. (The `ROLLBACK` keeps the dataset unchanged for later chapters.)

`UPDATE` and `DELETE` take subqueries in their `WHERE` too. Deactivate products that have never sold:

```
BEGIN;
UPDATE products SET is_active = false
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = products.id)
RETURNING name, is_active;
ROLLBACK;
```

```
name      | is_active
-----+-----
Whiteboard marker | f
```

Inside `UPDATE products`, there's no alias, so the outer row is referred to as `products.id`.

ANY and ALL

Two comparison helpers you'll see occasionally. `> ALL (list)` means "bigger than every value"; `> ANY (list)` means "bigger than at least one". Products priced above every Stationery item:

```
SELECT name, price FROM products
WHERE price > ALL (SELECT price FROM products WHERE category_id = 1)
ORDER BY price;
```

Six rows, from the Wireless mouse up. `= ANY (...)` is exactly `IN (...)`. These read well once in a while; most people reach for `IN`, `EXISTS`, or `max()` instead.

Join or subquery?

Often both work. Rough guide:

You want	Reach for
Columns from both tables in the result	A join
To filter one table by what's in another, showing only the first	<code>IN</code> or <code>EXISTS</code>
"Has none"	<code>NOT EXISTS</code>
A single computed number to compare against	A scalar subquery
To aggregate an aggregate	A derived table (or a CTE, next chapter)

When a query gets long, the next chapter's **CTEs** let you name each step. They're subqueries you can read top to bottom.

Common mistakes

1. A list where a value was needed

`WHERE id = (SELECT customer_id FROM orders)` fails with "more than one row". Use `IN`, or make the subquery return one row.

2. `NOT IN` with NULLs

Returns nothing, silently. Use `NOT EXISTS` when the inner column can be NULL.

3. Forgetting the outer reference in a correlated subquery

`WHERE price > (SELECT avg(price) FROM products WHERE category_id = category_id)` compares the column to itself: always true. Use aliases: `p2.category_id = p.category_id`.

4. Selecting real columns in `EXISTS`

`EXISTS (SELECT o.* ...)` works but misleads. `SELECT 1` says "I only care whether rows exist".

5. Nesting aggregates

`avg(sum(x))` is an error. Sum in a derived table, average outside.

6. Unreadable nesting

Three subqueries deep is hard for everyone. The next chapter fixes that.

Quick recap

- A subquery is a `SELECT` in brackets, used as a **value** (one row, one column), a **list** (`IN`), a **test** (`EXISTS`), or a **table** (in `FROM`).
- Scalar subqueries compare against a computed number: `WHERE price > (SELECT avg(price) ...)`.
- `IN (subquery)` picks from a list. `NOT IN` breaks if the list has a NULL; prefer `NOT EXISTS`.
- `EXISTS` / `NOT EXISTS` ask "is there any?" / "is there none?", usually correlated to the outer row.
- A derived table in `FROM` lets you aggregate an aggregate. Name it.
- Subqueries work inside `INSERT`, `UPDATE`, and `DELETE` too: look up the price, deactivate the unsold.

Next: try the [exercises](#), then move on to [26 Common Table Expressions](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
- Save each answer in `playground/ch25/ex1.sql`, `ex2.sql`, and so on.
- Any exercise that changes data must be wrapped in `BEGIN; ... ROLLBACK;`, so the dataset stays the same for the next chapters.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Compared to the average

1. Products that cost **less** than the average product price, cheapest first.
2. Orders whose total is **above the average order total**. Show order id and total, biggest first. (Total each order in a derived table, then compare against the average of another derived table.)

Expected output of query 2:

```
order_id | total
-----+-----
      8 | 173.99
      3 | 149.00
      9 |  90.00
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): `IN` and `NOT IN`

1. Names of customers who have at least one **shipped** order.
2. Names of products that have appeared in a **cancelled** order.
3. Names of customers who have bought anything from the **Stationery** category. (The inner query will need a join.)

Expected output of query 3:

```

      name
-----
Asha Rai
Bikram Shrestha
Dipesh Thapa
Farhan Ali
(4 rows)

```

Exercise 3 (Medium): EXISTS and NOT EXISTS

1. Rewrite "customers who have never ordered" with `NOT EXISTS`.
2. Customers for whom **every** order has shipped. (They must have at least one order, and no order that isn't shipped.) Expected: Bikram Shrestha and Chandra Gurung.
3. In a comment: why is `NOT EXISTS` safer than `NOT IN` for question 1, even though both give the same answer here?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Correlated

1. Each product with its price and its **category's average price**, in one query, no join.
2. Only the products priced **above** their own category's average.
3. Each customer's name and their **number of orders**, using a correlated scalar subquery in the `SELECT` (not a `GROUP BY`). Elina should show `0`.

Expected output of query 2:

```

      name      | price
-----+-----
Notebook       |   3.90
Stapler        |   8.00
Office chair   | 149.00
Backpack       |  45.00
USB hub        |  29.99
(5 rows)

```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Subqueries that change data

All inside `BEGIN; ... ROLLBACK;` .

1. Add a line to order 12: 4 Pens, with the unit price **looked up** from `products` , not typed. Use `RETURNING *` .
2. Deactivate every product that has never been sold, with `RETURNING name` .
3. Delete every order item belonging to a **cancelled** order, with `RETURNING *` . How many rows, and what was their value? (Use a nested `IN` : items whose `order_id` is in the list of cancelled order ids.)
4. Check with `SELECT count(*) FROM order_items;` that you still have 21 rows after the `ROLLBACK` .

Stuck? The hint is in the [Hints section](#) at the end.

26 Common Table Expressions

What is it?

A **common table expression** (everyone says **CTE**) is a named, temporary result that lives for one query. You define it at the top with `WITH`, then use it below as if it were a table:

```
WITH order_totals AS (  
  SELECT order_id, sum(quantity * unit_price) AS total  
  FROM order_items  
  GROUP BY order_id  
)  
SELECT * FROM order_totals WHERE total > 50;
```

It's a subquery that you've pulled out, given a name, and put first.

Why does it matter?

Chapter 25 ended with queries that were getting hard to read: a derived table inside a `FROM`, with another copy of the same derived table inside a scalar subquery. CTEs fix that in three ways:

1. **You read top to bottom.** Step one, step two, step three, final answer. Not inside-out.
2. **You write each step once** and reuse it by name.
3. **You can test each step** by itself: `WITH ... SELECT * FROM step_one; .`

Most professional SQL for reports is written as a chain of CTEs. It's the difference between a recipe with numbered steps and one long sentence.

Real-world example

A recipe:

1. **Sauce:** simmer tomatoes, garlic, and oil for 20 minutes.
2. **Pasta:** boil until al dente.
3. Combine the sauce and the pasta. Serve.

Each named step is a CTE. Step 3 refers to steps 1 and 2 by name. You could write the whole thing as one tangled sentence ("combine pasta that has been boiled until al dente with tomatoes that have been simmered with..."), and that's a nested subquery. Nobody wants to cook from that.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

One CTE

Order totals with customer names, biggest first:

```
WITH order_totals AS (  
  SELECT order_id, sum(quantity * unit_price) AS total  
  FROM order_items  
  GROUP BY order_id  
)  
SELECT o.id, c.name, o.status, ot.total  
FROM orders o  
JOIN customers c ON c.id = o.customer_id  
JOIN order_totals ot ON ot.order_id = o.id  
ORDER BY ot.total DESC  
LIMIT 5;
```

id	name	status	total
8	Asha Rai	shipped	173.99
3	Chandra Gurung	shipped	149.00
9	Chandra Gurung	shipped	90.00
5	Dipesh Thapa	shipped	56.24
4	Asha Rai	cancelled	45.00

(5 rows)

Step one: total each order (`order_totals`). Step two: join it to orders and customers. The main query reads like any other join, because `order_totals` behaves like a table.

Compare this with the chapter 22 version, which joined `order_items` directly and needed `count(DISTINCT o.id)` to avoid counting lines. Totalling first, in a CTE, sidesteps the whole trap: by the time you join, there's one row per order.

Several CTEs

Separate them with commas. Later ones can use earlier ones:

```

WITH order_totals AS (
    SELECT order_id, sum(quantity * unit_price) AS total
    FROM order_items
    GROUP BY order_id
),
customer_totals AS (
    SELECT o.customer_id, count(*) AS orders, sum(ot.total) AS spent
    FROM orders o
    JOIN order_totals ot ON ot.order_id = o.id
    WHERE o.status <> 'cancelled'
    GROUP BY o.customer_id
)
SELECT c.name, ct.orders, ct.spent
FROM customers c
JOIN customer_totals ct ON ct.customer_id = c.id
ORDER BY ct.spent DESC;

```

name	orders	spent
Chandra Gurung	2	239.00
Asha Rai	3	218.19
Dipesh Thapa	2	86.23
Farhan Ali	2	64.75
Bikram Shrestha	2	55.99

(5 rows)

Three clear steps: total the orders, total per customer, attach names. Each step is small enough to understand on its own, and `count(*)` is correct in step two because `order_totals` has one row per order.

Only one `WITH` at the top, then `name AS (...)`, comma, `name AS (...)`, and finally the main `SELECT` with no comma before it.

The "aggregate an aggregate" problem, solved

Chapter 25's average order value, as a CTE:

```

WITH order_totals AS (
    SELECT order_id, sum(quantity * unit_price) AS total
    FROM order_items
    GROUP BY order_id
)
SELECT round(avg(total), 2) AS avg_order, max(total) AS biggest, min(total) AS smallest
FROM order_totals;

```

avg_order	biggest	smallest
59.10	173.99	13.00

(1 row)

Same answer as the derived table, and now you can get three aggregates from the step without repeating it.

Testing a step

While building a long query, check the intermediate results:

```
WITH order_totals AS (
  SELECT order_id, sum(quantity * unit_price) AS total FROM order_items GROUP BY order_id
)
SELECT * FROM order_totals ORDER BY order_id;
```

Twelve rows, one per order, totals you recognize from chapter 21. Good. Now add the next step. Building queries this way, one verified step at a time, is how you avoid "the final number is wrong and I don't know which part did it".

Filling in the gaps: a calendar CTE

`GROUP BY month` only shows months that had orders (chapter 21). To show every month, including empty ones, generate the months first and left-join the data onto them:

```
WITH months AS (
  SELECT generate_series(date '2026-06-01', date '2026-10-01', interval '1 month')::date AS
    month
),
monthly AS (
  SELECT date_trunc('month', ordered_at)::date AS month, count(*) AS orders
  FROM orders
  GROUP BY 1
)
SELECT m.month, coalesce(mo.orders, 0) AS orders
FROM months m
LEFT JOIN monthly mo ON mo.month = m.month
ORDER BY m.month;
```

month	orders
2026-06-01	0
2026-07-01	4
2026-08-01	4
2026-09-01	4
2026-10-01	0

(5 rows)

`generate_series(start, stop, step)` is a built-in that produces a list of values: numbers or dates. It's the standard way to make a "calendar" to join against. June and October show zero instead of vanishing. This pattern appears in nearly every dashboard.

Recursive CTEs

A CTE can refer to **itself**. That lets a query repeat until it runs out of rows: counting, walking a tree, following a chain. Start with the smallest possible example:

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n           -- the starting row
  UNION ALL
  SELECT n + 1 FROM numbers -- each round: take the previous rows and add one
  WHERE n < 5             -- until this stops being true
)
SELECT n FROM numbers;
```

n
1
2
3
4
5

(5 rows)

Two halves joined by `UNION ALL` ([chapter 27](#)): the starting row, and a step that builds on what's there so far. The `WHERE` is the brake. Forget it and the query runs forever (press Ctrl+C).

For plain numbers and dates, `generate_series` is simpler. Recursion earns its keep on **hierarchies**: an org chart, nested categories, folders inside folders. Using the employees from chapter 22, plus one more level:

```

CREATE TEMP TABLE employees (id integer PRIMARY KEY, name text, manager_id integer REFERENCES
    employees (id));
INSERT INTO employees VALUES
    (1, 'Maya Rai', NULL), (2, 'Dev Shrestha', 1), (3, 'Asha Thapa', 2),
    (4, 'Bikram Karki', 2), (5, 'Sita Gurung', 1), (6, 'Ram Lama', 3);

WITH RECURSIVE chain AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL           -- start at the top
    UNION ALL
    SELECT e.id, e.name, e.manager_id, chain.level + 1
    FROM employees e
    JOIN chain ON chain.id = e.manager_id -- each round: the people who report to the last
        round
)
SELECT repeat(' ', level - 1) || name AS org_chart, level
FROM chain
ORDER BY level, name;

```

org_chart	level
Maya Rai	1
Dev Shrestha	2
Sita Gurung	2
Asha Thapa	3
Bikram Karki	3
Ram Lama	4

(6 rows)

Round one finds Maya (no manager). Round two finds everyone who reports to Maya. Round three finds their reports, and so on until a round finds nobody. A plain join can only go a fixed number of levels; recursion goes as deep as the data does.

"Everyone under Dev, however far down" is the same shape with a different starting row (WHERE name = 'Dev Shrestha'), and gives Asha, Bikram, and Ram.

You won't write recursive CTEs often. But when you need one, nothing else will do, and now you'll recognize the shape.

CTEs that change data

INSERT , UPDATE , and DELETE can live inside a CTE too, with RETURNING feeding the rows to the main query. Remove the lines of cancelled orders and report what went:

```

BEGIN;
WITH cancelled_items AS (
  DELETE FROM order_items
  WHERE order_id IN (SELECT id FROM orders WHERE status = 'cancelled')
  RETURNING *
)
SELECT count(*) AS removed_lines, sum(quantity * unit_price) AS removed_value
FROM cancelled_items;
ROLLBACK;

```

removed_lines	removed_value
1	45.00

(1 row)

Delete and summarize in one statement. Handy for migrations and clean-up scripts.

A note on performance

PostgreSQL usually treats a CTE like a subquery: it can merge it into the main query and optimize the whole thing. If you ever need the opposite (compute the CTE once and store it, for example when the CTE is used several times and is expensive), write `WITH name AS MATERIALIZED (...)`. You won't need this for a long time. Write CTEs for clarity and let PostgreSQL worry about speed.

Common mistakes

1. A comma before the main `SELECT`

```

WITH a AS (...), b AS (...),
SELECT ...

```

Syntax error. Commas go between CTEs, not after the last one.

2. Forgetting `RECURSIVE`

A CTE that refers to itself without the keyword gives `relation "numbers" does not exist`. If it recurses, say `WITH RECURSIVE`.

3. No brake on a recursive CTE

Without a `WHERE` in the recursive half, it never ends. Ctrl+C, add the condition.

4. Rewriting a simple query as five CTEs

A query that was clear as one `SELECT` doesn't get clearer with a `WITH`. Use CTEs when there are real steps.

5. Using a CTE in a different statement

A CTE lives for one statement. `WITH t AS (...) SELECT ...;` then `SELECT * FROM t;` fails. If you need a result across statements, that's a view ([chapter 33](#)) or a temp table.

6. `count(*)` after joining line-level data

The chapter 22 trap. The CTE way around it: total per order first, then join and count.

Quick recap

- `WITH name AS (SELECT ...) SELECT ... FROM name` names a step and uses it like a table. Read top to bottom.
 - Chain steps with commas; later CTEs can use earlier ones. No comma before the final `SELECT`.
 - CTEs solve "aggregate an aggregate" and the join-and-count trap cleanly: total first, then join.
 - Test each step with `SELECT * FROM step` before adding the next.
 - `generate_series` + `LEFT JOIN` fills in missing months or days.
 - `WITH RECURSIVE` repeats until no rows are left: for hierarchies like org charts and nested folders. Always include a brake.
 - A CTE can wrap `DELETE / UPDATE / INSERT ... RETURNING` and report on what changed.
-

Next: try the [exercises](#), then move on to [27 Combining Results](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
 - Save each answer in `playground/ch26/ex1.sql`, `ex2.sql`, and so on.
 - Build each query one CTE at a time, checking each step with `SELECT * FROM step` before moving on.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Rewrite with a CTE

Take chapter 25's "orders above the average order total" (Exercise 1, question 2). Rewrite it so the "totals per order" query appears **once**, as a CTE, and is used twice in the main query.

Expected output:

```
order_id | total
-----+-----
      8 | 173.99
      3 | 149.00
      9 |  90.00
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Monthly revenue

Using an `order_totals` CTE, show for each month: the number of orders, and the revenue, **excluding cancelled orders**. In month order.

Expected output:

```
month      | orders | revenue
-----+-----+-----
2026-07-01 |      3 | 194.99
2026-08-01 |      4 | 288.48
2026-09-01 |      4 | 180.69
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Share of revenue

Show each category's revenue (excluding cancelled orders) and its **percentage of total revenue**, rounded to one decimal place, biggest first.

Expected output:

category	revenue	pct
Furniture	347.98	52.4
Bags	129.00	19.4
Stationery	95.70	14.4
Electronics	91.48	13.8

(4 rows)

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Every month, even the empty ones

1. Build a `months` CTE with `generate_series` from June to October 2026. Left-join the monthly order counts onto it so June and October show `0`.
2. Extend it: add a second data column, revenue excluding cancelled orders, also showing `0.00` for empty months.

Expected output of query 1:

month	orders
2026-06-01	0
2026-07-01	4
2026-08-01	4
2026-09-01	4
2026-10-01	0

(5 rows)

Exercise 5 (Challenge): Recursion

1. Build the `employees` temp table from the notes. Write a recursive CTE that lists **everyone who reports to Dev Shrestha, directly or indirectly**, alphabetical. Expected: Asha Thapa, Bikram Karki, Ram Lama.
2. Write a recursive CTE that produces every date from `2026-10-01` to `2026-10-05`, with the weekday name next to each. Then write a comment: why would `generate_series` be the better tool for this one?
3. **Repeat customers:** using a CTE that finds each customer's first order date, show each customer's name and how many orders they placed **after** their first one. Most repeat orders first.

Expected output of query 3:

name	later_orders
Asha Rai	3
Bikram Shrestha	1
Chandra Gurung	1
Dipesh Thapa	1
Farhan Ali	1

(5 rows)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

27 Combining Results

What is it?

Joins combine tables **side by side**: more columns. The three operators in this chapter combine query results **top to bottom**: more rows.

Operator	Gives you
UNION ALL	Every row from both queries, stacked
UNION	The same, with duplicate rows removed
INTERSECT	Only rows that appear in both results
EXCEPT	Rows in the first result that are not in the second

They're called **set operations**, because they treat each result as a set of rows and combine the sets.

Why does it matter?

Some questions are about two lists:

- "Give me every email address we have, from customers *and* newsletter subscribers, each once." (UNION)
- "Which customers ordered in July *and* in August?" (INTERSECT)
- "Which products were sold in August but *not* in July?" (EXCEPT)
- "Add a TOTAL row to the bottom of this report." (UNION ALL)

You can often get the same answers with joins or EXISTS . But when the question is naturally "this list and that list", the set operators say exactly what you mean, in the order you'd say it.

Real-world example

Two guest lists, for a wedding and for a birthday party:

You want	Operator
One combined mailing list, each person once	UNION
The combined list, even if some people appear twice	UNION ALL
People invited to both	INTERSECT

You want	Operator
People invited to the wedding but not the birthday	EXCEPT

Same two lists, four different questions.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

UNION ALL : stack two results

```
SELECT name, 'customer' AS kind FROM customers
UNION ALL
SELECT name, 'category' FROM categories
ORDER BY kind, name;
```

name	kind
Bags	category
Electronics	category
Furniture	category
Stationery	category
Asha Rai	customer
Bikram Shrestha	customer
...	

(10 rows)

Six customers plus four categories, ten rows. The second query's rows are appended under the first's.

Three rules, which apply to all four operators:

1. **Same number of columns.** `SELECT name, email ... UNION SELECT name ...` fails with `each UNION query must have the same number of columns`.
2. **Matching types, position by position.** `SELECT name ... UNION SELECT id ...` fails with `UNION types text and integer cannot be matched`.
3. **Column names come from the first query.** The second query's names are ignored.

The `'customer' AS kind` column is a habit worth copying: a literal label so you can tell which rows came from where. Without it, the stacked rows are anonymous.

UNION : stack and remove duplicates

```
SELECT city FROM customers
UNION ALL
SELECT city FROM customers;
```

12 rows: every city, twice. Now with UNION :

```
SELECT city FROM customers
UNION
SELECT city FROM customers
ORDER BY city;
```

```
city
-----
Kathmandu
Lalitpur
Pokhara
(3 rows)
```

UNION removes **every** duplicate row, including duplicates that were already inside one of the queries. It's UNION ALL followed by DISTINCT .

Which to use? UNION ALL **unless you specifically need duplicates removed**. It's faster (no sorting to find duplicates), and it never silently merges two rows that happened to look the same but meant different things.

ORDER BY and LIMIT with set operations

An ORDER BY at the end applies to the **combined** result. You can't put one inside a part:

```
SELECT name FROM customers ORDER BY name
UNION
SELECT name FROM categories;
```

```
ERROR:  syntax error at or near "UNION"
```

If you want a LIMIT or ORDER BY on just one part (say, the two most expensive and the two cheapest products), wrap that part in brackets:

```
(SELECT name, price FROM products ORDER BY price DESC LIMIT 2)
UNION ALL
(SELECT name, price FROM products ORDER BY price ASC LIMIT 2);
```

name	price
Office chair	149.00
Backpack	45.00
Pen	1.20
Whiteboard marker	1.80

(4 rows)

INTERSECT : in both

Customers who ordered in July **and** in August:

```
SELECT customer_id FROM orders WHERE ordered_at >= '2026-07-01' AND ordered_at < '2026-08-01'
INTERSECT
SELECT customer_id FROM orders WHERE ordered_at >= '2026-08-01' AND ordered_at < '2026-09-01'
ORDER BY customer_id;
```

customer_id
1
2

(2 rows)

Asha and Bikram. The first query lists July's customers, the second August's, and `INTERSECT` keeps the ids on both lists. Say it as a sentence and it's exactly the question.

You can chain them. Customers active in **all three** months:

```
SELECT customer_id FROM orders WHERE ordered_at < '2026-08-01'
INTERSECT
SELECT customer_id FROM orders WHERE ordered_at >= '2026-08-01' AND ordered_at < '2026-09-01'
INTERSECT
SELECT customer_id FROM orders WHERE ordered_at >= '2026-09-01';
```

One row: customer 1, Asha.

EXCEPT : in the first, not the second

Customers who ordered in July but **not** in September:

```
SELECT customer_id FROM orders WHERE ordered_at >= '2026-07-01' AND ordered_at < '2026-08-01'
EXCEPT
SELECT customer_id FROM orders WHERE ordered_at >= '2026-09-01'
ORDER BY customer_id;
```

```
customer_id
-----
          2
(1 row)
```

Bikram. Order matters with `EXCEPT` : *first list minus second list*. Swap them and you get a different answer.

`EXCEPT` is a third way to write "never sold", after `LEFT JOIN ... IS NULL` and `NOT EXISTS` :

```
SELECT id FROM products
EXCEPT
SELECT product_id FROM order_items;
```

One row: product 5, the Whiteboard marker. All three ways are correct. Pick the one that reads best for the question in front of you.

Comparing two lists

The newsletter table from chapter 23, with all three operators:

```
CREATE TEMP TABLE newsletter (email text PRIMARY KEY);
INSERT INTO newsletter VALUES ('asha@example.com'), ('elina@example.com'),
    ('guest@example.com');

-- Every address we have, once
SELECT email FROM customers UNION SELECT email FROM newsletter ORDER BY email;      -- 7 rows

-- Customers who also subscribe
SELECT email FROM customers INTERSECT SELECT email FROM newsletter ORDER BY email;  -- asha,
    elina

-- Subscribers who aren't customers
SELECT email FROM newsletter EXCEPT SELECT email FROM customers;                --
    guest@example.com
```

That's the chapter 23 `FULL JOIN` broken into three plain questions, each one line.

A report with a total row

The classic `UNION ALL` trick:

```
SELECT status, count(*) AS orders, 1 AS sort_key FROM orders GROUP BY status
UNION ALL
SELECT 'TOTAL', count(*), 2 FROM orders
ORDER BY sort_key, status;
```

status	orders	sort_key
cancelled	1	1
paid	3	1
pending	2	1
shipped	6	1
TOTAL	12	2

(5 rows)

The `sort_key` column exists only to force `TOTAL` to the bottom (without it, `TOTAL` would sort alphabetically among the statuses). It's a bit ugly in the output; in a real report you'd leave it out of the final `SELECT` by wrapping the whole thing in a CTE. Same idea gives subtotals per category, or a "summary" block at the top of a dashboard query.

Getting names back

Set operations work on whatever columns you give them, and ids are the safest thing to compare. To show names, wrap the operation in a subquery:

```
SELECT name FROM customers
WHERE id IN (
  SELECT customer_id FROM orders WHERE ordered_at < '2026-08-01'
  INTERSECT
  SELECT customer_id FROM orders WHERE ordered_at >= '2026-08-01' AND ordered_at < '2026-09-01'
)
ORDER BY name;
```

Asha Rai, Bikram Shrestha. The set operation makes the id list; `IN` turns it into names.

INTERSECT ALL and EXCEPT ALL

Like `UNION ALL`, these keep duplicates. `EXCEPT ALL` removes one matching row from the first list for each row in the second, instead of removing all of them. Rare. Know they exist.

Set operations or joins?

Question shape	Reach for
"Combine columns from two tables about the same thing"	A join
"One list made from two sources"	<code>UNION ALL</code> / <code>UNION</code>
"Things on both lists"	<code>INTERSECT</code> (or <code>EXISTS</code>)
"Things on this list but not that one"	<code>EXCEPT</code> (or <code>NOT EXISTS</code>)
"Add a total row"	<code>UNION ALL</code>

Common mistakes

1. Different numbers of columns

Count them. Every part must have the same number.

2. Types that don't line up

`SELECT name, id ... UNION SELECT id, name ...` fails. Columns match **by position**, not by name. Put them in the same order.

3. `UNION` when you meant `UNION ALL`

Two genuinely different orders for the same amount on the same day look identical in a `SELECT amount, day` and `UNION` merges them. Default to `UNION ALL`.

4. No label column

Stacked rows with no way to tell them apart. Add `'something' AS kind`.

5. `ORDER BY` inside a part

Syntax error. Order the whole result at the end, or wrap the part in brackets.

6. Swapping the sides of `EXCEPT`

"July minus September" and "September minus July" are different questions. Say the sentence out loud before you write it.

Quick recap

- `UNION ALL` stacks results. `UNION` stacks and removes duplicates. Prefer `UNION ALL` unless duplicates must go.
 - `INTERSECT` keeps rows in both. `EXCEPT` keeps rows in the first but not the second. Order matters for `EXCEPT`.
 - All parts need the **same number of columns** with **matching types, by position**. Names come from the first part.
 - One `ORDER BY` at the very end. Brackets for a per-part `LIMIT`.
 - Add a literal label column so you know where each row came from.
 - Set operations work best on ids; wrap in `IN (...)` to get names.
-

Next: try the [exercises](#), then move on to [28 Window Functions](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
 - Save each answer in `playground/ch27/ex1.sql`, `ex2.sql`, and so on.
 - Before writing each query, say the question as a sentence with "and", "or", or "but not" in it. The sentence tells you which operator.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): One list from two

1. A single list of every product name and every category name, with a `kind` column saying which is which, sorted by kind then name. You should get 15 rows.
 2. Create the `newsletter` temp table from the notes. One list of every email address the shop knows about, each once, alphabetical. You should get 7.
-

Exercise 2 (Easy): UNION or UNION ALL ?

1. Stack `SELECT city FROM customers` on top of itself with `UNION ALL` , then with `UNION` . How many rows each time?
2. In a comment, give a real example (not cities) where `UNION` would quietly throw away a row you needed.
3. Three one-row queries stacked with `UNION ALL` to make a mini summary table with two columns, `what` and `n` : how many orders, customers, and products there are.

Expected output of query 3:

```
  what      | n
-----+-----
orders      | 12
customers   |  6
products    | 11
(3 rows)
```

Exercise 3 (Medium): Both, or one but not the other

1. Ids of customers who ordered in **both** July and August. Then wrap it so you get their **names**.
2. Ids of customers who ordered in July but **not** in September. Expected: just customer 2.
3. **Names** of products sold in **both** July and September.

Expected output of query 3:

```
name
-----
Backpack
Notebook
Pen
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): A report with a total

1. Orders per status, with a `TOTAL` row at the bottom, in that order. Use a sort key column.
2. Now hide the sort key: put the whole thing in a CTE and select only `status` and `orders` from it, still sorted correctly.
3. A **Furniture** price list with a `Subtotal` row: category, item, amount. Two products then the subtotal.

Expected output of query 3:

```
category | item | amount
-----+-----+-----
Furniture | Desk lamp | 24.99
Furniture | Office chair | 149.00
Furniture | Subtotal | 173.99
(3 rows)
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Harder set questions

1. **Names** of products that were sold in **August but not in July**. Expected four: Laptop sleeve, Sticky notes, USB hub, Wireless mouse.
2. The **name** of the one customer who placed an order in every one of the three months.
3. Write "products never sold" three ways: with `EXCEPT`, with `NOT EXISTS`, and with `LEFT JOIN ... IS NULL`. All three must return the Whiteboard marker. Then write a comment on which you find most readable and why.

Stuck? The hint is in the [Hints section](#) at the end.

28 Window Functions

What is it?

A **window function** calculates something for each row using a group of related rows (its "window"), **without collapsing the rows** the way `GROUP BY` does.

```
SELECT name, price, avg(price) OVER () AS avg_price FROM products;
```

Every product row stays. Each one gets the overall average next to it. The magic word is `OVER`.

Why does it matter?

`GROUP BY` answers "what's the average per category?" and gives you one row per category. But many real questions want **each row plus some context**:

- Each product, with its category's average beside it.
- Each order, with its rank among that customer's orders.
- Each month's revenue, with a running total.
- Each month, compared with the month before.
- The top 2 products in *each* category.

Before window functions, these needed correlated subqueries (slow) or horrible self-joins. With them, each is one `OVER (...)`. If you learn one "advanced" SQL feature, make it this one. It's what separates people who can write reports from people who can only write lists.

Real-world example

A race results sheet. One row per runner, and on each row:

Column	How it's calculated	Window function
Position	Rank among all runners by time	<code>rank() OVER (ORDER BY time)</code>
Position in age group	Rank within runners of the same age group	<code>rank() OVER (PARTITION BY age_group ORDER BY time)</code>
Gap to the runner ahead	This time minus the previous row's time	<code>time - lag(time) OVER (ORDER BY time)</code>
Team average	Average of their team's times	<code>avg(time) OVER (PARTITION BY team)</code>

Nobody is removed from the sheet. Every runner still has a row. That's the whole idea.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

`OVER ()` : the whole table as the window

```
SELECT name, price, round(avg(price) OVER (), 2) AS avg_price
FROM products
ORDER BY price DESC
LIMIT 4;
```

name	price	avg_price
Office chair	149.00	27.44
Backpack	45.00	27.44
USB hub	29.99	27.44
Desk lamp	24.99	27.44

(4 rows)

Empty brackets mean "the window is every row". `avg(price)` is the same aggregate you know; `OVER ()` turns it from "collapse into one row" into "compute it, then write it next to every row".

`PARTITION BY` : one window per group

```
SELECT name, category_id, price,
       round(avg(price) OVER (PARTITION BY category_id), 2) AS category_avg
FROM products
ORDER BY category_id, price DESC;
```

name	category_id	price	category_avg
-----+-----+-----+-----			
Stapler	1	8.00	3.53
Notebook	1	3.90	3.53
Sticky notes	1	2.75	3.53
Whiteboard marker	1	1.80	3.53
Pen	1	1.20	3.53
Office chair	2	149.00	87.00
Desk lamp	2	24.99	87.00
Backpack	3	45.00	32.25
Laptop sleeve	3	19.50	32.25
USB hub	4	29.99	22.87
Wireless mouse	4	15.75	22.87
(11 rows)			

`PARTITION BY category_id` splits the rows into one window per category. Each row sees the average of *its own* window. This is the chapter 25 correlated subquery, in one clause and much faster.

And now "how far is each product from its category average" is a one-liner: `round(price - avg(price) OVER (PARTITION BY category_id), 2)`.

Ranking: `row_number`, `rank`, `dense_rank`

```
SELECT name, price, row_number() OVER (ORDER BY price DESC) AS position
FROM products
LIMIT 5;
```

name	price	position
-----+-----+-----		
Office chair	149.00	1
Backpack	45.00	2
USB hub	29.99	3
Desk lamp	24.99	4
Laptop sleeve	19.50	5
(5 rows)		

`ORDER BY` **inside** the `OVER` says how to number the rows. It's separate from the query's own `ORDER BY` (which decides display order). Here they happen to agree.

The three ranking functions differ on **ties**. Rank customers by city, where each city has two people:

```

SELECT name, city,
       rank() OVER (ORDER BY city) AS rank,
       dense_rank() OVER (ORDER BY city) AS dense_rank,
       row_number() OVER (ORDER BY city, name) AS row_number
FROM customers;

```

name	city	rank	dense_rank	row_number
Asha Rai	Kathmandu	1	1	1
Dipesh Thapa	Kathmandu	1	1	2
Chandra Gurung	Lalitpur	3	2	3
Elina Maharjan	Lalitpur	3	2	4
Bikram Shrestha	Pokhara	5	3	5
Farhan Ali	Pokhara	5	3	6

(6 rows)

- `rank()` : ties share a number, and the next number **skips** (1, 1, 3, 3, 5, 5). Like sports: two silver medals, no bronze.
- `dense_rank()` : ties share a number, no skipping (1, 1, 2, 2, 3, 3).
- `row_number()` : no ties ever; each row gets its own number. Add a tie-breaker to its `ORDER BY` or the order among equals is arbitrary.

Use `row_number` when you need exactly one row per position (top-N per group, below). Use `rank` or `dense_rank` when ties should be honest.

Running totals: `ORDER BY` inside `OVER`

When an aggregate's `OVER` has an `ORDER BY`, the window becomes "from the first row **up to this one**". That's a running total:

```

WITH monthly AS (
  SELECT date_trunc('month', o.ordered_at)::date AS month,
         sum(oi.quantity * oi.unit_price) AS revenue
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.id
  WHERE o.status <> 'cancelled'
  GROUP BY month
)
SELECT month, revenue, sum(revenue) OVER (ORDER BY month) AS running_total
FROM monthly
ORDER BY month;

```


month	revenue	running_total
2026-07-01	194.99	194.99
2026-08-01	288.48	483.47
2026-09-01	180.69	664.16

(3 rows)

Notice the pattern: a CTE does the `GROUP BY`, and the window function runs over the grouped result. Window functions are evaluated **after** `GROUP BY` and `HAVING`, so this works, and it's the normal way to combine the two.

Without the `ORDER BY month` inside `OVER`, you'd get the grand total (664.16) on every row. The inner `ORDER BY` is what makes it *running*.

Looking at neighbours: `lag` and `lead`

`lag(column)` gives the value from the **previous** row in the window; `lead(column)` the next. Month-over-month change:

```
WITH monthly AS ( ...same as above... )
SELECT month, revenue,
       lag(revenue) OVER (ORDER BY month) AS previous,
       revenue - lag(revenue) OVER (ORDER BY month) AS change
FROM monthly
ORDER BY month;
```

month	revenue	previous	change
2026-07-01	194.99	[NULL]	[NULL]
2026-08-01	288.48	194.99	93.49
2026-09-01	180.69	288.48	-107.79

(3 rows)

July has no previous month, so `lag` is NULL there. August was up 93.49; September was down 107.79.

`lag(column, 2)` looks two rows back; `lag(column, 1, 0)` gives `0` instead of NULL for the first row.

With `PARTITION BY`, `lag` looks at the previous row *within the group*: `ordered_at::date -`

`lag(ordered_at::date) OVER (PARTITION BY customer_id ORDER BY ordered_at)` is "days since this customer's previous order".

Top N per group

The most-asked window question. The two best-selling products in **each** category:

```

WITH product_revenue AS (
  SELECT p.name, c.name AS category, sum(oi.quantity * oi.unit_price) AS revenue
  FROM order_items oi
  JOIN products p ON p.id = oi.product_id
  JOIN categories c ON c.id = p.category_id
  JOIN orders o ON o.id = oi.order_id
  WHERE o.status <> 'cancelled'
  GROUP BY p.name, c.name
),
ranked AS (
  SELECT *, row_number() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
  FROM product_revenue
)
SELECT category, name, revenue, rn
FROM ranked
WHERE rn <= 2
ORDER BY category, rn;

```

category	name	revenue	rn
Bags	Backpack	90.00	1
Bags	Laptop sleeve	39.00	2
Electronics	USB hub	59.98	1
Electronics	Wireless mouse	31.50	2
Furniture	Office chair	298.00	1
Furniture	Desk lamp	49.98	2
Stationery	Pen	42.00	1
Stationery	Notebook	29.20	2

(8 rows)

Three steps: aggregate, number the rows within each category, keep numbers 1 and 2. Change `<= 2` to `= 1` for "the best in each category".

Why the extra `ranked` CTE? Because you **can't filter on a window function directly**:

```

SELECT name, price FROM products WHERE row_number() OVER (ORDER BY price DESC) <= 3;

```

```

ERROR: window functions are not allowed in WHERE

```

Window functions run after `WHERE`, so `WHERE` can't see them. Compute the number in a CTE (or subquery), then filter in the next step.

Window functions keep every row

To see the difference from `GROUP BY` clearly:

```
SELECT o.id, o.customer_id, o.ordered_at::date AS day,  
       count(*) OVER (PARTITION BY o.customer_id) AS customer_orders  
FROM orders o  
ORDER BY o.customer_id, o.id  
LIMIT 6;
```

id	customer_id	day	customer_orders
1	1	2026-07-03	4
4	1	2026-07-25	4
8	1	2026-08-21	4
12	1	2026-09-26	4
2	2	2026-07-10	2
6	2	2026-08-09	2

(6 rows)

All twelve orders stay (six shown). `GROUP BY customer_id` would have given five rows. The window version lets you keep order-level detail *and* show the customer-level count.

Percent of total

Divide a row's value by the window's total:

```
SELECT name, price, round(100 * price / sum(price) OVER (), 1) AS pct_of_total  
FROM products  
ORDER BY price DESC  
LIMIT 3;
```

name	price	pct_of_total
Office chair	149.00	49.4
Backpack	45.00	14.9
USB hub	29.99	9.9

(3 rows)

Add `PARTITION BY` for "percent of its category". Add `ORDER BY` to the sum for a cumulative percentage, which is how you find "the products that make up 80% of revenue".

Frames: how far the window reaches

When `OVER` has an `ORDER BY`, the default window for aggregates is "from the start of the partition to the current row". You can change it. A three-order moving average:

```
WITH t AS (  
  SELECT o.id, o.ordered_at, sum(oi.quantity * oi.unit_price) AS total  
  FROM orders o JOIN order_items oi ON oi.order_id = o.id GROUP BY o.id  
)  
SELECT id, total,  
       round(avg(total) OVER (ORDER BY ordered_at ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2)  
       AS moving_avg_3  
FROM t ORDER BY ordered_at;
```

id	total	moving_avg_3
1	13.00	13.00
2	32.99	23.00
3	149.00	65.00
4	45.00	75.66
5	56.24	83.41
...		

`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` means "this row and the two before it". Frames are the fine print of window functions. The default is right for running totals; reach for an explicit frame only for moving averages and the like.

Other useful ones

- `first_value(name) OVER (PARTITION BY category_id ORDER BY price DESC)`: the name of the priciest product in each row's category.
- `ntile(3) OVER (ORDER BY price)`: splits rows into 3 equal-sized bands (cheap / mid / expensive by position, not by fixed prices).
- `sum(sum(x)) OVER ()`: a window over an aggregate, when you `GROUP BY` and want the grand total on the same row. Looks odd, works fine.

Naming a window

If several functions share the same `OVER (...)`, name it once:

```

SELECT name, price,
       rank() OVER w AS rank_in_category,
       round(avg(price) OVER w, 2) AS running_avg
FROM products
WINDOW w AS (PARTITION BY category_id ORDER BY price DESC)
ORDER BY category_id, rank_in_category;

```

The `WINDOW` clause goes after `WHERE / GROUP BY / HAVING` and before `ORDER BY`.

Common mistakes

1. Window function in `WHERE` or `HAVING`

Not allowed. Compute it in a CTE, filter in the next step.

2. Forgetting `ORDER BY` inside `OVER` for a running total

You get the grand total on every row. Running means ordered.

3. `rank()` when you need exactly one per group

Ties give two "number 1"s, and `WHERE rn = 1` returns both. Use `row_number()` with a tie-breaker.

4. Mixing up the two `ORDER BY`s

`OVER (ORDER BY ...)` controls the calculation. The query's `ORDER BY` controls display. They're independent. Set both.

5. `PARTITION BY` when you meant `GROUP BY`

If you want fewer rows, it's `GROUP BY`. If you want every row with extra columns, it's a window. Different tools.

6. Window directly over line-level rows

`sum(quantity * unit_price) OVER (PARTITION BY customer_id) on order_items` gives per-customer totals repeated on every *line*. Usually you want to aggregate per order first (CTE), then window.

Quick recap

- `function() OVER (...)` computes a value per row from a window of rows, **keeping every row**.
- `OVER ()` is the whole table. `PARTITION BY col` makes one window per group. `ORDER BY col` inside `OVER` orders the window, which turns aggregates into **running totals**.
- Ranking: `row_number()` (no ties), `rank()` (ties, gaps), `dense_rank()` (ties, no gaps).
- `lag()` / `lead()` read the previous / next row: month-over-month change, days since last order.

- **Top N per group:** CTE with `row_number() OVER (PARTITION BY group ORDER BY value DESC)`, then `WHERE rn <= N`.
 - Window functions run after `GROUP BY` and can't be used in `WHERE`. Aggregate in a CTE, window in the next step, filter in the step after.
-

Next: try the [exercises](#), then move on to the Level 3 project: [29 Project: Sales Report](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
 - Save each answer in `playground/ch28/ex1.sql`, `ex2.sql`, and so on.
 - Build in steps: a CTE to aggregate, then the window, then (if you need to filter on the window) one more step.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Context on every row

Every product with: its name, its price, the **overall** average price, and its **category's** average price, both rounded to 2 places. Sorted by category then price descending.

The first row should be `Stapler | 8.00 | 27.44 | 3.53`.

Exercise 2 (Easy): Rankings

1. Products numbered 1, 2, 3... from most to least expensive, using `row_number()`.
2. Customers ranked by total spending (excluding cancelled orders), with `rank()`. Chandra should be 1.
3. Customers ranked by **city** with `rank()`, `dense_rank()`, and `row_number()` side by side. In a comment, explain the three different numbers on the Lalitpur rows.

Expected output of query 2:

name	spent	rank
Chandra Gurung	239.00	1
Asha Rai	218.19	2
Dipesh Thapa	86.23	3
Farhan Ali	64.75	4
Bikram Shrestha	55.99	5

(5 rows)

Exercise 3 (Medium): Over time

1. Monthly revenue (excluding cancelled) with a **running total** column.
2. Extend it with the **previous month's** revenue and the **change** from the previous month.
3. Each order, with the number of **days since that customer's previous order**. The first order for each customer shows NULL. Sorted by customer then date.

Expected first rows of query 3:

customer_id	id	day	days_since_previous
1	1	2026-07-03	[NULL]
1	4	2026-07-25	22
1	8	2026-08-21	27
1	12	2026-09-26	36
2	2	2026-07-10	[NULL]
...			

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Shares

1. Each order's total and what **percentage of its customer's total spending** it represents (one decimal place). Order 8 should be 66.1% of Asha's spending.
2. Each order ranked **within its month** by total, biggest first.

Expected first rows of query 2:

month	id	total	rank_in_month
2026-07-01	3	149.00	1
2026-07-01	4	45.00	2
2026-07-01	2	32.99	3
2026-07-01	1	13.00	4
2026-08-01	8	173.99	1
...			

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Top N and the 80% rule

1. The **top 2 products by revenue in each category**, excluding cancelled orders. Eight rows.
2. Products sorted by revenue (excluding cancelled), with a **cumulative percentage of total revenue**. Then answer in a comment: which products together make up the first 80% of revenue?

Expected output of query 2:

name	revenue	cumulative_pct
Office chair	298.00	44.9
Backpack	90.00	58.4
USB hub	59.98	67.5
Desk lamp	49.98	75.0
Pen	42.00	81.3
Laptop sleeve	39.00	87.2
Wireless mouse	31.50	91.9
Notebook	29.20	96.3
Sticky notes	16.50	98.8
Stapler	8.00	100.0
(10 rows)		

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

29 Project: Sales Report

What you'll build

A complete sales report for the shop you've been querying since chapter 20: headline numbers, trends by month, breakdowns by category, customer, product, and city, operational checks, and a short written summary of what the numbers mean.

It's every tool from Level 3 used together: aggregates, grouping, joins, outer joins, functions, subqueries, CTEs, set operations, and window functions. More importantly, it's the thing people will actually ask you to produce: "can you pull the numbers?"

By the end, you'll have, in `playground/ch29/` :

- `report.sql` : one file, numbered sections, each a query answering one question, that runs clean from top to bottom,
- `findings.md` : five plain-English sentences a shop owner could act on, each backed by a number from the report.

Before you start

- Finish chapters 20 to 28.
- Read the starter README. It describes the dataset's quirks: the cancelled order, the price change, the product that never sold, the customer who never ordered. Every one of them is a trap for a careless query.
- Make `playground/ch29/` .
- Reload the dataset so you're starting clean: `\c sales then \i ../starter/seed.sql` .
- `\pset null '[NULL]'` .

Two rules for every query in this report:

1. **Revenue excludes cancelled orders.** A cancelled order is money that never arrived. Decide early how you'll exclude it (a `live_orders` CTE is the cleanest), and do it the same way everywhere.
2. **Every "per X" report lists every X**, including the ones with zero. The customer who never ordered and the product that never sold must appear. That means outer joins or `coalesce` , and you must check.

Milestone 1: Sanity checks

Goal: know your data before you summarize it.

Start `report.sql` with a section that prints row counts for all five tables, the grand total of all order lines, and the grand total excluding the cancelled order.

Check: 4 categories, 11 products, 6 customers, 12 orders, 21 lines. 709.16 for everything, 664.16 live. If those don't match, reload the seed.

Milestone 2: Headline numbers

Goal: one row, four numbers.

For live (non-cancelled) orders: how many orders, total revenue, average order value, and the biggest single order.

Check: 11 | 664.16 | 60.38 | 173.99 .

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 3: Month by month

Goal: the trend.

One row per month with: the month as Jul 2026 -style text, number of orders, revenue, running total, and change from the previous month.

Check:

month	orders	revenue	running_total	change
Jul 2026	3	194.99	194.99	[NULL]
Aug 2026	4	288.48	483.47	93.49
Sep 2026	4	180.69	664.16	-107.79

The trap in this milestone: if you ORDER BY the text label, you get Aug, Jul, Sep, because that's alphabetical. Sort by the real date (date_trunc), display the text. Your running total will look wrong until you fix this, which is how you'll notice.

Milestone 4: By category

Goal: where the money comes from.

Per category: units sold, revenue, and percentage of total revenue (one decimal). Biggest first.

Check: Furniture 52.4%, Bags 19.4%, Stationery 14.4%, Electronics 13.8%. Stationery sells 50 units for less money than Furniture makes on 4.

Milestone 5: By customer and by city

Goal: who buys.

1. Per customer, **all six**: name, city, live order count, total spent, average order. Highest spender first. Elina must appear with 0.
2. Per city: number of customers, orders, revenue.

Check: Chandra 239.00 on 2 orders (average 119.50); Asha 218.19 on 3; Elina 0. Kathmandu 304.42, Lalitpur 239.00, Pokhara 120.74.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 6: By product

Goal: what sells, and what doesn't.

1. Per product, **all eleven**: name, category, units, revenue from live orders. Best first. The Whiteboard marker must show 0.
2. Each product's rank within its category by revenue.
3. The top 2 products per category.

The trap in this milestone: the Backpack appears in the cancelled order (1 unit, 45.00) *and* a live one (2 units, 90.00). If your product report says Backpack sold 3 units for 135.00, the cancelled order leaked in. A `LEFT JOIN` to `order_items` then `JOIN orders ... WHERE status <> 'cancelled'` turns your left join back into an inner join and drops the Whiteboard marker too (chapter 23). The clean fix: a `live_items` CTE (order items joined to live orders), then `products LEFT JOIN live_items`.

Check: Office chair 298.00, Backpack **90.00** (2 units), ..., Whiteboard marker 0.

Milestone 7: Operations

Goal: things the shop needs to *do*.

1. Orders and total value per status.
2. How many pending orders, and how much money is waiting in them.
3. Average time from order to shipping, per month. The raw `avg()` of intervals can print odd things like `1 day 24:23:20`; wrap it in `justify_interval()` to tidy.
4. Active products with fewer than 20 in stock, lowest first.
5. Which weekdays orders arrive on.

Check: shipped 6 / 525.22, paid 3 / 78.24, pending 2 / 60.70, cancelled 1 / 45.00. Low stock: Office chair 6, Backpack 8, Desk lamp 15. Fridays and Saturdays get 5 orders each, Sundays 2.

Milestone 8: Customer behaviour

Goal: the questions a marketing person asks.

1. New customers per month (by the month of their **first** order). Jul 3, Aug 2.
2. Repeat vs one-time customers (2+ live orders vs 1).
3. Days between each customer's orders (chapter 28, exercise 3).
4. Which products make up the first 80% of revenue (cumulative percentage).
5. Customers who ordered in all three months (set operations), and customers who ordered in July but not since.

Milestone 9: Write it up

Goal: `findings.md`, five sentences.

Each sentence must contain a number from your report and say what the shop should do about it. Not "revenue was 664.16" but "September revenue fell 107.79 from August, mostly because no Furniture sold; the shop should check Office chair stock (6 left) before the next promotion." You're translating queries into decisions. That's the actual job.

Milestone 10: Run it clean

Reload the seed, then `\i report.sql`. Every section runs, no errors, every number matches the checks above. Read through the output once more as if you were the shop owner. If anything would confuse them (a NULL, an unlabeled column, 16 decimal places), fix it.

Common mistakes

1. Cancelled revenue leaking in

Decide once (a `live_orders` or `live_items` CTE), reuse everywhere. Check Backpack: 90.00, not 135.00.

2. Losing the zeroes

A `JOIN` after a `LEFT JOIN`, or a `WHERE` on the right table, drops Elina and the Whiteboard marker. Every "per X" needs a check that all X are there.

3. Sorting by a text label

`Aug` before `Ju1`. Sort by the date, show the text.

4. Counting lines as orders

Total per order first (CTE), then count. Or `count(DISTINCT o.id)` .

5. Raw averages

`27.4436363636` in a report. `round(..., 2)` everything that's money or an average.

6. Numbers without sentences

A report nobody can act on is a spreadsheet. Milestone 9 is not optional.

Quick recap

- Decide how to exclude cancelled orders once, in a CTE, and reuse it.
 - Every per-X report includes all X: outer joins, `coalesce` , and a check.
 - Aggregate in a CTE, then join, window, or filter in the next step.
 - Sort by real dates, display formatted text.
 - Round money and averages. Label every column.
 - Finish with sentences a person can act on.
-

Congratulations! You've finished Level 3. 🎉 You can now answer almost any question about data spread across many tables, which is the skill most people mean when they say "knows SQL". Try the [exercises](#) for extra report features, then head back to the roadmap. Level 4 is about keeping data safe and fast: transactions, indexes, permissions, and backups.

Exercises

How to do these:

- These extend your `report.sql` . Finish all 10 milestones first.
 - Add each as a new numbered section. After each, the whole file must still run clean after a seed reload.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to review the file.
-

Exercise 1 (Easy): Basket size

Add a section showing the average number of **lines** per order and the average number of **units** per order, both to 2 decimal places, across all orders.

Expected: `1.75` lines, `5.25` units.

Exercise 2 (Easy): Who to email

A section listing customers who have **never ordered**, with the month they joined (`Jun 2026` -style), so marketing can send them a welcome offer. Then a second list: customers whose **last** order was before 1 September (they may be drifting away).

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A month-by-category grid

Build a **pivot table**: one row per month, one column per category, each cell the revenue for that month and category (live orders only), with a total column. Use `FILTER`.

Expected output:

month	stationery	furniture	bags	electronics	total
Jul 2026	21.00	173.99	0	0	194.99
Aug 2026	33.50	173.99	19.50	61.49	288.48
Sep 2026	41.20	0	109.50	29.99	180.69
(3 rows)					

The row totals must match Milestone 3. Then, in a comment: what's the downside of this shape when the shop adds a fifth category?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): What-if pricing

The owner asks: "If Pens had been 1.50 instead of 1.20 all along, how much more would we have made?"

Recompute live revenue with every Pen line repriced at 1.50, and show the difference from actual live revenue. Don't change any data; do it with a `CASE` inside the sum.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A parameterized report

psql can hold **variables**. At the top of a file:

```
\set month_start '2026-08-01'  
\set month_end '2026-09-01'
```

and in a query, `: 'month_start'` (with the quotes) pastes the value in as text.

Rewrite your month-by-month section as a **single-month report**: headline numbers, by category, and top products, all for the month between the two variables. Run it for August, then change the two lines to September and run it again. Nothing else in the file should change.

Then one more: `\copy` the top-products result for the chosen month to a CSV on your desktop (chapter 11, exercise 4), so the owner can open it in Excel.

Stuck? The hint is in the [Hints section](#) at the end.

Level 4: Keeping Data Safe and Fast

Goal: protect your data from mistakes, crashes, and strangers, and keep queries fast as your data grows.

30 Transactions

What is it?

A **transaction** is a group of statements that the database treats as **one unit**: either all of them take effect, or none of them do. You've been using the three commands since chapter 10:

Command	Meaning
<code>BEGIN;</code>	Start the unit. Changes from here on are pending.
<code>COMMIT;</code>	Make every pending change permanent, all at once.
<code>ROLLBACK;</code>	Throw every pending change away.

This chapter is about *why* that matters, what happens when something goes wrong in the middle, and what other people see while you're working.

Why does it matter?

The classic example: moving 200 from Asha's account to Bikram's. That's two statements:

```
UPDATE accounts SET balance = balance - 200 WHERE id = 1;  -- Asha
UPDATE accounts SET balance = balance + 200 WHERE id = 2;  -- Bikram
```

Now imagine the power goes out between them. Asha has lost 200. Bikram hasn't received it. The money is gone, and no one can tell where. A transaction makes that impossible: both updates happen, or neither does.

The same problem is everywhere. An order with no items because the insert of the items failed. A user account created but the profile row missing. A stock count reduced but the sale not recorded. Every multi-step change to

data is a transfer in disguise, and transactions are how you keep it whole.

The second reason: **other people**. A real database has many users and programs working at once. Transactions decide what they see of each other's half-finished work (nothing), and what happens when two of them touch the same row (one waits).

Real-world example

An ATM withdrawal:

1. Check the balance.
2. Deduct the amount.
3. Dispense the cash.

If step 3 fails (the machine jams), step 2 must be undone, or you've paid for cash you never got. If step 2 fails, step 3 must not happen. The bank treats the whole thing as one operation. That's a transaction.

And while your withdrawal is in progress, someone checking your balance online sees the *old* balance until it completes. They don't see a half-done number. That's isolation.

How it works

Work in `sales`. Make a small table to play with:

```
DROP TABLE IF EXISTS accounts;
CREATE TABLE accounts (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  owner text NOT NULL,
  balance numeric(12, 2) NOT NULL CHECK (balance >= 0)
);
INSERT INTO accounts (owner, balance) VALUES ('Asha', 500.00), ('Bikram', 100.00);
```

A transfer that works

```
BEGIN;
UPDATE accounts SET balance = balance - 200 WHERE id = 1;
UPDATE accounts SET balance = balance + 200 WHERE id = 2;
COMMIT;

SELECT * FROM accounts ORDER BY id;
```

```

id | owner  | balance
----+-----+-----
 1 | Asha   |  300.00
 2 | Bikram |  300.00
(2 rows)

```

Two updates, one commit. Between `BEGIN` and `COMMIT`, nobody else could see Asha's balance at 300 with Bikram's still at 100. The change appeared to the rest of the world in one step.

A transfer that fails halfway

Asha tries to send 1000 she doesn't have:

```

BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;

```

```

ERROR:  new row for relation "accounts" violates check constraint "accounts_balance_check"
DETAIL:  Failing row contains (1, Asha, -700.00).

```

The `CHECK (balance >= 0)` from chapter 13 refuses. Now watch what happens to the *rest* of the transaction:

```

UPDATE accounts SET balance = balance + 1000 WHERE id = 2;

```

```

ERROR:  current transaction is aborted, commands ignored until end of transaction block

```

After an error, the transaction is **aborted**. Every further statement is ignored, even ones that would have worked, even a plain `SELECT`. PostgreSQL is refusing to let you build on a broken foundation. The only way out is to end the transaction:

```

COMMIT;

```

```

ROLLBACK

```

You typed `COMMIT`, and PostgreSQL replied `ROLLBACK`. An aborted transaction cannot be committed. Everything since `BEGIN` is undone. Check:

```

SELECT * FROM accounts ORDER BY id;

```

```

id | owner  | balance
----+-----+-----
 1 | Asha   |  300.00
 2 | Bikram |  300.00

```

Nothing changed. No half-transfer, no lost money. This is the whole point.

In psql, the prompt shows you the state: `sales=*` means a transaction is open, `sales=!*` means it's open **and aborted**. When you see `!`, type `ROLLBACK;`.

The four promises: ACID

You'll hear "transactions are ACID". It's four words, each one a promise:

Letter	Promise	Plain English
A tomtic	All or nothing	The transfer can't half-happen. You just saw this.
C onsistent	Rules hold before and after	Every constraint is true when the transaction ends. A commit can never leave a negative balance.
I solated	Others don't see your half-done work	While your transaction is open, other sessions see the data as it was before you began. Below.
D urable	Once committed, it's safe	When <code>COMMIT</code> returns, the change is written to disk. A crash one millisecond later won't lose it.

You don't have to do anything to get these. You just have to put the related statements inside one `BEGIN ... COMMIT`.

Autocommit: every statement is a tiny transaction

When you run a statement *without* `BEGIN`, PostgreSQL wraps it in its own transaction and commits it immediately. That's why a single `UPDATE` without a `WHERE` in chapter 10 was instantly permanent. `BEGIN` is you saying "hold on, I want several statements in the same unit".

Partial undo: `SAVEPOINT`

Sometimes one step failing shouldn't sink the whole transaction. A **savepoint** is a marker you can roll back to:

```

BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE id = 1;
SAVEPOINT before_bonus;
UPDATE accounts SET balance = balance - 10000 WHERE id = 2;    -- fails: would go negative
ROLLBACK TO SAVEPOINT before_bonus;                          -- undo just that, un-abort
UPDATE accounts SET balance = balance + 50 WHERE id = 2;
COMMIT;

SELECT * FROM accounts ORDER BY id;

```

id	owner	balance
1	Asha	250.00
2	Bikram	350.00

The failed update was undone, the transaction came back to life, and the rest committed. Savepoints are how a program can try something optional inside a transaction and carry on if it fails. Most of the time you won't need them; when you do, nothing else will do.

What other sessions see: isolation

This needs two psql windows. Open SQL Shell twice, connect both to `sales`, and run `\pset null '[NULL]'` in each. Call them **A** and **B**.

In **A**:

```

BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
SELECT balance FROM accounts WHERE id = 1;    -- A sees 150.00

```

Don't commit. Now in **B**:

```

SELECT balance FROM accounts WHERE id = 1;    -- B sees 250.00

```

B sees the **old** value. A's change is pending, invisible to everyone else. Reading never waits: B got an answer immediately, just not A's unfinished one.

Now in **B**, try to change the same row:

```

UPDATE accounts SET balance = balance - 50 WHERE id = 1;

```

B **hangs**. Nothing comes back. The row is **locked**: A's uncommitted update holds it, and B's update must wait until A finishes, because the result depends on what A does.

In **A**:

```
COMMIT;
```

The instant A commits, B's update runs on top of A's result, and B gets `UPDATE 1`. Check in either window: balance is 100.00 (250 – 100 – 50). Both changes applied, in order, nothing lost.

Three rules fall out of this:

1. **Readers never block, and are never blocked.** A `SELECT` always gets an answer: the latest *committed* data.
2. **Two writers on the same row take turns.** The second waits for the first to commit or roll back.
3. **Rows, not tables.** B could have updated Bikram's row (id 2) at the same time with no waiting. Only the *same row* conflicts.

This default behaviour is called the **read committed** isolation level: each statement sees whatever was committed before it started. PostgreSQL has stricter levels (`REPEATABLE READ` , `SERIALIZABLE`) for special cases, like a report that must see one consistent snapshot across many queries. You won't need them for a long time; the default is right for almost everything.

The lost update, and why `balance = balance - 50` matters

Suppose an app does the transfer the "obvious" way: read the balance into a variable, subtract in code, write the new number back.

```
Session A reads 250.    Session B reads 250.  
A writes 250 - 100 = 150.  
B writes 250 - 50 = 200.    ← overwrites A's change. The 100 is lost.
```

Two sessions, two reads of the same old value, and the second write clobbers the first. This is the **lost update**, and it's a real bug in real apps.

The SQL you've been writing avoids it: `SET balance = balance - 50` reads and writes the current value in **one** statement, inside the row lock, so the two sessions queue up properly. When a program really must read first and decide in code (check a balance, then withdraw), lock the row while deciding:

```
BEGIN;
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;    -- Lock it now
-- ...decide in code...
UPDATE accounts SET balance = balance - 50 WHERE id = 1;
COMMIT;
```

`FOR UPDATE` makes the `SELECT` take the same lock an `UPDATE` would, so a second session's `SELECT ... FOR UPDATE` waits. You'll use this in the [bank project](#).

Deadlocks

Two sessions can get stuck waiting for each other. A locks Asha's row then wants Bikram's; B locks Bikram's row then wants Asha's. Neither can proceed:

```
ERROR: deadlock detected
DETAIL: Process 343 waits for ShareLock on transaction 819; blocked by process 350.
Process 350 waits for ShareLock on transaction 818; blocked by process 343.
```

PostgreSQL notices within about a second, kills **one** of the two transactions with this error (that one is rolled back), and lets the other finish. Your program should catch the error and retry.

The usual cure is to lock rows **in a consistent order** everywhere: always the lower `id` first, say. If both sessions update Asha before Bikram, the second just waits for the first; no cycle, no deadlock.

Two small things worth knowing

`now()` **is frozen inside a transaction**. It returns the time the transaction *started*, so every row you insert in one transaction gets the same timestamp. That's usually what you want. `clock_timestamp()` gives the real current time if you need it.

Structure changes are transactional too. As you saw in chapter 15, `ALTER TABLE` and `DROP TABLE` can be rolled back. Most other databases can't do this. Use it.

Habits

1. **Group related writes in one transaction.** If two statements must both happen, `BEGIN` before the first, `COMMIT` after the last.
2. **Keep transactions short.** A transaction holds locks until it ends. Never open one, then go and read your email. Never wait for user input inside one.
3. **Always end it.** `COMMIT` or `ROLLBACK`. Watch for `*` and `!` in the prompt.
4. **In code, `ROLLBACK` on any error.** [Chapter 40](#) shows the try/catch pattern.
5. **Change values in place** (`balance = balance - 50`), or lock with `FOR UPDATE` when you must read first.

6. **Lock in a consistent order** to avoid deadlocks.

Clean up:

```
DROP TABLE accounts;
```

Common mistakes

1. Carrying on after an error inside a transaction

Every statement is ignored until you `ROLLBACK`. The `!` in the prompt is telling you.

2. Expecting `COMMIT` to save an aborted transaction

It rolls back instead. If anything failed, nothing is kept.

3. Leaving a transaction open

Locks held, other sessions waiting, and in a program, a connection stuck forever. Short and closed.

4. Read-then-write in application code without a lock

The lost update. Use `SET x = x - n`, or `SELECT ... FOR UPDATE`.

5. Thinking a `SELECT` will show you another session's uncommitted work

It never does. If you don't see a change, the other session hasn't committed.

6. Fearing deadlocks

They're rare, PostgreSQL detects them, and consistent lock ordering prevents most. Catch the error, retry.

Quick recap

- A transaction is **all or nothing**: `BEGIN`, statements, `COMMIT`. Any error aborts it, and `COMMIT` then becomes `ROLLBACK`.
- **ACID**: atomic, consistent, isolated, durable. You get it by grouping statements; nothing else to do.
- `SAVEPOINT` lets you undo part of a transaction and carry on.
- Other sessions see only **committed** data. Readers never wait. Two writers on the same row take turns; the second waits for the first to finish.
- `SET balance = balance - 50` is safe against lost updates. `SELECT ... FOR UPDATE` locks a row when you must read before writing.
- Deadlocks are detected and one side is rolled back. Lock rows in a consistent order.
- Keep transactions short, and always end them.

Next: try the [exercises](#), then move on to [31 Indexes](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Create the `accounts` table from the notes (Asha 500, Bikram 100) at the start of each exercise file so it's re-runnable.
 - Save each answer in `playground/ch30/ex1.sql`, `ex2.sql`, and so on. For the two-window exercises, write down what you saw as comments.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Two transfers

1. Transfer 150 from Asha to Bikram in one transaction. Show both balances afterwards.
2. Try to transfer 900 from Bikram to Asha in one transaction. Read every message PostgreSQL prints, including the reply to your `COMMIT`. Show both balances afterwards and confirm nothing changed.

Expected balances after step 1: Asha 350.00, Bikram 250.00. After step 2: the same.

Exercise 2 (Easy): The aborted state

1. `BEGIN`, then run a statement that fails (any constraint violation). Then run `SELECT 1;`. What happens, and why?
 2. Look at your psql prompt. What character shows the transaction is broken?
 3. Get out of it. Then, in a comment, explain in one sentence why PostgreSQL refuses to run even a harmless `SELECT` in an aborted transaction.
-

Exercise 3 (Medium): A savepoint in a batch

A nightly job pays 10 interest to every account, then tries to charge a 500 fee to account 2 (which can't afford it), then records today's date in a `last_processed` column (add it with `ALTER TABLE`).

Write this as **one** transaction where the fee step is wrapped in a savepoint, so that when it fails, the interest and the date still commit.

Expected: both balances up by 10, no fee charged, `last_processed` set on both rows.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Two windows

Open two psql sessions, A and B, both on `sales`. Work through the isolation demo from the notes and record, as comments in `ex4.sql`:

1. What B's `SELECT` showed while A's update was uncommitted.
 2. What happened when B ran an `UPDATE` on the **same** row (and roughly how long it waited).
 3. What happened when B ran an `UPDATE` on the **other** row instead while A was still open. Did it wait?
 4. The final balances after both committed, and why they're what they are.
-

Exercise 5 (Challenge): Deadlock, then prevent it

1. Cause a deadlock on purpose with two windows: A updates row 1 then (after you switch windows) row 2; B updates row 2 then row 1. Copy the `deadlock detected` message. Which session got the error, and what happened to the other?
2. Now repeat, but make **both** sessions update row 1 first, then row 2. Describe what happens instead of a deadlock.
3. Write the "check balance, then withdraw" pattern safely: a transaction that `SELECT`s Asha's balance `FOR UPDATE`, and then (pretending you decided in code) updates it. In a second window, try to `SELECT ... FOR UPDATE` the same row while the first transaction is open. What happens?

Stuck? The hint is in the [Hints section](#) at the end.

31 Indexes

What is it?

An **index** is a separate, sorted structure that lets PostgreSQL find rows by a column's value without reading the whole table. You create one with a single line:

```
CREATE INDEX idx_orders_customer_id ON orders (customer_id);
```

After that, "find the orders for customer 42" jumps straight to them instead of checking every row.

Why does it matter?

On your 12-row `orders` table, every query is instant and indexes change nothing. On a real table with ten million rows, a query that checks every row takes seconds, and the same query with the right index takes a millisecond. That's not a small difference. It's the difference between an app that feels instant and one that feels broken.

Indexes are the single biggest performance tool you have. They're also the one most beginners don't know exists, which is why "the database is slow" is so often fixed by one `CREATE INDEX`.

Real-world example

The index at the back of a textbook.

To find every mention of "mitochondria" without one, you'd read all 600 pages. With the index, you look up "mitochondria" in a sorted list, see "pages 41, 97, 215", and go straight there. The index is extra pages (it takes space), and every time the author adds a paragraph the index has to be updated (writes get slower). But lookups go from minutes to seconds.

A database index is exactly that: a sorted list of values with pointers to the rows that have them.

How it works

You need a big table to feel this. Work in `sales` and build one with a million rows using `generate_series` and `random()`:

```
CREATE TABLE big_orders AS
SELECT g AS id,
       (random() * 1000)::integer + 1 AS customer_id,
       timestampz '2025-01-01' + (random() * 600)::integer * interval '1 day' AS ordered_at,
       round((random() * 500)::numeric, 2) AS total
FROM generate_series(1, 1000000) AS g;

ALTER TABLE big_orders ADD PRIMARY KEY (id);
```

A million fake orders, spread over about a thousand customers. It takes a few seconds and about 70 MB (`SELECT pg_size_pretty(pg_total_relation_size('big_orders'));`).

(`CREATE TABLE ... AS SELECT` makes a table from a query's result. Handy for test data and for saving a report's output.)

Measure first

Turn on timing in psql:

```
\timing on
```

Now every statement prints how long it took. Find one customer's orders:

```
SELECT count(*) FROM big_orders WHERE customer_id = 42;
```

```
count
-----
 1050
(1 row)

Time: 33.627 ms
```

(Your count and time will differ: the data is random and your machine is different.) Thirty-odd milliseconds. Not slow, exactly. But ask PostgreSQL *how* it did it:

```
EXPLAIN SELECT * FROM big_orders WHERE customer_id = 42;
```

QUERY PLAN

```
-----  
Gather  (cost=1000.00..13108.33 rows=5000 width=48)  
  Workers Planned: 2  
    -> Parallel Seq Scan on big_orders  (cost=0.00..11608.33 rows=2083 width=48)  
        Filter: (customer_id = 42)  
(4 rows)
```

`EXPLAIN` shows the **plan**: the steps PostgreSQL will take. The key words are **Seq Scan**: a sequential scan, meaning "read every row of the table and check the filter". A million rows checked to find a thousand. (It split the work across workers to go faster, which is why it says Parallel and Gather. [Chapter 32](#) reads plans in detail.)

Add an index

```
CREATE INDEX idx_big_orders_customer_id ON big_orders (customer_id);
```

```
CREATE INDEX  
Time: 343.969 ms
```

A third of a second to sort a million values. Now the same query:

```
SELECT count(*) FROM big_orders WHERE customer_id = 42;
```

```
count  
-----  
1050  
(1 row)
```

```
Time: 0.961 ms
```

Thirty-five times faster, and the plan shows why:

```
EXPLAIN SELECT * FROM big_orders WHERE customer_id = 42;
```

QUERY PLAN

```
-----  
Bitmap Heap Scan on big_orders  (cost=59.17..6419.17 rows=5000 width=48)  
  Recheck Cond: (customer_id = 42)  
    -> Bitmap Index Scan on idx_big_orders_customer_id  (cost=0.00..57.92 rows=5000 width=0)  
        Index Cond: (customer_id = 42)  
(4 rows)
```

Index Scan instead of Seq Scan. PostgreSQL looked up 42 in the index, got the list of matching rows, and fetched only those. No more checking a million rows. (The "Bitmap" variety is how it fetches a batch of scattered rows efficiently; a plain Index Scan appears when fetching one or a few.)

That's the whole idea. One line, and a query category goes from "reads everything" to "reads what it needs".

Naming and seeing indexes

Name indexes `idx_table_column` (or `idx_table_col1_col2`). PostgreSQL will invent a name if you don't give one, but yours will be easier to recognize in plans and in `\d`:

```
\d big_orders
```

Indexes:

```
"big_orders_pkey" PRIMARY KEY, btree (id)  
"idx_big_orders_customer_id" btree (customer_id)
```

`\di` lists all indexes in the database. `btree` is the index type: a balanced tree, the default, right for almost everything (equality, ranges, sorting). Other types exist for special data; chapters 37 and 39 use one called GIN.

You already have some

`PRIMARY KEY` and `UNIQUE` create indexes automatically. That's how PostgreSQL enforces uniqueness fast, and why `WHERE id = 500000` is instant on any table:

```
EXPLAIN SELECT * FROM big_orders WHERE id = 500000;
```

```
Index Scan using big_orders_pkey on big_orders  (cost=0.42..8.44 rows=1 width=48)  
Index Cond: (id = 500000)
```

Foreign key columns are not indexed automatically. Look at `\d order_items` in `sales`: the primary key covers `(order_id, product_id)`, so lookups by `order_id` are indexed (it's the first column), but lookups by

`product_id` are not. "Which orders contain this product?" would scan the table. In a real system, **index every foreign key column** unless you have a reason not to. It's the most common missing index.

What indexes help with

An index on a column speeds up:

- `WHERE column = value` and `WHERE column IN (...)`
- `WHERE column BETWEEN a AND b`, `<`, `>` (ranges)
- `JOIN ... ON column = other` (joins look things up too)
- `ORDER BY column` (the index is already sorted, so `ORDER BY ... LIMIT 10` can read just 10 entries)

It does **not** help with:

- `WHERE lower(email) = ...` or any function or arithmetic on the column. The index holds `email`, not `lower(email)`. Watch:

```
EXPLAIN SELECT * FROM big_orders WHERE customer_id + 0 = 42;
```

Back to `Parallel Seq Scan`. The `+ 0` does nothing mathematically and completely defeats the index. Keep the indexed column bare on one side of the comparison, or make an expression index (below).

- `WHERE text_column LIKE '%word%'`. A leading `%` means "anything first", which a sorted list can't help with. Chapter 39 has the fix.
- Queries that match most of the table. If a condition matches 80% of rows, reading them all is faster than hopping around via the index, and PostgreSQL will choose a Seq Scan on purpose. That's not a bug.

Indexes on more than one column

```
CREATE INDEX idx_big_orders_customer_date ON big_orders (customer_id, ordered_at);
```

This index is sorted by `customer_id`, then by `ordered_at` within each customer, like a phone book sorted by surname then first name. It helps queries on `customer_id` alone, and on `customer_id` **and** `ordered_at` together:

```
EXPLAIN SELECT * FROM big_orders WHERE customer_id = 42 AND ordered_at >= '2026-01-01';  
-- Bitmap Index Scan on idx_big_orders_customer_date
```

But **not** on `ordered_at` alone:

```
EXPLAIN SELECT * FROM big_orders WHERE ordered_at >= '2026-06-01' AND ordered_at < '2026-06-02';
-- Parallel Seq Scan
```

You can't find everyone named "Asha" in a phone book sorted by surname. **Column order matters:** put the column you always filter on first, the one you sometimes add second.

Partial indexes

Index only the rows you'll actually look up:

```
CREATE INDEX idx_orders_pending ON orders (ordered_at) WHERE status = 'pending';
```

A tiny index containing only pending orders. "Show me pending orders, oldest first" flies, and the index doesn't grow with the millions of shipped orders nobody queries that way. Great for "active", "unprocessed", "unread" style flags.

Expression indexes

If you always search `lower(email)`, index that:

```
CREATE INDEX idx_customers_lower_email ON customers (lower(email));
```

Now `WHERE lower(email) = 'asha@example.com'` can use an index, because the index holds the lowered values. (On the 6-row `customers` table PostgreSQL will still choose a Seq Scan, because reading 6 rows is faster than opening an index. It'd matter at 60,000.)

Unique indexes

`CREATE UNIQUE INDEX` is a constraint and an index in one, and it works on expressions, which plain `UNIQUE` can't:

```
CREATE UNIQUE INDEX idx_customers_email_unique ON customers (lower(email));
INSERT INTO customers (name, email, city, joined_on) VALUES ('Dup', 'ASHA@example.com', 'X',
    current_date);
```

```
ERROR: duplicate key value violates unique constraint "idx_customers_email_unique"
DETAIL: Key (lower(email))=(asha@example.com) already exists.
```

Case-insensitive unique emails, the problem chapter 13 left open, solved in one line.

The cost

Indexes aren't free:

- **Space.** The index on `customer_id` is about 7 MB next to a 50 MB table. Each index is a copy of one or more columns.
- **Slower writes.** Every `INSERT`, `UPDATE` of an indexed column, or `DELETE` must update every index on the table too. A table with eight indexes writes much slower than one with two.
- **Maintenance.** Indexes that nobody uses are pure cost. PostgreSQL tracks index usage (`pg_stat_user_indexes`), and it's worth checking occasionally.

The rule: **index the columns your queries filter, join, and sort on. Don't index everything.** A table with a primary key, indexes on its foreign keys, and one or two for its hot queries is typical.

Removing one is easy:

```
DROP INDEX idx_big_orders_customer_date;
```

Keep the statistics fresh

PostgreSQL decides whether to use an index based on statistics about your data (how many rows, how many distinct values). It updates them automatically in the background, but after you load a lot of data at once, give it a nudge:

```
ANALYZE big_orders;
```

If a plan looks wrong after a big import, `ANALYZE` first.

Clean up when you're done with this chapter and the next:

```
DROP TABLE big_orders;
```

Common mistakes

1. No indexes on foreign keys

Joins and "find children of this parent" scan the table. Index them.

2. A function on the indexed column

`WHERE lower(email) = ...`, `WHERE date(ordered_at) = ...`, `WHERE price * 1.2 > 100`. The index can't help. Rewrite the condition (`ordered_at >= '...' AND ordered_at < '...'`) or make an expression index.

3. Indexing everything

Slow writes, wasted disk, no benefit. Index what you query.

4. Wrong column order in a multi-column index

`(ordered_at, customer_id)` doesn't help `WHERE customer_id = 42`. Lead with the column you always filter on.

5. Judging on a tiny table

PostgreSQL ignores indexes on tables with a few hundred rows, correctly. Test on realistic data sizes.

6. Expecting `LIKE '%text%'` to use an index

It can't. Chapter 39.

Quick recap

- An index is a sorted copy of a column with pointers to rows. `CREATE INDEX idx_table_col ON table (col);`
- It turns a **Seq Scan** (read everything) into an **Index Scan** (read what matches). On big tables, that's 10 to 1000 times faster.
- Primary keys and `UNIQUE` are indexed automatically. **Foreign keys are not**: add them.
- Indexes help `WHERE =`, ranges, joins, and `ORDER BY` on the column. They don't help functions on the column or `LIKE '%...'`.
- Multi-column indexes work left to right. Partial indexes cover a subset. Expression indexes cover `lower(email)` and friends. Unique indexes enforce uniqueness on expressions.
- Every index costs space and slows writes. Index what you query, and `ANALYZE` after big loads.

Next: try the [exercises](#), then move on to [32 Why Is My Query Slow?](#).

Exercises

How to do these:

- Work in psql, connected to `sales`, with `big_orders` from the notes. Run `\timing on`.
- Save each answer in `playground/ch31/ex1.sql`, `ex2.sql`, and so on, and write the timings and plan node names you observed as comments. Your exact numbers will differ from anyone else's; the *pattern* is what matters.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Before and after

1. Drop the `idx_big_orders_customer_id` index if it exists. Time `SELECT count(*) FROM big_orders WHERE customer_id = 7;` twice (the second run is fairer: the first includes reading from disk).
 2. `EXPLAIN` the same query. Write down the scan type.
 3. Create the index. Time the query twice more, and `EXPLAIN` it again.
 4. In a comment: how many times faster was it, and what changed in the plan?
-

Exercise 2 (Easy): Sizes and lists

1. Show the table's size and the index's size side by side with `pg_size_pretty(pg_relation_size(...))`.
 2. List all indexes in the database with `\di`.
 3. Run `\d order_items in sales` and answer in a comment: which lookups on this table are indexed, and which foreign key column isn't? Write the `CREATE INDEX` that would fix it (and run it).
-

Exercise 3 (Medium): Column order

1. Create `idx_big_orders_customer_date` ON `big_orders` (`customer_id`, `ordered_at`).
2. `EXPLAIN` three queries: filter on `customer_id` only; on `customer_id` and `ordered_at`; on `ordered_at` only. Which use the index?
3. Drop it and create the reverse, (`ordered_at`, `customer_id`). Run the same three `EXPLAIN`s. What changed?
4. In a comment: the shop's most common query is "this customer's orders, newest first". Which column order is right, and why?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Case-insensitive uniqueness

1. On the real `customers` table, create a **unique expression index** on `lower(email)`.
 2. Try to insert a customer with email `'BIKRAM@example.com'`. It must fail. Copy the error.
 3. Try to insert one with `'bikram.s@example.com'`. It must succeed. Then delete it again.
 4. In a comment: why couldn't a plain `UNIQUE (email)` constraint from chapter 13 do this?
-

Exercise 5 (Challenge): The price of writing

1. Create a fresh copy: `CREATE TABLE big_copy AS SELECT * FROM big_orders;` (no indexes at all). Time an insert of 100,000 new rows from `generate_series`.
2. Add four indexes to `big_copy` (on `customer_id`, `ordered_at`, `total`, and `(customer_id, ordered_at)`). Time the same insert again.
3. Create a partial index on `big_copy (ordered_at) WHERE total > 490`, and `EXPLAIN SELECT * FROM big_copy WHERE total > 495 ORDER BY ordered_at LIMIT 10;`. Does it use the partial index? What about `WHERE total > 100`?
4. Drop `big_copy`. In a comment: write the rule of thumb you'd give a teammate about how many indexes a table should have.

Stuck? The hint is in the [Hints section](#) at the end.

32 Why Is My Query Slow?

What is it?

`EXPLAIN` shows you the **plan** PostgreSQL will use to run a query: which tables it reads, in what order, how (scan the whole thing or use an index), and how it joins and sorts them. `EXPLAIN ANALYZE` actually runs the query and shows what *really* happened, with timings.

Reading a plan is how you find out *why* a query is slow, instead of guessing.

Why does it matter?

Someday a page in your app will take four seconds to load, and the database will be the reason. You'll have two choices: add indexes at random and hope, or look at the plan, see "Seq Scan on orders, 2 million rows, filter removed 1,999,990", and know exactly what to do.

The second way takes five minutes. The first can take a week. This chapter is the five-minute version.

Real-world example

A sat-nav that shows its route before you drive:

Sat-nav	PostgreSQL
"Take the motorway, then exit 12, then the B road"	The plan: scan this, join that, sort
Estimated time: 45 minutes	<code>cost=...</code> numbers (an estimate, in made-up units)
Actual drive: 52 minutes, because of a jam at exit 12	<code>EXPLAIN ANALYZE</code> 's <code>actual time</code>
"Why did it go the long way?"	Reading the plan to find the slow step

`EXPLAIN` is the preview. `EXPLAIN ANALYZE` is the drive with a stopwatch.

How it works

Work in `sales`, with `big_orders` still around from chapter 31 (recreate it if you dropped it) and its index on `customer_id`.

EXPLAIN : the plan

```
EXPLAIN SELECT * FROM products WHERE price > 20;
```

QUERY PLAN

```
-----  
Seq Scan on products (cost=0.00..18.38 rows=223 width=93)  
  Filter: (price > '20'::numeric)  
(2 rows)
```

One step: read all of `products` , keep rows where `price > 20` . The numbers in brackets are **estimates**:

- `cost=0.00..18.38` : effort to produce the first row, then all rows, in PostgreSQL's own units. Only useful for comparing plans against each other.
- `rows=223` : how many rows it *expects*. (It guessed 223 for an 11-row table because the statistics are stale; that's normal on tiny tables.)
- `width=93` : average bytes per row.

Nothing ran. `EXPLAIN` only asks the planner what it *would* do.

EXPLAIN ANALYZE : the plan plus reality

```
EXPLAIN ANALYZE SELECT * FROM products WHERE price > 20;
```

QUERY PLAN

```
-----  
Seq Scan on products (cost=0.00..18.38 rows=223 width=93) (actual time=0.008..0.008 rows=4.00  
loops=1)  
  Filter: (price > '20'::numeric)  
  Rows Removed by Filter: 7  
  Buffers: shared hit=1  
Planning Time: 0.023 ms  
Execution Time: 0.012 ms  
(6 rows)
```

Now there's a second bracket: **actual** . `rows=4.00` really came out (not 223). `Rows Removed by Filter: 7` says how many were read and thrown away. `Execution Time` is the real total.

EXPLAIN ANALYZE runs the query. For a `SELECT` , that's harmless. For an `UPDATE` or `DELETE` , it really changes data. Wrap those in `BEGIN ... ROLLBACK` .

Reading a plan

Plans are trees, printed with the **innermost steps indented furthest**. Read from the most indented line outwards: that's the order things happen.

```
EXPLAIN SELECT o.id, c.name FROM orders o JOIN customers c ON c.id = o.customer_id;
```

```
Hash Join (cost=1.14..23.44 rows=970 width=36)
  Hash Cond: (o.customer_id = c.id)
    -> Seq Scan on orders o (cost=0.00..19.70 rows=970 width=8)
    -> Hash (cost=1.06..1.06 rows=6 width=36)
      -> Seq Scan on customers c (cost=0.00..1.06 rows=6 width=36)
```

Bottom up: scan `customers` (6 rows), build a hash table from them (a fast in-memory lookup), scan `orders`, and for each order look up its customer in the hash. That's a **Hash Join**, the usual way to join a big table to a small one.

The node names you'll see most:

Node	What it does	Good or bad?
Seq Scan	Reads every row of a table	Fine on small tables. Suspicious on big ones with a <code>Filter</code> .
Index Scan	Uses an index to fetch matching rows	Good for a few rows.
Bitmap Index Scan + Bitmap Heap Scan	Uses an index to find many scattered rows, then fetches them in disk order	Good for hundreds to thousands of rows.
Index Only Scan	Answers from the index alone, never touching the table	Best case.
Hash Join	Builds a lookup table from one side, probes with the other	Normal for big-to-small joins.
Nested Loop	For each row on one side, look up the other side	Great when the inner side is indexed and the outer is small. Terrible otherwise.
Merge Join	Both sides sorted, walked together	Normal for two big sorted inputs.
Sort	Sorts rows (for <code>ORDER BY</code> , <code>DISTINCT</code> , merge joins)	Fine for small sets. Expensive on big ones.

Node	What it does	Good or bad?
HashAggregate / GroupAggregate	Does the <code>GROUP BY</code>	Normal.
Limit	Stops after N rows	Good, if what's beneath it can stop early.
Gather / Parallel ...	Work split across CPU cores	Means the table was big enough to bother.

You don't need to memorize these. You need to spot two things: a **Seq Scan on a big table with a Filter**, and a **Sort over a big input**. Those are where time goes.

Finding the slow step

A real one. Newest five orders:

```
EXPLAIN ANALYZE SELECT * FROM big_orders ORDER BY ordered_at DESC LIMIT 5;
```

```
Limit (cost=20232.10..20232.69 rows=5 width=22) (actual time=56.539..59.345 rows=5.00 loops=1)
  -> Gather Merge (...)
        -> Sort (cost=19232.08..20377.91 rows=458333 width=22) (actual time=52.909..52.910
rows=5.00 loops=3)
          Sort Key: ordered_at DESC
          Sort Method: top-N heapsort Memory: 25kB
          -> Parallel Seq Scan on big_orders (cost=0.00..11619.33 rows=458333 width=22)
(actual time=0.196..24.575 rows=366666.67 loops=3)
Planning Time: 0.073 ms
Execution Time: 59.400 ms
```

Read from the bottom: scan **all** million rows (24 ms), **sort** them all by date (another 28 ms), then keep 5. Sixty milliseconds to get five rows, because there's no way to find "the newest" without looking at everything. The slow step is the Sort fed by a full scan.

The fix is an index on the sort column, because an index *is* a sorted list:

```
CREATE INDEX idx_big_orders_ordered_at ON big_orders (ordered_at);
EXPLAIN ANALYZE SELECT * FROM big_orders ORDER BY ordered_at DESC LIMIT 5;
```

```

Limit (cost=0.43..0.64 rows=5 width=22) (actual time=0.030..0.032 rows=5.00 loops=1)
  -> Index Scan Backward using idx_big_orders_ordered_at on big_orders (cost=0.43..47180.85
rows=1100000 width=22) (actual time=0.029..0.030 rows=5.00 loops=1)
Planning Time: 0.218 ms
Execution Time: 0.041 ms

```

From 59 ms to 0.04 ms: over a thousand times faster. PostgreSQL walked the index backwards from the end, read five entries, and stopped. No scan, no sort. This is the single most common fix in web apps: "latest N things" needs an index on the date column.

Estimates vs reality

Compare the two `rows=` numbers on each line. In the plan above, the planner estimated `rows=996` for customer 42 and the real answer was `1050`: close, so it made a good decision. When the estimate is wildly off (expects 10, gets 100,000), the planner may pick a plan that's great for 10 rows and awful for 100,000. The fix is usually `ANALYZE table_name` to refresh statistics.

Buffers: disk or memory?

```

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM big_orders WHERE customer_id = 42;

```

```

Bitmap Heap Scan on big_orders (...) (actual time=0.188..1.102 rows=1050.00 loops=1)
  Buffers: shared hit=972
  ...

```

`shared hit` means pages found in memory; `read` means fetched from disk, which is far slower. A query that's fast the second time and slow the first is paying for disk reads. Not much to do about it except have enough memory and not read more than you need.

The checklist

When a query is slow, in this order:

1. **EXPLAIN ANALYZE it.** Find the step with the biggest `actual time`.
2. **Seq Scan with a Filter on a big table?** Index the filtered column (chapter 31).
3. **Sort over a big input, feeding a Limit?** Index the `ORDER BY` column.
4. **A function around an indexed column?** `WHERE lower(email) = ...`, `WHERE date(created_at) = ...`. Rewrite as a range (`created_at >= ... AND created_at < ...`) or add an expression index.
5. **LIKE '%word%' ?** No normal index can help. Chapter 39.
6. **Nested Loop with a Seq Scan inside?** The inner table needs an index on the join column.

7. **Estimates way off?** `ANALYZE` the table.
8. **Selecting columns you don't use?** `SELECT *` on wide rows moves more data than needed. Ask for what you need. (It also lets the planner use Index Only Scans.)
9. **Running the query a thousand times?** See below.

The N+1 problem

The slowest "query" is often not one query but a thousand small ones. A program fetches 12 orders, then loops and runs `SELECT * FROM order_items WHERE order_id = ?` for each. That's $1 + 12 = 13$ round trips to the database. With 10,000 orders it's 10,001. Each is fast; together they take forever.

The database can't fix this; it only sees fast little queries. The fix is in the program: one query with a `JOIN` (or `WHERE order_id IN (...)`) that fetches everything at once. [Chapter 42](#) shows how ORMs make this mistake easy, and how to avoid it.

Finding slow queries in the first place

In a real system you don't know which query is slow until a user complains. PostgreSQL ships an extension, `pg_stat_statements`, that records every query's total and average time. Turning it on is a configuration change (adding it to `shared_preload_libraries` and restarting), and then `SELECT * FROM pg_stat_statements ORDER BY mean_exec_time DESC LIMIT 10` shows your ten worst. Worth knowing exists; you'll meet it again when you run a real server.

Clean up when you're done:

```
DROP INDEX idx_big_orders_ordered_at;  
DROP TABLE big_orders;
```

Common mistakes

1. Guessing instead of explaining

Adding indexes "to be safe" slows writes and may not touch the real problem. Look first.

2. Reading the plan top-down

The first line is the *last* step. Start at the deepest indent.

3. Panicking about Seq Scans on small tables

Reading 11 rows directly is faster than any index. Seq Scan on a small table is correct.

4. `EXPLAIN ANALYZE` on a `DELETE`

It runs. Use `BEGIN ... ROLLBACK`.

5. Trusting `cost` as time

Costs are relative units for comparing plans. Only `actual time` is milliseconds.

6. Fixing one query, forgetting the thousand

N+1 looks fine in `EXPLAIN` and kills the app. Count the round trips.

Quick recap

- `EXPLAIN` shows the plan; `EXPLAIN ANALYZE` runs it and shows actual rows and times. Read from the deepest indent outward.
 - Spot **Seq Scan + Filter** on big tables and **Sort** over big inputs. Those are the slow steps.
 - An index on the filtered column turns Seq Scan into Index Scan. An index on the `ORDER BY` column turns Sort + Limit into a quick Index Scan.
 - Functions around indexed columns and `LIKE '%.%.%'` block index use. Rewrite or use an expression index.
 - Compare estimated and actual rows; `ANALYZE` when they disagree badly.
 - The N+1 problem lives in the program, not the plan. One join beats a thousand lookups.
-

Next: try the [exercises](#), then move on to [33 Views](#).

Exercises

How to do these:

- Work in psql, connected to `sales`, with `big_orders` (chapter 31) and its `customer_id` index.
 - Save each answer in `playground/ch32/ex1.sql`, `ex2.sql`, and so on. Paste the relevant plan lines as comments, and note the `Execution Time` before and after each fix.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Read the plan

Run `EXPLAIN ANALYZE` on these and, for each, write down: the node type at the deepest indent, the estimated vs actual rows on that line, and the total execution time.

```
SELECT * FROM customers WHERE city = 'Pokhara';
SELECT * FROM big_orders WHERE id = 123456;
SELECT * FROM big_orders WHERE customer_id = 123;
SELECT count(*) FROM big_orders WHERE total > 499;
```

Which of the four reads the whole table? Is that a problem for each one? Say why or why not.

Exercise 2 (Easy): The newest N

1. `EXPLAIN ANALYZE SELECT * FROM big_orders ORDER BY total DESC LIMIT 10;` Find the Sort and note the time.
2. Add the index that fixes it. Run again. How many times faster?
3. Now `EXPLAIN ANALYZE SELECT * FROM big_orders ORDER BY total DESC, id LIMIT 10;` (a tie-breaker added). Does your index still help fully? What index would?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): The function trap

A developer wrote these to find a customer's orders on a given day:

```
EXPLAIN ANALYZE SELECT * FROM big_orders WHERE customer_id = 42 AND date(ordered_at) = '2026-03-15';
```

1. Does it use the `customer_id` index? Does it use anything for the date? What does the plan have to do with the 1000-odd rows for customer 42?
 2. Rewrite the date condition as a **range** on `ordered_at` (from midnight to midnight) and compare plans.
 3. Create a two-column index `(customer_id, ordered_at)`. Run the range version again. Describe the plan.
 4. In a comment: why does the `date(...)` version block the index even after step 3?
-

Exercise 4 (Medium): Joins big and small

1. Create a `big_customers` table with 1001 rows: `CREATE TABLE big_customers AS SELECT g AS id, 'Customer ' || g AS name FROM generate_series(1, 1001) g;` (no primary key yet).
2. `EXPLAIN ANALYZE` a join of `big_orders` to `big_customers` on `customer_id = id`, filtered to `big_orders.customer_id BETWEEN 1 AND 3`, returning order id and customer name. Note the join type and time.

3. Add a primary key to `big_customers`. Run again. Did the plan change?
4. Drop `big_customers`. In a comment: explain why indexing the **small** table's join column mattered here.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): The N+1 problem, measured

1. Write a `.sql` file that does the N+1 version by hand for the real `sales` tables: one `SELECT id FROM orders;` followed by **twelve** separate `SELECT * FROM order_items WHERE order_id = N;` statements. Run it with `\timing on` and add up the times.
2. Write the one-query version with a `JOIN`. Time it.
3. In a comment: the twelve small queries each took well under a millisecond. Explain to a teammate why the N+1 pattern is still a serious problem in a web app with 10,000 orders. Mention round trips.
4. Drop `big_orders` and any indexes you made. You're done with it.

33 Views

What is it?

A **view** is a saved query that you can use like a table:

```
CREATE VIEW order_totals AS
SELECT o.id AS order_id, o.customer_id, o.ordered_at, o.status,
       sum(oi.quantity * oi.unit_price) AS total
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
GROUP BY o.id;

SELECT * FROM order_totals WHERE total > 50;
```

The view stores the *query*, not the data. Every time you select from it, PostgreSQL runs the saved query against the live tables. So it's always up to date, and it takes no space.

Why does it matter?

You wrote "total each order" as a CTE in chapters 26, 28, and 29, over and over. A view writes it **once**, gives it a name, and every query in the project can use it. When the definition needs to change (say, to exclude cancelled orders), you change it in one place.

Views also hide complexity and hide columns. A `public_products` view that shows `name` and `price` but not `cost_price` or `supplier_id` can be handed to someone who should never see those. [Chapter 34](#) uses that.

Real-world example

A **saved search** or a **smart playlist**. You define "songs I rated 5 stars added this year" once. Every time you open it, it shows the current matches. It isn't a copy of the songs; it's a rule that's re-applied each time. Add a song that matches, and it appears. That's a view.

A **materialized view**, which you'll meet below, is the same playlist burned to a CD: fast to play, but it doesn't change until you burn it again.

How it works

Work in `sales`, with `\pset null '[NULL]'`.

Creating and using a view

Create `order_totals` as above. Then:

```
SELECT * FROM order_totals ORDER BY order_id LIMIT 3;
```

order_id	customer_id	ordered_at	status	total
1	1	2026-07-03 10:15:00+05:45	paid	13.00
2	2	2026-07-10 14:30:00+05:45	shipped	32.99
3	3	2026-07-18 09:05:00+05:45	shipped	149.00

(3 rows)

It looks like a table with five columns. Join to it, filter it, group it:

```
SELECT c.name, count(*) AS orders, sum(ot.total) AS spent
FROM order_totals ot
JOIN customers c ON c.id = ot.customer_id
WHERE ot.status <> 'cancelled'
GROUP BY c.name
ORDER BY spent DESC
LIMIT 3;
```

name	orders	spent
Chandra Gurung	2	239.00
Asha Rai	3	218.19
Dipesh Thapa	2	86.23

(3 rows)

The chapter 26 two-CTE query, with the first CTE replaced by a view. Shorter, and the "total each order" logic now lives in exactly one place.

Views are always current

```
BEGIN;
UPDATE order_items SET quantity = 10 WHERE order_id = 1 AND product_id = 1;
SELECT order_id, total FROM order_totals WHERE order_id = 1;  -- 41.00
ROLLBACK;
SELECT order_id, total FROM order_totals WHERE order_id = 1;  -- 13.00
```

The view reflected the change instantly, and the rollback instantly too. There's no copy to get stale. It's just the query, re-run.

Seeing what's there

```
\dv          -- list views
\d+ order_totals  -- columns, and the saved query
```

The `\d+` output ends with `View definition:` and the SQL, slightly reformatted by PostgreSQL. Useful when you've forgotten what a view does.

Hiding columns and rows

```
CREATE VIEW public_products AS
SELECT id, name, price FROM products WHERE is_active;
```

Anyone using `public_products` sees three columns of active products. They can't see `stock`, `sku`, or discontinued items, because the view doesn't include them. Combined with permissions (next chapter), this is how you give a reporting tool or a junior colleague exactly what they need and nothing more.

Changing a view

```
CREATE OR REPLACE VIEW public_products AS
SELECT id, name, price, stock FROM products WHERE is_active;
```

`CREATE OR REPLACE` swaps the definition. One limit: you can **add** columns at the end but not **remove** or reorder existing ones, because other views or queries may depend on them:

```
CREATE OR REPLACE VIEW public_products AS SELECT id, name FROM products WHERE is_active;
```

```
ERROR:  cannot drop columns from view
```

To shrink a view, `DROP VIEW` and `CREATE VIEW` again.

Views on views

A view can select from another view:

```
CREATE VIEW customer_spending AS
SELECT c.id AS customer_id, c.name, c.city,
       count(ot.order_id) AS orders,
       coalesce(sum(ot.total), 0) AS spent
FROM customers c
LEFT JOIN order_totals ot ON ot.customer_id = c.id AND ot.status <> 'cancelled'
GROUP BY c.id;

SELECT * FROM customer_spending ORDER BY spent DESC;
```

customer_id	name	city	orders	spent
3	Chandra Gurung	Lalitpur	2	239.00
1	Asha Rai	Kathmandu	3	218.19
4	Dipesh Thapa	Kathmandu	2	86.23
6	Farhan Ali	Pokhara	2	64.75
2	Bikram Shrestha	Pokhara	2	55.99
5	Elina Maharjan	Lalitpur	0	0

(6 rows)

Layers of views are like layers of CTEs: each one small and readable. Two or three layers is fine. Ten layers deep makes plans hard to read and slow; keep an eye on it.

Views create dependencies

```
DROP TABLE orders;
```

```
ERROR: cannot drop table orders because other objects depend on it
DETAIL: constraint order_items_order_id_fkey on table order_items depends on table orders
view order_totals depends on table orders
view customer_spending depends on view order_totals
HINT: Use DROP ... CASCADE to drop the dependent objects too.
```

PostgreSQL won't pull a table out from under a view. That's protection. `DROP TABLE orders CASCADE` would drop the views too, which is rarely what you want. Also, renaming a column that a view uses is blocked the same way. In a project with many views, every schema change has to consider them, which is one reason not to make a view for everything.

Can you write through a view?

Simple views (one table, no `GROUP BY`, no aggregates, no `DISTINCT`) are **automatically updatable**:


```
BEGIN;
UPDATE public_products SET price = 4.00 WHERE id = 1 RETURNING *;
SELECT id, name, price FROM products WHERE id = 1;    -- 4.00: the real table changed
ROLLBACK;
```

PostgreSQL translates the update onto `products`. Views with grouping can't be updated this way:

```
UPDATE order_totals SET total = 0 WHERE order_id = 1;
```

```
ERROR:  cannot update view "order_totals"
DETAIL:  Views containing GROUP BY are not automatically updatable.
```

Which makes sense: `total` is computed, so what would it mean to set it? Mostly you'll use views for reading. Updatable simple views are a nice bonus for the "hide some columns" case.

Materialized views: a saved copy

A normal view re-runs its query every time. If the query is slow (a complex report over millions of rows) and the data doesn't need to be live to the second, store the result instead:

```
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT date_trunc('month', ordered_at)::date AS month, count(*) AS orders, sum(total) AS revenue
FROM order_totals
WHERE status <> 'cancelled'
GROUP BY 1;

SELECT * FROM monthly_revenue ORDER BY month;
```

month	orders	revenue
2026-07-01	3	194.99
2026-08-01	4	288.48
2026-09-01	4	180.69

(3 rows)

That result is **stored**, like a table. Reading it is instant no matter how slow the query behind it is. The cost is that it goes stale:

```

BEGIN;
INSERT INTO orders (customer_id, ordered_at, status) VALUES (5, '2026-09-28 10:00+05:45',
    'paid');
INSERT INTO order_items VALUES (currval(pg_get_serial_sequence('orders', 'id')), 2, 100, 1.20);
SELECT * FROM monthly_revenue WHERE month = '2026-09-01';    -- still 4 orders, 180.69
REFRESH MATERIALIZED VIEW monthly_revenue;
SELECT * FROM monthly_revenue WHERE month = '2026-09-01';    -- now 5 orders, 300.69
ROLLBACK;
REFRESH MATERIALIZED VIEW monthly_revenue;                    -- back to 4

```

New data doesn't appear until `REFRESH MATERIALIZED VIEW`. In practice you schedule the refresh (hourly, nightly) and accept that the dashboard is a little behind. `\dm` lists materialized views. They can have indexes, like tables.

Use a materialized view when the query is expensive and slightly-old data is fine. Use a plain view everywhere else.

Dropping views

```

DROP MATERIALIZED VIEW monthly_revenue;
DROP VIEW customer_spending;
DROP VIEW public_products;
DROP VIEW order_totals;

```

Drop dependents first (or use `CASCADE` on the one they depend on).

Where views fit

Use a view for	Example
A calculation many queries share	<code>order_totals</code>
Hiding columns or rows from some users	<code>public_products</code> , with permissions
Giving a long join a short name	<code>order_details</code> with customer, product, and item columns
A stable interface while tables change underneath	Rename a column, update the view, nobody's queries break

Don't use a view for	Why
Something used by one query	A CTE is simpler and has no dependency
Storing data	Views store queries. Use a table (or a materialized view)

Don't use a view for	Why
Replacing every join in the project	Ten layers of views are slow to read and slow to run

Common mistakes

1. Expecting a view to be faster

A plain view is exactly as fast as its query. It saves typing, not time. For speed, materialize or index.

2. Forgetting a materialized view is stale

The dashboard shows yesterday's numbers because nobody refreshed. Schedule the refresh, and show the refresh time on the dashboard.

3. CASCADE on a DROP TABLE

Takes all dependent views with it, silently. Read the `DETAIL` list first.

4. Too many layers

A view on a view on a view. Each layer is a query; the planner has to merge them all. Flatten when it gets deep.

5. Trying to drop a column via CREATE OR REPLACE

Not allowed. `DROP VIEW` then `CREATE VIEW`.

6. Views as the only security

A view hides columns, but only if the user can't also read the base table. Pair it with the permissions in the next chapter.

Quick recap

- `CREATE VIEW name AS SELECT ...` saves a query to use like a table. It stores the query, not the data, so it's always current.
- Use views for shared calculations, hiding columns, naming long joins, and a stable interface.
- `CREATE OR REPLACE` can add columns, not remove them. Views block dropping or renaming what they depend on.
- Simple one-table views can be updated; grouped views can't.
- **Materialized views** store the result: fast, but stale until `REFRESH MATERIALIZED VIEW`.
- `\dv`, `\dm`, `\d+ view_name`.

Next: try the [exercises](#), then move on to [34 Users, Roles and Permissions](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
 - Save each answer in `playground/ch33/ex1.sql`, `ex2.sql`, and so on. Start files with `DROP VIEW IF EXISTS ...` so they're re-runnable.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): The three views every report needs

Create these three views, then prove each works with a short `SELECT`:

1. `live_orders`: all columns of `orders` where status isn't cancelled.
2. `order_totals`: one row per order with `order_id`, `customer_id`, `ordered_at`, `status`, `total` (as in the notes).
3. `order_details`: one row per order **line** with order id, order date, customer name, product name, category name, quantity, unit price, and line total. Every join from Level 3 in one place.

Then write "revenue per category, live orders only" as a `SELECT` from `order_details`. It should be about four lines.

Exercise 2 (Easy): Hiding things

1. Create `customer_directory` showing each customer's id, name, and city but **not** email or join date.
 2. Try to `UPDATE customer_directory SET city = 'Bhaktapur' WHERE id = 5;` inside a transaction, then roll back. Did it work? Why?
 3. Create `public_products` (active products, no stock or SKU). Try `UPDATE public_products SET stock = 0 WHERE id = 1;`. What happens, and why is that good?
-

Exercise 3 (Medium): Rewrite the report

Take **three** sections of your chapter 29 `report.sql` (headline numbers, monthly trend, per-customer spending) and rewrite them to use `order_totals` and `live_orders` (or `order_details`) instead of their CTEs. They must

produce exactly the same numbers as before.

In a comment: which version is easier to read? What's one disadvantage of the view version?

Exercise 4 (Medium): Dependencies

1. Create a view `customer_spending` on top of `order_totals` (as in the notes).
 2. Try to drop `order_totals`. Copy the error.
 3. Try to rename `orders.status` to `state` with `ALTER TABLE`. Copy the error.
 4. Try to add a column to `order_totals` with `CREATE OR REPLACE` (say, `count(*) AS lines`). Does it work? Now try to remove `status` from it the same way. Does that?
 5. In a comment: explain to a teammate what they need to check before changing a table in a database that has views.
-

Exercise 5 (Challenge): A dashboard that refreshes

1. Create a **materialized view** `dashboard` with one row: live order count, live revenue, average order value, and `now()` as `refreshed_at`.
2. Select from it. Note `refreshed_at`.
3. Inside a transaction, add an order with one item worth 50.00, select from `dashboard` (unchanged), refresh it, select again (changed), then roll back and refresh once more.
4. Create a plain view `dashboard_live` with the same query (no `refreshed_at`). Select from both after an insert-and-rollback. In a comment: when would you choose each one?
5. Add an index to the materialized view on `refreshed_at`. (Yes, you can.) Then drop everything you made in this exercise.

Stuck? The hint is in the [Hints section](#) at the end.

34 Users, Roles and Permissions

What is it?

So far you've done everything as `postgres`, the **superuser**, who can do anything to anything. That's fine for learning. It's dangerous for a real system.

A **role** is an account in PostgreSQL. It can be a person, a program, or a group. **Permissions** (PostgreSQL calls them privileges) say what each role may do: read this table, write that one, nothing at all to a third.

```
CREATE ROLE reporter LOGIN PASSWORD 'report123';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO reporter;
```

Now `reporter` can read every table and change nothing.

Why does it matter?

Three reasons, in order of how soon they'll bite you:

- 1. **Mistakes.** A report script that connects as a read-only role can't accidentally `DELETE FROM orders`. The database says no, before the damage.
- 2. **Attackers.** If your web app's database password leaks (it happens), an app role that can only read and insert does far less harm than a superuser.
- 3. **Privacy.** The analyst needs sales numbers, not customer emails. A role that can see a view without the email column solves it.

The principle is called **least privilege**: every role gets exactly what it needs to do its job, and nothing else. It's the single most effective security habit in databases, and it costs nothing.

Real-world example

Keys in an office building:

Office	PostgreSQL
The master key that opens everything	The <code>postgres</code> superuser
A visitor badge: read the noticeboards, open no doors	A read-only role
A staff key: your own office and the shared kitchen	An app role: write to its tables, read the rest

Office	PostgreSQL
"All cleaners get these doors"	A group role, with people added to it
Someone leaves: take the key back	REVOKE , then DROP ROLE

Nobody gives the visitor the master key "because it's easier".

How it works

Work as `postgres` in `sales` , in one psql window. You'll open a second window to connect as the new roles.

Creating a role that can log in

```
CREATE ROLE reporter LOGIN PASSWORD 'report123';
```

`LOGIN` means this role can connect (a role without `LOGIN` is a group, below). The password is for connecting over the network, which includes your own laptop's SQL Shell.

See all roles:

```
\du
```

Role name	Attributes
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS
reporter	

The new role has no attributes: it's an ordinary account.

Connecting as the new role

Open a second SQL Shell. When it asks for the username, type `reporter` instead of accepting `postgres` , then the password `report123` . (Or in a terminal: `psql -U reporter -d sales` .) The prompt will be `sales=>` , with `>` instead of `#` : you're not a superuser.

Try to read something:

```
SELECT count(*) FROM orders;
```

```
ERROR: permission denied for table orders
```

A brand-new role can connect, and that's all. It can't read a single table. This is least privilege by default, and it's exactly right.

Granting read access

Back in the `postgres` window:

```
GRANT CONNECT ON DATABASE sales TO reporter;  
GRANT USAGE ON SCHEMA public TO reporter;  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO reporter;
```

Three layers:

- `CONNECT ON DATABASE` : may connect to this database at all. (Often granted to everyone by default, which is why `reporter` could connect before, but say it explicitly.)
- `USAGE ON SCHEMA public` : may see the tables in the `public` schema (the section of the database where your tables live).
- `SELECT ON ALL TABLES` : may read them.

Now in the `reporter` window:

```
SELECT count(*) FROM orders;           -- 12. Works.  
INSERT INTO categories (name) VALUES ('Hacked');
```

```
ERROR: permission denied for table categories
```

```
DELETE FROM orders;
```

```
ERROR: permission denied for table orders
```

```
DROP TABLE products;
```

```
ERROR: must be owner of table products
```

Reading works. Everything else is refused. The report script can't hurt anything.

Seeing who can do what

```
\dp orders
```

Schema	Name	Type	Access privileges	Column privileges	Policies
public	orders	table	postgres=arwdDxtm/postgres+ reporter=r/postgres		

The letters: **r** read (SELECT), **a** append (INSERT), **w** write (UPDATE), **d** DELETE, **D** TRUNCATE, **x** REFERENCES, **t** TRIGGER, **m** MAINTAIN. `postgres` has all of them; `reporter` has **r**. The `/postgres` says who granted it.

Tables you create later

`GRANT ... ON ALL TABLES` covers the tables that exist **now**. Make a new one:

```
CREATE TABLE audit_log (id integer);
```

and `reporter` gets permission denied for table `audit_log`. To cover future tables too:

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO reporter;
```

From now on, every table `postgres` creates in `public` is readable by `reporter` automatically. Do both: `GRANT ... ON ALL TABLES` for what exists, `ALTER DEFAULT PRIVILEGES` for what's coming. Forgetting the second is the classic "it worked yesterday" permissions bug.

A role for your app

Your web app shouldn't be a superuser either. It needs to read everything, create orders, and adjust stock. Nothing more:

```
CREATE ROLE shop_app LOGIN PASSWORD 'app123';  
GRANT CONNECT ON DATABASE sales TO shop_app;  
GRANT USAGE ON SCHEMA public TO shop_app;  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO shop_app;  
GRANT INSERT, UPDATE ON orders, order_items TO shop_app;  
GRANT UPDATE (stock) ON products TO shop_app;
```

That last line is a **column-level** grant: `shop_app` may update `stock` and no other column of `products`. Connect as `shop_app` and try:

```
INSERT INTO orders (customer_id, ordered_at, status) VALUES (5, now(), 'pending') RETURNING id;
-- works
UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING name, stock;
-- works
UPDATE products SET price = 0 WHERE id = 1;
```

ERROR: permission denied for table products

```
DELETE FROM orders WHERE customer_id = 5;
```

ERROR: permission denied for table orders

The app can take orders and adjust stock. It cannot change prices or delete history. If its password leaks, an attacker gets the same limits. (Clean up that test order as `postgres` afterwards.)

Identity columns need no extra grants: inserting into `orders` worked without touching its sequence. (Older `serial` columns sometimes need `GRANT USAGE ON SEQUENCE`; another small reason to prefer `IDENTITY`.)

Group roles

Grant permissions to a **group**, then put people in it:

```
CREATE ROLE readonly NOLOGIN;
GRANT USAGE ON SCHEMA public TO readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;

CREATE ROLE analyst LOGIN PASSWORD 'an123' IN ROLE readonly;
GRANT CONNECT ON DATABASE sales TO analyst;
```

`readonly` can't log in; it's a bundle of permissions. `analyst` is a member (`IN ROLE readonly`) and inherits them. When a second analyst joins, it's one line: `CREATE ROLE ... IN ROLE readonly` . When the read-only set needs a new table, you grant it to `readonly` once. `GRANT readonly TO existing_role` adds an existing role to the group.

This is how real teams manage it: a handful of group roles (`readonly` , `app_writer` , `admin`), and every person or program is a member of one.

Taking it away

```
REVOKE SELECT ON ALL TABLES IN SCHEMA public FROM reporter;
```

reporter is back to permission denied. To remove the role entirely:

```
DROP ROLE reporter;
```

```
ERROR:  role "reporter" cannot be dropped because some objects depend on it
DETAIL:  privileges for schema public
         privileges for database sales
         privileges for default privileges on new relations belonging to role postgres in schema public
```

A role that owns anything or holds any grant can't be dropped until those are cleared. The shortcut:

```
DROP OWNED BY reporter;    -- removes its privileges and anything it owns, in this database
DROP ROLE reporter;
```

(If the role has grants in several databases, run `DROP OWNED BY` in each first.)

Who am I?

```
SELECT current_user;
SELECT rolsuper FROM pg_roles WHERE rolname = current_user;
```

Handy in scripts, and in the panic moment of "wait, which account am I running this as?"

Views plus permissions

Chapter 33's `public_products` view hides columns. With permissions, it becomes real protection:

```
GRANT SELECT ON public_products TO analyst;
-- and do NOT grant SELECT on products itself
```

Now `analyst` can read names and prices, and literally cannot read `sku` or `stock`, because the only path to them is a table they can't touch. For truly sensitive data (emails, phone numbers, addresses), this is the pattern: a view with the safe columns, granted; the table, not.

Where connections are allowed from

Permissions decide what a role may *do*. A separate file, `pg_hba.conf` ("host-based authentication"), decides who may *connect*, from where, and how they prove it. On your Windows install it's in the data directory (`SHOW hba_file`; tells you the exact path). Its default lines say roughly "local connections must give a password". When

you run a real server, you'll edit it to allow your app server's address and nothing else. Just know it's there; you won't need to touch it in this course.

Two more things, briefly

- **Row-level security** lets a table show different rows to different roles ("customers see only their own orders") with policies attached to the table. Powerful, and a chapter of its own in a more advanced course.
- **Never put passwords in SQL files you share.** `CREATE ROLE ... PASSWORD 'report123'` in a committed file is a leak. Set passwords interactively (`\password reporter` in `psql`) or from a secrets store.

Common mistakes

1. Running the app as `postgres`

The most common mistake in small projects. One leaked password, total loss. Make an app role on day one.

2. Forgetting `ALTER DEFAULT PRIVILEGES`

New tables are invisible to everyone but their owner. Grant for existing tables *and* future ones.

3. Granting more than needed "for now"

"For now" lasts years. Grant the minimum, add when asked.

4. `DROP ROLE` without `DROP OWNED BY`

Fails with a dependency error. Clear the privileges first.

5. Protecting a view but not the table

If the role can read the table, the view hides nothing. Grant on the view only.

6. Passwords in version control

Set them interactively. Treat SQL files as public.

Quick recap

- A **role** is an account. `LOGIN` roles connect; `NOLOGIN` roles are groups.
- New roles can do **nothing** until granted. Grant `CONNECT` on the database, `USAGE` on the schema, then table privileges (`SELECT` , `INSERT` , `UPDATE` , `DELETE`), even per column.
- `GRANT ... ON ALL TABLES` covers existing tables; `ALTER DEFAULT PRIVILEGES` covers future ones. Do both.
- Make group roles (`readonly` , `app_writer`) and add people with `IN ROLE` or `GRANT group TO role` .
- Give your app its own role with the minimum it needs. Never run it as `postgres` .

- Views + grants hide sensitive columns for real. `REVOKE` takes rights away; `DROP OWNED BY` then `DROP ROLE` removes an account.
 - `\du` , `\dp table` , `current_user` .
-

Next: try the [exercises](#), then move on to [35 Backup and Restore](#).

Exercises

How to do these:

- Work as `postgres` in one psql window on `sales` , and open a second window to connect as each new role (SQL Shell asks for the username; or `psql -U rolename -d sales`).
 - Save each answer in `playground/ch34/ex1.sql` , `ex2.sql` , and so on. Record what each role could and couldn't do as comments, with the exact error messages.
 - **Clean up** at the end of each exercise: `DROP OWNED BY role; DROP ROLE role;` .
 - Use throwaway passwords, and don't commit these files anywhere public.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): A read-only reporter

1. Create `reporter` with `LOGIN` and a password. Connect as it in the second window. Try `SELECT count(*) FROM products;` and copy the error.
 2. Grant the three layers (connect, schema usage, select on all tables). Try again: it works.
 3. As `reporter` , try to `INSERT` , `UPDATE` , `DELETE` , and `DROP TABLE` . Copy each error. Which one is worded differently, and why?
 4. `\dp products` as `postgres` . Which letter does `reporter` have?
-

Exercise 2 (Easy): Future tables

1. As `postgres` , create a table `notes (id integer)` . As `reporter` , try to read it. Copy the error.
 2. Fix it so this never happens again, with `ALTER DEFAULT PRIVILEGES` . Create another table, `notes2` . Can `reporter` read it now? Can it read `notes` ? (Fix that one too.)
 3. Drop both tables. In a comment: explain the difference between the two kinds of grant to a teammate.
-

Exercise 3 (Medium): The app's own key

Create `shop_app` for the web shop. It must be able to:

- read everything,
- create orders and order items,
- update an order's `status` and `shipped_at`, but **not** its `customer_id` or `ordered_at`,
- reduce stock, but not change prices or names.

As `shop_app`, prove each of these with a working statement and a refused one. Then, as `postgres`, undo any test data the app inserted.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Groups

1. Create a `NOLOGIN` role `readonly` with schema usage and select on all tables (and default privileges for future ones).
 2. Create two login roles, `analyst1` and `analyst2`, as members of `readonly`. Connect as each; both should read, neither should write.
 3. Grant `readonly` select on a brand new table. Do both analysts get it with no further grants?
 4. Remove `analyst2` from the group with `REVOKE readonly FROM analyst2;`. What can `analyst2` do now?
 5. In a comment: why is this better than granting tables to each analyst directly?
-

Exercise 5 (Challenge): Hide the emails, for real

The marketing analyst must be able to see customer names and cities and all sales data, but **never** email addresses.

1. Create a view `customer_directory` (id, name, city) and a role `marketing` that can select from the view, from `orders`, `order_items`, `products`, and `categories`, but **not** from `customers`.
2. As `marketing`: select from the view (works), select `email` from `customers` (fails), and write a per-city revenue query using the view instead of `customers` (works).
3. Now deliberately make the mistake: `GRANT SELECT ON customers TO marketing;`. Show that the view no longer protects anything. Revoke it.
4. In a comment: write the two-sentence rule you'd put in the team's wiki about views and permissions.
5. Clean up every role from this chapter. `\du` should show only `postgres`.

Stuck? The hint is in the [Hints section](#) at the end.

35 Backup and Restore

What is it?

A **backup** is a copy of your database saved somewhere safe. **Restoring** is rebuilding a database from that copy. PostgreSQL ships two command-line tools for this:

- `pg_dump` writes one database to a file.
- `pg_restore` (or plain `psql`) loads it back.

Unlike everything so far, these run in your **terminal**, not inside `psql`.

Why does it matter?

Every database eventually meets one of these:

- A `DELETE` without a `WHERE`, committed.
- A migration that dropped the wrong column.
- A disk that dies.
- A laptop that gets stolen.
- A cloud account that gets suspended.

Transactions protect you from half-done changes. Constraints protect you from bad data. Nothing protects you from a committed mistake or a dead disk **except a backup made before it happened**. People who've lost data back up religiously. People who haven't, don't yet.

There's a second, happier use: moving a database. From your laptop to a server, from one cloud to another, or to a teammate who needs a copy of the real data. That's a dump and a restore too.

Real-world example

Photographing every page of your notebook and putting the photos in a safe:

Notebook	Database
Photograph every page	<code>pg_dump</code>
The photos in a folder	The dump file
Spill coffee on the notebook	Drop the wrong table
Rewrite the pages from the photos	<code>pg_restore</code>

Notebook	Database
Photos from last month don't have this month's notes	A backup only contains what existed when it was taken

The last row is the important one. A backup is a snapshot. Take them often enough that losing "everything since the last one" is bearable.

How it works

These examples assume `pg_dump` and `pg_restore` are on your PATH (chapter 03's optional step). If not, use the full path, in quotes:

```
"C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" ...
```

Every command below will ask for the `postgres` password. To stop that, create a file at `%APPDATA%\postgresql\pgpass.conf` containing one line, `localhost:5432*:postgres:yourpassword`, and the tools will read it. (Treat that file like a password, because it is one.)

Work in a terminal, in a folder where you want the backup files to land, like `playground/ch35/`.

A plain SQL dump

```
pg_dump -U postgres -d sales -f sales.sql
```

That's it. `-U` user, `-d` database, `-f` output file. For the `sales` database it finishes instantly and produces a text file of a few hundred lines. Open it in VS Code:


```
--
-- PostgreSQL database dump
--

-- Dumped from database version 18.6
...
CREATE TABLE public.categories (
    id integer NOT NULL,
    name text NOT NULL
);
...
COPY public.categories (id, name) FROM stdin;
1 Stationery
2 Furniture
3 Bags
4 Electronics
\.
...
SELECT pg_catalog.setval('public.categories_id_seq', 5, true);
...
ALTER TABLE ONLY public.order_items
    ADD CONSTRAINT order_items_order_id_fkey FOREIGN KEY (order_id) REFERENCES public.orders(id)
    ON DELETE CASCADE;
```

It's SQL you could have written yourself: `CREATE TABLE` for every table, the data as `COPY ... FROM stdin` blocks (a fast bulk-load format, tab-separated), the identity counters reset to where they were, and the constraints added **last**, after the data, so there are no ordering problems. Run this file against an empty database and you get your database back, exactly.

Because it's plain text, you can read it, search it, and even edit it before restoring. That's the big advantage of this format.

Restoring a plain dump

A plain dump is just SQL, so `psql` runs it:

```
psql -U postgres -c "CREATE DATABASE sales_copy;"
psql -U postgres -d sales_copy -f sales.sql
psql -U postgres -d sales_copy -c "SELECT count(*) FROM orders;"
```

```
count
-----
    12
```

The dump doesn't create the database itself (by default), so you make an empty one first, then load into it. Now you have an exact copy to experiment on, without touching the original. This is also how you'd hand a database to a teammate: send the file, they run these three lines.

Two useful flags when dumping: `--clean --if-exists` adds `DROP TABLE IF EXISTS ...` lines at the top, so you can restore over an existing copy without emptying it first.

The custom format

For anything bigger than a toy, use `-Fc` ("format: custom"):

```
pg_dump -U postgres -d sales -Fc -f sales.dump
```

The file is **compressed** (a quarter the size or better), and it's **restorable in pieces**: one table, schema only, data only. You can't read it in an editor, but you can list it:

```
pg_restore -l sales.dump
```

```
230; 1259 16386 TABLE public categories postgres
234; 1259 16424 TABLE public customers postgres
...
3494; 0 16386 TABLE DATA public categories postgres
3498; 0 16424 TABLE DATA public customers postgres
...
3344; 2606 16468 FK CONSTRAINT public order_items order_items_order_id_fkey postgres
```

Every table, every chunk of data, every constraint, as a separate item. Restore all of it:

```
psql -U postgres -c "CREATE DATABASE sales_copy;"
pg_restore -U postgres -d sales_copy sales.dump
```

Or just one table:

```
pg_restore -U postgres -d sales_copy -t order_items sales.dump
```

That last one is the everyday rescue: someone wiped `order_items` at 3pm, and you restore just that table from last night's dump into a scratch database, then copy the rows you need back. With a plain dump you'd be hunting through a text file for the right `COPY` block.

Use `-Fc` for backups. Use plain SQL when you want to read or share the file.

Partial dumps

You want	Command
One table	<code>pg_dump -U postgres -d sales -t customers -f customers.sql</code>
Structure, no data	<code>pg_dump -U postgres -d sales --schema-only -f schema.sql</code>
Data, no structure	<code>pg_dump -U postgres -d sales --data-only -f data.sql</code>
Data as <code>INSERT</code> statements instead of <code>COPY</code>	add <code>--column-inserts</code>

`--schema-only` gives you a `schema.sql` like the ones you wrote by hand in the projects, generated from the real database. `--column-inserts` makes a data file that other databases can read, at the cost of being much slower to load:

```
INSERT INTO public.customers (id, name, email, city, joined_on) OVERRIDING SYSTEM VALUE VALUES
(1, 'Asha Rai', 'asha@example.com', 'Kathmandu', '2026-01-15');
```

(Note the `OVERRIDING SYSTEM VALUE`: chapter 14's escape hatch for inserting explicit ids into an identity column, used here so the restored rows keep their original ids.)

Roles aren't in the dump

`pg_dump` copies one database. Roles and their passwords live outside any database, so they're not included. For those:

```
pg_dumpall -U postgres --globals-only -f roles.sql
```

```
CREATE ROLE postgres;
ALTER ROLE postgres WITH SUPERUSER ... PASSWORD 'SCRAM-SHA-256$...';
CREATE ROLE shop_app;
...
```

A full backup of a server is: `pg_dumpall --globals-only` for the roles, plus `pg_dump -Fc` per database. (`pg_dumpall` with no flags dumps everything, every database, as one big plain-SQL file. Fine for small servers.)

Where the data actually lives

Your databases are files under the data directory (`SHOW data_directory;` in psql shows the path, something like `C:\Program Files\PostgreSQL\18\data`). You might think "I'll just copy that folder". **Don't**, at least not while the server is running: the files change constantly and a copy taken mid-write is corrupt. `pg_dump` exists precisely so you get a consistent snapshot while the database stays live. (There is a way to copy the files safely, called a base backup, and it's the foundation of the fancier methods below.)

The habit

A backup you've never restored is a hope, not a backup. The routine that works:

1. **Automate the dump.** On Windows, Task Scheduler can run a `.bat` file nightly containing `pg_dump -U postgres -d sales -Fc -f "D:\backups\sales_%DATE%.dump"`. On a server, `cron`.
2. **Copy it somewhere else.** A backup on the same disk as the database dies with the disk. Another drive, another machine, a cloud bucket.
3. **Keep several.** Last 7 days, last 4 weeks. The mistake you discover on Thursday might have happened on Monday.
4. **Test the restore.** Once a month, restore the latest dump into a scratch database and run a few queries. If you can't restore it, you don't have a backup.
5. **Before every risky change**, take a one-off dump. A migration that drops a column, a bulk update, an upgrade. Thirty seconds now saves a very bad day later.

Beyond dumps

A nightly dump means you can lose up to a day of data. Systems that can't accept that use **continuous archiving** (PostgreSQL keeps a log of every change, and that log is backed up as it's written), which allows **point-in-time recovery**: "restore to exactly 14:59, one minute before the bad `DELETE`". It's set up on the server, not with these commands, and it's the kind of thing a managed cloud database ([chapter 43](#)) gives you with a checkbox. Know it exists; for your own projects, scheduled dumps are the right start.

Common mistakes

1. No backup

The default state of every hobby project. Set up the scheduled dump before you have data you'd miss.

2. Never testing a restore

Dumps can be incomplete, truncated, or taken of the wrong database. Restore into a scratch database and look.

3. Backup on the same disk

One failure takes both. Copy it elsewhere.

4. Copying the data directory while the server runs

Corrupt. Use `pg_dump`.

5. Forgetting roles

The dump restores, the app can't log in, because `shop_app` doesn't exist on the new server. `pg_dumpall --globals-only`.

6. Restoring over the live database by accident

`pg_restore -d sales` with `--clean` *drops and recreates* your real tables. Always restore into a copy first, check it, and only then decide what to do with the original.

Quick recap

- `pg_dump -U postgres -d name -f file.sql` makes a readable SQL backup. Restore it with `psql -d copy -f file.sql` into a fresh database.
- `pg_dump -Fc -f file.dump` makes a compressed backup you can restore in pieces with `pg_restore (-l to list, -t table for one table)`.
- `--schema-only`, `--data-only`, `-t table`, `--column-inserts` for partial dumps.
- Roles aren't in a database dump. `pg_dumpall --globals-only` for those.
- Never copy the data directory while the server is running.
- Automate, copy elsewhere, keep several, **test the restore**, and dump before every risky change.

Next: try the [exercises](#), then move on to the Level 4 project: [36 Project: Bank Account System](#).

Exercises

How to do these:

- Work in a **terminal** (not psql), in `playground/ch35/`. Keep a `commands.md` there with every command you ran and what happened.
- Never restore over `sales` itself. Always into a copy.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your notes.

Exercise 1 (Easy): Dump, read, restore

1. Dump `sales` to `sales.sql` (plain format). Open it in VS Code. Find the `CREATE TABLE` for `orders`, its `COPY` block, and the line that adds the foreign key from `order_items` to `orders`. In what order do these three appear, and why does that order matter?
 2. Create `sales_copy` and restore the dump into it. Run the chapter 29 headline-numbers query against the copy. Same answers?
 3. Drop `sales_copy`.
-

Exercise 2 (Easy): Custom format

1. Dump `sales` with `-Fc` to `sales.dump`. Compare the file sizes of `sales.sql` and `sales.dump`.
 2. List the dump's contents with `pg_restore -l`. How many `TABLE DATA` items are there?
 3. Restore it into a fresh `sales_copy`. Check `\dt` and a row count.
-

Exercise 3 (Medium): The 3pm rescue

Simulate the classic disaster and recovery:

1. Take a `-Fc` backup of `sales`.
2. In psql, on `sales_copy` (a restore of that backup, so you're not risking the real one), "accidentally" run `DELETE FROM order_items;` and commit.
3. Restore **only** `order_items` from the dump into `sales_copy`. You'll need to drop the empty table first (why?). Confirm 21 rows are back.
4. In `commands.md`: write the four commands a colleague would need to do this at 3pm on a bad day, with no explanation needed.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Pieces

1. Produce a `schema.sql` (structure only) and a `data.sql` (data only, as `INSERT` statements) for `sales`.
2. Create an empty `sales_rebuilt`, load `schema.sql`, then `data.sql`. Does it work? Check the foreign keys held.
3. Open `data.sql` and find `OVERRIDING SYSTEM VALUE`. In a comment, explain what it does and why `pg_dump` needs it (chapter 14).

4. Dump roles with `pg_dumpall --globals-only`. Which roles appear? (You may have cleaned up chapter 34's; that's fine.)
-

Exercise 5 (Challenge): Automate it

1. Write `backup.bat` (Windows) that dumps `sales` in custom format to a `backups` folder, with today's date in the filename. Run it by double-clicking. (If it asks for a password, set up `pgpass.conf` as described in the notes.)
2. Open Task Scheduler and schedule it to run daily at a time your laptop is usually on. Run the task manually once to prove it works.
3. Write a second script, `restore-test.bat`, that drops and recreates `sales_copy` and restores the **newest** file from `backups` into it, then runs `psql -c "SELECT count(*) FROM orders"` against the copy.
4. In `commands.md`: your backup policy in five lines. How often, kept where, how many kept, how often tested, and what you do before a risky change.

Stuck? The hint is in the [Hints section](#) at the end.

36 Project: Bank Account System

What you'll build

The database for a small savings co-operative: customers, their accounts, and every movement of money as a permanent ledger. Transfers that can't half-happen. Rules that stop overdrafts and frozen accounts. Statements with running balances. A read-only auditor, a teller who can move money but not rewrite history, and a backup you've proven you can restore.

It's Level 4 end to end: transactions, locking, indexes, views, roles, and backups, applied to the one domain where getting it wrong costs real money.

By the end, you'll have, in `playground/ch36/` :

- `schema.sql` : three tables, every constraint, every index,
- `seed.sql` : customers, accounts, opening deposits,
- `operations.sql` : transfers (good and bad), fees, interest, all as proper transactions,
- `views.sql` : statements, overview, and a reconciliation check,
- `roles.sql` : teller and auditor, least privilege,
- `backup.md` : the commands you ran to back up and restore, and proof it worked,
- `concurrency.md` : what you saw with two windows open.

Before you start

- Finish chapters 30 to 35.
- Read the starter README: the co-op's brief, the sample data, and the expected balances after every operation, so you can check yourself.
- Make `playground/ch36/`. Create a database called `bank`.
- `\pset null '[NULL]'`.

The one design rule for this project: money never just *changes*. Every change to a balance has a matching row in a **ledger** (`transactions`) saying what happened, when, and why. The balance on the account is a convenience copy of "the sum of the ledger". If they ever disagree, something is wrong, and you'll write the query that proves they agree.

Milestone 1: Design

Goal: three tables and their rules, in `plan.md` first.

- `customers` : who.

- `accounts` : one customer has many; each has a unique account number, a kind (`current` or `savings`), a balance, and a frozen flag.
- `transactions` : the ledger. One row per movement: which account, how much (**negative for money out, positive for money in**), what kind (`deposit` , `withdrawal` , `transfer_in` , `transfer_out` , `fee` , `interest`), when, and a note. For transfers, both legs share a `transfer_id` so you can find the pair.

Decide and write down: types for money (chapter 12), which `CHECK` s (balance never negative; amount never zero; kinds from a fixed list), the primary key type for a ledger that will grow forever (chapter 14), which columns need indexes (chapter 31: every foreign key, and `created_at` because statements are date ranges).

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 2: Schema and seed

Goal: `schema.sql` builds it; `seed.sql` fills it.

Schema: drop in child-first order, create in parent-first order, constraints inline, indexes at the end. Seed: the three customers and four accounts from the brief, then the **opening deposits**. Each opening deposit is a ledger row *and* a balance update, inside **one transaction**: the ledger and the balance must never disagree, even for a moment.

Check: `SELECT account_number, balance FROM accounts ORDER BY id;` gives 5000.00, 20000.00, 1500.00, 300.00. `SELECT count(*) FROM transactions;` gives 4.

Milestone 3: A transfer, done right

Goal: 1200 from Asha's current account (ACC-1001) to Bikram (ACC-2001), as one transaction in `operations.sql`.

The shape, from chapters 27 and 30:

```
BEGIN;
SELECT id, balance, is_frozen FROM accounts WHERE id IN (1, 3) ORDER BY id FOR UPDATE;
UPDATE accounts SET balance = balance - 1200 WHERE id = 1 AND NOT is_frozen;
UPDATE accounts SET balance = balance + 1200 WHERE id = 3 AND NOT is_frozen;
WITH t AS (SELECT gen_random_uuid() AS tid)
INSERT INTO transactions (account_id, amount, kind, transfer_id, note)
SELECT 1, -1200, 'transfer_out', tid, 'Rent' FROM t
UNION ALL
SELECT 3, 1200, 'transfer_in', tid, 'Rent' FROM t;
COMMIT;
```

Three things to notice and write in a comment: why the `FOR UPDATE` comes first and is `ORDER BY id` (deadlocks, chapter 30); why both `UPDATE`s say `AND NOT is_frozen` (keep reading); why the two ledger rows come from one CTE (they must share the same `transfer_id`).

Check: balances 3800.00 and 2700.00. Six ledger rows. `SELECT transfer_id, count(*), sum(amount) FROM transactions WHERE transfer_id IS NOT NULL GROUP BY 1;` shows 2 legs netting to 0.00.

Milestone 4: A transfer that must fail

Goal: Chandra (300.00) tries to send 500 to Asha.

Write it exactly like Milestone 3. Run it. Read every line PostgreSQL prints, and write them as comments: the `CHECK` violation, the "current transaction is aborted" lines, and `ROLLBACK` as the reply to your `COMMIT`.

Check: nothing changed. Still 6 ledger rows, Chandra still 300.00. Then write one sentence: what would have happened to Chandra's money if these statements had run without `BEGIN`?

Milestone 5: Frozen accounts

Goal: freeze Bikram's account and try to take 100 out.

Set `is_frozen = true` on ACC-2001. Then `BEGIN`, and run `UPDATE accounts SET balance = balance - 100 WHERE id = 3 AND NOT is_frozen;`. PostgreSQL says `UPDATE 0`. **No error.** The row simply didn't match.

That means the person or program running this must *look at the row count* and `ROLLBACK` when it's zero, or the ledger insert that follows will record a withdrawal that never happened. Write the `ROLLBACK`, and a comment explaining why a `CHECK` constraint can't express this rule and what [chapter 38](#) will offer instead (a trigger that raises an error). Unfreeze the account afterwards.

Milestone 6: Batch operations

Goal: the monthly run. Two transactions in `operations.sql`:

1. A **50.00 fee** on every `current` account: one `INSERT ... SELECT` that writes a `fee` ledger row per current account, and one `UPDATE` that reduces their balances. Same transaction.
2. **1% interest** on every `savings` account, the same way, rounded to 2 decimal places.

Also do the internal transfer from the brief first (3000 from Asha's savings to her current account), so the numbers line up.

Check: ACC-1001 6750.00, ACC-1002 17170.00, ACC-2001 2650.00, ACC-3001 250.00. Total held 26820.00.

Milestone 7: Views

Goal: `views.sql`, three of them.

1. `account_statements`: every ledger row with its account number and a **running balance** per account (`sum(amount) OVER (PARTITION BY account_id ORDER BY created_at, id)`, chapter 28). For ACC-1001 it should read 5000 → 3800 → 6800 → 6750.
2. `customer_overview`: each customer with number of accounts and total balance, including customers with no accounts.
3. `reconciliation`: each account's stored balance next to the sum of its ledger, and the difference. **Every difference must be 0.00**. This view is your alarm. Run it after every operation for the rest of the project.

Milestone 8: Indexes and plans

Goal: prove the statement lookup is fast.

`EXPLAIN SELECT * FROM transactions WHERE account_id = 1 ORDER BY created_at;` . You should see your `account_id` index being used. Then, in a comment, name the two other indexes you created and the query each one is for. If you can't name a query, drop the index.

Milestone 9: Roles

Goal: `roles.sql`. Two roles, least privilege (chapter 34):

- `teller`: may read everything, insert into `transactions`, and update **only** `balance` and `is_frozen` on `accounts`. May not delete or update anything in `transactions`, ever. May not change account numbers or kinds.
- `auditor`: may read the three views and nothing else. Not even the tables.

Connect as each in a second window and prove it: the teller can run a transfer; the teller gets `permission denied` trying to `DELETE FROM transactions`; the auditor can read `reconciliation` and gets `permission denied` on `SELECT * FROM accounts`.

Milestone 10: Two windows

Goal: `concurrency.md`. Open two sessions as `postgres` (or `teller`).

1. In A: `BEGIN;` and the `SELECT ... FOR UPDATE` for accounts 1 and 3. Don't commit. In B: start the same transfer. B waits. Commit A. B proceeds. Record what you saw.
2. Now the dangerous one. Set ACC-3001 to 300.00 again if needed. In A: begin a 200 withdrawal from ACC-3001 (`FOR UPDATE`, `UPDATE`, don't commit). In B: begin a 200 withdrawal from the same account. B waits

on the `FOR UPDATE`. Commit A. What happens to B's `UPDATE`? Why is that exactly right? Roll B back.

3. Write two sentences on what would have gone wrong in step 2 if the app had checked the balance with a plain `SELECT`, decided " $300 \geq 200$, fine", and then updated.

Milestone 11: Backup and restore

Goal: `backup.md`.

Dump `bank` in custom format. Create `bank_copy`, restore into it, and run `SELECT sum(balance) FROM accounts;` on the copy: 26820.00. Then delete all ledger rows in the copy (on purpose), restore **only** `transactions` from the dump, and show the reconciliation view on the copy is all zeroes again. Write every command and its output.

Common mistakes

1. Updating a balance without a ledger row, or the reverse

The reconciliation view catches it. Always both, always one transaction.

2. Forgetting the amount sign

Money out is negative. A `transfer_out` of `+1200` makes the ledger lie and the reconciliation fail.

3. Not checking the row count

`UPDATE 0` is not an error. Frozen account, wrong id, typo: all silent. Look at the count.

4. Locking rows in different orders

Transfer A locks 1 then 3; transfer B locks 3 then 1: deadlock. `ORDER BY id` in the `FOR UPDATE`, every time.

5. A teller who can edit the ledger

If `transactions` rows can be updated or deleted, the ledger is worthless. Insert only.

6. A backup nobody restored

Milestone 11 is the proof. Do it.

Quick recap

- Money moves are **ledger row + balance change, in one transaction**. The balance is a copy; the ledger is the truth; a reconciliation view keeps them honest.
- Lock the rows you're about to change with `SELECT ... FOR UPDATE ORDER BY id` before changing them.

- Rules the data can express go in `CHECK` s. Rules that need a decision (frozen) need a row-count check now, and a trigger later.
 - Views give the auditor exactly what they need. Roles give the teller exactly what they need. Nobody uses `postgres` .
 - Back it up, restore it somewhere else, and check the total.
-

Congratulations! You've finished Level 4. 🎉 You can now keep data safe from mistakes, from concurrent users, from strangers, and from disks, and keep it fast as it grows. Try the [exercises](#), then head back to the roadmap. Level 5 connects all of this to the apps you build.

Exercises

How to do these:

- These extend your `bank` database. Finish all 11 milestones first.
 - Each exercise is a new `.sql` file in `playground/ch36/` . Run the reconciliation view after each one; it must still be all zeroes.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to review your files.
-

Exercise 1 (Easy): Deposits and withdrawals

Write the two simplest operations as transactions: a **deposit** of 1000 into ACC-3001, and a **withdrawal** of 75 from ACC-1001. Each is one ledger row plus one balance update. Then try to withdraw 5000 from ACC-3001 and watch it fail cleanly.

Expected afterwards: ACC-3001 1250.00, ACC-1001 6675.00.

Exercise 2 (Easy): A monthly statement

Write a query (then make it a view, `monthly_statement`) that, for one account and one calendar month, shows: the **opening balance** (sum of all ledger rows before the month), each transaction in the month with its running balance, and the **closing balance**. Test it for ACC-1001 and the current month.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): A daily limit

The co-op limits withdrawals and outgoing transfers to **2000 per account per day**. Write a withdrawal transaction that:

1. Locks the account row.
2. Adds up today's `withdrawal` and `transfer_out` amounts for that account.
3. Only proceeds if today's total plus this withdrawal stays within the limit. (For now, "doesn't proceed" means you look at the result and `ROLLBACK`. Chapter 38 will let the database refuse by itself.)

Test it by withdrawing 1500, then trying 600 more on the same day.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Closing an account

Add an `is_closed` boolean `NOT NULL DEFAULT false` to `accounts`, with a **constraint** that a closed account must have a zero balance. Then write the "close account" transaction: it must refuse (via the constraint) if there's money left, so the right procedure is: transfer the remaining balance to another account of the same customer, *then* close.

Close ACC-1002 (Asha's savings) by moving its balance to ACC-1001 first. Prove that trying to close an account with money in it fails, and that a closed account can't receive a deposit (add `AND NOT is_closed` to your updates, and check the row count, as in Milestone 5).

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): The auditor's toolkit

Give the auditor three more views, and grant them:

1. `unbalanced_transfers`: any `transfer_id` whose legs don't sum to zero or don't come in pairs. (Should be empty. Prove it catches problems by inserting a lone `transfer_out` row inside a transaction, selecting from the view, and rolling back.)
2. `large_movements`: every ledger row with an absolute amount over 1000, newest first, with the customer's name.
3. `daily_totals`: per day, total in, total out, and net, for the whole co-op.

Then, as the auditor, run all three. Finally, dump just the **ledger** (`transactions`, data only, as `INSERT` statements) to a file, and write a comment: why might an auditor want exactly that file?

Stuck? The hint is in the [Hints section](#) at the end.

Level 5: PostgreSQL in Real Apps

Goal: use PostgreSQL's special features, and connect it to the apps you build.

Before chapter 40: you'll need the JavaScript course up to Node.js Basics (chapter 49), and the TypeScript course. From chapter 40 on, you'll write code that talks to your database.

37 JSON and Arrays

What is it?

Two column types that hold **more than one value** in a single cell:

- **jsonb** holds a whole JSON object (or array, or anything JSON): `{"color": "black", "pages": 200, "ruled": true}`. If you've done the JavaScript course, it's an object, stored in a column.
- **Arrays** like `text[]` or `integer[]` hold a list of values of one type: `{eco, bestseller}`.

Yes, this breaks chapter 02's "one value per cell" rule. On purpose, in specific situations. This chapter is about what those situations are, and how to work with the data once it's there.

Why does it matter?

Three real problems that plain columns handle badly:

1. **Attributes that differ by product.** A notebook has a page count and ruling. A lamp has bulbs and dimensions. A USB hub has ports. You can't make a column for every attribute of every kind of product; most would be NULL for most rows. A `specs jsonb` column holds whatever each product needs.
2. **Data that arrives as JSON.** APIs send JSON. Webhooks send JSON. Sometimes you want to store exactly what arrived and pick it apart later.
3. **Small, simple lists.** A handful of tags on a product. Chapter 17 taught the proper many-to-many table, and that's still right when tags need their own rows (descriptions, counts, foreign keys). For a free-form list that's only ever read with its parent row, an array is simpler.

And one more: your API will *send* JSON. PostgreSQL can build the exact JSON document your React app expects, nested items and all, in one query.

Real-world example

A paper form with fixed boxes (name, date, amount) and a blank area at the bottom labelled **"Other details"**. The fixed boxes are columns: everyone fills them in, you can sort and total them. The blank area is `jsonb`: each form has whatever's relevant written there, and you can still search it, but you wouldn't design the whole form as one blank area.

How it works

Work in `sales`, with `\pset null '[NULL]'`. Everything here is cleaned up at the end so later chapters see the normal dataset.

`json` vs `jsonb`

PostgreSQL has both. **Always use `jsonb`**. `json` stores the exact text you typed (spaces, key order, duplicate keys). `jsonb` parses it into a binary form that's faster to query and can be indexed:

```
SELECT '{"a":1, "a":2, "b": 3}'::json AS json_keeps, '{"a":1, "a":2, "b": 3}'::jsonb AS
      jsonb_normalizes;
```

json_keeps	jsonb_normalizes
-----+-----	
{ "a":1, "a":2, "b": 3 }	{ "a": 2, "b": 3 }

The only reason to use `json` is when you must preserve the exact original text. That's rare.

Adding JSON data

```
ALTER TABLE products ADD COLUMN specs jsonb;

UPDATE products SET specs = '{"color": "black", "pages": 200, "ruled": true}' WHERE id = 1;
UPDATE products SET specs = '{"color": "blue", "pack_size": 10}' WHERE id = 2;
UPDATE products SET specs = '{"color": "white", "bulbs": 1, "dimensions": {"width_cm": 15,
    "height_cm": 40}}' WHERE id = 6;
UPDATE products SET specs = '{"color": "black", "ports": 4, "usb_c": true}' WHERE id = 10;
```

JSON goes in as text inside single quotes, with double quotes for keys and string values (that's JSON's rule, not SQL's). Four products, four different shapes, one column. Invalid JSON is refused:

```
UPDATE products SET specs = '{"color": black}' WHERE id = 2;
```

```
ERROR: invalid input syntax for type json
DETAIL: Token "black" is invalid.
```

Reading values: `->` and `->>`

```
SELECT name, specs->>'color' AS color, specs->'pages' AS pages_json, specs->>'pages' AS
       pages_text
FROM products WHERE specs IS NOT NULL ORDER BY id;
```

name	color	pages_json	pages_text
Notebook	black	200	200
Pen	blue	[NULL]	[NULL]
Desk lamp	white	[NULL]	[NULL]
USB hub	black	[NULL]	[NULL]

Two arrows, and the difference matters:

- `->` gives you the value **as JSON** (still `jsonb`). Use it to go deeper.
- `->>` gives you the value **as text**. Use it when you want to display, compare, or cast.

Missing keys give NULL, not an error.

For nested values, chain `->` and finish with `->>`, or use `#>>` with a path:

```
SELECT specs->'dimensions'->>'height_cm' AS height, specs #>> '{dimensions,height_cm}' AS
       height2
FROM products WHERE id = 6;
```

Both give `40`.

Everything from `->>` is **text**. To do maths, cast:

```
SELECT name, (specs->>'pages')::integer * 2 AS double_pages FROM products WHERE specs ? 'pages';
```

name	double_pages
Notebook	400

Filtering on JSON

Three ways, from simplest to most powerful:

```
SELECT name FROM products WHERE specs->>'color' = 'black';           -- compare extracted text
SELECT name FROM products WHERE specs @> '{"color": "black"}';       -- containment
SELECT name FROM products WHERE specs ? 'ports';                     -- key exists
```

The first two both return Notebook and USB hub. The `@>` operator means "contains this JSON": the row's `specs` must have everything on the right. It can check several keys at once (`'{"color": "black", "usb_c": true}'`) and, crucially, **it can use an index**:

```
CREATE INDEX idx_products_specs ON products USING gin (specs);
```

A **GIN** index (generalized inverted index) indexes every key and value inside the JSON, so `@>` and `?` queries stay fast on millions of rows. `->>` comparisons can't use it (same rule as chapter 31: a function on the column defeats the index). When a JSON query matters for speed, write it with `@>`.

Changing JSON

```
UPDATE products SET specs = specs || '{"ruled": false, "cover": "hard"}' WHERE id = 1 RETURNING specs;
```

```
{"color": "black", "cover": "hard", "pages": 200, "ruled": false}
```

`||` merges: new keys are added, existing keys are overwritten. For a nested value, `jsonb_set(target, path, new_value)`:

```
UPDATE products SET specs = jsonb_set(specs, '{dimensions,height_cm}', '42') WHERE id = 6
RETURNING specs->'dimensions' AS dimensions;
```

```
{"width_cm": 15, "height_cm": 42}
```

And `-` removes a key:

```
UPDATE products SET specs = specs - 'cover' WHERE id = 1 RETURNING specs;
```

Note that all of these **replace the whole column value**. JSON isn't edited in place; you compute the new document and store it. That's fine for documents of a few kilobytes, and a reason not to store huge ones.

Looking inside

```
SELECT jsonb_pretty(specs) FROM products WHERE id = 6;           -- indented, readable
SELECT key, value FROM products, jsonb_each(specs) WHERE id = 10; -- one row per key
SELECT jsonb_object_keys(specs) FROM products WHERE id = 6;     -- just the keys
```

`jsonb_each` turns an object into rows, which means you can `GROUP BY key` or count how many products have each attribute: the "schemaless" data becomes queryable again.

Building JSON for your API

This is the part that'll change how you write back ends. One product as JSON:

```
SELECT jsonb_build_object('id', id, 'name', name, 'price', price) AS product FROM products WHERE
       id = 1;
```

```
{"id": 1, "name": "Notebook", "price": 3.90}
```

`jsonb_build_object` takes key, value, key, value. `jsonb_agg` collects many rows into a JSON array. Together, with a subquery, they build a nested document:

```
SELECT jsonb_pretty(jsonb_build_object(
  'order_id', o.id,
  'customer', c.name,
  'items', (
    SELECT jsonb_agg(jsonb_build_object('product', p.name, 'quantity', oi.quantity,
      'unit_price', oi.unit_price) ORDER BY p.name)
    FROM order_items oi JOIN products p ON p.id = oi.product_id
    WHERE oi.order_id = o.id
  )
)) AS order_document
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.id = 5;
```

```
{
  "items": [
    {"product": "Notebook", "quantity": 3, "unit_price": 3.50},
    {"product": "USB hub", "quantity": 1, "unit_price": 29.99},
    {"product": "Wireless mouse", "quantity": 1, "unit_price": 15.75}
  ],
  "customer": "Dipesh Thapa",
  "order_id": 5
}
```

That's exactly what a `GET /orders/5` endpoint would return, built by the database in one round trip, with no loop in your code to fetch the items (the N+1 problem from chapter 32, avoided). [Chapter 40](#) sends this straight to a browser.

Arrays

```
ALTER TABLE products ADD COLUMN tags text[] NOT NULL DEFAULT '{}';

UPDATE products SET tags = ARRAY['eco', 'bestseller'] WHERE id = 1;
UPDATE products SET tags = '{bestseller,new}' WHERE id = 2;
UPDATE products SET tags = ARRAY['new'] WHERE id = 10;

SELECT name, tags FROM products WHERE id IN (1, 2, 10) ORDER BY id;
```

name	tags
Notebook	{eco,bestseller}
Pen	{bestseller,new}
USB hub	{new}

Two ways to write one: `ARRAY['a', 'b']` or the text form `'{a,b}'`. `'{}'` is an empty array, which is what the default gives every other product (not NULL: an empty list is a known fact, "no tags").

Searching arrays:

```
SELECT name FROM products WHERE 'bestseller' = ANY (tags);      -- has this tag
SELECT name FROM products WHERE tags @> ARRAY['new'];           -- contains all of these
```

`= ANY` is "equals at least one element". `@>` is containment again, and again it's the one a GIN index can speed up (`CREATE INDEX ... USING gin (tags)`).

Working with arrays:

```
SELECT name, cardinality(tags) AS tag_count, tags[1] AS first_tag FROM products WHERE
    cardinality(tags) > 0;
```

`cardinality()` is the length. **Array positions start at 1**, not 0: `tags[1]` is the first tag.

```
UPDATE products SET tags = array_append(tags, 'sale') WHERE id = 1 RETURNING tags;    --
    {eco,bestseller,sale}
UPDATE products SET tags = array_remove(tags, 'sale') WHERE id = 1 RETURNING tags;    --
    {eco,bestseller}
```

Turning an array into rows (and back) is the key trick:

```
SELECT tag, count(*) FROM products, unnest(tags) AS tag GROUP BY tag ORDER BY 2 DESC, 1;
```

tag	count
bestseller	2
new	2
eco	1

`unnest` makes one row per element, and then every tool from Level 3 applies. The reverse, `array_agg`, collects rows into an array; you've been using its cousin `string_agg` since chapter 21:

```
SELECT c.name AS category, array_agg(p.name ORDER BY p.name) AS products
FROM products p JOIN categories c ON c.id = p.category_id
GROUP BY c.name ORDER BY c.name;
```

category	products
Bags	{Backpack,"Laptop sleeve"}
Electronics	{"USB hub","Wireless mouse"}
Furniture	{"Desk lamp","Office chair"}
Stationery	{Notebook,Pen,Stapler,"Sticky notes","Whiteboard marker"}

Array or table?

Chapter 17's `book_genres` table and a `genres text[]` column solve the same problem. Pick by asking:

Question	Array	Junction table
Do tags need their own data (description, colour, owner)?	No	Yes
Must every tag be a valid, known one?	Only with a <code>CHECK (tags <@ ARRAY[...])</code>	Yes , via foreign key
Is "all products with tag X" a hot query?	Fine with a GIN index	Fine with the FK index
Do you rename a tag everywhere at once?	Painful	One <code>UPDATE</code>
Is it a short list read with its row and rarely queried alone?	Yes	Overkill

Rule of thumb: arrays for small lists of plain values that belong to one row. Tables for anything that's a *thing* in its own right. The same logic applies to `jsonb` vs columns: if you filter, join, or `GROUP BY` a value regularly, it wants to be a real column.

Clean up:

```
ALTER TABLE products DROP COLUMN specs, DROP COLUMN tags;
```

Common mistakes

1. `->` when you meant `->>`

`specs->'color' = 'black'` compares JSON to text and fails or misbehaves. Comparing or displaying: `->>`.
Going deeper: `->`.

2. Forgetting that `->>` is text

`specs->>'pages' > 100` compares text to a number. Cast: `(specs->>'pages')::integer`.

3. Everything in one `jsonb` column

"We'll just store the whole form as JSON." Six months later, nothing can be validated, indexed, or joined. Columns for what you know, JSON for the genuinely variable rest.

4. `json` instead of `jsonb`

Slower, no indexing, no containment operators. `jsonb`.

5. Arrays for relationships that need integrity

An `integer[]` of product ids has no foreign key: it can hold ids that don't exist. Lists of *references* belong in a junction table.

6. Zero-based indexing

`tags[0]` is NULL. Arrays start at 1.

Quick recap

- `jsonb` stores a JSON document in one column. Use it for attributes that vary by row and for data that arrives as JSON. Never `json`.
 - `->` gives JSON (go deeper), `->>` gives text (compare, display, cast). `#>>` takes a path.
 - Filter with `@>` (containment) and `?` (key exists); those can use a **GIN index**. Change with `||`, `jsonb_set`, and `-`.
 - `jsonb_build_object` and `jsonb_agg` build nested API responses in one query.
 - Arrays (`text[]`) hold small lists. `ANY`, `@>`, `cardinality`, `unnest` (array to rows), `array_agg` (rows to array). Positions start at 1.
 - If you filter, join, or group on it regularly, make it a real column or a real table.
-

Next: try the [exercises](#), then move on to [38 Functions and Triggers](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
 - Save each answer in `playground/ch37/ex1.sql`, `ex2.sql`, and so on. End each file by dropping the columns or tables it added, so the dataset stays clean.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Specs

1. Add `specs jsonb` to `products` and fill it for the four products in the notes, plus two more of your own design (the Office chair and the Backpack).
2. List every product that has a `color`, with the colour as text.
3. List every product whose specs **contain** `{"color": "black"}`, using the operator that can use an index.

4. For the Desk lamp, show the width and height as two separate text columns.

Exercise 2 (Easy): Tags

1. Add `tags text[] NOT NULL DEFAULT '{}'` and tag at least five products, some with two or three tags.
2. Products with the tag `new`. Products with **both** `eco` and `bestseller`.
3. How many products have each tag? (Use `unnest`.)
4. Each category with an array of its product names, alphabetical.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): An API response

Write one query that returns a single JSON document for customer 1 shaped like this:

```
{
  "customer": { "id": 1, "name": "Asha Rai", "city": "Kathmandu" },
  "orders": [
    { "order_id": 1, "status": "paid", "total": 13.00, "items": [ { "product": "Notebook",
      "quantity": 2 }, ... ] },
    ...
  ]
}
```

Use `jsonb_pretty` to check the shape. Every order Asha has placed should be in the array, each with its own nested `items` array.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Changing JSON

With the `specs` from Exercise 1:

1. Add `"warranty_months": 12` to **every** product that has specs, in one `UPDATE`, without losing existing keys.
2. Change the Desk lamp's height to 45 with `jsonb_set`.
3. Remove `ruled` from the Notebook.
4. Create a GIN index on `specs`. `EXPLAIN` a `@>` query and a `->>` query. On this tiny table both will be Seq Scans; in a comment, say which one *could* use the index on a big table, and why the other can't.

Exercise 5 (Challenge): Array or table?

Build the same feature twice and compare.

1. **Array version:** `tags text[]` on products, with a `CHECK` that only allows tags from a fixed list (`eco` , `new` , `bestseller` , `sale`). Prove the check rejects `'random'` .
2. **Table version:** `tags (id, name UNIQUE)` and `product_tags (product_id, tag_id)` as in chapter 17.
3. Write "products per tag, including tags with zero products" for **both** versions. Which one can even do it? (Think about where the list of all possible tags lives in each design.)
4. Write "rename `sale` to `clearance` everywhere" for both.
5. In a comment, decide which design this shop should use, in three sentences.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

38 Functions and Triggers

What is it?

A **function** is code saved inside the database that you call from SQL, like the built-ins from chapter 24, but written by you:

```
SELECT id, order_total(id) FROM orders;
```

A **trigger** is a function that runs **automatically** when something happens to a table: a row is inserted, updated, or deleted. You don't call it; the event does.

Why does it matter?

Four jobs that are awkward or impossible without them:

- 1. **Reuse.** "Total of an order" is written in a dozen queries. As a function, it's written once and can't drift.
- 2. **Rules a CHECK can't express.** "A price can't drop by more than half in one update" compares the new value to the old one. "Frozen accounts can't move money" (chapter 36) needs to *refuse*, not silently update zero rows. A trigger can raise an error.
- 3. **Bookkeeping that must never be forgotten.** `updated_at` on every change. A history row for every price change. Stock reduced when an item is sold. If the app does it, one forgotten code path breaks it. If a trigger does it, it happens every time, from every client.
- 4. **Operations that must be one step.** A money transfer as a single function call that locks, checks, updates, and raises, so no caller can do it halfway.

Functions and triggers put logic *in* the database, next to the data it protects. That's powerful, and it has a cost you'll read about at the end.

Real-world example

An office with standing rules:

Office	Database
"To work out the invoice total, use the calculator on the shelf" (a shared procedure anyone can run)	A function
"Every letter that arrives gets date-stamped" (nobody decides; it just happens)	A <code>BEFORE INSERT</code> trigger setting a timestamp

Office	Database
"Every price change gets written in the log book"	An <code>AFTER UPDATE</code> trigger inserting a history row
"Refuse any refund over 50% unless a manager signs"	A <code>BEFORE UPDATE</code> trigger that raises an error

The stamp and the log book aren't optional and nobody has to remember them. That's what triggers buy you.

How it works

Work in `sales`, with `\pset null '[NULL]'`. Everything is cleaned up at the end.

A function in plain SQL

```
CREATE FUNCTION order_total(p_order_id integer) RETURNS numeric
LANGUAGE sql STABLE
AS $$
    SELECT coalesce(sum(quantity * unit_price), 0)
    FROM order_items
    WHERE order_id = p_order_id;
$$;
```

Piece by piece:

- `order_total(p_order_id integer)`: the name, and one parameter. Prefixing parameters with `p_` keeps them distinct from column names.
- `RETURNS numeric`: what comes back.
- `LANGUAGE sql`: the body is ordinary SQL. The result of the last statement is returned.
- `STABLE`: a promise that the function doesn't change data and gives the same answer within one statement. (`IMMUTABLE` is for pure calculations with no table access; `VOLATILE`, the default, is for anything that changes data or depends on `now()`. PostgreSQL uses these to optimize; when unsure, leave the default.)
- `$$ ... $$`: **dollar quoting**. The body contains single quotes and semicolons, so it's wrapped in `$$` instead of `'`. You'll see `$body$ ... $body$` too; same idea.

Use it like any function:

```
SELECT id, status, order_total(id) AS total FROM orders ORDER BY id LIMIT 4;
```

id	status	total
1	paid	13.00
2	shipped	32.99
3	shipped	149.00
4	cancelled	45.00

`order_total(999)` gives `0`, thanks to the `coalesce`. One definition, used everywhere, no CTE to repeat.

A function can return a whole table too:

```
CREATE FUNCTION customer_orders(p_customer_id integer)
RETURNS TABLE (order_id integer, ordered_on date, total numeric)
LANGUAGE sql STABLE
AS $$
    SELECT o.id, o.ordered_at::date, sum(oi.quantity * oi.unit_price)
    FROM orders o JOIN order_items oi ON oi.order_id = o.id
    WHERE o.customer_id = p_customer_id
    GROUP BY o.id ORDER BY o.id;
$$;

SELECT * FROM customer_orders(1);
```

A parameterized view, in effect.

PL/pgSQL: functions with logic

For `IF`, variables, loops, and errors, PostgreSQL has a procedural language called **PL/pgSQL**:

```
CREATE FUNCTION price_tier(p_price numeric) RETURNS text
LANGUAGE plpgsql IMMUTABLE
AS $$
BEGIN
    IF p_price < 5 THEN
        RETURN 'cheap';
    ELSIF p_price < 50 THEN
        RETURN 'mid';
    ELSE
        RETURN 'premium';
    END IF;
END;
$$;

SELECT name, price, price_tier(price) AS tier FROM products ORDER BY price DESC LIMIT 3;
```

The body sits between `BEGIN` and `END` (nothing to do with transactions; it's just the block). `IF ... THEN ...`, `ELSIF ... ELSE ... END IF`. Every `IF` ends with `END IF`, and statements end with `;`. It reads like a stiff version of any programming language.

A function that does things, and refuses

The bank transfer from chapter 36, as one call. Make a small `accounts` table (two rows: Asha 500, Bikram 100), then:

```
CREATE FUNCTION transfer(p_from integer, p_to integer, p_amount numeric) RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
    v_balance numeric;
BEGIN
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'Amount must be positive, got %', p_amount;
    END IF;

    SELECT balance INTO v_balance FROM accounts WHERE id = p_from FOR UPDATE;
    IF v_balance IS NULL THEN
        RAISE EXCEPTION 'Account % does not exist', p_from;
    END IF;
    IF v_balance < p_amount THEN
        RAISE EXCEPTION 'Insufficient funds: balance is %, tried to send %', v_balance, p_amount;
    END IF;

    UPDATE accounts SET balance = balance - p_amount WHERE id = p_from;
    UPDATE accounts SET balance = balance + p_amount WHERE id = p_to;
END;
$;
```

New pieces: `DECLARE` introduces variables (prefixed `v_`). `SELECT ... INTO v_balance` puts a query result into one. `FOR UPDATE` locks the row, as in chapter 30. And `RAISE EXCEPTION` stops everything with an error message; each `%` is filled from the values after the comma.

```
SELECT transfer(1, 2, 100);      -- works: 400 / 200
SELECT transfer(1, 2, 10000);
```

```
ERROR: Insufficient funds: balance is 400.00, tried to send 10000
CONTEXT: PL/pgSQL function transfer(integer,integer,numeric) line 14 at RAISE
```

A clear, specific error, and nothing changed. The function runs **inside the caller's transaction**: if you `BEGIN;` `SELECT transfer(1, 2, 50); ROLLBACK;`, the transfer is undone. And if anything inside the function fails, every

change it made is undone too. That's the "operation that must be one step".

Triggers

A trigger needs two things: a function that `RETURNS trigger`, and the trigger itself, which says when to run it.

`updated_at`, the classic:

```
ALTER TABLE products ADD COLUMN updated_at timestamptz NOT NULL DEFAULT now();

CREATE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER products_set_updated_at
BEFORE UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

Inside a trigger function, `NEW` is the row as it's about to be written, and `OLD` is the row as it was. A `BEFORE` trigger can **change** `NEW` before it's saved; this one stamps the time. `RETURN NEW` says "go ahead with this row". `FOR EACH ROW` means the function runs once per affected row (an `UPDATE` of 50 rows runs it 50 times).

```
UPDATE products SET stock = stock WHERE id = 1 RETURNING name, updated_at;
```

The timestamp moves, though the statement never mentioned `updated_at`. Every client, every path, forever.

An audit log, with `AFTER`:

```

CREATE TABLE price_history (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  product_id integer NOT NULL REFERENCES products (id),
  old_price numeric(10, 2) NOT NULL,
  new_price numeric(10, 2) NOT NULL,
  changed_at timestamptz NOT NULL DEFAULT now(),
  changed_by text NOT NULL DEFAULT current_user
);

CREATE FUNCTION log_price_change() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.price <> OLD.price THEN
    INSERT INTO price_history (product_id, old_price, new_price)
    VALUES (OLD.id, OLD.price, NEW.price);
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER products_log_price_change
AFTER UPDATE OF price ON products
FOR EACH ROW EXECUTE FUNCTION log_price_change();

```

AFTER triggers run once the row is written; they're for side effects on *other* tables. **UPDATE OF price** means only updates that mention **price** fire it. The **IF** skips updates that set the same price again.

```

UPDATE products SET price = 4.20 WHERE id = 1;
UPDATE products SET price = 4.20 WHERE id = 1;    -- same price: no history row
UPDATE products SET price = 3.90 WHERE id = 1;
SELECT product_id, old_price, new_price, changed_by FROM price_history;

```

product_id	old_price	new_price	changed_by
1	3.90	4.20	postgres
1	4.20	3.90	postgres

Who changed what, from what, to what, when. The chapter 19 exercise about order status history: this is how you make it automatic.

Keeping stock in sync:


```

CREATE FUNCTION reduce_stock() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE products SET stock = stock - NEW.quantity WHERE id = NEW.product_id;
    RETURN NEW;
END;
$$;

CREATE TRIGGER order_items_reduce_stock
AFTER INSERT ON order_items
FOR EACH ROW EXECUTE FUNCTION reduce_stock();

```

Now inserting an order item reduces the product's stock, automatically. And watch what happens when the trigger's update breaks a constraint:

```

BEGIN;
INSERT INTO order_items VALUES (12, 10, 2, 29.99);    -- USB hub stock: 30 -> 28
INSERT INTO order_items VALUES (12, 7, 100, 149.00); -- 100 chairs, stock is 6

```

```

ERROR: new row for relation "products" violates check constraint "products_stock_check"
CONTEXT: SQL statement "UPDATE products SET stock = stock - NEW.quantity WHERE id =
NEW.product_id"
PL/pgSQL function reduce_stock() line 3 at SQL statement

```

The `CHECK (stock >= 0)` from chapter 13 fired *inside the trigger*, which fails the `INSERT` that caused it. You can't sell what you don't have, and no app code had to check. (`ROLLBACK` to undo the first insert too.)

Validation, with `BEFORE` and `RAISE` :

```

CREATE FUNCTION block_big_price_cut() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.price < OLD.price * 0.5 THEN
        RAISE EXCEPTION 'Price cut too large for %: % -> %', OLD.name, OLD.price, NEW.price;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER products_block_big_price_cut
BEFORE UPDATE OF price ON products
FOR EACH ROW EXECUTE FUNCTION block_big_price_cut();

UPDATE products SET price = 1.00 WHERE id = 7;

```

```

ERROR: Price cut too large for Office chair: 149.00 -> 1.00

```

A rule that compares old and new, something no `CHECK` can do. This is the proper answer to chapter 36's frozen-account problem: a `BEFORE UPDATE` trigger on `accounts` that raises when `OLD.is_frozen` and the balance changes.

BEFORE or AFTER ?

Use BEFORE when you want to	Use AFTER when you want to
Change the row being written (<code>NEW.x := ...</code>)	Write to <i>other</i> tables (audit, stock, totals)
Reject the row (<code>RAISE EXCEPTION</code>)	React to something that definitely happened
Fill in defaults that depend on other columns	

`BEFORE` triggers must `RETURN NEW` (or `RETURN NULL` to silently skip the row, which is rarely a good idea). `AFTER` triggers' return value is ignored, but `RETURN NEW` is the convention.

Seeing and removing them

```

\df          -- your functions
\d products  -- ends with a "Triggers:" section

```

```
DROP TRIGGER products_block_big_price_cut ON products;
DROP FUNCTION block_big_price_cut;
```

Drop the trigger before the function it uses, or use `DROP FUNCTION ... CASCADE`.

The cost of putting logic in the database

Triggers are invisible. A developer runs `UPDATE products SET price = 1.00` and gets an error about price cuts from code they've never seen. Six months later nobody remembers that inserting an order item changes stock. Debugging "why did this row change?" means hunting through triggers.

So:

- **Prefer a `CHECK` or a foreign key** when one can express the rule. Constraints are visible in `\d`.
- **Use triggers for bookkeeping that must be universal:** timestamps, audit logs, derived counts, and rules that compare old and new. Keep them small and obvious.
- **Keep business decisions in application code** where the team can read them: pricing logic, workflow, notifications.
- **Name triggers `table_what_it_does`** and keep all of them in one `triggers.sql` in your project, so they're discoverable.
- **Document them in the table's comment or your README.** A surprise trigger is a bug waiting to be written.

Clean up: drop the four triggers and seven functions, `price_history`, `accounts`, and the `updated_at` column, and set the Notebook back to 3.90 and the Office chair to 149.00 if you changed them.

Common mistakes

1. Forgetting `RETURN NEW`

A `BEFORE` trigger that returns nothing fails with `control reached end of trigger procedure without RETURN`. Always return.

2. Side effects in `BEFORE`, changes to `NEW` in `AFTER`

Writing to other tables works in either, but belongs in `AFTER`. Changing `NEW` in an `AFTER` trigger does nothing: the row's already written.

3. A trigger that triggers itself

An `AFTER UPDATE` trigger on `products` that runs `UPDATE products ...` fires itself again, forever (until PostgreSQL stops it with a stack error). Use `BEFORE` and set `NEW` instead, or guard with a condition.

4. Single quotes instead of \$\$

The body has quotes in it. Dollar-quote it.

5. Too much logic in the database

If a trigger is 80 lines of business rules, it's application code hiding in the wrong place.

6. Not documenting triggers

The next developer (or you, next year) will be surprised. Write them down.

Quick recap

- `CREATE FUNCTION name(params) RETURNS type LANGUAGE sql|plpgsql AS $$ ... $$;` saves code in the database. SQL bodies for queries, PL/pgSQL for `IF`, variables, and `RAISE EXCEPTION`.
- Functions run inside the caller's transaction; if they fail, everything they did is undone.
- A **trigger** = a function `RETURNS trigger` + `CREATE TRIGGER name BEFORE|AFTER INSERT|UPDATE|DELETE ON table FOR EACH ROW EXECUTE FUNCTION f()`.
- `BEFORE` triggers modify `NEW` or reject the row. `AFTER` triggers write to other tables. `OLD` is the previous row.
- Use triggers for universal bookkeeping (timestamps, audit logs, stock) and old-vs-new rules. Prefer constraints when possible. Keep business logic in the app. Document every trigger.

Next: try the [exercises](#), then move on to [39 Full-Text Search](#).

Exercises

How to do these:

- Exercises 1, 2, 4 and 5 use `sales`; Exercise 3 uses your `bank` database from chapter 36.
- Save each answer in `playground/ch38/ex1.sql`, `ex2.sql`, and so on. Start each file with the matching `DROP TRIGGER IF EXISTS ... ON ...;` and `DROP FUNCTION IF EXISTS ...;` lines so it's re-runnable.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): SQL functions

1. `line_total(p_order_id integer, p_product_id integer)` returning that line's `quantity * unit_price`.

2. `customer_spend(p_customer_id integer)` returning the customer's total over non-cancelled orders, `0` for customers with none.
3. `customer_orders(p_customer_id)` returning a table, as in the notes. Call it for customer 5. What comes back?

Expected: `customer_spend(3)` is `239.00`; `customer_spend(5)` is `0`.

Exercise 2 (Easy): PL/pgSQL

1. `stock_status(p_stock integer)` returning `'out'` for 0, `'low'` for under 20, otherwise `'ok'`. Use it to count products per status.
 2. `safe_divide(a numeric, b numeric)` returning `a / b`, or NULL when `b` is zero, with an `IF`. Compare with chapter 24's `nullif` trick.
 3. `shout(p_text text)` returning the text uppercased with `'!'` on the end. Then `RAISE EXCEPTION` if the text is empty.
-

Exercise 3 (Medium): The bank, properly

In `bank`, replace chapter 36's hand-written transfer with a function `transfer(p_from, p_to, p_amount, p_note)` that:

- rejects non-positive amounts,
- locks both accounts `FOR UPDATE` in id order,
- rejects if either account is missing or **frozen**, with a message saying which,
- rejects insufficient funds with the balance in the message,
- updates both balances **and** inserts both ledger rows with a shared `transfer_id`.

Then add a `BEFORE UPDATE` trigger on `accounts` that raises an exception if `OLD.is_frozen` is true and `NEW.balance <> OLD.balance`. Freeze an account and prove both the function and a plain `UPDATE` are refused with a real error (not `UPDATE 0`).

Run your reconciliation view afterwards. Still all zeroes?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Timestamps and audit

1. Add `updated_at` to `customers` with a `BEFORE UPDATE` trigger that sets it.

2. Add an `email_history` table and an `AFTER UPDATE OF email` trigger that records old and new email, when, and who. Change a customer's email twice and show the history.
3. Add a `BEFORE UPDATE OF email` trigger that lowercases `NEW.email` before saving. Update an email to `'ASHA@Example.com'` and show what was stored.
4. In a comment: list the order in which the three triggers fire for one `UPDATE customers SET email = ...`, and explain how you know. (Hint: `\d customers`, and the rule that `BEFORE` runs before `AFTER`, alphabetically by name within each.)

Clean up: drop the triggers, functions, table, and column, and restore the original emails.

Exercise 5 (Challenge): The shop's bookkeeping

Three triggers on the `sales` tables, each tested inside a transaction you roll back:

1. **Stock:** `AFTER INSERT ON order_items` reduces `products.stock` (as in the notes), **and** `AFTER DELETE ON order_items` puts it back. Insert and delete a line; stock must end where it started.
2. **Status history:** an `order_status_history` (`order_id`, `status`, `changed_at`, `changed_by`) table filled by an `AFTER UPDATE OF status ON orders` trigger, only when the status actually changed. Move an order from `pending` to `paid` to `shipped` and show three... no, two history rows (why not three?).
3. **Protect history:** a `BEFORE DELETE ON orders` trigger that raises an exception if `OLD.status = 'shipped'`. Try to delete order 2.

Then, in a comment: of these three, which would you keep as a trigger in a real shop, which would you move to application code, and why?

Stuck? The hint is in the [Hints section](#) at the end.

39 Full-Text Search

What is it?

Full-text search finds documents by the **words** in them, the way a search engine does: it understands that "organizing" and "organized" are the same word, ignores "the" and "a", lets users type "desk -lamp" to exclude things, and ranks the best matches first.

It's built into PostgreSQL. No extra server, no extension for the basics. The core is one operator, @@, that asks "does this text match this query?"

Why does it matter?

Every app with a search box starts with `WHERE title ILIKE '%' || :term || '%'`, and it fails in four ways:

- 1. **Word forms.** Searching "organize" misses "organizing" and "Organised".
- 2. **Multiple words.** "desk lamp" with `ILIKE` means those two words *in that order, adjacent*. A document saying "lamp for your desk" is missed.
- 3. **No ranking.** Twenty results come back in whatever order. The best match might be last.
- 4. **No index.** `LIKE '%word%'` can't use a normal index (chapter 31), so it scans every row, every search.

Full-text search fixes all four. And for the fifth problem, typos, PostgreSQL has a second tool (`pg_trgm`) covered at the end.

Real-world example

Finding a topic in a textbook:

Approach	Database
Flip through every page looking for the exact letters "organizing"	<code>LIKE '%organizing%'</code>
Look up "organize" in the index at the back, which already groups "organizing", "organized", and "organization" under one heading, and lists the pages that matter most first	Full-text search with a GIN index

The book's index was built once, when the book was printed. The database's index is built once, when the row is saved. Both make every lookup fast.

How it works

Work in `sales`, with `\pset null '[NULL]'`. We'll make a small `articles` table for the shop's blog, and clean it up at the end.

Two new types: `tsvector` and `tsquery`

A `tsvector` is a document broken into normalized words:

```
SELECT to_tsvector('english', 'The quick brown foxes are jumping over the lazy dogs');
```

```
'brown':3 'dog':10 'fox':4 'jump':6 'lazi':9 'quick':2
```

Look at what happened: "foxes" became `fox`, "jumping" became `jump`, "lazy" became `lazi`, and "the", "are", "over" vanished. Each word is reduced to its **stem** (a root form, which is why `lazi` looks odd; it's internal, not shown to users), **stop words** (common words that carry no meaning) are dropped, and each stem records where it appeared. The `'english'` tells PostgreSQL which language's rules to use.

A `tsquery` is a search request in the same normalized form:

```
SELECT to_tsquery('english', 'jumping & fox');
```

```
'jump' & 'fox'
```

And `@@` matches them:

```
SELECT to_tsvector('english', 'The quick brown foxes are jumping over the lazy dogs') @@  
       to_tsquery('english', 'fox');    -- t  
SELECT to_tsvector('english', 'The quick brown foxes are jumping over the lazy dogs') @@  
       to_tsquery('english', 'cat');    -- f
```

"fox" matched "foxes" because both became `fox`. That's the whole trick.

Some articles

```
CREATE TABLE articles (  
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  title text NOT NULL,  
  body text NOT NULL  
);  
  
INSERT INTO articles (title, body) VALUES  
  ('Choosing the right notebook', 'Dotted, ruled or plain? We compare paper weights and binding  
    styles so your notes look great.'),  
  ('Five ways to organize your desk', 'A tidy desk starts with a good lamp, a pen holder and a  
    weekly reset. Organizing is a habit.'),  
  ('Why fountain pens are back', 'Fountain pens make writing slower and more deliberate. Here is  
    how to pick your first one.'),  
  ('Sticky notes for planning', 'Use sticky notes to plan a week on the wall. Move them, group  
    them, throw them away.'),  
  ('Backpacks that fit a laptop', 'Padded sleeves, water resistance and good straps: what to  
    look for in a work backpack.'),  
  ('Desk lamps and eye strain', 'Warm light in the evening, cool light in the day. A dimmable  
    lamp protects your eyes.'),  
  ('Our stapler is discontinued', 'The classic stapler has been retired. Here are the  
    alternatives we now stock.'),  
  ('Caring for your notebooks', 'Keep notebooks away from damp, and organise finished ones on a  
    shelf by year.');
```

Where LIKE gives up

```
SELECT title FROM articles WHERE body LIKE '%organize%';      -- 0 rows: the body says  
    "Organizing"  
SELECT title FROM articles WHERE body ILIKE '%organi%';      -- 2 rows, by luck of a shared  
    prefix
```

Now with search:

```
SELECT title FROM articles  
WHERE to_tsvector('english', title || ' ' || body) @@ to_tsquery('english', 'organize');
```

Five ways to organize your desk

"organize" found "organize" in the title and "Organizing" in the body, because all three stem to `organ`. (The British "organise" in the last article stems differently, to `organis`, and isn't found. Dictionaries are language-specific. Honest limitation; see the end.)

Three ways to write the query

`to_tsquery` wants the raw syntax (`&` and, `|` or, `!` not, `:*` prefix) and errors on plain sentences. For user input, use one of these:

```
-- plainto_tsquery: all the words, ANDed together
SELECT title FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ plainto_tsquery('english', 'notebook
    paper');
-- Choosing the right notebook

-- websearch_to_tsquery: Like a search engine box. Quotes for phrases, - to exclude, "or"
SELECT title FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ websearch_to_tsquery('english', 'desk
    lamp');
-- Five ways to organize your desk, Desk lamps and eye strain

SELECT title FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ websearch_to_tsquery('english', 'desk -
    lamp');
-- 0 rows: both desk articles mention lamps

SELECT title FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ websearch_to_tsquery('english', 'notebook
    or backpack');
-- 3 rows
```

Use `websearch_to_tsquery` for anything a user types. It never errors on strange input, and people already know its syntax from Google.

Ranking

```
SELECT title, round(ts_rank(to_tsvector('english', title || ' ' || body), q)::numeric, 4) AS
    rank
FROM articles, websearch_to_tsquery('english', 'notebook') AS q
WHERE to_tsvector('english', title || ' ' || body) @@ q
ORDER BY rank DESC;
```

title	rank
Caring for your notebooks	0.0760
Choosing the right notebook	0.0608

`ts_rank` scores how well a document matches: more occurrences, closer together, score higher. (Putting the query in the `FROM` as `q` lets you write it once.) The absolute numbers mean nothing; only the order does. To

weight titles higher than bodies, `setweight(to_tsvector(title), 'A') || setweight(to_tsvector(body), 'B')`.

Showing why it matched

```
SELECT title, ts_headline('english', body, websearch_to_tsquery('english', 'lamp'),
                          'MaxWords=12, MinWords=6') AS snippet
FROM articles WHERE to_tsvector('english', body) @@ websearch_to_tsquery('english', 'lamp');
```

title	snippet
Five ways to organize your desk	lamp, a pen holder and a weekly
Desk lamps and eye strain	light in the day. A dimmable lamp protects your eyes.

`ts_headline` cuts out the relevant fragment and wraps the matches in `<b>` tags (configurable). That's the snippet under each result on a search page.

Making it fast: a stored vector and a GIN index

Everything above recomputes `to_tsvector(...)` for every row on every search. On 8 rows, fine. On 80,000, slow. The production pattern is to **store the vector** in a column that PostgreSQL maintains, and **index it**:

```
ALTER TABLE articles ADD COLUMN search tsvector
GENERATED ALWAYS AS (to_tsvector('english', coalesce(title, '') || ' ' || coalesce(body, '')))
STORED;

CREATE INDEX idx_articles_search ON articles USING gin (search);

SELECT title FROM articles WHERE search @@ websearch_to_tsquery('english', 'organizing desks');
```

A **generated column** (`GENERATED ALWAYS AS (...) STORED`) is computed from other columns whenever the row changes. You never write to it; it's always right. The **GIN index** on it makes `@@` a lookup instead of a scan. Searches now take milliseconds no matter how many articles you have.

The `coalesce` guards against NULLs: `'text' || NULL` is NULL (chapter 07), which would blank out the whole vector.

Prefix search

For "search as you type":

```
SELECT title FROM articles WHERE search @@ to_tsquery('english', 'note:*');
```

Three rows: anything with a word starting "note". Combine with `websearch_to_tsquery` by appending `:*` to the user's last word in your code.

Language and spelling, honestly

'english' is a **text search configuration**: stemming rules and stop words for one language. PostgreSQL ships with about two dozen (`SELECT cfgname FROM pg_ts_config;`). There's also 'simple', which does no stemming and drops no words, for codes, names, and languages without a built-in dictionary:

```
SELECT to_tsvector('simple', 'The quick brown foxes are jumping');
```

```
'are':5 'brown':3 'foxes':4 'jumping':6 'quick':2 'the':1
```

Two things full-text search does **not** do: fix typos ("notebok" matches nothing), or unify spellings across variants ("organise" vs "organize"). For those, the next section.

Fuzzy matching: `pg_trgm`

An **extension** adds features to PostgreSQL. `pg_trgm` ("trigrams") compares strings by their three-letter chunks, which makes it tolerant of typos:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

SELECT similarity('notebook', 'notebok') AS close, similarity('notebook', 'backpack') AS far;
-- 0.7, 0
SELECT name FROM products WHERE name % 'notebok';
-- Notebook
SELECT name, round(similarity(name, 'staplr')::numeric, 2) AS sim FROM products ORDER BY sim
DESC LIMIT 3;
```

name	sim
Stapler	0.50
Sticky notes	0.11
Laptop sleeve	0.05

`similarity()` is 0 to 1; `%` means "similar enough" (default threshold 0.3). "Did you mean *Stapler*?" is `ORDER BY similarity(name, :term) DESC LIMIT 1`.

`pg_trgm` also gives you the thing chapter 31 said was impossible: an index for `ILIKE '%word%'`:

```
CREATE INDEX idx_products_name_trgm ON products USING gin (name gin_trgm_ops);
```

On a big table, `WHERE name ILIKE '%note%'` now uses `idx_products_name_trgm` instead of scanning. (On our 11 products the planner still picks a Seq Scan, correctly.) For short fields like product names, where users expect substring matching and typo tolerance more than stemming, trigrams are often the better tool.

Which tool?

You're searching	Use
Articles, descriptions, comments: real sentences	Full-text search: <code>tsvector</code> column + GIN + <code>websearch_to_tsquery</code>
Names, titles, codes: short fields, typos likely	<code>pg_trgm: %</code> , <code>similarity()</code> , trigram GIN for <code>ILIKE</code>
Both (a product search box)	Full-text first, trigram as the "did you mean" fallback
Many languages at once, faceted filters, millions of documents with complex relevance tuning	A dedicated search engine (Elasticsearch, Meilisearch, Typesense), fed from PostgreSQL

For most apps, PostgreSQL alone is plenty, and it saves running and syncing a second system.

Clean up:

```
DROP INDEX IF EXISTS idx_products_name_trgm;  
DROP TABLE articles;
```

Common mistakes

1. `LIKE` for search

Misses word forms, can't rank, can't index. Use `@@`.

2. `to_tsquery` on user input

`to_tsquery('english', 'desk lamp')` is a syntax error (no operator between words). `websearch_to_tsquery` handles anything.

3. Recomputing `to_tsvector` per row

Works, then slows to a crawl. Store it in a generated column and index it.

4. Mismatched configurations

An index built with `'english'` and a query written with `'simple'` (or no config at all, which uses a server default) may not use the index and may not match. Say `'english'` in both.

5. Expecting typo tolerance from full-text search

It's about word forms, not spelling mistakes. `pg_trgm` for typos.

6. Forgetting `coalesce` in the generated column

One NULL `body` and that article's whole search vector is NULL, so it never matches anything.

Quick recap

- `to_tsvector('english', text)` normalizes a document into stems; `websearch_to_tsquery('english', input)` normalizes a user's search; `@@` matches them.
- Stemming means "organize" finds "organizing". Stop words are dropped. Multiple words match in any order.
- `ts_rank` orders results by relevance; `ts_headline` makes snippets.
- Production pattern: a `GENERATED ALWAYS AS (to_tsvector(...)) STORED` column with a **GIN index**.
- `pg_trgm` adds typo tolerance (`similarity, %`) and makes `ILIKE '%x%'` indexable.
- Sentences: full-text. Short names with typos: trigrams. Enormous multilingual catalogues: a dedicated engine.

Next: try the [exercises](#), then move on to [40 Connecting from Node.js](#).

Exercises

How to do these:

- Work in psql, connected to `sales`. Run `\pset null '[NULL]'`.
- Save each answer in `playground/ch39/ex1.sql`, `ex2.sql`, and so on. Build the `articles` table from the notes at the top of files that need it, and drop it at the end.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work.

Exercise 1 (Easy): Stems and stop words

1. Run `to_tsvector('english', ...)` on three sentences of your own. For each, write down which words disappeared and which changed form.
 2. Show that `'running'`, `'runs'` and `'ran'` do or don't end up as the same stem. (One of them won't. Why might that be?)
 3. Convert these user searches with `websearch_to_tsquery('english', ...)` and look at the result: `desk lamp`, `"sticky notes"`, `pen -fountain`, `notebook or backpack`.
-

Exercise 2 (Easy): Searching the blog

With the `articles` table:

1. Articles about notebooks (any form of the word).
 2. Articles mentioning desks but **not** lamps. Then articles mentioning desks **or** lamps.
 3. Articles matching `pen`. Does "pens" match? Does "pencil"? Explain.
-

Exercise 3 (Medium): A results page

Write **one** query that takes a search term (use a psql variable: `\set term 'organizing your notebooks'`) and returns, best match first, at most 5 rows of: title, a rank rounded to 3 places, and a snippet from the body with matches in `<b>` tags.

Then add the stored `search` column and GIN index from the notes, and rewrite the query to use the column. `EXPLAIN` it. On 8 rows the index won't be used; in a comment, explain what would be different with 80,000 articles.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Product search with typos

1. Add a generated `search tsvector` column to `products` built from `name` (and index it). Search for `note`, `notes`, and `notebooks`. Which match?
2. Enable `pg_trgm`. For the misspellings `notebok`, `staplr`, `bakpack`, and `wireles mouse`, show the closest product name and its similarity score.
3. Write "did you mean": a query that, for a term that matches nothing with full-text search, returns the single most similar product name if its similarity is above 0.3, else nothing.
4. Drop the column and index when done.

Exercise 5 (Challenge): One search function

Using chapter 38, write `search_products(p_term text)` returning `(id, name, price, match_kind text, score numeric)` that:

1. Returns full-text matches on `name` first (`match_kind = 'word'`, `score = ts_rank`),
2. and, if there are **no** full-text matches, returns trigram matches with similarity above 0.3 instead (`match_kind = 'fuzzy'`, `score = similarity`),
3. ordered by score descending, at most 10 rows.

Test with `'pen'`, `'notebok'`, and `'xyz'`. The last should return nothing.

Stuck? The hint is in the [Hints section](#) at the end.

40 Connecting from Node.js

What is it?

Until now, *you* have been the client: typing SQL into psql. From this chapter on, **your code** is the client. A Node.js program opens a connection to PostgreSQL, sends SQL, and gets rows back as JavaScript objects. The library that does this is called `pg` (also known as node-postgres). It's small, it's been around for over a decade, and nearly everything else in the Node world (including the ORMs in chapter 42) is built on top of it.

Why does it matter?

This is where the two halves of your learning meet. Your React apps fetch data from an API (React chapter 18). That API is a Node program. This chapter is what that program does on the inside: run a query, turn the rows into JSON, send them back. It's also where the single most dangerous security mistake in web development lives. You'll make it on purpose, watch it work, and then learn the one habit that makes it impossible.

Real-world example

psql is you walking up to the librarian's desk. A Node program is a **phone line** to the librarian, kept open so your app can ask questions all day without walking over each time:

Phone	pg
Dial the number once, keep the line open	A connection
Several lines, so many callers don't queue behind one	A pool of connections
"Find me the book called __", with the blank read out separately so nothing you say is mistaken for an instruction	A parameterized query (<code>\$1</code>)
Hang up when you're done	<code>pool.end()</code>

How it works

Set up a project

Make a folder `PostgreSQL/playground/node-shop/`, open a terminal in it, and run:

```
npm init -y
npm pkg set type=module
npm install pg
npm install -D @types/pg
```

"type": "module" lets you use `import` and top-level `await`, as in the JavaScript and TypeScript courses. `@types/pg` gives VS Code autocomplete and type checking for the library.

Create a file called `.env` holding your connection details:

```
DATABASE_URL=postgres://postgres:yourpassword@localhost:5432/sales
```

A **connection string** packs everything into one line: `postgres:// user : password @ host : port / database`. Node reads `.env` for you with `--env-file`. **Never commit** `.env`: create a `.gitignore` with `.env` and `node_modules` in it right now, before you forget.

Your first query

`list-products.ts`:

```
import pg from "pg";

const { Pool } = pg;

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

const result = await pool.query("SELECT id, name, price FROM products ORDER BY id LIMIT 3");

console.log(result.rowCount, "rows");
console.log(result.rows);

await pool.end();
```

Run it:

```
node --env-file=.env list-products.ts
```

```
3 rows
[
  { id: 1, name: 'Notebook', price: '3.90' },
  { id: 2, name: 'Pen', price: '1.20' },
  { id: 3, name: 'Sticky notes', price: '2.75' }
]
```

Each row is a plain object with one property per column. That's the whole idea: SQL in, objects out.

Three things about this file:

- **Pool**. A pool keeps a few connections open and hands one out per query. Opening a connection takes time (tens of milliseconds), so you open a handful once and reuse them. Make **one pool per program**, at the top, and share it.
- `pool.query(sql)` returns a promise, so you `await` it. The result has `rows` (the data) and `rowCount` (how many).
- `pool.end()` closes the connections so the program can exit. In a script, call it at the end. In a server, don't: the pool lives as long as the server does.

Node 24 runs `.ts` files directly (as in the TypeScript course), stripping the types. The import is written `import pg from "pg"` then `const { Pool } = pg`, because `pg` is an older-style package; that's the form its own documentation uses.

What comes back, and the `numeric` surprise

Look again: `price: '3.90'`. That's a **string**, not a number. Check a few more types:

```
const r = await pool.query(
  "SELECT id, ordered_at, status, shipped_at, (SELECT sum(quantity * unit_price) FROM
    order_items WHERE order_id = o.id) AS total, count(*) OVER () AS total_orders FROM
    orders o WHERE id = 1",
);
for (const [key, value] of Object.entries(r.rows[0])) {
  console.log(key.padEnd(13), typeof value, value instanceof Date ? "(Date)" : "",
    JSON.stringify(value));
}
```

id	number	1
ordered_at	object	(Date) "2026-07-03T04:30:00.000Z"
status	string	"paid"
shipped_at	object	(Date) "2026-07-05T03:15:00.000Z"
total	string	"13.00"
total_orders	string	"1"

PostgreSQL type	Becomes	Why
integer	number	Fits safely
text	string	
boolean	boolean	
timestampz , date	Date	A JavaScript Date (shown in UTC when printed as JSON, hence the z)
numeric	string	JavaScript numbers can't hold decimals exactly (chapter 12's <code>0.1 + 0.2</code>). pg refuses to lose money in the conversion.
bigint , count(*)	string	Can be bigger than JavaScript's safe integer range

So money and counts arrive as strings. When you need maths, convert on purpose: `Number(row.total) * 2` gives `26`. When you're sending the value to a browser as JSON, a string is often fine (and safer). Forgetting this causes the classic bug where `'13.00' + '5.00'` gives `'13.005.00'`.

Parameters, and the attack they prevent

Suppose a search box on your site sends a customer name, and you write:

```
const userInput = "' OR 1=1 --"; // what an attacker typed into the box

const unsafeSql = `SELECT name, email FROM customers WHERE name = '${userInput}'`;
const unsafe = await pool.query(unsafeSql);
console.log(unsafe.rowCount);
```

6

Six rows: **every customer's name and email**, from a search for one name. Look at the SQL that was actually sent:

```
SELECT name, email FROM customers WHERE name = '' OR 1=1 --'
```

The attacker's quote closed your string early, `OR 1=1` made the condition always true, and `--` turned the rest into a comment. This is **SQL injection**, and it has leaked more data than any other bug in the history of the web. With a different input it can delete tables or change passwords. It works against any app that glues user input into SQL text.

The fix is to never glue. Use a **placeholder**, and pass the value separately:

```
const safe = await pool.query("SELECT name, email FROM customers WHERE name = $1", [userInput]);
console.log(safe.rowCount);
```

```
0
```

Zero rows: no customer is literally named ' OR 1=1 --'. The value travelled to PostgreSQL *as a value*, separately from the SQL, so it could never be mistaken for SQL. `$1`, `$2`, `$3` match the positions in the array:

```
await pool.query(
  "SELECT name FROM customers WHERE city = $1 AND joined_on >= $2 ORDER BY name",
  ["Kathmandu", "2026-01-01"],
);
// rows: Asha Rai, Dipesh Thapa
```

The rule has no exceptions: user input goes in the array, never in the string. Not for numbers, not for "trusted" input, not "just this once". Every library you'll use (including ORMs) does this for you underneath. Your only job is to not go around it.

Errors

A failing query throws. Catch it where you can do something useful:

```
try {
  await pool.query("INSERT INTO categories (name) VALUES ('Bags')");
} catch (err) {
  const e = err as { code: string; detail?: string };
  console.log(e.code, e.detail);
}
```

```
23505 Key (name)=(Bags) already exists.
```

PostgreSQL errors carry a five-character **code**. The ones you'll handle most:

Code	Meaning	Typical response
23505	Unique violation	"That email is already registered"
23503	Foreign key violation	"That customer doesn't exist"
23514	Check violation	"Quantity must be positive"

Code	Meaning	Typical response
23502	Not-null violation	"Name is required"
42P01	Table doesn't exist	A bug or a missing migration

Switch on `e.code` to turn database refusals into friendly messages. For anything else: log it on the server and return a generic "something went wrong". Never send the raw message to a user, because it can contain table names or data.

Transactions in code

A transaction must run on **one** connection. So you borrow one from the pool, use it for every statement, and give it back:

```

async function placeOrder(customerId: number, lines: { productId: number; quantity: number }[])
{
  const client = await pool.connect();
  try {
    await client.query("BEGIN");

    const order = await client.query<{ id: number }>(
      "INSERT INTO orders (customer_id, ordered_at, status) VALUES ($1, now(), 'pending')
      RETURNING id",
      [customerId],
    );
    const orderId = order.rows[0].id;

    for (const line of lines) {
      await client.query(
        `INSERT INTO order_items (order_id, product_id, quantity, unit_price)
        VALUES ($1, $2, $3, (SELECT price FROM products WHERE id = $2))`,
        [orderId, line.productId, line.quantity],
      );
      await client.query("UPDATE products SET stock = stock - $1 WHERE id = $2", [line.quantity,
        line.productId]);
    }

    await client.query("COMMIT");
    return orderId;
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();
  }
}

```

The shape to memorize: `connect`, then `try { BEGIN ... COMMIT } catch { ROLLBACK; rethrow } finally { release }`. Every statement goes through `client`, not `pool`. The `finally` guarantees the connection goes back to the pool even when something fails. Forget it, and your pool slowly runs dry.

Test it with a good order and a bad one:

```
const okId = await placeOrder(5, [{ productId: 2, quantity: 2 }, { productId: 10, quantity: 1 }]);
console.log("Placed order", okId);

try {
  await placeOrder(5, [{ productId: 1, quantity: 1 }, { productId: 7, quantity: 1000 }]);
} catch (err) {
  console.log("Order failed:", (err as Error).message);
}
```

```
Placed order 13
Order failed: new row for relation "products" violates check constraint "products_stock_check"
```

The second order inserted a notebook line, then tried to take 1000 chairs from a stock of 6. The `CHECK` refused, the `catch` rolled back, and the notebook line and the order row vanished with it. Nothing half-done. (Chapter 30 explains why. The `RETURNING id` and the price-lookup subquery are from chapters 06 and 25.) Reload the seed afterwards if you want the dataset back to normal.

A tiny API

Here's the bridge to your React apps: an HTTP server that answers with JSON straight from the database. No framework, just Node's built-in `http` module. `api.ts`:

```

import { createServer } from "node:http";
import pg from "pg";

const { Pool } = pg;
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

const server = createServer(async (req, res) => {
  const url = new URL(req.url ?? "/", "http://localhost");
  res.setHeader("Content-Type", "application/json");

  try {
    // GET /products or /products?category=Bags
    if (url.pathname === "/products") {
      const category = url.searchParams.get("category"); // null if not given
      const result = await pool.query(
        `SELECT p.id, p.name, p.price, c.name AS category
         FROM products p JOIN categories c ON c.id = p.category_id
         WHERE p.is_active AND ($1::text IS NULL OR c.name = $1)
         ORDER BY p.name`,
        [category],
      );
      res.end(JSON.stringify(result.rows));
      return;
    }

    // GET /orders/5
    const match = url.pathname.match(/^\/orders\/(\d+)\$/);
    if (match) {
      const result = await pool.query(
        `SELECT jsonb_build_object(
          'order_id', o.id,
          'customer', c.name,
          'status', o.status,
          'items', (SELECT jsonb_agg(jsonb_build_object('product', p.name, 'quantity',
            oi.quantity, 'unit_price', oi.unit_price) ORDER BY p.name)
            FROM order_items oi JOIN products p ON p.id = oi.product_id WHERE
            oi.order_id = o.id)
        ) AS doc
        FROM orders o JOIN customers c ON c.id = o.customer_id
        WHERE o.id = $1`,
        [match[1]],
      );
      if (result.rowCount === 0) {
        res.statusCode = 404;
        res.end(JSON.stringify({ error: "Order not found" }));
        return;
      }
      res.end(JSON.stringify(result.rows[0].doc));
      return;
    }
  }
});

```



```

    }

    res.statusCode = 404;
    res.end(JSON.stringify({ error: "Not found" }));
  } catch (err) {
    console.error(err);
    res.statusCode = 500;
    res.end(JSON.stringify({ error: "Something went wrong" }));
  }
});

server.listen(3000, () => console.log("API listening on http://localhost:3000"));

```

Run it with `node --env-file=.env api.ts`, then open these in your browser:

- `http://localhost:3000/products?category=Bags` gives the two bags as a JSON array.
- `http://localhost:3000/orders/5` gives the nested order document from chapter 37, built by the database and sent as-is.
- `http://localhost:3000/orders/999` gives `{"error":"Order not found"}` with status 404.

A React component can `fetch` any of those URLs today.

Notice: the `category` from the URL goes in as `$1`, never into the string. The `$1::text IS NULL OR c.name = $1` trick makes the filter optional: with no category, the first half is true and every product comes back. The pool is created once and never ended, because the server runs until you stop it (Ctrl+C). Errors are logged on the server and hidden from the client.

Real projects use a framework like Express or Fastify for the routing instead of `if` statements on the path, but the database part is exactly this.

Habits

1. **One pool, created once.** Not one per request.
2. **Parameters, always.** `$1`, never string concatenation.
3. **Transactions on a client**, with `try/catch/finally` and `release()`.
4. **Convert numeric and bigint on purpose.** They're strings.
5. **Connection string in .env**, and `.env` in `.gitignore`.
6. **Never run your app as postgres**. Chapter 34: make a `shop_app` role and put *its* details in `DATABASE_URL`.
7. **Map error codes to messages.** Don't leak raw errors to users.

Common mistakes

1. String concatenation

The injection. Even once. Even for a number.

2. Running a transaction through `pool.query`

Each call may land on a different connection, so `BEGIN` on one and `UPDATE` on another means no transaction at all. Use `pool.connect()` and one `client`.

3. Forgetting `client.release()`

After about ten leaked connections (the pool's default size), every query waits forever. Put it in `finally`.

4. Treating `numeric` as a number

`'13.00' + '5.00'`. Convert with `Number()`, or keep it as a string for display.

5. Ending the pool in a server

`pool.end()` is for scripts. A server keeps the pool open.

6. Committing `.env`

Your password is now on GitHub, and deleting the commit later doesn't remove it from history. Make the `.gitignore` first, then the `.env` file.

Quick recap

- `npm install pg`, then `import pg from "pg"; const { Pool } = pg;`. One `Pool` per program, built from `DATABASE_URL` in `.env`.
- `const result = await pool.query(sql, [params]);` then use `result.rows` and `result.rowCount`.
- `$1`, `$2` placeholders with a values array are the only defence against SQL injection, and the rule has no exceptions.
- `numeric`, `bigint` and `count(*)` come back as **strings**; dates come back as `Date`.
- Transactions: `pool.connect()`, `BEGIN / COMMIT` on that `client`, `ROLLBACK` in `catch`, `release()` in `finally`.
- Errors have codes (`23505` unique, `23503` foreign key, `23514` check). Turn them into friendly messages.
- A `node:http` server plus `pool.query` plus `JSON.stringify(rows)` is a working API for your React app.

Next: try the [exercises](#), then move on to [41 Migrations](#).

Exercises

How to do these:

- Work in `PostgreSQL/playground/node-shop/`, set up as in the notes, with `DATABASE_URL` pointing at `sales`.
 - One `.ts` file per exercise (`ex1.ts`, `ex2.ts`, ...). Run with `node --env-file=.env ex1.ts`.
 - **Every** value that comes from outside the program (a command-line argument, a URL) goes in the `parameters` array. If you catch yourself writing `${...}` inside SQL, stop.
 - Undo any test data you insert, or reload the seed afterwards, so `sales` stays as the seed left it.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): List and count

1. Print every customer's name and city, one per line, formatted like `Asha Rai (Kathmandu)`.
2. Print the number of orders and the total revenue, both excluding cancelled orders, as one sentence. The revenue comes back as a string; turn it into a number before formatting it with two decimals.

Expected second output: `11 orders, revenue 664.16`.

Exercise 2 (Easy): A command-line search

Read a city from the command line (`process.argv[2]`) and print the customers who live there, newest member first. Run it as `node --env-file=.env ex2.ts Pokhara`.

Then run it with the argument `"' OR 1=1 --"` (in double quotes). If it prints all six customers, you have an injection. Fix it.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Friendly errors

Write a function `addCategory(name)` that inserts a category and returns its new id. Call it three times: with a new name, with `'Bags'` (already exists), and with an empty string, after adding `CHECK (length(name) > 0)` to the table in `psql`.

For each failure, catch the error, look at `code`, and print a **friendly** message (A category called Bags already exists, Category name can't be empty), never the raw error. For any code you didn't expect, print `Unexpected error` and rethrow.

Clean up afterwards: delete the category you added, and drop the check.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): A transaction that moves stock

Write `placeOrder(customerId, lines)` from the notes, then extend it:

1. Throw, before touching the database, if `lines` is empty.
2. Record the unit price from `products` with a subquery, as in the notes.
3. After the loop, `SELECT` the order total inside the same transaction, and return `{ orderId, total }`.

Test it: a good order for Elina (customer 5), then an order that oversells a product. Afterwards, prove with a query that Elina has exactly one order and the oversold product's stock is unchanged. Then delete the test order and put the stock back (or reload the seed).

Exercise 5 (Challenge): An API for your React app

Build the `node:http` server from the notes, then add:

1. `GET /customers/:id/orders`: that customer's orders as a JSON array, each with its total and number of items. Return `404` if the customer doesn't exist.
2. `GET /search?q=term`: products whose name contains the term, ignoring case, using a parameterized `ILIKE`.
3. Proper status codes: `200`, `404`, and `500` for unexpected errors, with the real error only in the server's log.

Test every route in the browser, including a bad id and a search for `' OR 1=1 --`. Then, if you have an app from the React course handy, `fetch /products` from it and render the list. You've just built your first full stack.

Stuck? The hint is in the [Hints section](#) at the end.

41 Migrations

What is it?

A **migration** is a file holding one change to your database's structure: create this table, add that column, build this index. Migrations are **numbered**, kept in your project next to the code, and **applied in order** by a small program that remembers which ones have already run.

```
migrations/
├─ 001_create_notes.sql
├─ 002_add_is_done_to_notes.sql
└─ 003_create_tags.sql
```

You've been doing this by hand since chapter 15: an `ALTER TABLE` saved in a file, run inside a transaction. Migrations make it a system.

Why does it matter?

Your `schema.sql` files from the projects rebuild a database from nothing. That's perfect for a database with no data in it. It's useless for one with real customers in it: you can't drop and recreate `orders` just to add a column.

And a real project has **many copies** of its database: your laptop, each teammate's laptop, a test server, production. Every copy must end up with the same structure. "Please run this `ALTER` on your machine too" in a chat message doesn't work for long. Someone forgets, someone runs it twice, someone's copy is slightly different, and a query that works for you fails for them.

Migrations fix both problems. Every change is a file in version control, applied exactly once, in the same order, on every copy. The history of your database becomes as easy to review as the history of your code.

Real-world example

Building permits for a house people live in:

House	Database
You can't knock the house down to add a window	You can't drop a table full of data to add a column
Each change is a numbered permit: "Permit 12: add a window to the east wall"	Each change is a numbered migration file

House	Database
The council keeps a list of permits that have been carried out	The <code>schema_migrations</code> table
Every copy of the house (the show home, yours, your neighbour's) gets the same permits in the same order	Dev, test and production all run the same files
You don't edit a permit after the work is done; you file a new one	Never change a migration that has run; add another

How it works

The idea in three steps

1. A table in the database, `schema_migrations`, records which files have been applied.
2. A small program (the **runner**) reads the `migrations/` folder, sorted by name.
3. For each file not yet in the table: run it inside a transaction, and record it. Stop at the first failure.

That's all a migration tool is. Let's build one. Twenty lines will teach you more than any tool's documentation.

Two migrations

In your `node-shop` folder from chapter 40, make `migrations/001_create_notes.sql`:

```
-- 001: notes table for the migrations demo
CREATE TABLE notes (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  body text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

and `migrations/002_add_is_done_to_notes.sql`:

```
-- 002: notes can be ticked off
ALTER TABLE notes ADD COLUMN is_done boolean NOT NULL DEFAULT false;
CREATE INDEX idx_notes_is_done ON notes (is_done) WHERE NOT is_done;
```

Plain SQL, one change each, numbered so they sort in the right order. The number is what matters; the rest of the name is a description for humans.

The runner

`migrate.ts` :

```

import { readdir, readFile } from "node:fs/promises";
import path from "node:path";
import pg from "pg";

const { Client } = pg;

const migrationsDir = path.join(import.meta.dirname, "migrations");

const client = new Client({ connectionString: process.env.DATABASE_URL });
await client.connect();

// 1. The bookkeeping table: which migrations have already run?
await client.query(`
  CREATE TABLE IF NOT EXISTS schema_migrations (
    version text PRIMARY KEY,
    applied_at timestamptz NOT NULL DEFAULT now()
  )
`);

const applied = new Set(
  (await client.query<{ version: string }>("SELECT version FROM
    schema_migrations")).rows.map((r) => r.version),
);

// 2. Every .sql file in the folder, in name order
const files = (await readdir(migrationsDir)).filter((f) => f.endsWith(".sql")).sort();

// 3. Run each one that hasn't been applied, inside its own transaction
for (const file of files) {
  if (applied.has(file)) {
    console.log("skip ", file);
    continue;
  }
  const sql = await readFile(path.join(migrationsDir, file), "utf8");
  try {
    await client.query("BEGIN");
    await client.query(sql);
    await client.query("INSERT INTO schema_migrations (version) VALUES ($1)", [file]);
    await client.query("COMMIT");
    console.log("apply", file);
  } catch (err) {
    await client.query("ROLLBACK");
    console.error("FAILED", file, "-", (err as Error).message);
    process.exitCode = 1;
    break;
  }
}

```



```
await client.end();
```

It uses a single `Client` rather than a `Pool`, because migrations run one at a time on one connection. The file's SQL and the `INSERT` into `schema_migrations` are in the **same transaction**: either the change happens *and* is recorded, or neither happens. That works because PostgreSQL's `ALTER TABLE` can be rolled back (chapter 15). In some other databases it can't, and migration tools there have a harder life.

Running it

Make a fresh database to try it on, and point `DATABASE_URL` at it:

```
psql -U postgres -c "CREATE DATABASE migrations_demo;"  
node --env-file=.env migrate.ts
```

```
apply 001_create_notes.sql  
apply 002_add_is_done_to_notes.sql
```

Run it again:

```
skip 001_create_notes.sql  
skip 002_add_is_done_to_notes.sql
```

Nothing happens, because both are recorded. **Running migrations is safe to repeat**, which is exactly what you want from something that runs on every deploy.

```
SELECT * FROM schema_migrations ORDER BY version;
```

version		applied_at
001_create_notes.sql		2026-10-01 17:51:41.143656+05:45
002_add_is_done_to_notes.sql		2026-10-01 17:51:41.155609+05:45

The database now knows its own history.

When a migration fails

Add `migrations/003_bad_migration.sql`, broken on purpose:

```
-- 003: deliberately broken, to show the rollback
ALTER TABLE notes ADD COLUMN priority integer NOT NULL DEFAULT 3;
ALTER TABLE notes ADD COLUMN priority integer; -- duplicate column: this line fails
```

```
skip 001_create_notes.sql
skip 002_add_is_done_to_notes.sql
FAILED 003_bad_migration.sql - column "priority" of relation "notes" already exists
```

Now look at `\d notes` in psql: there is **no** `priority` column. The first `ALTER` in the file worked, the second failed, and the `ROLLBACK` undid the first. The file isn't recorded either, so once you fix it, the runner will try it again. A failed migration leaves the database exactly as it was. That's the whole reason for the transaction.

Delete or fix the bad file, run again, and you get two skips and no failures.

The rules

- 1. Never edit a migration that has been applied anywhere.** Your laptop has run `002`; so has your teammate's. If you change `002` now, their database and yours drift apart, and the runner won't notice, because `002` is already marked "done" on both. Want to change something? Write `003`.
- 2. One change per file**, named for what it does. `005_add_phone_to_customers.sql`, not `005_misc.sql`. The folder listing becomes your schema's changelog.
- 3. Migrations are code.** Commit them together with the feature that needs them, and review them. A pull request that adds a column has a migration in it.
- 4. Run them when you deploy, before the new code starts.** The new code expects the column, so the column must exist first.
- 5. Test them on a copy of real data** before production. A migration that's instant on your 12-row table can lock a 50-million-row table for an hour.

Up and down

Some teams write a **down** migration for each one: `002_add_is_done_to_notes.down.sql` containing `ALTER TABLE notes DROP COLUMN is_done;`. It lets you undo the last change if a deploy goes wrong. Chapter 15's exercise 5 had you write a pair.

An honest opinion: down migrations are handy in development and rarely used in production. By the time you'd want to roll back, the new column usually has data in it that the down migration would destroy. Many teams write them anyway for the convenience; many don't. Both are defensible. Find out what your team does.

Data migrations

Not every migration changes structure. Sometimes it's "lowercase every existing email", or "mark every product with stock as active". These are **data migrations**, and they're migrations too: numbered, recorded, run once.

The safe way to add a required column to a table that already has rows (chapter 15) fits in one file:

```
ALTER TABLE customers ADD COLUMN phone text;           -- 1. nullable, so existing rows
    are fine
UPDATE customers SET phone = 'unknown' WHERE phone IS NULL; -- 2. fill it (the data part)
ALTER TABLE customers ALTER COLUMN phone SET NOT NULL;  -- 3. now lock it down
```

Starting from an existing database

If you already have a database (your `shop` from chapter 19, say), its first migration is a **baseline**:

`000_baseline.sql`, containing the output of `pg_dump --schema-only` (chapter 35). On copies that already have those tables, mark it as applied without running it (`INSERT INTO schema_migrations (version) VALUES ('000_baseline.sql')`). On brand-new copies, let it run. From then on, every change is a new numbered file.

Big tables and locks

Two things to know before running migrations against a table with millions of rows:

- **ALTER TABLE takes a lock** that blocks reads and writes while it runs. Adding a nullable column, or one with a fixed default, is instant. Changing a column's type, or adding `NOT NULL` without a default, reads or rewrites the whole table, and blocks everyone until it's done. Do those at quiet times.
- **CREATE INDEX blocks writes** while it builds. `CREATE INDEX CONCURRENTLY` doesn't, but it can't run inside a transaction, so your runner needs a way to run a file *without* `BEGIN` and `COMMIT` (real tools have a flag or a special comment for this). For the table sizes in this course you can ignore it. For a big production table, remember it exists.

Real tools

The runner above is a teaching version. Real tools add a command that creates the next numbered file for you, down migrations, a lock so two deploys can't migrate at the same time, and a way to mark the files that can't run in a transaction. In the Node world you'll meet:

Tool	Style
node-pg-migrate	Migrations in JavaScript or SQL, built on <code>pg</code> . Small and popular.
dbmate	One program, plain SQL files with <code>-- migrate:up</code> and <code>-- migrate:down</code> sections. Works with any language.

Tool	Style
Flyway / Liquibase	The Java-world standards. Plain SQL works with both. Common in big companies.
Prisma Migrate, Drizzle Kit	Part of the ORMs in chapter 42 . They <i>generate</i> the SQL by comparing your schema definition with the database.

They all do what your twenty lines do, plus extras. Pick the one your team or ORM uses, and read its migration files with the understanding you now have.

Common mistakes

1. Editing an applied migration

Databases drift apart, silently. Always a new file.

2. Running SQL by hand on production

The change isn't in a file, isn't on anyone else's copy, and isn't in version control. At the next deploy, something breaks. Every change goes through a migration.

3. Migrations that depend on today's date or data

`UPDATE prices SET ... WHERE created_at > now() - interval '1 day'` does different things on different days. A migration must do the same thing wherever and whenever it runs.

4. No transaction

A half-applied change with no record of it. Keep `BEGIN / COMMIT` around every file, unless the file truly can't run in one (concurrent indexes).

5. Deploying code before running migrations

The new code asks for a column that doesn't exist yet. Migrate first.

6. One giant "fix everything" migration

Hard to review, and if line 40 fails, lines 1 to 39 roll back too. Small files.

Quick recap

- A migration is a numbered SQL file holding one change. A runner applies the files that haven't run yet, in order, and records each one in `schema_migrations`.
- A file's SQL and its record go in **one transaction**, so a failure leaves the database untouched.

- Running migrations is repeatable: files that already ran are skipped.
 - **Never edit an applied migration.** Add a new one. One change per file, committed with the code, run before the new code starts.
 - Data changes (backfills) are migrations too. For required columns on live tables: add nullable, fill, then `SET NOT NULL`.
 - Real tools (node-pg-migrate, dbmate, Prisma Migrate, Drizzle Kit) do exactly this, with extras.
-

Next: try the [exercises](#), then move on to [42 ORMs](#).

Exercises

How to do these:

- Work in `PostgreSQL/playground/node-shop/`. Create a database called `migrations_demo` and point `DATABASE_URL` at it for these exercises. (Or make a second file, `.env.migrations`, and run with `--env-file=.env.migrations`.)
 - Write down what each run printed in `playground/ch41/notes.md`.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Build and run the runner

1. Create `migrate.ts` and the two migrations from the notes. Run it. Run it again. Copy both outputs into your notes.
 2. Look at `schema_migrations` and `\d notes` in psql.
 3. Add `003_create_tags.sql`: a `tags` table and a `note_tags` junction table (chapter 17). Run the migrator. Only `003` should apply.
-

Exercise 2 (Easy): Watch a failure

1. Write `004_broken.sql` with two statements where the **second** one fails (a duplicate column, a type that doesn't exist, anything). Run the migrator and copy the error.
2. Check in psql that the first statement's change did **not** survive, and that `004` isn't in `schema_migrations`.
3. Fix the file (keep the same name, since it never ran anywhere) and run again.
4. In your notes: why was it fine to edit `004` here, when editing `002` would not be?

Exercise 3 (Medium): Migrate the shop

Turn your chapter 19 project into migrations, in a fresh database called `shop_migrated`:

1. `001_create_shop.sql`: the contents of your `schema.sql`, without the `DROP` lines. (A migration creates. It never drops what came before it.)
2. `002_seed_reference_data.sql`: the categories. Reference data that the app needs in order to work belongs in a migration. Sample customers and orders don't; those stay in a seed file.
3. `003_` to `006_`: one file per change request from chapter 19's Milestone 7.
4. Run them all. Then run your old `seed.sql` (without the categories) and `queries.sql` against the result. Same answers as before?

Exercise 4 (Medium): A required column on a live table

In `shop_migrated`, with customers in it, add a **required** column `country text NOT NULL`.

1. First try the naive way: one migration with `ALTER TABLE customers ADD COLUMN country text NOT NULL;`. Watch it fail, and check that the runner left the table unchanged.
2. Write it properly as one migration with three statements: add it nullable, fill it with `'Nepal'`, then set `NOT NULL`.
3. In your notes: on a table with 50 million rows, which of the three statements would be slow, and what could you do about it?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Down migrations

Extend `migrate.ts`:

1. When run as `node --env-file=.env migrate.ts down`, it should find the **most recently applied** migration, run the matching `NNN_name.down.sql` file inside a transaction, and delete that migration's row from `schema_migrations`.
2. Write `.down.sql` files for your `notes` migrations (001 to 003). Run `down` three times, then run normally once. The database should end up complete again.
3. In your notes: write the down migration for Exercise 4's `country` column. What data does it destroy? When, if ever, would you run it in production?

Stuck? The hint is in the [Hints section](#) at the end.

42 ORMs

What is it?

An **ORM** (object-relational mapper) is a library that lets you talk to the database in your programming language instead of in SQL strings. You describe your tables once in TypeScript, and then write things like:

```
const cheap = await db.select().from(products).where(lt(products.price, "5"));
```

and the ORM writes and runs the SQL for you:

```
select * from "products" where "products"."price" < $1
```

The two you'll meet most in the Node world are **Drizzle** and **Prisma**. This chapter shows what they do, what they're good at, and when plain SQL (chapter 40) is the better tool.

Why does it matter?

Three real benefits:

1. **Types.** With `pg`, `result.rows` is `any`. You can misspell a column and find out at runtime. With an ORM, `products.name` is a known column with a known type, and your editor catches the typo as you type. For a TypeScript developer, this is the big one.
2. **Less boilerplate.** Inserting an object, updating a few fields, building a `WHERE` from optional filters: with SQL strings these mean fiddly string-building. ORMs make them one-liners.
3. **Migrations included.** Both tools compare your TypeScript schema with the real database and generate the migration SQL (chapter 41) for you.

And one real cost, which the whole second half of this chapter is about: **an ORM hides the SQL**, and everything you learned in Levels 3 and 4 is about the SQL. Developers who only know the ORM write slow queries without knowing it. You know better. Now you'll learn to use an ORM *and* keep seeing through it.

Real-world example

A **translator** between you and the librarian.

You can learn the librarian's language (SQL) and speak it yourself. Or you can tell a translator in your own language what you want, and they'll say it to the librarian. The translator is faster for everyday requests and never makes a grammar mistake. But for an unusual request, the translation may be clumsy or slow, and you

may not realize unless you listen to what they actually said. A good ORM lets you listen (log the SQL). A good developer does.

How it works

Drizzle: SQL-shaped TypeScript

Drizzle's idea is "if you know SQL, you know Drizzle". Install it in your `node-shop` project:

```
npm install drizzle-orm
```

(It uses the `pg` driver you already have.)

Describe your tables in `schema.ts`. This mirrors `\d` for each table:


```

import { pgTable, integer, text, numeric, boolean, timestamp, date } from "drizzle-orm/pg-core";

export const categories = pgTable("categories", {
  id: integer("id").primaryKey().generatedAlwaysAsIdentity(),
  name: text("name").notNull().unique(),
});

export const products = pgTable("products", {
  id: integer("id").primaryKey().generatedAlwaysAsIdentity(),
  categoryId: integer("category_id").notNull().references(() => categories.id),
  name: text("name").notNull(),
  sku: text("sku").notNull().unique(),
  price: numeric("price", { precision: 10, scale: 2 }).notNull(),
  stock: integer("stock").notNull().default(0),
  isActive: boolean("is_active").notNull().default(true),
});

export const customers = pgTable("customers", {
  id: integer("id").primaryKey().generatedAlwaysAsIdentity(),
  name: text("name").notNull(),
  email: text("email").notNull().unique(),
  city: text("city").notNull(),
  joinedOn: date("joined_on").notNull(),
});

export const orders = pgTable("orders", {
  id: integer("id").primaryKey().generatedAlwaysAsIdentity(),
  customerId: integer("customer_id").notNull().references(() => customers.id),
  orderedAt: timestamp("ordered_at", { withTimezone: true }).notNull(),
  status: text("status").notNull().default("pending"),
});

```

Every word here is something you learned in Level 2: identity primary keys (14), `notNull` and `unique` and `default` (13), `references` (16), `numeric(10, 2)` and `timestamp with time zone` (12). The TypeScript names (`categoryId`) can differ from the column names (`category_id`); Drizzle maps between them.

Connect and query. `drizzle-demo.ts`:

```

import pg from "pg";
import { drizzle } from "drizzle-orm/node-postgres";
import { eq, desc, sql } from "drizzle-orm";
import { products, categories, customers, orders } from "./schema.ts";

const { Pool } = pg;
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
const db = drizzle(pool);

// 1. Select with a where and an order
const cheap = await db
  .select({ name: products.name, price: products.price })
  .from(products)
  .where(sql`${products.price} < 5`)
  .orderBy(products.price);
console.log("cheap:", cheap);

// 2. A join
const bags = await db
  .select({ product: products.name, category: categories.name })
  .from(products)
  .innerJoin(categories, eq(categories.id, products.categoryId))
  .where(eq(categories.name, "Bags"));
console.log("bags:", bags);

// 3. Insert with returning
const inserted = await db
  .insert(categories)
  .values({ name: "Books" })
  .returning({ id: categories.id, name: categories.name });
console.log("inserted:", inserted);
await db.delete(categories).where(eq(categories.id, inserted[0].id)); // tidy up

// 4. Update
const updated = await db
  .update(products)
  .set({ stock: 119 })
  .where(eq(products.id, 1))
  .returning({ name: products.name, stock: products.stock });
console.log("updated:", updated);
await db.update(products).set({ stock: 120 }).where(eq(products.id, 1));

// 5. Group by, with a raw SQL fragment for the aggregate
const perCity = await db
  .select({ city: customers.city, customers: sql`<number>`count(*)` })
  .from(customers)
  .groupBy(customers.city)
  .orderBy(customers.city);
console.log("per city:", perCity);

```

```
// 6. The latest order
const latest = await db.select().from(orders).orderBy(desc(orders.orderedAt)).limit(1);
console.log("latest order id:", latest[0].id);

// 7. See the SQL Drizzle would send
const q = db.select({ name: products.name }).from(products).where(eq(products.isActive, true));
console.log("generated SQL:", q.toSQL().sql);

await pool.end();
```

```
cheap: [
  { name: 'Pen', price: '1.20' },
  { name: 'Whiteboard marker', price: '1.80' },
  { name: 'Sticky notes', price: '2.75' },
  { name: 'Notebook', price: '3.90' }
]
bags: [
  { product: 'Backpack', category: 'Bags' },
  { product: 'Laptop sleeve', category: 'Bags' }
]
inserted: [ { id: 7, name: 'Books' } ]
updated: [ { name: 'Notebook', stock: 119 } ]
per city: [
  { city: 'Kathmandu', customers: '2' },
  { city: 'Lalitpur', customers: '2' },
  { city: 'Pokhara', customers: '2' }
]
latest order id: 12
generated SQL: select "name" from "products" where "products"."is_active" = $1
```

Read it against the SQL you know: `select ... from ... where ... orderBy` is `SELECT ... FROM ... WHERE ... ORDER BY`. `innerJoin(table, eq(a, b))` is `JOIN table ON a = b`. `insert(...).values(...).returning(...)` is `INSERT ... VALUES ... RETURNING`. Nothing is hidden. The last line proves it: `toSQL()` shows exactly what will be sent, and it's parameterized (`$1`), so injection is impossible by construction.

Two things to notice from chapter 40 that haven't changed: `price` is still a **string** (it's `numeric`), and `count(*)` is still a string. The ORM uses the same driver underneath, so the same conversions apply.

The `sql` tag (`sql`${products.price} < 5``) is Drizzle's escape hatch: any SQL fragment, with the column reference and parameters handled safely. You'll use it for aggregates, window functions, `date_trunc`, and anything else Drizzle doesn't have a helper for. That's a feature, not a workaround: the ORM gets out of the way when you need it to.

Migrations with Drizzle Kit. `npm install -D drizzle-kit`, a small config file pointing at `schema.ts` and `DATABASE_URL`, and then `npx drizzle-kit generate` compares your schema file with the database and writes a

numbered SQL migration into a folder, exactly like the ones from chapter 41, which `npx drizzle-kit migrate` applies. You change `schema.ts`, generate, read the SQL it produced (always read it), commit, deploy. The tool's docs have the config details; they're short.

Prisma: a schema language and a generated client

Prisma takes a different route. You describe tables in its own language, in `prisma/schema.prisma`:

```
model Category {
  id      Int      @id @default(autoincrement())
  name    String   @unique
  products Product[]
}

model Product {
  id      Int      @id @default(autoincrement())
  name    String
  sku     String   @unique
  price   Decimal  @db.Decimal(10, 2)
  stock   Int      @default(0)
  isActive Boolean  @default(true) @map("is_active")
  category Category @relation(fields: [categoryId], references: [id])
  categoryId Int    @map("category_id")

  @@map("products")
}
```

Then `npx prisma migrate dev` writes and applies the migration, and `npx prisma generate` builds a **client** with a method for every model:

```
const cheap = await prisma.product.findMany({
  where: { price: { lt: 5 } },
  orderBy: { price: "asc" },
  select: { name: true, price: true },
});

const bags = await prisma.product.findMany({
  where: { category: { name: "Bags" } },
  include: { category: true }, // the join, expressed as "include the related row"
});

const order = await prisma.order.findUnique({
  where: { id: 5 },
  include: { customer: true, items: { include: { product: true } } }, // nested relations
});
```

Prisma reads less like SQL and more like describing the shape of the object you want back. `include` fetches related rows and nests them, which is lovely for the "order with its items with their products" case. For a one-off `GROUP BY` with a window function, you'll reach for `prisma.$queryRaw` and write SQL.

Honest note: Prisma's setup (how the client is generated, where it's imported from, whether a driver adapter is needed) has changed noticeably between its major versions. The model syntax and the query style above are stable; for the exact install and generate commands, follow the current "Getting started with PostgreSQL" guide on Prisma's site rather than any blog post.

Drizzle or Prisma?

	Drizzle	Prisma
Schema written in	TypeScript	Prisma's own language
Queries look like	SQL	Object shapes (<code>where</code> , <code>include</code> , <code>select</code>)
Joins	Explicit <code>innerJoin</code> , like SQL	Implicit via <code>include</code> (and a relational query API that also nests)
When SQL doesn't fit	<code>sql\ ...`</code> fragments anywhere	<code>\$queryRaw</code> for whole queries
Generated SQL	One query, visible via <code>toSQL()</code>	Can be several queries per <code>include</code> ; log to see
Learning curve for you	Small, since you know SQL	Different mental model, very good docs
Feels best for	People who think in SQL and want types	People who think in objects and want nesting for free

Both are good. Both are widely used. If you've done this course, Drizzle will feel like SQL with autocomplete, which is a nice place to be. Prisma is worth knowing because many teams use it. Try both in the exercises and pick the one that makes you *more* aware of the database, not less.

Seeing through the ORM

This is the part that matters more than which library you choose.

Turn on query logging. Drizzle: `drizzle(pool, { logger: true })` . Prisma: `new PrismaClient({ log: ["query"] })` . Now every query the ORM sends is printed. Leave it on in development, always.

Watch for N+1. The most common ORM disaster (chapter 32). Code that looks innocent:

```
const allOrders = await db.select().from(orders);
for (const o of allOrders) {
  const items = await db.select().from(orderItems).where(eq(orderItems.orderId, o.id)); // one
  query per order
}
```

sends 1 + 12 queries on our data and 1 + 10,000 in production. With logging on, you'll see the flood. The fix is one join (or one `IN (...)`), as it always was. Prisma's `include` does the right thing *for you* in most cases, which is its great strength; Drizzle makes you write the join, which means you'll notice.

Know when to drop to SQL. Reports with `GROUP BY`, `HAVING`, window functions, CTEs, `date_trunc`, full-text search: these are what Level 3 taught you, and ORMs are clumsy at most of them. The right move is a `sql` fragment or a raw query, or a **view** (chapter 33) that the ORM reads like a table. Nobody will be impressed by a 40-line ORM expression that replaces a 6-line CTE.

Still design the database. ORMs generate migrations from your schema file, but they don't choose indexes for foreign keys (chapter 31), decide `ON DELETE` behaviour (16), write `CHECK` constraints that compare columns (13), or normalize (18). Those are still yours, in the schema file or in hand-written migrations alongside the generated ones.

Still use a least-privilege role. The ORM connects with whatever `DATABASE_URL` you give it. Chapter 34 still applies.

The shape of a real project

Most production Node projects end up like this:

- An ORM for the everyday work: `findMany`, `insert`, `update`, simple joins, with full types.
- Hand-written SQL (through the ORM's escape hatch, or `pg` directly) for reports, searches, bulk operations, and anything performance-sensitive.
- Views and functions in the database for logic that belongs there (chapters 33 and 38).
- The ORM's migration tool for structure changes, with the generated SQL **read before it's run**.
- Query logging on in development, and `EXPLAIN ANALYZE` (chapter 32) on anything slow.

That mix is what "knows the database *and* the ORM" looks like, and it's rarer and more valuable than either alone.

Common mistakes

1. Learning the ORM instead of SQL

Then every performance problem is a mystery. You've done it the right way round.

2. Never looking at the generated SQL

Turn on logging. Read `toSQL()`. If you can't explain what the ORM sent, you don't know what your code does.

3. N+1 in a loop

A query inside a `for` over rows. Join instead.

4. Forcing a report through the ORM

Forty lines of nested objects to avoid a CTE. Use `sql`, `$queryRaw`, or a view.

5. Treating the schema file as the whole design

Indexes on foreign keys, `ON DELETE` choices, check constraints, roles: still your job.

6. Trusting generated migrations blindly

A rename can be generated as drop-column-plus-add-column, which destroys the data. Read every generated migration before running it.

Quick recap

- An ORM lets you query in TypeScript with types; underneath it's `pg` and parameterized SQL.
- **Drizzle** is SQL-shaped: `select().from().where().orderBy()`, explicit joins, a `sql` tag for anything else, `toSQL()` to see the output.
- **Prisma** is object-shaped: a schema language, a generated client, `include` for nested relations, `$queryRaw` for SQL.
- Both generate migrations from your schema. Read the SQL they produce.
- `numeric` and `count(*)` are still strings. The driver hasn't changed.
- Turn on query logging, watch for N+1, drop to SQL for reports, and keep designing the database yourself.

Next: try the [exercises](#), then move on to [43 PostgreSQL in the Cloud](#).

Exercises

How to do these:

- Work in `PostgreSQL/playground/node-shop/` with `npm install drizzle-orm` done and the `schema.ts` from the notes.
- One `.ts` file per exercise. Turn on query logging (`drizzle(pool, { logger: true })`) in every file, and paste the SQL that was logged into comments next to your code.

- Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your work.
-

Exercise 1 (Easy): Translate from SQL

Write each of these with Drizzle, run it, and compare the logged SQL with what you'd have written by hand:

1. `SELECT name, price FROM products WHERE is_active ORDER BY price DESC LIMIT 3;`
2. `SELECT name, email FROM customers WHERE city = 'Pokhara';`
3. `SELECT o.id, c.name FROM orders o JOIN customers c ON c.id = o.customer_id WHERE o.status = 'pending';`

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Write, then read back

1. Insert a new customer with `.insert(customers).values({...}).returning()`. Note the `id`.
 2. Update their city with `.update(...).set(...).where(...)`.
 3. Delete them with `.delete(...).where(...)`.
 4. Try to insert a customer with an email that already exists. Catch the error and print its `code`. Is it the same code as in chapter 40?
-

Exercise 3 (Medium): Where the ORM runs out

1. Write "revenue per category, excluding cancelled orders" (chapter 22) in Drizzle. You'll need `sql` fragments for `sum(...)` and probably for the whole aggregate. Count how many lines it took.
2. Write the same query as one `sql` template passed to `db.execute(sql\...\`)`. Count the lines.
3. Create the `order_totals` view from chapter 33 in psql, add it to `schema.ts` with `pgView`, and select from it in Drizzle as if it were a table.
4. In a comment: for this shop's reporting, which of the three approaches would you use, and why?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Catch an N+1 in the act

1. Write the loop from the notes: fetch all orders, then fetch each order's items inside a `for` loop. With logging on, count the queries.
 2. Rewrite it as one join that returns every order line with its order's status and the customer's name. Count the queries.
 3. Rewrite it a third way: fetch all orders, collect their ids, then fetch all items `WHERE order_id IN (...)` in **one** query with `inArray`, and group them in JavaScript. Count the queries.
 4. In a comment: with 10,000 orders, how many queries does each version send? Which would you ship?
-

Exercise 5 (Challenge): Migrations with Drizzle Kit, or Prisma

Pick one:

Option A: Drizzle Kit. Install `drizzle-kit`, write its config file pointing at `schema.ts` and your `DATABASE_URL`, and run `generate` against a **fresh** database. Read the migration it produced. Then add a `phone` column to `customers` in `schema.ts`, generate again, and read that migration. Apply both with `migrate`. In a comment: what did the tool get right, and what would you still add by hand (chapter 31, chapter 16)?

Option B: Prisma. Follow Prisma's current PostgreSQL getting-started guide to set up a project against a **fresh** database. Model `categories` and `products`. Run the migration, generate the client, and write the three queries from Exercise 1 in Prisma's style. Turn on query logging and compare the SQL with Drizzle's. In a comment: which style did you find clearer, and what did each hide from you?

Either way, finish with one paragraph: how will you decide, on your next project, between plain `pg`, an ORM, and a mix?

43 PostgreSQL in the Cloud

What is it?

So far your database lives on your laptop. Your deployed React app (React chapter 40) lives on the internet, and it can't reach your laptop. A **cloud database** is PostgreSQL running on someone else's servers, reachable from anywhere with the right connection string, and looked after by them: updates, backups, disks, uptime.

You don't install anything. You sign up, click "create database", copy a connection string, and everything you've learned in this course works exactly the same.

Why does it matter?

Two reasons:

- 1. **Your apps need it.** A deployed API has to talk to a database that's also deployed. The moment your project leaves your laptop, so must its data.
- 2. **Someone else does the hard parts.** Chapter 35 told you to back up nightly, keep copies elsewhere, test restores, and think about point-in-time recovery. Chapter 34 mentioned `pg_hba.conf`. A managed service does all of that with checkboxes, and also handles things this course didn't cover: version upgrades, taking over when a machine dies, monitoring, scaling. For one developer or a small team, it's the difference between *running* a database and *using* one.

Real-world example

Owning a car versus using a taxi service:

Your laptop's PostgreSQL	Managed cloud PostgreSQL
You own the car, service it, insure it, and it sits in your driveway	A fleet someone else maintains, available wherever you are
Free, once bought	Pay for what you use (often free for small use)
Only you can drive it, from your house	Anyone with the key (connection string) can connect, from anywhere
Breaks down: your problem, your weekend	Breaks down: their on-call engineer

The car (local PostgreSQL) is still right for learning and development. The taxi is for going somewhere.

How it works

The providers

An honest note first: this is the part of the course most likely to go out of date, because these companies change their offers often. The names below were accurate when this was written; check current free tiers and prices before choosing.

Kind	Examples	Good for
PostgreSQL specialists	Neon, Supabase, Crunchy Bridge	Developer-friendly, generous free tiers, quick to start. Neon adds database "branching" (a copy per git branch). Supabase adds login, file storage, and an automatic API on top of plain PostgreSQL.
App platforms with a database add-on	Railway, Render, Fly.io, Heroku	You deploy your Node API <i>and</i> click "add PostgreSQL" in the same place. The simplest full-stack setup.
The big cloud providers	AWS RDS and Aurora, Google Cloud SQL, Azure Database for PostgreSQL, DigitalOcean	What companies use at scale. More settings, more cost, more to learn. You'll meet these at work.

For this course and your own projects: a specialist or an app platform. For a job interview: know that RDS exists and what "managed" means.

What "managed" gives you

Every line here is something you'd otherwise do yourself:

- **Automatic backups**, usually daily, kept for 7 to 35 days.
- **Point-in-time recovery**: restore to any second inside that window (the fancier method chapter 35 mentioned).
- **Updates and security patches** to PostgreSQL itself.
- **High availability**: a standby copy that takes over if the main one fails.
- **Monitoring**: CPU, connections, slow queries, disk space, on a dashboard.
- **Scaling**: more CPU or disk with a slider, often without downtime.
- **Encrypted connections** by default.

What it does **not** give you: good table design, indexes, sensible queries, or least-privilege roles. Those are still yours. Everything in Levels 2 to 4 matters just as much in the cloud.

Step by step

The flow is the same everywhere; only the buttons differ.

1. **Sign up** and create a project or database. Pick a **region** close to where your API server will run. Every query crosses that distance; a database on another continent adds 100 to 200 ms to every request.
2. **Copy the connection string.** It looks like your local one, with a real hostname and a long random password:

```
postgres://shop_app:Xk9...@ep-cool-name-123456.ap-southeast-1.aws.neon.tech/sales?
sslmode=require
```

The new part is `?sslmode=require`: the connection is encrypted. Your local connection didn't need it; one that crosses the internet must have it. `pg` understands the parameter.

3. **Connect from psql** to check it works:

```
psql "postgres://shop_app:Xk9...@ep-cool-name.../sales?sslmode=require"
```

Same prompt, same `\dt`, same everything, just a little slower.

4. **Create your structure.** Run your migrations (chapter 41) against it by pointing `DATABASE_URL` at the new string. Or restore a dump from your laptop (chapter 35): `pg_restore -d "postgres://...?sslmode=require" sales.dump`.
5. **Point your app at it.** On the platform that hosts your Node API (Render, Railway, Vercel, wherever), set the **environment variable** `DATABASE_URL` to the connection string. Your code doesn't change at all; it already reads `process.env.DATABASE_URL`. This is why chapter 40 insisted on `.env` instead of a hard-coded string.
6. **Deploy.** Your React app calls your API, your API talks to the cloud database. The full stack is live.

Secrets

The connection string **is** the password. Rules:

- It lives in the hosting platform's environment variables and in your local `.env`. Nowhere else.
- Never in code, never in git, never in a screenshot, never in a chat message.
- If it leaks, **rotate** it (every provider has a "reset password" button) and update the environment variable.
- Make an app role (chapter 34) and put *its* string in the app. Keep the admin string for yourself, for migrations and psql.

Connection limits and pooling

A cloud database allows a fixed number of connections at once, often 20 to 100 on small plans. One Node server with a pool of 10 is fine. But **serverless** platforms (Vercel functions, AWS Lambda, Cloudflare Workers) start a fresh copy of your code for every burst of traffic, and each copy makes its own pool. Fifty copies times ten connections is 500, and the database refuses: `too many connections`.

The fix is a **connection pooler** sitting between your app and the database. PgBouncer is the classic; Neon, Supabase, and others run one for you and give you a second, "pooled" connection string. Use the pooled string from serverless code, and the direct string for migrations and `psql`. (Poolers in their fastest mode don't support everything, which is why migrations use the direct one.)

If you deploy a normal, always-running Node server (Render, Railway, Fly), you don't have this problem: one process, one pool.

Latency, and why it changes how you query

Locally, a query takes under a millisecond and the network adds nothing. In the cloud, the network adds a few milliseconds per round trip even within the same region. So:

- The **N+1 problem** (chapters 32 and 42) hurts far more: 100 small queries is now $100 \times 5 \text{ ms}$ = half a second, before any work is done. One join wins by a mile.
- Building JSON **in the database** (chapter 37) instead of assembling it in Node from several queries saves round trips.
- `\timing` in `psql` against the cloud database shows you the real number your users will feel.

Costs

Free tiers typically give you a small database (a few hundred MB to a few GB), limited compute, and sometimes they pause after inactivity (the first query after a pause takes a second or two to wake up). That's fine for learning, portfolios, and small apps. Paid plans start at a few dollars a month. The things that cost money: storage, compute hours, data transferred out, and backups kept longer. **Set a billing alert** before you give anyone a card number.

Moving data around

Everything from chapter 35 works across the internet:

```
pg_dump -Fc -d "postgres://...local.../sales" -f sales.dump
pg_restore -d "postgres://...cloud...?sslmode=require" --no-owner sales.dump
```

That's how you seed a new cloud database from your laptop, move from one provider to another, or pull a copy of production down to debug locally (after scrubbing sensitive columns; `pg_dump --exclude-table-data` helps). The `--no-owner` avoids errors when the cloud server doesn't have a role called `postgres`.

Who can connect

Managed databases replace `pg_hba.conf` with an **IP allow list** in the dashboard: only connections from these addresses are accepted. Lock it to your API server's address and your own, if the provider supports it. Some providers accept connections from anywhere by default and rely on the password and SSL alone, which is acceptable for a small project and not for a company.

Checklist before you go live

- ☐ The app connects as a **least-privilege role**, not the admin.
- ☐ The connection string is only in environment variables; `.env` is gitignored.
- ☐ `sslmode=require` on every connection.
- ☐ Migrations run on deploy, before the new code starts.
- ☐ Backups confirmed in the dashboard, and you've done one test restore.
- ☐ Indexes on every foreign key and every filtered or sorted column (chapter 31).
- ☐ The database's region matches your API server's region.
- ☐ The pooled connection string, if you're on serverless.
- ☐ A billing alert set.

Common mistakes

1. The admin connection string in the app

One leak, total loss. App role.

2. `sslmode` missing

Some providers refuse the connection; others accept it unencrypted. Always `?sslmode=require`.

3. Database and API in different regions

Every query crosses an ocean. Put them together.

4. Serverless without a pooler

`too many connections` under load. Use the pooled string.

5. Assuming "managed" means "designed"

The provider keeps the server alive. Your schema, indexes, and queries are still yours to get right.

6. Only ever testing locally

Local is fast and forgiving. Run your report queries against the cloud database once and look at the timings.

Quick recap

- A managed cloud database is PostgreSQL run and backed up by someone else, reached through a connection string with a real host and `?sslmode=require`.
 - Pick a provider (specialist, app platform, or big cloud), a region near your API, and a free tier to start. Check current offers; they change.
 - Your code doesn't change: `DATABASE_URL` becomes an environment variable on the hosting platform.
 - Build the structure with migrations or a restored dump. Connect as an app role. Keep the string secret and rotate it if it leaks.
 - Serverless needs a pooled connection string. Network latency makes $N+1$ and chatty code hurt, so join and build JSON in the database.
 - Managed means the server is looked after. Design, indexes, roles, and queries are still your job.
-

Next: try the [exercises](#), then move on to the last chapter: [44 Final Project](#).

Exercises

How to do these:

- You'll need to sign up for a free-tier PostgreSQL provider. Any from the notes is fine; the steps are the same everywhere. Keep `playground/ch43/notes.md` with what you did and what you saw.
 - **Never paste a connection string into these notes or into git.** Write `postgres://...` and leave it at that.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your notes.
-

Exercise 1 (Easy): Hello, cloud

1. Create a free database. Note the region you chose and why.
2. Connect to it from psql with the connection string. Run `SELECT version(), now();`. Is the version the same as your local one? Is the time zone?

3. Turn on `\timing` and run `SELECT 1;` ten times. Write down the typical time, and compare it with the same query on your laptop.
-

Exercise 2 (Easy): Move the shop up

1. Dump your local `sales` database in custom format (chapter 35).
2. Restore it into the cloud database.
3. Run your chapter 29 headline-numbers query against the cloud. Still `664.16` ?
4. Run the whole `report.sql` with `\timing`. Which section is slowest, and roughly how much slower is it than locally?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Least privilege, remotely

1. In the cloud database, create `shop_app` with the chapter 34 grants (read everything, write orders and order items, update stock).
 2. Give your `node-shop` project a second env file, `.env.cloud` (gitignored), whose `DATABASE_URL` connects as `shop_app` to the cloud database.
 3. Run chapter 40's `list-products.ts` and `transaction.ts` with `--env-file=.env.cloud`. Both should work. Then try a script that deletes a category. It should be refused.
 4. In your notes: where exactly does the admin connection string live now, and where does the app's?
-

Exercise 4 (Medium): Feel the latency

Run chapter 42's N+1 exercise against the cloud database: twelve separate item queries versus one join. Time both from Node with `console.time` and `console.timeEnd`.

Then do the same for `GET /orders/5` two ways: the single `jsonb_build_object` query from chapter 40, versus one query for the order plus one for its items, assembled in JavaScript.

In your notes: the local difference was tiny. What's the cloud difference, and what does it tell you about how to write queries for a deployed app?

Exercise 5 (Challenge): Ship it

1. Deploy your chapter 40 API (the `node:http` server, or an Express version if you've learned it) to a platform that runs Node (Render, Railway, Fly.io, or similar), with `DATABASE_URL` set as an environment variable to the `shop_app` cloud string.
2. Open `/products?category=Bags` on the public URL, from your phone.
3. If you have a React app from the React course deployed, point its `fetch` at your API's public URL and render the products. (You'll need the CORS header from chapter 40's exercise 5.)
4. Go through the "checklist before you go live" in the notes and tick each item honestly in your notes, with one sentence of evidence for each.

Stuck? The hint is in the [Hints section](#) at the end.

44 Final Project

What you'll build

The database for an app of **your own**, from a one-page brief to a deployed system your React front end can talk to. You choose the app. The checklist below makes sure it uses everything in this course.

There's no starter folder this time. The brief is yours to write.

By the end, you'll have a repository with:

- `BRIEF.md` : what the app does, written like the client briefs in chapters 19 and 36,
- `plan.md` : tables, columns, types, rules, relationships, and a diagram,
- `migrations/` : numbered files that build the whole structure from nothing,
- `seed.sql` : realistic sample data, including the awkward cases,
- `queries.sql` : the questions the app will ask, each one checked,
- `views.sql` , `functions.sql` , `roles.sql` : the database-side logic and access,
- `api/` : a Node service that answers at least five routes from the database,
- `backup.md` and `deploy.md` : proof you backed it up, restored it, and put it in the cloud,
- `README.md` : how to run it, and what you learned.

Choosing your app

Pick something you'd actually use, with at least **five** kinds of thing in it and at least one **many-to-many** relationship. Good candidates from your React course:

App	Things in it	The many-to-many
Task board (React chapter 37)	users, boards, lists, cards, labels, comments	cards ↔ labels
Online bookstore (React chapter 29)	books, authors, customers, orders, reviews	books ↔ authors
Recipe finder (React chapter 21)	recipes, ingredients, users, collections, ratings	recipes ↔ ingredients, collections ↔ recipes
Something of your own	A habit tracker, a club's membership system, a small clinic's appointments, a library	Find it in your nouns

Avoid two things: anything that holds real people's private data, and anything so large you can't finish it in a few weeks. A small thing done completely beats a big thing done halfway.

The checklist

Your project must include each of these, and your `README.md` must say **where** each one is. The chapter numbers tell you where you learned it.

Design (Level 2)

- ☐ At least 5 tables, each with a primary key, in third normal form (18)
- ☐ At least one one-to-many and one many-to-many relationship, with foreign keys and deliberate `ON DELETE` choices (16, 17)
- ☐ `NOT NULL`, `UNIQUE`, and `CHECK` constraints wherever the brief implies a rule (13)
- ☐ The right types: `numeric` for money, `timestampz` for moments, `date` for days (12)
- ☐ At least one deliberate snapshot or stored total, explained in `plan.md` (18)

Queries (Level 3)

- ☐ At least 10 queries in `queries.sql` answering real questions from the brief, using joins, grouping, an outer join, a CTE, and a window function (20 to 28)
- ☐ Every "per X" report includes the X that has nothing (23)

Safety and speed (Level 4)

- ☐ Every multi-step write wrapped in a transaction, and one place where `FOR UPDATE` matters (30)
- ☐ Indexes on every foreign key and every column your queries filter or sort on, with one `EXPLAIN ANALYZE` before and after saved as evidence (31, 32)
- ☐ At least two views, one of which hides something from someone (33)
- ☐ An app role with least privilege, and a read-only role (34)
- ☐ A backup taken, restored into a copy, and checked (35)

Real-app features (Level 5)

- ☐ One `jsonb` or array column where it's genuinely the right choice, and one API response built with `jsonb_build_object` (37)
- ☐ One function and one trigger, each with a comment in `functions.sql` saying why it belongs in the database rather than the app (38)
- ☐ Full-text or trigram search on at least one text field (39)
- ☐ A Node API with parameterized queries, a transaction, and error codes mapped to friendly messages (40)
- ☐ The whole structure built by numbered migrations (41)
- ☐ The database deployed to a cloud provider, with the API connecting as the app role over SSL (43)

Optional

- ☐ The same data access written with an ORM, for comparison (42)
- ☐ A React front end calling the API

Milestones

Work through these in order. Each is a day or two.

1. The brief

Write `BRIEF.md` in plain English, as the app's owner would: what it does, who uses it, what must never happen ("a card can't be in two lists at once", "a review needs a purchase"). One page. Include at least three rules that will become constraints and one that will need a trigger.

2. The plan

Nouns become tables. Columns, types, which can be NULL, which constraints. Relationships with their `ON DELETE` decisions. The diagram. Normalize, then name your deliberate exceptions. Check it against the Level 2 part of the checklist before writing any SQL.

3. Migrations and seed

`001_` creates the first tables, and so on. Run them with your chapter 41 runner into a fresh local database. Then write `seed.sql` with data that includes the awkward cases: the user with nothing, the thing that's never been used, the cancelled one, the item that appears in both a live and a dead parent. Those are what make your queries honest.

4. Break it on purpose

`checks.sql` : for every rule in the brief, a statement that tries to break it, and the error PostgreSQL gave. If a rule can't be broken, you haven't enforced it.

5. Queries

`queries.sql`, with a comment above each saying which question from the brief it answers. Check each one against the seed data by hand for at least one row.

6. Database logic

Views for the shared calculations and for hiding columns. A function for the operation that must be one step. A trigger for the bookkeeping that must never be forgotten. Search on the field people will actually search.

7. Protect it

Roles. Indexes, with evidence. A backup, restored, checked.

8. The API

Five routes minimum. One with an optional filter. One that writes, in a transaction. One that returns a nested JSON document built by the database. Friendly errors.

9. Deploy

Cloud database, migrations run against it, API deployed with `DATABASE_URL` as an environment variable, connecting as the app role. Open a route from your phone.

10. Write it up

`README.md` : how to run it locally (three commands), how it's deployed, the checklist with the location of each item, and a section called "**What I'd do differently**". That last section is the one a future employer will read first.

How to get unstuck

- **The design feels wrong?** Reread chapter 18's method: "what is this a fact about?"
- **A query gives the wrong number?** Chapter 29's traps: cancelled rows leaking in, zeroes dropped by a join, counting lines instead of orders.
- **Something's slow?** `EXPLAIN ANALYZE` it. Chapter 32.
- **Not sure whether it should be a constraint, a trigger, or app code?** Chapter 38's last section.
- **Stuck for more than an hour?** Ask Claude, with your `plan.md` and the exact error message.

Common mistakes

1. Too big

Fifteen tables and a half-finished API. Five tables, finished, deployed, written up.

2. Skipping the brief

Without the rules written down, there's nothing to enforce and nothing to test. The brief is where constraints come from.

3. Seed data that's too clean

If every user has orders and every product has sold, your outer joins are never tested. Add the empties.

4. Hand-edited production

Everything through migrations, even when it's "just one column".

5. Admin credentials in the deployed app

The last checklist item exists for a reason.

6. No write-up

The project isn't done until someone else could run it and understand your decisions.

Quick recap

- Pick a small app you care about, with five tables and a many-to-many.
 - Brief, plan, migrations, seed, checks, queries, then views and functions and triggers and search, then roles and indexes and backup, then the API, then deploy, then the README.
 - Every item on the checklist, with its location named in the README.
 - Finished and deployed beats ambitious and half-done.
-

Congratulations! You've reached the end of the PostgreSQL course. 🎉 You started with "what is a database?" and you've ended with a normalized, indexed, access-controlled, backed-up database in the cloud, with your own API in front of it. That's what back-end developers do. Try the [exercises](#) if you want to push further, then go back to the roadmap and tick the last box.

After this: a Node framework like Express or Fastify will tidy up your API's routing. React Native lets your database power a phone app. And nearly everything here transfers unchanged to MySQL, SQLite, and SQL Server.

Exercises

How to do these:

- These are stretch goals for after your final project is deployed and written up. Each one goes deeper into a chapter.
 - Add each one you do as a section in your README under "Stretch goals", with the evidence.
 - Pick the ones that fit your app. You don't need all five.
-

Stretch 1: A real dashboard

Build a **materialized view** (chapter 33) that summarizes your app's activity by day (sign-ups, orders, posts, whatever your app does), and an API route that returns it as JSON. Schedule a refresh: a small Node script run by

Task Scheduler or your platform's scheduler, or a trigger that refreshes on change if the table is small. Include the view's `refreshed_at` in the response so users know how fresh it is.

Stretch 2: Audit everything

Add an `audit_log` (`id`, `table_name`, `row_id`, `action`, `old_data jsonb`, `new_data jsonb`, `changed_by`, `changed_at`) and **one generic trigger function** (chapter 38) that writes to it, attached to your three most important tables. The trick: `to_jsonb(OLD)` and `to_jsonb(NEW)` turn any row into JSON, so one function serves every table. Then write a query that shows the full history of a single row.

Stretch 3: Row-level security

Chapter 34 mentioned it. Read the PostgreSQL docs for `CREATE POLICY`, then make one table in your app show each user only their own rows when the app role runs `SET app.current_user_id = ...` at the start of a request. Prove it with two users in psql. In your README: what did it make easier, and what did it make harder?

Stretch 4: The ORM comparison

Rewrite your five API routes with Drizzle or Prisma (chapter 42), side by side with the `pg` versions. Measure: lines of code, number of queries sent (turn on the ORM's query logging), and the time for your heaviest route. Write a paragraph: which would you choose for this app, and why?

Stretch 5: Load it up

Use `generate_series` (chapter 31) to inflate your biggest table to a million rows, in a copy of your database. Run your ten queries from `queries.sql` with `EXPLAIN ANALYZE`. Which ones got slow? Fix each with an index or a rewrite, and record the before and after. Then, in your README, write the one sentence you'd tell your past self about designing for scale.

Hints

Try each exercise on your own first. Come here only when you're stuck, and read one hint at a time.

01 What Is a Database?

Exercise 1 (Easy): Spot the database

[Back to the exercise](#)

Hint

Ask yourself: what does the app remember about me between sessions? What did I create? What did other people create that I can see?

Exercise 2 (Easy): CRUD in real life

[Back to the exercise](#)

Hint

Ticking a task as done is a change to an existing task. Which CRUD letter is that?

Exercise 3 (Medium): The spreadsheet problem

[Back to the exercise](#)

Hint

Think about: twelve people saving the same file, a student's name being spelled three different ways, and what happens to the report when someone accidentally deletes a row.

Exercise 4 (Challenge): Read SQL before you learn it

[Back to the exercise](#)

Hint

Read left to right, one word at a time. `FROM products` tells you which table. `WHERE ...` narrows down which rows. `SET` changes a value.

02 Tables, Rows and Columns

Exercise 2 (Easy): Design a contacts table

[Back to the exercise](#)

Hint

What does your phone show when you open a contact? Name, phone number, email... Also think about whether someone might not have an email. What goes in that cell?

Exercise 3 (Medium): Fix the bad table

[Back to the exercise](#)

Hint

Look at the `courses` cells. Look at the two phone columns together. And think about what happens to `age` next year.

Exercise 4 (Medium): Find the tables in a library

[Back to the exercise](#)

Hint

"Who has borrowed what" is a thing too: a *loan*. It happens on a date, involves one book and one member, and has a due date. How would a loan row point at its book and its member?

Exercise 5 (Challenge): A cinema booking app

[Back to the exercise](#)

Hint 1

A screening is *not* the same as a film. *Dune* is one film, but it might be shown 12 times this week. Each showing is one screening row.

Hint 2

A booking connects one customer to one seat at one screening. So a booking row will have three "pointer" columns.

03 Installing PostgreSQL

Exercise 2 (Easy): The database does maths

[Back to the exercise](#)

Hint

Compare the last two. What's different about `8` and `8.0`?

Exercise 4 (Medium): Text and time

[Back to the exercise](#)

Hint

Errors in PostgreSQL always start with `ERROR:` . Read the whole line. It usually tells you what it *thought* you meant.

04 Your First Database

Exercise 2 (Easy): Clean up

[Back to the exercise](#)

Hint

You can only drop a database you're *not* connected to. Where do you need to be first?

Exercise 3 (Easy): Escape the trap

[Back to the exercise](#)

Hint

If you get tangled up, Ctrl+C always gets you back to `=#`.

Exercise 4 (Medium): A file with several statements

[Back to the exercise](#)

Hint

Every statement needs its own semicolon, or PostgreSQL will try to read two lines as one statement and complain.

Exercise 5 (Challenge): Read the help

[Back to the exercise](#)

Hint

A safe one to try is `CREATE DATABASE test_db WITH OWNER = postgres;`. It creates a database owned by the `postgres` user, which is what you'd get anyway.

05 Your First Table

Exercise 1 (Easy): Fix the broken statements

[Back to the exercise](#)

Hint

Read the error message each time. It points at the spot where PostgreSQL got confused, which is usually *just after* the real mistake.

Exercise 3 (Medium): A shopping cart

[Back to the exercise](#)

Hint

Price is a decimal, so it's `numeric(10, 2)`, not `integer`. For "which product", a column called `product_id` of type `integer` will do for now. Chapter 16 shows how to link it properly.

Exercise 5 (Challenge): The library, in SQL

[Back to the exercise](#)

Hint 1

For "which book" and "which member" in the loans table, use `book_id integer` and `member_id integer`, the same trick as Exercise 3.

Hint 2

If you get an error about a name, check it isn't an SQL word. `date` is a type name, so a column called `date` is asking for trouble. Try `loan_date` or `borrowed_on`.

06 Adding Data

Exercise 1 (Easy): Fill the to-do list

[Back to the exercise](#)

Hint

For "no due date", either leave `due_date` out of the column list for that row... but you can't do that for just one row in a multi-row insert. So write `NULL` in its place instead.

Exercise 2 (Easy): Fix the broken inserts

[Back to the exercise](#)

Hint

Four different mistakes: the wrong kind of quotes, a date that doesn't exist, a count that doesn't match, and an apostrophe that ends the text too early.

Exercise 4 (Medium): Break the promises

[Back to the exercise](#)

Hint

Number 2 might surprise you. PostgreSQL rounds `12.5` to fit an `integer` rather than refusing it. Check what got stored, and delete that row afterwards with `DELETE FROM products WHERE name = '...'`; (you'll learn `DELETE` properly in chapter 10).

Exercise 5 (Challenge): A re-runnable seed file

[Back to the exercise](#)

Hint

Why do the ids start from 1 again after dropping and recreating? Because the automatic counter belongs to the table. Drop the table, and the counter goes with it.

07 Reading Data

Exercise 2 (Easy): Nicer headings

[Back to the exercise](#)

Hint

Capital letters in an alias need double quotes: `AS "Task"`.

Exercise 3 (Medium): Stock value

[Back to the exercise](#)

Hint

Multiplying by 1.2 adds 20%. Expressions can use more than one column: `price * stock`.

Exercise 4 (Medium): A sentence per row

[Back to the exercise](#)

Hint

Glue pieces together with `||`. Numbers join onto text without any conversion in PostgreSQL, so `'costs ' || price` works. Don't forget the spaces inside your quoted text.

Exercise 5 (Challenge): Days until due

[Back to the exercise](#)

Hint

Subtracting one date from another gives the number of days between them. You'll need to write the fixed date as a date, not text: `date '2026-09-30'`, or `'2026-09-30'::date`.

08 Filtering Rows

Exercise 1 (Easy): One condition each

[Back to the exercise](#)

Hint

A boolean column can be tested directly: `WHERE on_sale` means "where on_sale is true", and `WHERE NOT is_done` means "where is_done is false". `WHERE on_sale = true` works too.

Exercise 3 (Medium): Patterns

[Back to the exercise](#)

Hint

For number 3, `_` matches exactly one character. Three underscores in a row match exactly three characters.

Exercise 4 (Medium): Where's the NULL?

[Back to the exercise](#)

Hint

"Is unknown different from 2026-10-01?" can't be answered. Add a second condition with `OR` that catches the unknown case.

Exercise 5 (Challenge): The shop's front page

[Back to the exercise](#)

Hint

Without brackets, `AND` is checked first, so the condition becomes "on sale, OR (cheap and at least 10 in stock)". Which on-sale product has fewer than 10 in stock?

09 Sorting and Limiting

Exercise 2 (Easy): Top three

[Back to the exercise](#)

Hint

"Most expensive first" is descending. Then limit.

Exercise 5 (Challenge): Pages and a combined query

[Back to the exercise](#)

Hint

Page 2 of 2-per-page means skip 2, take 2. Check the clause order: `WHERE` , then `ORDER BY` , then `LIMIT / OFFSET` .

10 Changing and Deleting Data

Exercise 2 (Easy): Sale prices

[Back to the exercise](#)

Hint

`price * 1.1` gives the new price. The column is `numeric(10, 2)` , so PostgreSQL rounds the result to 2 decimal places when it stores it. That's why 27.489 becomes 27.49.

Exercise 5 (Challenge): The seatbelt

[Back to the exercise](#)

Hint

For step 3, look at the row count PostgreSQL replies with. Then look very carefully at the `WHERE`.

11 Project: My Book Collection

Milestone 1: Plan on paper

[Back to the lesson](#)

Hint

A rating is a whole number. "Finished?" is a `boolean`. "Finished on" is a `date`. "How long" is pages, an `integer`. Genre is `text`, and it's **one** genre per book, for now.

Milestone 4: Ask questions

[Back to the lesson](#)

Hint

Question 9 is two conditions joined with `OR`, and each condition is itself two things joined with `AND`. Use brackets. Question 10 needs `||`, and unfinished books will produce NULL sentences if you forget the `WHERE`.

Exercise 1 (Easy): A wishlist

[Back to the exercise](#)

Hint

If you use the starter data, its `INSERT` doesn't mention `is_owned`, so those rows will get NULL. Think about whether that's what you want, or whether the column should default to `true`. `DEFAULT` is coming in chapter 13, but you can peek: `is_owned boolean DEFAULT true`.

Exercise 2 (Easy): Recent reads

[Back to the exercise](#)

Hint

`current_date - 90` is a date 90 days ago. Compare `finished_on` to it.

Exercise 3 (Medium): Re-reads

[Back to the exercise](#)

Hint

`SET times_read = times_read + 1` uses the old value. Chapter 10.

Exercise 4 (Medium): Export to a spreadsheet

[Back to the exercise](#)

Hint

The whole `SELECT` goes inside the brackets, and it can have `WHERE` and `ORDER BY` like any other query. Forward slashes in the path, as always.

Exercise 5 (Challenge): Reading speed

[Back to the exercise](#)

Hint 1

`finished_on - started_on` is the number of days, as an integer. To get decimals, turn one side into a decimal first: `pages::numeric / (finished_on - started_on)`. The `::numeric` is a cast, covered in chapter 12.

Hint 2

`WHERE finished_on > started_on` keeps only books that took at least a day. The result will have many decimal places; chapter 24 has `round()` for that.

12 Data Types in Depth

Exercise 1 (Easy): Predict, then run

[Back to the exercise](#)

Hint

For `'5' + 3`, PostgreSQL sees a quoted value next to a `+` and a number, and works out that `'5'` must be meant as a number. For `'5' || 3`, it sees `||` and knows that means text. Same `'5'`, two different guesses.

Exercise 2 (Easy): Pick the type

[Back to the exercise](#)

Hint

Ask two questions for each: "would I ever do maths on this?" and "does it need to be exact?"

Exercise 3 (Medium): Watch `numeric` work

[Back to the exercise](#)

Hint

`numeric(6, 2)` has room for 6 digits in total, 2 of them after the point. Count the digits in `12345.67`.

Exercise 4 (Medium): Date arithmetic

[Back to the exercise](#)

Hint

Number 5: `current_date + interval '3 months'` gives you a `timestamp`, not a `date`. Cast it back with `::date` if you want just the day.

Exercise 5 (Challenge): Flight times

[Back to the exercise](#)

Hint

`timestampz` stores one exact moment and knows that 08:00 in Kathmandu is 02:15 in London and 06:15 in Dubai. Plain `timestamp` doesn't know where the clock was, so it just subtracts the numbers.

13 Constraints

Exercise 1 (Easy): A users table with rules

[Back to the exercise](#)

Hint

The fourth one is the sneaky one. Read the "Expecting a default to rescue an explicit NULL" mistake in the notes.

Exercise 3 (Medium): Reviews

[Back to the exercise](#)

Hint

`length(body) >= 10` is a valid `CHECK` condition. Because it's about one column, you can put it right after the column, but to give it a name you'll need the `CONSTRAINT name CHECK (...)` form at the end of the table.

Exercise 5 (Challenge): Meeting rooms

[Back to the exercise](#)

Hint 1

"Once per room per day" is uniqueness of a **pair** of columns. It goes at the end of the table: `UNIQUE (room_name, booked_on)`.

Hint 2

The fixed set of statuses is `CHECK (status IN ('pending', 'confirmed', 'cancelled'))`. Combine it with `NOT NULL` and `DEFAULT 'pending'` on the same column.

14 Primary Keys

Exercise 2 (Easy): Natural or surrogate?

[Back to the exercise](#)

Hint

Ask: can this value change? Can it be missing? Is it short and stable enough that you'd be happy typing it in a `WHERE` a hundred times? Country and currency codes usually pass; almost everything else doesn't.

Exercise 4 (Medium): The import trap

[Back to the exercise](#)

Hint

```
ALTER TABLE tickets ALTER COLUMN id RESTART WITH 4;
```

Exercise 5 (Challenge): Composite keys at the cinema

[Back to the exercise](#)

Hint

If you add `id ... PRIMARY KEY`, the pair is no longer the primary key, so you'd need `UNIQUE (screening_id, seat_number)` to keep the rule. Either design is valid; the rule is what matters.

15 Changing a Table's Shape

Exercise 1 (Easy): A category for every product

[Back to the exercise](#)

Hint

`WHERE name IN ('Notebook', 'Pen', 'Sticky notes', 'Stapler')` sets four at once. Step 3 would have failed if you'd done it *before* step 2. Why?

Exercise 2 (Easy): Better names, better types

[Back to the exercise](#)

Hint

A `date` converts to a `timestampz` automatically, at midnight in your time zone, so no `USING` is needed.

Exercise 3 (Medium): Rules the data must already follow

[Back to the exercise](#)

Hint

Sticky notes have 0 in stock. A shop can genuinely run out, so `>= 0` is the honest rule, and `> 0` was wrong.

`CHECK (length(name) > 0)` or `CHECK (name <> '')` both work for step 4.

Exercise 4 (Medium): Rewiring with the lights on

[Back to the exercise](#)

Hint

`ALTER COLUMN price TYPE integer USING round(price)`. The `round()` function is from chapter 24, but you can use it early.

Exercise 5 (Challenge): Your first migration, and its undo

[Back to the exercise](#)

Hint 1

The "down" file can't bring back the old `is_on_sale` values, because `DROP COLUMN` destroyed them. That's normal: a down migration restores the *shape*, not always the *data*. Say so in a comment. It's also why step 4 of the up migration deserves a `BEGIN` and a hard look before `COMMIT`.

Hint 2

The `UNIQUE` constraint gets an automatic name, probably `products_sku_key`. Check `\d products` so your down file drops the right name. Or give it your own name in the up file, which is why people name constraints.

16 Relationships and Foreign Keys

Exercise 1 (Easy): Authors get their own table

[Back to the exercise](#)

Hint

Parents first: insert the authors, note their ids (they'll be 1, 2, 3 on a fresh table), then insert the books using those ids.

Exercise 3 (Medium): Three ways to say goodbye

[Back to the exercise](#)

Hint

The invoice is the blocker, on purpose. A business doesn't delete invoices; it would delete the invoice row only if it was a mistake, or (more realistically) never delete the team at all. For this exercise, deleting the invoice row first is fine, but say in a comment what you'd really do.

Exercise 5 (Challenge): A small blog

[Back to the exercise](#)

Hint 1

A common answer: `posts.author_id` is `RESTRICT` (make the user delete or transfer their posts first, or mark the account inactive), `comments.post_id` is `CASCADE` (a comment is meaningless without its post), `comments.author_id` is `SET NULL` (the comment stays, shown as "deleted user"). Other answers are fine if you can defend them.

Hint 2

If you chose `RESTRICT` on posts, step 2 will refuse to delete a user who has posts. That's the design working. Delete a user who has only comments, or delete the posts first, and write down which you did.

17 Many-to-Many Relationships

Exercise 1 (Easy): Books and genres, properly

[Back to the exercise](#)

Hint

Look up ids with `SELECT id, title FROM books;` and `SELECT id, name FROM genres;` before writing the `book_genres` inserts.

Exercise 3 (Medium): Playlists

[Back to the exercise](#)

Hint

The rule is "one song per slot", not "one slot per song". So the unique pair is `(playlist_id, position)`. Give the junction table its own `id` as primary key, and put `UNIQUE (playlist_id, position)` on it.

Exercise 4 (Medium): Actors and films

[Back to the exercise](#)

Hint

`PRIMARY KEY (actor_id, film_id, role)` works, with `role text NOT NULL`. The lesson: the key is whatever combination must be unique *in the real world*, not automatically the two foreign keys.

Exercise 5 (Challenge): Order items with a price snapshot

[Back to the exercise](#)

Hint

For step 2, look up each price with `SELECT id, price FROM products;` and type it into the insert. In chapter 25 you'll learn to have the database copy it for you with a subquery.

18 Normalization

Exercise 1 (Easy): Name the anomaly

[Back to the exercise](#)

Hint

Ask "what is this a fact about?" for `department_phone` and `manager_name`. Neither is a fact about Asha.

Exercise 2 (Easy): Rule 1

[Back to the exercise](#)

Hint

Phones: a student has many, a phone belongs to one student. That's one-to-many (chapter 16). Courses: a student takes many, a course has many students. That's many-to-many (chapter 17).

Exercise 3 (Medium): Normalize an invoice sheet

[Back to the exercise](#)

Hint

`hourly_rate` on a line is probably a snapshot: the freelancer's rate may rise next year, but this invoice was at this rate. `line_total` is $\text{hours} \times \text{rate}$, so calculate it. `invoice_total` is the sum of lines, so calculate it too, unless you decide an issued invoice must be frozen, which is also defensible.

Exercise 4 (Medium): Duplication or snapshot?

[Back to the exercise](#)

Hint

Number 3 is the interesting one. If a product is renamed, should old receipts show the new name or the old one? There's a case either way. Pick one and say why. Number 1 is usually a mistake (you want the customer's *current* email), unless you have a rule that receipts go to the address used at the time.

Exercise 5 (Challenge): A school timetable

[Back to the exercise](#)

Hint 1

The `lessons` table is the junction in the middle, with foreign keys to `classes`, `subjects`, `teachers`, and `rooms`, plus `day` and `period`. Each of the three "can't double-book" rules is a `UNIQUE` on a different combination: `(room_id, day, period)`, `(teacher_id, day, period)`, `(class_id, day, period)`. One table can have several `UNIQUE` constraints.

Hint 2

`room_capacity` is a fact about the room, not the lesson. `teacher_email` is a fact about the teacher. Move both.

19 Project: Online Shop Database

Milestone 1: Find the things

[Back to the lesson](#)

Hint

Categories, products, customers, orders. And "which products, and how many, are in each order" is a thing in its own right: an order item. Chapter 17.

Milestone 2: Columns, types, and rules

[Back to the lesson](#)

Hint

Every table gets the `id` recipe, except possibly the one in the middle, which can use a composite key. Timestamps for "when" are `timestampz NOT NULL DEFAULT now()`. Money is `numeric(10, 2)`. Statuses are `text NOT NULL DEFAULT 'pending' CHECK (status IN (...))`.

Milestone 3: Draw the links

[Back to the lesson](#)

Hint

Four foreign keys. Two are one-to-many (product → category, order → customer). Two make up the many-to-many (order item → order, order item → product). The brief says the shop never wants to lose an order record. What does that mean for deleting a customer? For deleting a product that's been sold?

Milestone 8: Ask questions

[Back to the lesson](#)

Hint

Question 3 needs the three-table pattern from chapter 17: start from `order_items`, join `products`, filter by `order_id`. Question 2 is the two-table join from chapter 16 with an `ORDER BY`.

Exercise 1 (Easy): Product reviews

[Back to the exercise](#)

Hint

It's a junction table between customers and products with extra facts on it (chapter 17). The "only once" rule is the composite key or a `UNIQUE (customer_id, product_id)`.

Exercise 2 (Easy): Multiple addresses

[Back to the exercise](#)

Hint

`INSERT INTO addresses (customer_id, label, city, is_default) SELECT id, 'home', city, true FROM customers;` copies every customer's city into a new address row. `INSERT ... SELECT` is new, but it reads naturally: insert whatever this `SELECT` returns. Then `ALTER TABLE customers DROP COLUMN city;`.

Exercise 4 (Medium): Stock movements

[Back to the exercise](#)

Hint

`quantity_change integer NOT NULL CHECK (quantity_change <> 0)` and `reason text NOT NULL CHECK (reason IN ('delivery', 'sale', 'correction', 'return'))`. Movements are a history, so this table never gets `UPDATE` d or `DELETE` d, only `INSERT` ed.

Exercise 5 (Challenge): Order status history

[Back to the exercise](#)

Hint

The history table's `status` column should have the **same** `CHECK (... IN ...)` list as `orders.status`. If the list exists in two places, a future change has to update both. That's a small normalization smell, and some teams solve it with a tiny `order_statuses` lookup table and two foreign keys. Either approach is fine here; say which you chose and why.

20 Counting and Totals

Exercise 1 (Easy): Three counts

[Back to the exercise](#)

Hint

Number 3 is `count(column)`, not `count(*)`. Why does that work here?

Exercise 3 (Medium): Receipt totals

[Back to the exercise](#)

Hint

A "line" is a row in `order_items`. "Units" is the sum of `quantity`. The total is the sum of `quantity * unit_price`.

Exercise 5 (Challenge): Real business questions

[Back to the exercise](#)

Hint

For number 3, `shipped_at - ordered_at` is an interval, and `min()` and `max()` work on intervals. Think about whether you need a `WHERE` for the unshipped orders.

21 Grouping

Exercise 3 (Medium): `HAVING`

[Back to the exercise](#)

Hint

Question 3 needs both a `WHERE` (to drop cancelled rows) and a `HAVING` (to keep groups with 2+). Which comes first in the query?

Exercise 5 (Challenge): A customer summary

[Back to the exercise](#)

Hint

`min(ordered_at)::date` and `max(ordered_at)::date`. You can `ORDER BY` an alias.

22 Joins

Exercise 3 (Medium): Reports across tables

[Back to the exercise](#)

Hint

Query 2: `order_items` → `products` → `categories`. Query 3: `customers` → `orders` → `order_items`, with `count(DISTINCT o.id)`.

Exercise 4 (Medium): Questions with a `WHERE`

[Back to the exercise](#)

Hint

Query 1: `ordered_at >= '2026-09-01'` is enough, since nothing is later than September. In real data you'd add `AND ordered_at < '2026-10-01'`.

Exercise 5 (Challenge): Receipts, all at once

[Back to the exercise](#)

Hint 1

Query 1: every non-aggregated column in `SELECT` must be in `GROUP BY`: `GROUP BY o.id, c.name, o.status`.

Hint 2

Query 2 joins `employees` to itself **twice**, with three aliases: `e`, `m`, and `mm`. The second `ON` matches `mm.id = m.manager_id`. Employees without a manager's manager will vanish; that's expected for an inner join.

23 Outer Joins

Exercise 3 (Medium): Complete tables

[Back to the exercise](#)

Hint

Query 3: `max(o.ordered_at)::date`, and `ORDER BY last_order DESC NULLS LAST`.

Exercise 4 (Medium): ON or WHERE ?

[Back to the exercise](#)

Hint

Query 2 combines both tricks: the status condition goes in the `ON` (so customers with no pending orders survive), and then `WHERE o.id IS NULL` keeps only those.

Exercise 5 (Challenge): Two lists, and a chain

[Back to the exercise](#)

Hint 1

Query 1: `WHERE c.id IS NULL OR n.email IS NULL`.

Hint 2

Query 2: both joins must be `LEFT JOIN`, the count must be `count(DISTINCT o.id)` (chapter 22's trap), and the sum needs `coalesce(..., 0)`. For the `0.00` formatting, `coalesce(sum(...), 0.00)` or a cast to `numeric(10, 2)` both work.

24 Handy Built-in Functions

Exercise 1 (Easy): Text clean-up

[Back to the exercise](#)

Hint

Initials: `left(name, 1) || left(split_part(name, ' ', 2), 1)`.

Exercise 2 (Easy): Dates for humans

[Back to the exercise](#)

Hint

Three `to_char()` calls on `ordered_at` with patterns `'DD Mon YYYY'`, `'Mon YYYY'`, and `'Dy'`.

Exercise 3 (Medium): Price bands and status groups

[Back to the exercise](#)

Hint

Query 1 is a `CASE` expression with an alias, then `GROUP BY` that alias.

Exercise 4 (Medium): Filling in blanks

[Back to the exercise](#)

Hint

Query 2: `round(extract(epoch from shipped_at - ordered_at) / 3600)`. Query 3: `coalesce(nullif(city, 'Kathmandu'), '(local)')`. Say out loud what each layer does.

Exercise 5 (Challenge): A monthly report, nicely formatted

[Back to the exercise](#)

Hint 1

Group by `date_trunc('month', ordered_at)`, and `ORDER BY` that, but display `to_char(date_trunc('month', ordered_at), 'Mon YYYY')`. You can `GROUP BY` and `ORDER BY` an expression that isn't shown.

Hint 2

`round(100.0 * count(*) FILTER (WHERE status = 'shipped') / count(*), 1) || '%'`. The `100.0` (not `100`) is what makes the division decimal.

25 Subqueries

Exercise 1 (Easy): Compared to the average

[Back to the exercise](#)

Hint

Query 2 needs the "totals per order" query twice: once as the table you select from, once inside `avg()` in a scalar subquery. Chapter 26 will show how to write it once.

Exercise 3 (Medium): EXISTS and NOT EXISTS

[Back to the exercise](#)

Hint

Question 2 is two conditions: `EXISTS (some order for this customer)` AND `NOT EXISTS (an order for this customer whose status <> 'shipped')`.

Exercise 4 (Medium): Correlated

[Back to the exercise](#)

Hint

Query 3: `(SELECT count(*) FROM orders o WHERE o.customer_id = c.id) AS orders . count(*)` of nothing is 0, which is why Elina doesn't need a `coalesce`.

Exercise 5 (Challenge): Subqueries that change data

[Back to the exercise](#)

Hint

Query 3: `WHERE order_id IN (SELECT id FROM orders WHERE status = 'cancelled')`. One row, worth 45.00.

26 Common Table Expressions

Exercise 1 (Easy): Rewrite with a CTE

[Back to the exercise](#)

Hint

```
SELECT order_id, total FROM order_totals WHERE total > (SELECT avg(total) FROM order_totals).
```

Exercise 2 (Easy): Monthly revenue

[Back to the exercise](#)

Hint

Join `orders` to `order_totals`, filter on status, group by `date_trunc('month', o.ordered_at)::date`. Because there's one total per order, `count(*)` is correct here.

Exercise 3 (Medium): Share of revenue

[Back to the exercise](#)

Hint

Two CTEs: `category_revenue` (the chapter 22 query) and `total` (one row: `sum(revenue)` from the first CTE). The main query can select from both at once: `FROM category_revenue, total` is a tiny cross join against a one-row table, which is a common trick for "divide by the grand total".

Exercise 5 (Challenge): Recursion

[Back to the exercise](#)

Hint 1

Query 1: the starting row is `WHERE name = 'Dev Shrestha'`; the recursive half joins `employees e ON e.manager_id = team.id`. Exclude Dev himself in the final `SELECT`.

Hint 2

Query 3: `first_orders AS (SELECT customer_id, min(ordered_at) AS first_at FROM orders GROUP BY customer_id)`, then join `orders` on `customer_id` **and** `ordered_at > first_at`, and count.

27 Combining Results

Exercise 3 (Medium): Both, or one but not the other

[Back to the exercise](#)

Hint

Query 3's parts each need a join from `order_items` to `orders` to get at the date. Wrap the `INTERSECT` in `WHERE id IN (...)` on `products`.

Exercise 4 (Medium): A report with a total

[Back to the exercise](#)

Hint

Query 2: `WITH report AS (...the UNION ALL with sort_key...) SELECT status, orders FROM report ORDER BY sort_key, status; . ORDER BY` can use a column you don't show.

Exercise 5 (Challenge): Harder set questions

[Back to the exercise](#)

Hint

Query 2 is two `INTERSECT` s chained, wrapped in `WHERE id IN (...)`.

28 Window Functions

Exercise 3 (Medium): Over time

[Back to the exercise](#)

Hint

Query 3: `ordered_at::date - lag(ordered_at::date) OVER (PARTITION BY customer_id ORDER BY ordered_at) .`

Exercise 4 (Medium): Shares

[Back to the exercise](#)

Hint

Both start with the same CTE: one row per order with its total (and customer, and month). Then `sum(total) OVER (PARTITION BY customer_id)` for query 1, `rank() OVER (PARTITION BY month ORDER BY total DESC)` for query 2.

Exercise 5 (Challenge): Top N and the 80% rule

[Back to the exercise](#)

Hint 1

Query 1 is the three-step pattern from the notes: aggregate per product and category, `row_number() OVER (PARTITION BY category ORDER BY revenue DESC)`, then `WHERE rn <= 2`.

Hint 2

Query 2: `round(100 * sum(revenue) OVER (ORDER BY revenue DESC) / sum(revenue) OVER (), 1)`. The first `sum` is running (it has an `ORDER BY`); the second is the grand total (no `ORDER BY`).

29 Project: Sales Report

Milestone 2: Headline numbers

[Back to the lesson](#)

Hint

A CTE that totals each live order, then aggregates over it. The chapter 26 pattern.

Milestone 5: By customer and by city

[Back to the lesson](#)

Hint

Total live orders in a CTE, then `customers LEFT JOIN` that CTE. Count the CTE's id column, not `*`.

Exercise 2 (Easy): Who to email

[Back to the exercise](#)

Hint

The second one: a CTE with `max(ordered_at)` per customer, then keep rows where that's `< '2026-09-01'`. Bikram is the answer (last order 9 August).

Exercise 3 (Medium): A month-by-category grid

[Back to the exercise](#)

Hint

Start from a CTE of live order lines joined to products and categories, with the line total and the month. Then one column per category: `coalesce(sum(line_total) FILTER (WHERE category = 'Stationery'), 0) AS stationery`, and so on, plus `sum(line_total) AS total`. Group and sort by the real month, display the text.

Exercise 4 (Medium): What-if pricing

[Back to the exercise](#)

Hint

`sum(quantity * CASE WHEN product_id = 2 THEN 1.50 ELSE unit_price END)`. Pens sold 35 units in live orders, so the difference is 35×0.30 .

Exercise 5 (Challenge): A parameterized report

[Back to the exercise](#)

Hint

`WHERE o.ordered_at >= :month_start AND o.ordered_at < :month_end`. PostgreSQL will convert the text to a timestamp for the comparison. `\copy` can't use a CTE in brackets with variables easily on one line, so put the final query in a view (chapter 33 is coming) or keep it short.

30 Transactions

Exercise 3 (Medium): A savepoint in a batch

[Back to the exercise](#)

Hint

`SAVEPOINT before_fee; ... the failing UPDATE ... ROLLBACK TO SAVEPOINT before_fee; ... then the date update and COMMIT .`

Exercise 5 (Challenge): Deadlock, then prevent it

[Back to the exercise](#)

Hint

For step 1, type A's first update, then B's first update, then A's second update (A now waits), then B's second update. Within a second one of them errors. For step 2, the second session simply waits on row 1, then proceeds after the first commits: a queue, not a cycle.

31 Indexes

Exercise 3 (Medium): Column order

[Back to the exercise](#)

Hint

Think about the phone book. A query can use the index when it filters on the **first** column (and optionally more). It can't skip the first column.

Exercise 5 (Challenge): The price of writing

[Back to the exercise](#)

Hint

For the insert: `INSERT INTO big_copy SELECT g, 5, now(), 1 FROM generate_series(2000001, 2100000) g; .`
Step 3's second query asks for a range the partial index doesn't cover (`total > 100` includes rows with `total ≤`

490), so PostgreSQL can't use it.

32 Why Is My Query Slow?

Exercise 2 (Easy): The newest N

[Back to the exercise](#)

Hint

Step 3: a sort on two columns wants an index on both, in that order: `(total DESC, id)`. An index can be declared with a direction.

Exercise 4 (Medium): Joins big and small

[Back to the exercise](#)

Hint

Without an index on `big_customers.id`, each matching order (a few thousand) may trigger a scan of the 1001-row table, or PostgreSQL builds a hash of it. With the primary key, it can look each one up directly. Compare the inner node.

33 Views

Exercise 5 (Challenge): A dashboard that refreshes

[Back to the exercise](#)

Hint

Materialized views can't be refreshed inside the same transaction in a way that others see until commit, but within your own session the refreshed data is visible, which is enough for this experiment. `CREATE INDEX ON dashboard (refreshed_at);` works like on a table.

34 Users, Roles and Permissions

Exercise 3 (Medium): The app's own key

[Back to the exercise](#)

Hint

Column-level grants: `GRANT UPDATE (status, shipped_at) ON orders TO shop_app;` and `GRANT UPDATE (stock) ON products TO shop_app;`. Remember `INSERT` on `orders` and `order_items` too.

Exercise 5 (Challenge): Hide the emails, for real

[Back to the exercise](#)

Hint

The view is owned by `postgres`, who can read `customers`, so `marketing` reading the view works even without access to the table. That's the feature. Step 3 shows the one way to break it.

35 Backup and Restore

Exercise 3 (Medium): The 3pm rescue

[Back to the exercise](#)

Hint

`pg_restore -t order_items` tries to `CREATE TABLE`, which fails if the table exists. Either `DROP TABLE order_items` first, or add `--clean` to the restore. Both are fine; know what each does.

Exercise 5 (Challenge): Automate it

[Back to the exercise](#)

Hint

In a `.bat` file, `%DATE%` contains slashes on some systems, which aren't allowed in filenames. `set TODAY=%DATE:\/=-%` replaces them. For the newest file: `for /f %f in ('dir /b /o-d backups\*.dump') do (set NEWEST=%f & goto :found)`. Ask Claude if the batch syntax fights you; the point is the policy, not the scripting.

36 Project: Bank Account System

Milestone 1: Design

[Back to the lesson](#)

Hint

`balance numeric(14, 2) NOT NULL DEFAULT 0 CHECK (balance >= 0) . transactions.id bigint . transfer_id uuid (nullable: deposits don't have one). gen_random_uuid() makes one.`

Exercise 2 (Easy): A monthly statement

[Back to the exercise](#)

Hint

Opening balance is a scalar subquery: `(SELECT coalesce(sum(amount), 0) FROM transactions WHERE account_id = 1 AND created_at < date_trunc('month', now()))`. The running balance within the month starts from that number: add it to the window sum.

Exercise 3 (Medium): A daily limit

[Back to the exercise](#)

Hint

`SELECT coalesce(-sum(amount), 0) FROM transactions WHERE account_id = $x AND kind IN ('withdrawal', 'transfer_out') AND created_at >= current_date;` gives today's outgoing total as a positive number.

Exercise 4 (Medium): Closing an account

[Back to the exercise](#)

Hint

`CONSTRAINT closed_accounts_are_empty CHECK (NOT is_closed OR balance = 0)`. Read it as "either it's open, or it's empty".

Exercise 5 (Challenge): The auditor's toolkit

[Back to the exercise](#)

Hint

Unbalanced: `GROUP BY transfer_id HAVING count(*) <> 2 OR sum(amount) <> 0`. Daily totals: `sum(amount)`
`FILTER (WHERE amount > 0)` and `FILTER (WHERE amount < 0)`, grouped by `created_at::date`.

37 JSON and Arrays

Exercise 2 (Easy): Tags

[Back to the exercise](#)

Hint

"Both" is containment: `tags @> ARRAY['eco', 'bestseller']`.

Exercise 3 (Medium): An API response

[Back to the exercise](#)

Hint

Three levels of `jsonb_build_object`, with two `jsonb_agg` subqueries: one over orders (correlated to the customer), and inside it one over items (correlated to the order). The `total` is `(SELECT sum(quantity * unit_price) FROM order_items WHERE order_id = o.id)`.

Exercise 5 (Challenge): Array or table?

[Back to the exercise](#)

Hint 1

The check: `CHECK (tags <@ ARRAY['eco', 'new', 'bestseller', 'sale'])`. `<@` is "is contained by": every element of `tags` must be in the allowed list.

Hint 2

Renaming in the array version: `UPDATE products SET tags = array_replace(tags, 'sale', 'clearance') WHERE 'sale' = ANY (tags)`; and you also have to change the `CHECK`. In the table version it's one `UPDATE tags SET`


```
name = ...
```

38 Functions and Triggers

Exercise 3 (Medium): The bank, properly

[Back to the exercise](#)

Hint

To lock in id order regardless of which is `p_from`: `PERFORM id FROM accounts WHERE id IN (p_from, p_to) ORDER BY id FOR UPDATE;` (`PERFORM` is PL/pgSQL for "run this query and ignore the result"). Then `SELECT ... INTO` the two balances and frozen flags separately. `gen_random_uuid()` into a `v_transfer_id uuid` variable, used in both inserts.

Exercise 5 (Challenge): The shop's bookkeeping

[Back to the exercise](#)

Hint

For `AFTER DELETE`, `NEW` is NULL; use `OLD.product_id` and `OLD.quantity`. Question 2 has two rows because the status started as `pending`; only *changes* are logged. If you want the initial status recorded too, you'd add an `AFTER INSERT` trigger on `orders`.

39 Full-Text Search

Exercise 3 (Medium): A results page

[Back to the exercise](#)

Hint

```
FROM articles, websearch_to_tsquery('english', :term') AS q WHERE search @@ q ORDER BY  
ts_rank(search, q) DESC LIMIT 5.
```

Exercise 5 (Challenge): One search function

[Back to the exercise](#)

Hint

PL/pgSQL with `RETURN QUERY SELECT ...;` for the full-text part, then `IF NOT FOUND THEN RETURN QUERY SELECT ... (trigram) ...; END IF;` `FOUND` is true if the previous query returned rows.

40 Connecting from Node.js

Exercise 2 (Easy): A command-line search

[Back to the exercise](#)

Hint

```
const city = process.argv[2]; then pool.query("... WHERE city = $1 ...", [city]).
```

Exercise 3 (Medium): Friendly errors

[Back to the exercise](#)

Hint

`23505` is unique, `23514` is check. A `switch (e.code)` reads well here.

Exercise 5 (Challenge): An API for your React app

[Back to the exercise](#)

Hint

For the search pattern: `const pattern = "%" + q + "%";` then `WHERE name ILIKE $1 with [pattern]`. The `%` signs are part of the value, not the SQL, so this is still safe. Your browser may block a `fetch` from a React dev server on a different port. That's **CORS**, and adding the header `Access-Control-Allow-Origin: *` to your responses fixes it for local development.

41 Migrations

Exercise 4 (Medium): A required column on a live table

[Back to the exercise](#)

Hint

The `UPDATE` touches every row. On a huge table you'd fill it in batches (`WHERE id BETWEEN ...`), spread over several runs, and set `NOT NULL` last. Alternatively, `ADD COLUMN country text NOT NULL DEFAULT 'Nepal'` is instant in modern PostgreSQL, because the default is stored once instead of being written into every row.

Exercise 5 (Challenge): Down migrations

[Back to the exercise](#)

Hint

`SELECT version FROM schema_migrations ORDER BY applied_at DESC LIMIT 1`, then `version.replace(".sql", ".down.sql")` gives the file name. `process.argv[2] === "down"` chooses the mode. Make sure the `.down.sql` files are skipped by the normal "up" loop (filter out names ending in `.down.sql`).

42 ORMs

Exercise 1 (Easy): Translate from SQL

[Back to the exercise](#)

Hint

```
import { eq, desc } from "drizzle-orm". Number 1 is .where(eq(products.isActive, true)).orderBy(desc(products.price)).limit(3).
```

Exercise 3 (Medium): Where the ORM runs out

[Back to the exercise](#)

Hint

```
import { pgView } from "drizzle-orm/pg-core" and export const orderTotals = pgView("order_totals", {  
  orderId: integer("order_id"), ... }).existing();
```

 tells Drizzle the view already exists in the database.

43 PostgreSQL in the Cloud

Exercise 2 (Easy): Move the shop up

[Back to the exercise](#)

Hint

```
pg_restore -d "postgres://...?sslmode=require" --no-owner sales.dump.
```

Exercise 5 (Challenge): Ship it

[Back to the exercise](#)

Hint

Most platforms need a `start` script in `package.json`. `"start": "node --env-file=.env api.ts"` won't work there, because no `.env` file exists on the server; the variables are injected for you. Use `"start": "node api.ts"` and let `process.env.DATABASE_URL` come from the platform. If the platform's Node version can't run `.ts` files directly, compile with `tsc` first or rename the file to `.js` for the deploy.